



Construire avec Nkentseu

De la première ligne de C
à dix applications complètes

Cours complet — débutant à professionnel

Rêvons ensemble et réinventons le futur

Rihen — 5 août 2026

Table des matières

I	Les outils	1
0	Avant-propos	3
0.1	À qui s'adresse ce cours	3
0.1.1	Ce que ce cours couvre, et ce qu'il ne couvre pas	3
0.2	Comment lire les exemples.	4
0.3	Comment le dépôt est organisé	4
0.3.1	Kernel/Foundation — les briques de base	4
0.3.2	Kernel/System — les services système	5
0.3.3	Kernel/Runtime — les modules du moteur	5
0.3.4	Engine et Integrations	5
0.3.5	Applications et Ressources	5
0.4	Les outils de base du moteur	6
0.4.1	Les types primitifs	6
0.4.2	Chaînes et conteneurs — NKContainers	6
0.4.3	Mathématiques — NKMath	7
0.4.4	Fichiers et chemins — NKFileSystem	7
0.4.5	Mémoire — NKMemory	7
0.4.6	Les habitudes à prendre	8
0.5	Compiler : le système Jenga	8
0.5.1	À quoi ressemble un fichier .jenga	9
0.5.2	Les deux commandes que vous taperez	9
0.5.3	Où sont les journaux.	10
0.6	Le point d'entrée : nkmain, pas main	10
0.7	Exercices	11
1	NKCode : l'éditeur du dépôt.	13
1.1	Le lancer	13
1.2	Les raccourcis	14
1.3	IntelliSense : clangd	14
1.4	Construire sans quitter l'éditeur	14
1.5	Les autres services	15
II	Les langages	17
2	Le C : la matière dont tout est fait	19
2.1	Compiler, lier.	19
2.2	Premier programme	19
2.3	Variables et types.	20
2.3.1	const, et pourquoi on le met partout	20
2.4	Opérateurs.	21
2.4.1	Les opérateurs de bits, en pratique	21
2.5	Contrôle de flux	22
2.6	Fonctions.	22
2.6.1	Portée et durée de vie	22

2.7	Pointeurs	23
2.7.1	Pointeurs de fonction	23
2.8	Tableaux et chaînes.	24
2.9	Structures, unions, énumérations	24
2.10	Conversions	25
2.11	Mémoire : pile et tas	25
2.12	Le préprocesseur	25
2.13	Compilation séparée	26
3	Le C++ : ce qu'il ajoute, et ce que Nkentsu en garde	28
3.1	La norme employée	28
3.2	Classes et structures	28
3.3	Construire et détruire : RAII	29
3.4	Références	29
3.5	Surcharge et valeurs par défaut	30
3.6	Héritage et polymorphisme	30
3.7	Templates	31
3.8	Espaces de noms.	31
3.9	Ce que Nkentsu emploie, et ce qu'il refuse	32
3.9.1	La règle : zéro bibliothèque standard	32
3.9.2	Et l'état réel, mesuré	32
3.9.3	Deux mots qui trompent.	33
3.10	Ce qu'on ne trouve presque pas ici	33
4	Templates : écrire une fois pour tous les types	35
4.1	Modèle de fonction	35
4.2	Modèle de classe.	36
4.3	Paramètres qui ne sont pas des types	36
4.4	Spécialisation	36
4.5	Modèles variadiques	37
4.6	Références universelles et transfert parfait	37
4.7	Traits de type.	38
4.8	Ce que les templates coûtent	38
5	C++ moderne : lambdas, déplacement, possession	40
5.1	Les lambdas : une fonction écrite là où on s'en sert	40
5.2	Le déplacement : transférer au lieu de copier	41
5.3	La possession : qui libère, et quand	42
5.4	Deux types qui remplacent des conventions.	43
5.5	Le reste du confort, en une page.	43
III	Les fondations	46
6	NKMemory : qui alloue, qui libère	48
6.1	La règle, et ce qu'elle coûte de violer	48
6.2	Les quatre fonctions de base.	48
6.3	Allouer un objet, pas des octets	49
6.4	Neuf allocateurs, et pourquoi il en faut plusieurs	50
6.4.1	Le cas le plus utile : l'allocateur linéaire	50
6.5	Savoir ce qu'on a alloué	51
6.6	Ce que vous écrirez vraiment	51

7	NKContainers : ranger des données sans la bibliothèque standard	54
7.1	Pourquoi ne pas employer la bibliothèque standard	54
7.2	Le tableau dynamique : <code>NkVector<T></code>	54
7.3	Les chaînes : <code>NkString</code>	55
7.4	Les tables	56
7.5	Le reste, et à quoi ça sert.	57
8	NKMath : vecteurs, matrices, angles	59
8.1	Les vecteurs	59
8.2	Matrices et quaternions	59
8.3	Angles, aléatoire, géométrie	60
8.4	Les fonctions	60
8.5	Les constantes	60
8.6	La virgule flottante, et ce qu'elle ne sait pas faire	61
9	NKLogger : dire ce qui se passe	63
9.1	Les six niveaux	63
9.2	□ Deux familles de messages, et on ne les mélange pas	63
9.3	Où le journal atterrit	64
9.4	Ce qu'un bon message contient	65
10	NKThreading : faire deux choses à la fois	67
10.1	Ce qu'est une course, et pourquoi c'est si dur	67
10.2	Les fils	67
10.3	Protéger une donnée : mutex et verrou à portée.	68
10.4	Le reste de la boîte à outils.	69
10.5	Ce que le rendu impose	69
IV	Le moteur	71
11	Le décor : la pile complète	73
11.1	Vue d'ensemble	73
11.2	Étage 1 — la fenêtre : <code>NKWindow</code>	73
11.3	Étage 2 — les événements : <code>NKEvent</code>	74
11.4	Étage 3 — le GPU. Attention, il y a deux abstractions	75
11.4.1	<code>NKRHI</code> — l'abstraction GPU bas niveau.	75
11.4.2	<code>NKCanvas</code> — sa propre abstraction, pour la 2D.	75
11.4.3	Les cinq backends 2D de <code>NKCanvas</code> , et leur état réel	76
11.5	Étage 4 — <code>NKCanvas</code> , le rendu 2D	77
11.6	Étage 5 — <code>NKGui</code> , les widgets	77
11.6.1	Le point capital : <code>NKGui</code> ne connaît aucun GPU	78
11.6.2	Ce que <code>NKGui</code> produit : <code>NKGuiDrawList</code>	78
11.7	Le pont : marier <code>NKGui</code> et <code>NKCanvas</code>	79
11.7.1	<code>NKGuiCanvasBackend</code> — le pont vers <code>NKCanvas</code>	79
11.7.2	<code>NKGuiRHIBackend</code> — le pont vers <code>NKRHI</code>	80
11.7.3	Pourquoi cette séparation ?	80
11.8	Le chemin complet d'un clic, de bout en bout	81
11.9	Deux mots sur les macros d'export.	81
11.10	Exercices	82

12	NkCanvas : dessiner en deux dimensions	83
12.1	Ce que NkCanvas est, en une phrase.	83
12.2	Les deux niveaux d'API.	83
12.2.1	NkIRenderer2D : l'interface.	83
12.2.2	NkRenderer2D : la façade.	84
12.3	La cible de rendu : NkRenderWindow	84
12.3.1	Le redimensionnement	85
12.4	Le repère : Y vers le bas	86
12.4.1	La caméra 2D	86
12.5	Les primitives disponibles	87
12.5.1	Le sommet.	87
12.5.2	Les primitives composites, réalisées dans l'en-tête	87
12.5.3	Les types de primitives, et l'absence de QUADS	88
12.6	Ce que NkCanvas n'a PAS	88
12.7	Les couleurs	88
12.8	Les textures	89
12.8.1	Créer et remplir	89
12.8.2	La texture blanche de 1 pixel	89
12.8.3	Les pixels conservés côté CPU	90
12.9	Les polices et le texte	90
12.9.1	Comment un texte devient des triangles	90
12.10	Les formes persistantes	91
12.11	Le batching : le vrai sujet du module.	92
12.11.1	L'accumulation.	92
12.11.2	La capacité, et l'histoire qui va avec	93
12.11.3	La soumission : Flush()	93
12.11.4	Le vrai appel de dessin.	94
12.12	Le découpage (scissor) et sa pile	94
12.12.1	Changer de découpe force un vidage	95
12.12.2	Le retournement d'axe sous OpenGL	95
12.13	La surface réellement utilisée en pratique	96
12.14	Un programme complet	96
12.14.1	L'ossature	97
12.14.2	Fenêtre, cible, état	97
12.14.3	La boucle	98
12.14.4	Le rendu	98
12.14.5	Ce que ce programme raconte	99
12.15	Récapitulatif des pièges du chapitre	99
12.16	Exercices	100
13	NKGui : l'interface immédiate	102
13.1	L'idée : on ne crée rien, on redéclare tout	102
13.1.1	Le mode immédiat.	102
13.1.2	Ce que ce choix vous épargne	102
13.1.3	Une note d'honnêteté sur la documentation du module	103
13.2	L'identité d'un widget	103
13.2.1	La pile d'identifiants	104
13.3	Le contexte : NkGuiContext	104
13.3.1	Cycle de vie	105
13.3.2	Le contexte courant	105
13.3.3	Le thème.	106

13.4	L'entrée : NkGuiInput.	106
13.4.1	Ce que l'application pose	107
13.4.2	Ce que NKGui calcule	107
13.4.3	La répétition au maintien	107
13.4.4	Les touches suivies.	107
13.5	Le cycle d'une image.	108
13.5.1	BeginFrame	108
13.5.2	EndFrame.	109
13.5.3	L'invariant d'ordre	109
13.6	Comment un widget conserve son état	110
13.7	Comment un widget reçoit les événements	110
13.7.1	Le problème	110
13.7.2	La solution : glouton, avec une image de retard	111
13.7.3	ButtonBehavior : la sémantique du clic	112
13.7.4	La machine à états d'interaction	113
13.8	Le routeur d'occlusion : gérer vos propres surfaces flottantes	113
13.8.1	Les quatre couches	114
13.8.2	L'API	114
13.8.3	L'usage type	114
13.9	La liste de dessin	116
13.9.1	Choisir la bonne couche : ctx.DL()	116
13.9.2	Les primitives	117
13.9.3	Le découpage	117
13.9.4	Le texte, et le piège du flou	118
13.9.5	La formule de la ligne de base	119
13.10	La mise en page	119
13.10.1	Le modèle : un curseur qui descend	120
13.10.2	Les sept modes de flux	120
13.10.3	Les helpers de curseur	120
13.10.4	Les conteneurs.	121
13.10.5	Le DPI	122
13.11	Le catalogue des widgets	122
13.11.1	Texte et boutons	122
13.11.2	Cases à cocher	123
13.11.3	Valeurs numériques	123
13.11.4	Saisie de texte	124
13.11.5	Graphiques et couleur	124
13.11.6	Images.	125
13.11.7	Arbres, sélection, onglets	125
13.11.8	Zones défilables et tables	126
13.11.9	Popups, combos, menus, infobulles	126
13.11.10	Le contexte utilisé comme un widget	127
13.12	Panneaux, fenêtres, docking	127
13.12.1	BeginPanel / EndPanel	127
13.12.2	Begin / EndWindow	128
13.12.3	Le docking	129
13.13	Le hook de style	129
13.14	Les limites dures	130
13.15	Récapitulatif des idiomes.	131
13.16	Exercices	131

14	Construire une application, de zéro à l'exécutable	133
14.1	Vue d'ensemble du squelette	133
14.2	Étape 1 — la fenêtre	133
14.3	Étape 2 et 3 — contexte GPU et cible de rendu	134
14.4	Étape 4 — le contexte NKGui.	135
14.5	Étape 5 — le pont vers NkCanvas	135
14.6	Étape 6 — la police, et l'upload de son atlas	136
14.6.1	Ce que fait LoadEmbedded	136
14.6.2	Ce que fait UploadFontGray8	137
14.6.3	Les polices de repli.	138
14.7	Étape 7 — brancher l'entrée	138
14.7.1	Souris et fermeture	138
14.7.2	La molette	139
14.7.3	Le double-clic	139
14.7.4	Le texte et les touches	140
14.7.5	Le presse-papiers	140
14.8	La boucle principale	140
14.8.1	Le temps.	141
14.8.2	Les événements — avant tout le reste.	141
14.8.3	Le redimensionnement	141
14.8.4	L'image d'interface.	142
14.8.5	Le curseur	143
14.8.6	La présentation	143
14.8.7	Ce que fait Submit, en détail.	144
14.9	Le programme complet	145
14.10	Ajouter une image	148
14.11	Le fichier de build	149
14.11.1	Pourquoi chaque dépendance est là.	150
14.11.2	Enregistrer la cible et compiler.	150
14.12	Diagnostiquer les silences	151
14.13	Pour aller plus loin	151
14.14	Exercices	152
15	La coquille d'application : ce que vous n'écrivez plus	154
15.1	Le plus petit programme complet	154
15.2	Les méthodes que vous remplissez	155
15.3	Les trois services que la coquille rend	155
15.3.1	La boucle cède la main, et sur le Web c'est vital	156
15.3.2	La zone sûre, mesurée et non devinée.	156
15.3.3	Le pointeur : souris et doigt unifiés	157
15.4	Deux coquilles, et il faut choisir	157
15.5	L'écran d'ouverture, offert	158
V	Les modules	161
16	NKImage : charger, fabriquer, afficher des images	163
16.1	Ce qu'est NKImage, et ce qu'il n'est pas	163
16.1.1	Aucune dépendance externe.	163
16.1.2	NKImage ne connaît aucun GPU.	164
16.1.3	L'arborescence.	164

16.2	Les deux vocabulaires de formats	164
16.3	Deux familles d'API, et pourquoi il faut choisir	165
16.3.1	L'API instance : l'objet vit sur la pile	166
16.3.2	L'API statique : vous possédez le pointeur	166
16.3.3	Copie interdite, déplacement autorisé	167
16.4	Charger une image.	167
16.4.1	La signature qui compte	167
16.4.2	Les accesseurs	167
16.4.3	Le stride : la cause numéro un des images « penchées »	168
16.5	Fabriquer une image de toutes pièces	168
16.5.1	Le squelette canonique de l'API statique	168
16.5.2	Créer une image remplie d'une couleur	169
16.5.3	Dessiner dans une image (sur le processeur)	169
16.6	Écrire sur le disque	170
16.6.1	Le dispatch par extension	170
16.6.2	Les deux dernières lignes sont des mensonges	170
16.6.3	Encoder en mémoire.	171
16.7	Transformer une image.	172
16.7.1	Redimensionner	172
16.7.2	Recadrer, convertir, recopier	173
16.7.3	Le cas HDR.	173
16.8	Le cas particulier du GIF animé.	173
16.9	Libérer correctement : les quatre cas.	174
16.10	Afficher une image dans une interface NKGui	175
16.10.1	Le widget	175
16.10.2	L'upload	175
16.10.3	La séquence complète.	176
16.10.4	Le chemin plus court : NkTexture de NkCanvas	177
16.11	Décoder ailleurs, uploader ici	178
16.12	État réel du module	178
16.12.1	Ce qui fonctionne	178
16.12.2	Ce qui ne fonctionne pas	179
16.12.3	Les deux classes utilitaires exportées	179
16.13	Exercices	180
17	NKFont : polices, atlas et métriques	182
17.1	Le module ne connaît ni image, ni GPU, ni fenêtre.	182
17.2	Les quatre objets à connaître	183
17.2.1	NkFontAtlas — le propriétaire de tout	183
17.2.2	NkFont — une face rastérisée à une taille	184
17.2.3	NkFontGlyph — la structure qui fait tout le travail	184
17.2.4	NkFontConfig — les réglages, posés avant l'ajout	185
17.3	Les dix polices embarquées : écrire du texte sans aucun fichier	186
17.4	La séquence canonique en quatre temps	187
17.4.1	Le mécanisme de repli en action.	188
17.5	Dessiner une chaîne de caractères	188
17.5.1	Le patron universel	188
17.5.2	Le pixel-snap, et l'invariant subtil	189
17.5.3	La ligne de base, pas le haut du texte	189
17.5.4	Mesurer avant de dessiner.	190

17.6	De l'atlas à l'écran	190
17.6.1	L'atlas est en alpha 8 bits	190
17.6.2	Le piège d'upload	191
17.6.3	Le chemin NKGui, celui que vous emploierez	191
17.6.4	Changer d'échelle : recharger et ré-uploader	192
17.7	Le mode SDF : du texte net à toute taille.	193
17.8	Les limites dures, et l'invariant du <code>Build()</code>	193
17.8.1	Des tableaux de taille fixe	193
17.8.2	<code>Build()</code> n'est ni réentrant ni parallélisable.	194
17.8.3	Le <code>mergeMode</code> duplique les pointeurs.	194
17.8.4	Le cache de tailles rend <code>nullptr</code> la première fois	195
17.9	Ce que le module ne fait pas.	195
17.10	L'autre wrapper : <code>renderer::NkFont</code>	195
17.11	Récapitulatif des trois chemins.	196
17.12	Exercices	197
18	NKAudio : faire du bruit	199
18.1	Le module, et sa dépendance surprenante.	199
18.2	L'invariant fondateur : un seul format interne	200
18.3	Le patron canonique en cinq temps	200
18.3.1	Temps 1 — initialiser le moteur	201
18.3.2	Temps 2 — décoder le fichier	202
18.3.3	Temps 3 — lancer une voix	202
18.3.4	Temps 4 — piloter pendant la lecture	203
18.3.5	Temps 5 — l'ordre de fermeture, qui n'est pas négociable.	204
18.4	Jouer un son au clic d'un bouton.	204
18.4.1	Au démarrage — une seule fois	204
18.4.2	Dans la frame — quand le widget renvoie <code>true</code>	205
18.4.3	À la fermeture	205
18.5	Les bus : régler des groupes de sons ensemble	205
18.6	Les fichiers longs : le streaming	207
18.7	Le <i>callback</i> procédural, et le fil audio.	208
18.7.1	L'ordre de destruction avec un lecteur de flux	208
18.8	Le micro	209
18.9	Tester sans carte son	210
18.10	État réel du module	211
18.10.1	Ce qui fonctionne	211
18.10.2	Ce qui ne fonctionne pas	211
18.10.3	Le reste de la surface, en survol	212
18.11	Relier le son à l'image	212
18.12	Exercices	213
19	NKMedia : lire et écrire de la vidéo	215
19.1	Le module, et une inversion inhabituelle.	215
19.1.1	Ici, les commentaires <i>sous-estiment</i> le module.	216
19.2	<code>NkMediaProbe</code> : qu'y a-t-il dans ce fichier?	217
19.3	Écrire une vidéo	218
19.3.1	L'API	218
19.3.2	Le squelette de référence	219
19.3.3	Deux limites de l'écriture.	220
19.3.4	La séquence d'images, à la manière de Blender	220

19.4	Lire une vidéo	221
19.4.1	L'API	221
19.4.2	La boucle de lecture	221
19.4.3	Le seek n'est pas ce que vous croyez	222
19.4.4	Décoder coûte cher	222
19.5	Afficher la vidéo	223
19.5.1	Dans une fenêtre NkCanvas	223
19.5.2	Dans une interface	224
19.6	Enregistrer ce que l'application affiche	225
19.6.1	L'API	225
19.6.2	La capture, de bout en bout	225
19.6.3	Trois pièges du recorder	226
19.6.4	Le sens des pixels	227
19.7	Le démux audio, en deux mots	227
19.8	Vérifier que tout marche sur votre machine	227
19.9	Une leçon de performance qui dépasse la vidéo	228
19.10	Exercices	229
20	NKNetwork : faire parler deux machines	231
20.1	Où il vit, et ce dont il a besoin	231
20.1.1	Deux avertissements sur la documentation du module	232
20.2	Le périmètre de ce chapitre	232
20.3	Le socle : la plateforme	233
20.3.1	Les codes de retour	233
20.3.2	Les adresses	234
20.4	NkConnectionManager : la classe du chapitre	234
20.4.1	Trois invariants de fil d'exécution	235
20.4.2	Le bout-en-bout, tel qu'il est testé	235
20.4.3	On ne reçoit pas, on draine	236
20.5	Choisir son canal	237
20.6	NkBitStream : ne pas gaspiller le réseau	238
20.6.1	L'argument : la quantification	238
20.7	NkNetWorld : répliquer des entités	240
20.8	Le socket brut : la découverte sur le réseau local	241
20.9	HTTP, et pourquoi HTTPS échoue	243
20.10	Comment structurer une application en réseau	244
20.11	L'ordre de fermeture	245
20.12	Exercices	246
21	NKCamera : lire une caméra	248
21.1	La façade, en huit appels	248
21.2	Le format n'est pas un choix	248
21.3	L'image reçue, et ce qu'elle déclare	249
21.4	Le miroir : une question de géométrie, pas de goût	250
21.5	Ce qu'il faut faire, dans l'ordre	250
VI	Cas pratiques	253
22	Construire un jeu de plateau, de A à Z	255
	Ce que nous allons construire	255
22.1	Étape 1 — le dossier et le fichier de projet	255

22.1.1	Les six autres plateformes	257
22.2	Étape 2 — la fenêtre qui s'ouvre	257
22.3	Étape 3 — les règles : les types.	258
22.4	Étape 4 — l'initialisation	261
22.5	Étape 5 — les coups simples	261
22.6	Étape 6 — la rafle, et c'est le seul endroit délicat.	262
22.7	Étape 7 — rafle maximale, jouer, fin de partie	264
22.8	Étape 8 — le banc. Avant l'écran.. . . .	265
22.9	Étape 9 — le thème, puis la géométrie.	267
22.10	Étape 10 — le dessin.	268
22.11	Étape 11 — le clic devient un coup.	268
22.12	Étape 12 — l'adversaire	269
22.13	Étape 13 — l'ouverture, et les modes	270
22.14	Construire, lancer, emballer	270
22.15	Ce qui change pour les échecs et le ludo	271
23	Construire un jeu animé, de A à Z	273
23.1	Étape 14 — la gemme, et ce qu'elle ne fait pas	273
23.2	Étape 15 — la machine à phases.	274
23.3	Étape 16 — trouver les alignements	275
23.4	Étape 17 — la cascade	275
23.5	Étape 18 — les quatre écrans	276
23.6	Étape 19 — la progression	276
23.7	Étape 20 — le son, sans un seul fichier.	277
23.8	Étape 21 — le didacticiel	278
23.9	Étape 22 — les cas de recette d'un jeu animé	278
24	Construire un lecteur, de A à Z	281
24.1	Étape 1 — ouvrir un fichier, et le dire quand ça rate	281
24.2	Étape 2 — le son : réduire un million d'échantillons	282
24.3	Étape 3 — dessiner, et comprendre ce qu'on dessine	283
24.4	Étape 4 — la vidéo : la chaîne complète	283
24.5	Étape 5 — la cadence, et pourquoi elle n'est pas un Sleep	284
24.6	Étape 6 — traiter les pixels, et à quel endroit	285
24.7	Étape 7 — l'image : le pas de ligne.	285
24.8	Étape 8 — rendre les pixels, et les quatre cas	285
24.9	Étape 9 — la barre de transport	286
24.10	Ce que NkImageDemo enseigne malgré lui	286
25	Construire un éditeur de texte, puis de code	288
25.1	Étape 1 — décider comment ranger le texte.	288
25.2	Étape 2 — le document, et où vit l'état	289
25.3	Étape 3 — charger et enregistrer.	289
25.4	Étape 4 — ne dessiner que ce qui se voit	290
25.5	Étape 5 — le curseur, et la conversion des coordonnées.	290
25.6	Étape 6 — la saisie	291

25.7	Étape 7 — le défilement automatique	291
25.8	Étape 8 — l’annulation	292
25.9	Étape 9 — la gouttière	292
25.10	Étape 10 — décrire un langage plutôt que le programmer	293
25.11	Étape 11 — tokeniser une ligne, sans allouer	293
25.12	Étape 12 — l’état d’entrée de ligne	294
26	Construire un mini terminal, de A à Z	296
	Niveau 1 — lancer une commande sans geler	296
26.1	Étape 1 — le fil d’arrière-plan et la file	296
26.2	Étape 2 — afficher, et savoir s’arrêter	297
	Niveau 2 — un shell qui reste vivant	298
26.3	Étape 3 — pourquoi un tube ne suffit pas	298
26.4	Étape 4 — le pseudo-terminal	298
26.5	Étape 5 — isoler le code de plateforme	299
	Niveau 3 — comprendre ce que le shell renvoie	299
26.6	Étape 6 — la grille de cellules	300
26.7	Étape 7 — l’automate, et son état persistant	300
26.8	Étape 8 — dessiner la grille	301
26.9	Étape 9 — le clavier, et le sens du flux	301
26.10	Étape 10 — redimensionner, et l’historique	302
27	Projet final : NkAtelier	304
27.1	Ce que nous allons construire	304
27.2	Étape 0 — l’arborescence	305
27.3	Étape 1 — le fichier de build	305
27.3.1	Le contenu	305
27.3.2	Ce qu’il faut comprendre dans ce fichier	306
27.3.3	Enregistrer le projet dans l’espace de travail	306
27.4	Étape 2 — le squelette : fenêtre, contexte, backend, police	307
27.4.1	Les inclusions	307
27.4.2	Fenêtre et cible de rendu	307
27.4.3	Contexte NKGui et backend	308
27.4.4	La police	309
27.5	Étape 3 — les événements et la boucle	309
27.5.1	Le branchement des entrées	309
27.5.2	La boucle, et l’invariant d’ordre	310
27.6	Étape 4 — l’image (NKImage)	310
27.6.1	Fabriquer l’image	311
27.6.2	L’appeler, et l’afficher	312
27.7	Étape 5 — le son (NKAudio)	312
27.7.1	Synthétiser trois sons	312
27.7.2	Initialiser, jouer, couper	313
27.8	Étape 6 — la vidéo (NKMedia)	314
27.8.1	Écrire la vidéo	314
27.8.2	Relire la vidéo	315
27.8.3	Cadencer et afficher	316

27.9	Étape 7 — la fermeture	317
27.10	Compiler et lancer	318
27.11	Le catalogue des silences	319
27.12	Le programme, vu de haut	320
27.13	Pour aller plus loin	320
27.13.1	Enregistrer ce que l'atelier affiche	320
27.13.2	Ajouter le réseau.	321
27.13.3	Faire du décodage un travail de fond	321
27.14	Exercices	321

PARTIE I

Les outils

Avant d'écrire une ligne

On ne commence pas un métier par son sujet, on le commence par ses outils. Un menuisier apprend l'établi avant le meuble — non par tradition, mais parce qu'un outil qu'on ne maîtrise pas transforme chaque erreur en énigme.

Cette partie installe les deux outils dont tout le reste dépend : **Jenga**, qui décrit et construit un projet, et **NKCode**, l'environnement où l'on écrit. Vous y apprendrez ce que coûte un projet *sans* Jenga — la compilation à la main, ligne de commande comprise — avant de voir ce qu'il apporte. C'est délibéré : un outil dont on n'a pas senti le manque reste une formalité qu'on subit.

À la fin de cette partie, vous saurez créer un projet, le construire, le lancer, et lire ce que la construction vous dit quand elle échoue. C'est peu ; c'est exactement ce qui manque à ceux qui abandonnent.

Avant-propos

0.1 À qui s'adresse ce cours

Ce cours s'adresse à une personne qui *sait déjà programmer en C++* et qui ne connaît *rien* au moteur Nkntseu. C'est une distinction importante : nous n'expliquerons pas ce qu'est un pointeur, une référence, une lambda ou un constructeur. En revanche, nous expliquerons *tout* du moteur — y compris les choses qui paraissent évidentes une fois qu'on les a comprises, et particulièrement celles-là, parce que ce sont elles qui font perdre une soirée quand personne ne les a écrites.

Ce que vous devez savoir avant d'ouvrir ce document :

- du C++ moderne, niveau C++17 : auto, lambdas, références, constexpr, noexcept, énumérations fortement typées (enum class);
- la notion de compilation séparée : en-têtes, unités de traduction, édition de liens;
- ce qu'est un pointeur non possédant (*non-owning*) par opposition à un pointeur possédant. Ce cours en parlera souvent : le moteur en est rempli et c'est la première source de plantages au démarrage;
- quelques notions vagues de rendu : un sommet (*vertex*), un triangle, une texture, une matrice de projection. Vous n'avez pas besoin de savoir écrire un *shader* : nous n'en écrirons aucun.

Ce que vous n'avez *pas* besoin de savoir : OpenGL, Vulkan, Direct3D, Metal. Tout le cours se tient au-dessus de ces API ; le moteur les cache, et c'est précisément son intérêt.

0.1.1 Ce que ce cours couvre, et ce qu'il ne couvre pas

Les cinq premiers chapitres — ceux que vous tenez — forment un parcours complet et fermé : à la fin du chapitre 4, vous savez écrire, compiler et lancer une application graphique interactive complète avec le moteur. Le parcours est le suivant :

1. **Chapitre 0** (celui-ci) : le dépôt, les règles du projet, la compilation.
2. **Chapitre 1** : la pile logicielle complète, de la fenêtre du système d'exploitation jusqu'aux boutons.
3. **Chapitre 2** : **NkCanvas**, le moteur de rendu 2D. Comment on dessine des triangles, et ce que le module ne sait *pas* faire.
4. **Chapitre 3** : **NKGui**, le framework d'interface. C'est le cœur du cours et le chapitre le plus long.
5. **Chapitre 4** : construire une application finie, du fichier de build jusqu'à l'exécutable qui s'affiche.

Ne sont *pas* couverts ici : le rendu 3D (NKRenderer, NKRI), l'audio (NKAudio), la vidéo (NKMedia), le réseau (NKNetwork), l'intelligence artificielle (Kernel/AI). Certains de ces modules font l'objet de chapitres ultérieurs de ce même cours.

Un avertissement d'honnêteté

Ce cours dit aussi ce qui *ne marche pas*. Le moteur est un projet vivant : certains backends graphiques sont validés à l'exécution, d'autres non ; certaines fonctions déclarées retournent une valeur neutre. Chaque fois que c'est le cas, il est écrit noir sur blanc, avec le fichier et la ligne. Un cours qui promet un moteur parfait vous ferait perdre plus de temps qu'il ne vous en ferait gagner.

0.2 Comment lire les exemples

Tous les blocs de code de ce cours portent leur **provenance** en titre, sous la forme `chemin/vers/fichier.cpp:ligne`. Ce n'est pas de la coquetterie : cela signifie que vous pouvez ouvrir le fichier et vérifier. Les extraits sont pris tels quels dans le dépôt, parfois abrégés (les coupures sont signalées par des points de suspension ou un commentaire). Quand un exemple est *écrit pour le cours* et n'existe pas tel quel dans le dépôt, c'est dit explicitement.

Trois encadrés reviennent régulièrement :

- **Ce qu'il faut retenir** — la phrase à emporter, si vous ne deviez retenir qu'une chose de la section ;
- **Le piège** — un comportement qui coûte une soirée. Presque tous ceux de ce cours sont issus de bugs réellement rencontrés dans le dépôt et documentés en commentaire dans les sources ;
- **Exercice** — à faire, pas à lire.

Le dépôt de référence est `D:/Projets/2026/Nkentseu/Nkentseu`. Tous les chemins de ce cours sont relatifs à cette racine.

0.3 Comment le dépôt est organisé

Le moteur est découpé en **modules**. Un module est un dossier autonome avec ses sources, son fichier de build (`.jenga`) et, souvent, sa documentation (`README.md`, `ARCHITECTURE.md`, `ROADMAP.md`, `USAGE.md`). Les modules sont rangés par **couche**, et le principe fondamental est énoncé dans `ARCHITECTURE.md` : *chaque couche ne connaît que les couches situées en dessous d'elle*.

0.3.1 Kernel/Foundation — les briques de base

Module	Rôle
NKCore	Types primitifs (<code>int32</code> , <code>float32</code> , <code>usize...</code>), macros, asserts
NKMath	<code>NkVec2/3/4</code> , <code>NkMat4</code> , <code>NkQuat</code> , <code>NkColor</code> , <code>NkRect</code>
NKMemory	Allocateurs, <code>NkUniquePtr</code> — la porte d'entrée de toute allocation
NKContainers	<code>NkVector</code> , <code>NkString</code> , <code>NkHashMap</code>
NKPlatform	Détection de plateforme à la compilation, macros <code>NKENTSEU_PLATFORM_*</code>

Ces cinq modules n'ont *aucune* dépendance externe. Tout le reste du moteur repose dessus.

0.3.2 Kernel/System — les services système

NKFileSystem (chemins, fichiers), NKLogger (journalisation), NKStream (lecture/écriture binaire et texte), NKThreading (fils d'exécution, mutex, atomiques), NKTime (horloges, *delta time*), NKNetwork, NKReflection, NKSerialization.

0.3.3 Kernel/Runtime — les modules du moteur

C'est là que vivent les deux vedettes de ce cours :

- [Kernel/Runtime/NKCanvas](#) — le rendu 2D (chapitre 2);
- [Kernel/Runtime/NKGui](#) — le framework d'interface (chapitre 3).

et leurs voisins immédiats : NKWindow (la fenêtre native), NKEvent (les événements typés), NKFont (rasterisation de polices), NKImage (décodage PNG/JPG/SVG...), NKRHI (abstraction GPU bas niveau), NKRenderer (rendu 3D), NKAudio, NKMedia et NKSL (langage de *shaders*).

Un module que vous verrez, et qu'il ne faut pas employer

Kernel/Runtime/NKUI existe encore dans l'arbre. C'est l'« ancien » module d'interface, et il est **déprécié** : son retrait est en cours, module par module. N'écrivez aucune interface dessus, et ne prenez pas ses fichiers comme modèles — l'interface d'aujourd'hui, c'est NKGui.

Ce qui a tranché entre les deux n'est pas l'âge, c'est la sûreté des entrées : NKGui possède un routeur d'occultation par couches, interrogé avant tout test de survol. NKUI n'en a pas — son `IsOccluded` rend `false`. Un widget NKUI réagit donc à un clic tombé sur une *modale dessinée au-dessus de lui* : le dessin se superpose, l'entrée non.

0.3.4 Engine et Integrations

[Engine/NKEditorKit](#) est une *coquille* d'application d'édition : fenêtre sans décoration, barre de titre maison, docking, palette de commandes, préférences. C'est ce qui permet à l'IDE NKCode de n'écrire que ses panneaux. Ce cours ne s'en sert pas — nous construirons l'application à la main au chapitre 4 — mais il faut savoir qu'il existe, parce que la plupart des exemples réels du dépôt passent par lui.

[Integrations/NKGui](#) contient le pont alternatif entre NKGui et NKRHI ([NKGuiRHIBackend](#)), pour les applications qui font de la 3D. Nous y reviendrons au chapitre 1.

0.3.5 Applications et Resources

[Applications/](#) contient toutes les applications et démos, une par dossier. Trois nous serviront de référence tout au long du cours :

- [Applications/NKGuiDemo](#) — la démo de référence de NKGui (environ 1 067 lignes dans un seul `main.cpp`). Elle prouve le pipeline complet et couvre presque toute l'API. C'est le fichier à ouvrir à côté de ce cours;
- [Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp](#) — la démo `NkCanvas` *seul*, sans interface, à laquelle nous consacrerons la fin du chapitre 2;
- [Applications/NKCode](#) — l'IDE complet, l'application la plus grosse du dépôt. Elle sert d'étude de cas.

`Resources/` contient les données partagées (polices, textures, icônes, modèles, shaders). `Documentation/` et `Guides/` contiennent la documentation écrite. `Build/` reçoit tout ce que la compilation produit, et `logs/` les journaux d'exécution.

Ce qu'il faut retenir

Le dépôt suit une règle simple : **un dossier = un module = un fichier .jenga**. Pour comprendre ce dont un module dépend, il suffit d'ouvrir son .jenga et de lire l'appel à `nkentseudependson`. Nous le ferons souvent : c'est le moyen le plus fiable de savoir qui connaît qui.

0.4 Les outils de base du moteur

Avant d'écrire du code Nkentseu, il faut connaître la poignée d'outils qu'on retrouve dans chaque fichier du dépôt. Ils viennent tous de `Kernel/Foundation` et de `Kernel/System`, et ils reviendront à toutes les pages de ce cours.

0.4.1 Les types primitifs

Le moteur nomme explicitement la taille de ses entiers, dans `NKCore` : `int8`, `int16`, `int32`, `int64`, `uint8`, `uint16`, `uint32`, `uint64`, `float32`, `float64`, et `usize` pour une taille en octets. Ils sont disponibles dans l'espace de noms `nkentseu`. Une largeur de fenêtre est un `uint32`, une coordonnée un `float32`, un compteur de boucle un `int32` : on lit la taille dans le nom, sans avoir à connaître la plateforme.

0.4.2 Chaînes et conteneurs — `NKContainers`

Type	Rôle
<code>NkString</code>	chaîne de caractères, <code>CStr()</code> donne le <code>const char *</code>
<code>NkStringView</code>	vue non possédante sur une chaîne
<code>NkVector<T></code>	tableau dynamique — le conteneur le plus utilisé du moteur
<code>NkHashMap</code>	table associative

Les méthodes suivent la convention de nommage du moteur, en `PascalCase` : `Size()`, `Data()`, `PushBack()`, `PopBack()`, `Resize()`, `Reserve()`, `Clear()`, `Erase()`, `Begin()`, `End()`. L'accès indexé s'écrit comme partout, avec les crochets. Un parcours typique, tel qu'on en croise des centaines dans le dépôt :

`Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:122-125 (extrait)`

```
1 mScratch.Resize(dl.vtx.Size());
2 for (uint32 i = 0; i < dl.vtx.Size(); ++i) {
3     const nkgui::NkGuiVertex &s = dl.vtx[i];
4     renderer::NkVertex2D &d = mScratch[i];
```

Deux réflexes visibles ici et à reprendre : on dimensionne d'abord (`Resize`), on remplit ensuite ; et on prend une *référence* sur l'élément plutôt qu'une copie.

Pour construire une chaîne à partir de valeurs, le moteur fournit `NkPrintf`, qui renvoie une `NkString` :

Applications/NKCode/src/NKCode/Editor/NkCodeEditor.h:4585-4586 (extrait)

```
1 const NkString nb = NkPrintf("%d", i + 1);
2 const float32 nw = ctx.font->MeasureWidth(nb.CStr());
```

0.4.3 Mathématiques — NKMath

Tout vit dans l'espace de noms `nkentseu::math` : `NkVec2f`, `NkVec2i`, `NkVec2u`, `NkVec3f`, `NkVec4f` pour les vecteurs; `NkMat4f` pour les matrices; `NkQuat` pour les quaternions; `NkRect2f`, `NkRect2i` pour les rectangles; `NkColor` pour les couleurs; et les fonctions trigonométriques `NkCos`, `NkSin`, etc. Les vecteurs se construisent en agrégat, ce qui donne un code très compact :

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:90-92

```
1 math::NkVec2f ball{220.f, 200.f};
2 math::NkVec2f vel{260.f, 215.f}; // pixels / seconde
3 const float32 R = 42.f;
```

0.4.4 Fichiers et chemins — NKFileSystem

`NkPath` manipule les chemins, `NkFile` lit et écrit. Deux appels que ce cours croquera : `NkPath::GetExecutableDirectory()`, qui donne le dossier de l'exécutable, et `NkFile::ReadAllBytes`, qui charge un fichier entier — c'est ainsi que `NkFont` lit une fonte ([Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkFont.cpp:65-81](#)).

0.4.5 Mémoire — NKMemory

Toute la mémoire du moteur passe par `NKMemory`, dans l'espace de noms `nkentseu::memory`. Deux outils suffisent pour ce cours :

- `memory::NkMakeUnique<T>(args...)` crée un objet possédé par un `NkUniquePtr<T>` : il se libère tout seul en fin de portée;
- `memory::NkGetDefaultAllocator()` donne l'allocateur par défaut, dont les méthodes `New<T>(args...)` et `Delete(ptr)` servent quand on gère la durée de vie soi-même.

Il existe aussi des allocateurs spécialisés — linéaire pour les données « par image », *pool* pour des objets identiques, arène pour un niveau de jeu — détaillés dans [Guides/01-NKMemory.md](#).

La règle qui va avec, et qui n'admet pas d'exception : **un objet obtenu d'un allocateur se rend au même allocateur**. Le dépôt en porte la cicatrice :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DFactory.cpp:41-46

```
1 // IMPORTANT : on alloue via l'allocateur NKMemory (NkGetDefaultAllocator)
2 // et NON via `new`. Raison : Le NkUniquePtr<NkIRenderer2D> (CreateUnique)
3 // desalloue via NkDefaultDelete -> ce MEME allocateur (-> _aligned_free).
4 // Allouer avec `new` (malloc) puis Libérer avec _aligned_free = mismatch
5 // d'allocateur -> heap corruption c0000374 au free du renderer (shutdown).
6 // cf BugReports/NKCanvas/heap-c0000374-renderer-alloc-mismatch.md.
```

Le mélange d'allocateurs

Rendre à un allocateur un bloc qu'il n'a pas donné produit une **corruption de tas**. Sous Windows, cela se manifeste par un plantage `0xc0000374` — et, ce qui rend le bug diabolique, ce plantage survient *à la fermeture de l'application*, très loin de la ligne fautive. Le fichier `Kernel/Runtime/NKCanvas/src/NKCanvas/Factory/NkContextFactory.h` le dit en une ligne (:39) : « RÈGLE : ne JAMAIS faire `delete` (objet alloué par `NKMemory`) ». Un objet créé par une *factory* du moteur se détruit par la méthode `Destroy` correspondante, et par elle seule.

0.4.6 Les habitudes à prendre

Voici la forme que prend tout cela dans le code que vous écrirez au chapitre 4 :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:262-269 (extrait)

```
1 auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
2 if (!target || !target->IsValid())
3     return -1;
4
5 auto ctxPtr = memory::NkMakeUnique<NkGuiContext>();
6 if (!ctxPtr)
7     return -1;
8 NkGuiContext &ctx = *ctxPtr;
```

Trois habitudes à prendre immédiatement :

- on crée les objets à durée de vie longue avec `memory::NkMakeUnique<T>` : ils se libèrent seuls, au bon endroit;
- on teste toujours le résultat. `NkMakeUnique` peut retourner un pointeur nul; les constructeurs du moteur ne lancent pas d'exception — ils posent un état « invalide » qu'on interroge par `IsValid()`;
- quand un objet est possédé par un `NkUniquePtr`, on passe partout ailleurs un *pointeur brut non possédant* obtenu par `.Get()`. Il faut alors garantir la durée de vie à la main. Nous verrons au chapitre 4 que c'est exactement le cas de la police.

Ce qu'il faut retenir

`NkString` et `NkVector` pour les données, `math::Nk*` pour la géométrie, `NkFileSystem` pour les fichiers, `NKMemory` pour la mémoire : ce sont les outils du moteur, et ils suffisent. Les noms de méthodes sont en `PascalCase` et les tailles d'entiers sont écrites dans le nom du type.

0.5 Compiler : le système Jenga

Le moteur n'utilise ni `CMake`, ni `Meson`, ni `Visual Studio` directement. Il utilise **Jenga**, un système de build maison écrit en Python. Chaque module porte un fichier `<NomDuModule>.jenga` qui est, littéralement, un script Python.

0.5.1 À quoi ressemble un fichier .jenga

Applications/NKGuiDemo/NKGuiDemo.jenga:41-58 (extrait)

```
1 with project("NKGuiDemo"):
2     windowedapp()
3     language("C++")
4     cppdialect("C++17")
5     location(".")
6
7     files(["src/**/*.cpp"])
8
9     nkentseudependson(
10         ["NKGui", "NKReflection", "NKCanvas", "NKWindow", "NKEvent", "NKGlad",
11          "NKFont", "NKImage", "NKStream", "NKFileSystem", "NKLogger", "NKMath",
12          "NKTime", "NKContainers", "NKMemory", "NKCore", "NKPlatform", "NKThreading"],
13         extra_includes=["src", "src/NKGuiDemo", "%{NKGlad.location}/include"],
14     )
15
16     objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
17     targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
```

Les quatre appels à connaître :

- `project("Nom")` — ouvre la déclaration d'une cible. Le nom est celui qu'on passera à `--target`;
- `windowedapp()` — le type de cible : application fenêtrée. Une bibliothèque déclarerait `staticlib()`;
- `files([...])` — les sources à compiler. Notez le motif `"src/**/*.cpp"` : *seuls les .cpp*. Un module entièrement en `.h` avec des fonctions `inline` ne compile rien du tout — c'est le cas du pont `NKGui` → `NkCanvas`, et c'est délibéré (chapitre 1);
- `nkentseudependson([...])` — la liste des modules dont on dépend. C'est *la* ligne à lire pour savoir qui connaît qui.

0.5.2 Les deux commandes que vous taperez

Commande — à taper depuis la racine du dépôt

```
1 jenga build --target NKGuiDemo --config Release
2 jenga build --target NKGuiDemo --config Debug
```

L'exécutable produit atterrit à un emplacement déterminé par le `targetdir` ci-dessus :

Chemin de l'exécutable produit (Windows)

```
1 Build\Bin\Release-Windows\NKGuiDemo\NKGuiDemo.exe
2 Build\Bin\Debug-Windows\NKGuiDemo\NKGuiDemo.exe
```

La configuration `Debug` garde les symboles et les assertions; `Release` optimise. Un réflexe du dépôt, énoncé dans les règles de travail du projet : quand on livre un lot, on construit **les deux** configurations, parce qu'un bug qui n'existe qu'en `Release` (ordre d'initialisation, dépassement de tampon masqué) est le pire de tous.

Lancer depuis la racine du dépôt

Beaucoup d'applications du dépôt cherchent leurs ressources par des chemins *relatifs au répertoire courant* : `Resources/Fonts/...`, `Applications/NKCode/data/textures/...`. Lancer l'exécutable depuis son propre dossier (`Build/Bin/...`) donne alors une fenêtre sans polices, sans icônes, parfois entièrement noire — et *aucun* message d'erreur explicite. La parade adoptée par le dépôt est de chercher dans plusieurs dossiers candidats, dont celui de l'exécutable, comme le documente [Applications/NKCode/src/NKCode/Shell/NkAppFonts.h:18-21](#) : « Candidats RELATIFS AU CWD (dev, lancement depuis la racine du repo) PUIS relatifs à l'EXECUTABLE ». Tant que vous êtes en développement : **lancez depuis la racine du dépôt**.

0.5.3 Où sont les journaux

Le moteur journalise via `NKLogger`. La trace d'exécution est écrite dans `logs/app.log`, *relative-ment au répertoire courant* — donc, si vous lancez depuis la racine, dans `logs/app.log` à la racine du dépôt. La trace précédente est conservée sous `logs/app.prev.log`.

C'est le premier endroit à regarder quand une application se lance et n'affiche rien : le choix du backend graphique, les échecs de création de contexte, les polices introuvables y sont écrits. Dans votre propre code, la journalisation s'écrit ainsi :

`Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:319 et 1055 (extraits)`

```
1 logger.Info("Image chargée : {0}x{1}", imgW, imgH);
2 logger.Error("Capture écran ECHEC ({0})", path);
```

(l'objet global `logger` vient de `NKLogger/NkLog.h` ; il existe aussi des variantes `Infof/Warnf/Errorf` au format `printf`).

0.6 Le point d'entrée : `nkmain`, pas `main`

Dernière particularité à connaître avant d'écrire la moindre ligne : le moteur fournit lui-même le `main` natif de chaque plateforme (`winMain` sous Windows, `main` ailleurs, un point d'entrée Java/JNI sur Android). Votre programme, lui, déclare :

`Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:29-33 et 57`

```
1 NKENTSEU_DEFINE_APP_DATA(() {
2     NkAppData d{};
3     d.appName = "NkCanvas Demo";
4     d.appVersion = "1.0.0";
5     return d;
6 })();
7
8 int nkmain(const NkEntryState &state) {
```

Il faut pour cela inclure `NKWindow/NKMain.h`. Le paramètre `state` porte notamment les arguments de la ligne de commande, sous forme d'un `NkVector<NkString>` :

`Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:36-40 (extrait)`

```
1 static NkGraphicsApi ParseBackend(const NkVector<NkString> &args) {
2     for (usize i = 1; i < args.Size(); ++i) {
```

```

3     const NkString &a = args[i];
4     if (a == "--backend=vulkan" || a == "-bvk")
5         return NkGraphicsApi::NK_GFX_API_VULKAN;

```

Ce qu'il faut retenir

Trois choses à retenir de ce chapitre, et rien d'autre :

1. Les outils du moteur : `NkString`, `NkVector`, `math::Nk*`, `NkFileSystem`, et `memory::NkMakeUnique` pour la mémoire.
2. `jenga build --target <Cible> --config Release` depuis la racine, exécutable dans `Build/Bin/Release-Windows/<Cible>/<Cible>.exe`, lancé **depuis la racine**.
3. Le point d'entrée s'appelle `nkmain(const NkEntryState &)`, et les journaux sont dans `logs/app.log`.

Ce chapitre en bref

Le dépôt est bâti en couches — Foundation, System, Runtime, Engine — et chacune n'emploie que celles du dessous. Vous savez lire un `.jenga`, construire une cible, lancer le binaire produit, et retrouver son journal dans `logs/`. Trois habitudes valent pour tout le reste du livre : des types primitifs de *taille connue*, aucune allocation hors `NKMemory`, et un point d'entrée `nkmain` qui rend le programme identique sur huit plateformes.

0.7 Exercices

1 — Reconnaître la carte

Sans compiler quoi que ce soit, ouvrez `Kernel/Runtime/NKGui/NKGui.jenga` et `Kernel/Runtime/NKCanvas/NKCanvas.jenga`. Relevez la liste passée à `nkentseudependson` dans chacun. Question : **NKGui dépend-il de NKCanvas**? Notez votre réponse ; le chapitre 1 y revient et en tire toute une architecture.

2 — Le premier build

Construisez et lancez la démo de référence :

```

cd D:\Projets\2026\Nkentseu\Nkentseu
jenga build --target NKGuiDemo --config Release
.\Build\Bin\Release-Windows\NKGuiDemo\NKGuiDemo.exe

```

Puis relancez-la depuis *son propre dossier* (`cd Build\Bin\Release-Windows\NKGuiDemo` puis `.\NKGuiDemo.exe`) et comparez : qu'est-ce qui change à l'écran ? Ouvrez ensuite `logs/app.log` et retrouvez la ligne qui indique le backend graphique retenu.

3 — Chasse aux allocations

Cherchez dans `Kernel/Runtime/NKCanvas/src/` toutes les occurrences de `NkGetDefaultAllocator`. Pour chacune, identifiez qui libère l'objet et par quel appel. Vous devriez tomber sur le destructeur du pont `NKGui`, que le chapitre 1 détaille : `Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NKGuiCanvasBackend.h:24-32`.

Questions de cours

1. Nommez les quatre couches du dépôt, de la plus basse à la plus haute, et donnez la règle qui les relie.
2. Pourquoi le point d'entrée s'appelle-t-il `nkmain` et non `main`?
3. Où une application écrit-elle son journal, et pourquoi n'écrit-elle pas sur la sortie standard?
4. Que déclare `nkntseudependson` dans un fichier `.jenga`?
5. Citez trois types primitifs du dépôt et dites ce qu'ils garantissent que `int` ne garantit pas.

NKCode : l'éditeur du dépôt

Vous pouvez écrire du Nkcode dans n'importe quel éditeur de texte. Mais le dépôt en fournit un, écrit avec le moteur lui-même, et qui connaît Jenga : c'est NKCode. Ce chapitre le prend en main — assez pour que vous n'ayez plus à quitter l'éditeur pour compiler, chercher ou déboguer.

Ce qu'il faut retenir

NKCode est écrit avec **NKCanvas** et **NKGui**, les deux modules que ce cours enseigne. C'est donc aussi le plus gros exemple à votre disposition : 56 fichiers d'une application réelle. Quand une question du cours vous restera, ouvrez son code — il répond souvent.

1.1 Le lancer

Commandes – à taper dans un terminal à la racine du dépôt

```
1 jenga build --target NKCode --config Release
2 .\Build\Bin\Release-Windows\NKCode\NKCode.exe
```

Le piège

Un NKCode fraîchement construit ne démarre pas sous Windows, et le build ne le dit pas. Il meurt sur `python312.dll` est introuvable : quatre DLL du runtime Python embarqué sont posées par `scripts/MakeNkCodeDist.py`, pas par `jenga build`. Pour un essai rapide, préfixez le PATH sans rien copier :

Contournement

```
1 $env:PATH = "<depot>\Externals\Libs\PythonEmbed\runtime;$env:PATH"
2 .\Build\Bin\Release-Windows\NKCode\NKCode.exe
```

Le diagnostic qui l'a trouvé : comparer le dossier de sortie avec celui d'un poste où NKCode tourne — il contient quatre fichiers de plus. *Un build vert et un exécutable mort ne sont pas contradictoires* : ils mesurent deux choses différentes.

1.2 Les raccourcis

Raccourci	Action
Ctrl+P	palette de commandes — <i>commencez par là</i>
Ctrl+N	nouveau fichier
Ctrl+S	enregistrer
Ctrl+F	rechercher (non modal)
Ctrl+G	aller à la ligne
Ctrl+D	dupliquer la ligne ou la sélection
Ctrl+L	sélectionner la ligne (s'étend si répété)
F2	renommer l'identifiant
F9	poser un point d'arrêt
F5	déboguer — lance jenga gdb, points d'arrêt transmis
F1	documentation
Ctrl+Q	quitter

Ce qu'il faut retenir

Ctrl+P rend tous les autres facultatifs. La palette liste les commandes par leur nom : si vous ne vous rappelez pas d'un raccourci, tapez le verbe. C'est aussi la façon de *découvrir* ce que l'éditeur sait faire.

1.3 IntelliSense : clangd

NKCode parle le protocole LSP et s'appuie sur clangd. Vous obtenez donc les diagnostics en temps réel, le survol qui montre un type, et l'aller-à-la définition — les mêmes que dans un IDE commercial, parce que c'est le même moteur d'analyse.

Le piège

clangd a besoin de savoir *comment* chaque fichier est compilé : quels chemins d'inclusion, quelles macros. Sans cette information, il souligne en rouge des inclusions parfaitement valides.

Un fichier souligné en rouge qui compile parfaitement n'est pas un défaut du code : c'est un défaut de configuration de l'analyseur. Ne « corrigez » rien avant d'avoir lancé un vrai build — c'est jenga qui a raison, pas le soulignement.

1.4 Construire sans quitter l'éditeur

NKCode connaît Jenga. Le point d'arrêt que vous posez avec F9 est transmis à jenga gdb par F5 : vous n'écrivez aucune ligne de commande de débogage.

Ce qu'il faut retenir

Ce que le débogueur vous donne, et que l'affichage ne donne pas : l'état au moment de l'arrêt — toutes les variables, la pile d'appels, qui a appelé qui.

Un `printf` répond à une question que vous avez déjà formulée. Un point d'arrêt répond à celles que vous n'aviez pas pensé à poser. Quand vous ne comprenez pas ce qui se passe, le débogueur est plus rapide, toujours.

1.5 Les autres services

Service	Ce qu'il fait
Recherche / remplacement	Ctrl+F, Ctrl+H, non modaux
Multilingue	8 langues, changement sans redémarrer
Assistant IA	discussion, contexte du fichier, commandes
Émulateurs	lance Android et HarmonyOS depuis l'éditeur

Ce qu'il faut retenir

« Non modal » veut dire que la barre de recherche ne bloque pas l'édition : vous tapez dans le fichier pendant qu'elle est ouverte. C'est un détail qui change l'usage — une boîte de dialogue modale interrompt le geste, une barre non modale l'accompagne.

Exercice pratique

Ouvrez le dépôt dans NKCode et faites les cinq gestes qui reviennent tous les jours :

1. Ctrl+P, puis tapez « ouvrir » — lisez les commandes proposées ;
2. ouvrez `Kernel/Runtime/NKEvent/src/NKEvent/NkPointerEvent.h` et survolez `NkPointerPhase` : le type doit s'afficher ;
3. posez un point d'arrêt (F9) dans le `nkmain` d'une petite application, lancez F5, et regardez la pile d'appels ;
4. cherchez `NkCanvasApp` dans tout le dépôt — combien de fichiers ?
5. changez la langue de l'interface, et vérifiez que rien ne redémarre.

Ce chapitre en bref

NKCode est un confort, jamais une obligation : tout ce que fait ce livre se fait aussi avec un éditeur quelconque et un terminal. Ce qu'il apporte est concentré — la palette de commandes, l'auto-complétion réelle par clangd, la construction sans quitter l'éditeur. Retenez surtout que son IntelliSense lit le *même* projet que le compilateur : s'il ne trouve pas un en-tête, c'est souvent le projet qui est mal décrit, pas l'éditeur qui se trompe.

Questions de cours

1. Que lit clangd pour savoir où sont vos en-têtes ?
2. Que fait Ctrl+P, et en quoi diffère-t-il d'une recherche de fichier ordinaire ?
3. Pourquoi une barre de recherche *non modale* change-t-elle l'usage ?
4. Peut-on construire un projet sans quitter l'éditeur ? Par quel moyen ?
5. NKCode est-il nécessaire pour suivre ce cours ? Justifiez.

Exercice de démonstration

Prouvez que l'auto-complétion lit bien votre projet, et non une supposition. Créez un en-tête à vous, dans un dossier que le projet ne déclare pas. Incluez-le depuis un fichier source : l'auto-complétion ne doit rien proposer. Ajoutez ensuite le dossier aux chemins d'inclusion du `.jenga`, rechargez, et vérifiez qu'elle propose désormais vos symboles.

Ce que la démonstration établit : que le service dépend d'une *donnée* — la description du projet — et non d'une magie de l'éditeur. C'est ce qui vous dira quoi corriger le jour où il ne trouve plus rien.

Exercice de logique

Un collègue vous dit : « l'auto-complétion ne marche pas, NKCode est cassé ». Énumérez au moins quatre causes possibles, en distinguant celles qui viennent de l'éditeur de celles qui viennent du projet. Pour chacune, donnez la *première chose à regarder* — et non le correctif.

Puis dites laquelle est la plus probable, et pourquoi c'est justement celle qu'on soupçonne en dernier.

Ce que cette partie vous laisse

Un projet Nkentseu est décrit par un fichier `.jenga`, construit par une commande, et lancé par une autre. Vous savez maintenant ce que cette commande fait à votre place — et donc ce qu'il faut relire quand elle se plaint. NKCode n'est pas obligatoire : c'est un confort, et vous pouvez tout faire sans lui. Ce qui suit ne suppose que trois choses : un projet qui se construit, un éditeur, et un terminal.

PARTIE II

Les langages

Le C, le C++, et ce que Nkentseu en garde

Nkentseu est écrit en C++, mais pas dans le C++ qu'on enseigne d'ordinaire : la bibliothèque standard y est *écartée*, remplacée par des équivalents maison. Apprendre le C++ « normal » puis découvrir cette règle donnerait l'impression d'une contrainte arbitraire. Cette partie fait donc les deux à la fois : le langage, et la raison de chaque écart.

Elle commence par le **C** — non par nostalgie, mais parce que les pointeurs, la mémoire et la compilation séparée s'expliquent mieux sans les objets par-dessus. Puis le **C++** : classes, héritage, RAII, exceptions. Puis les **templates**, qui sont le mécanisme sur lequel repose la totalité de NKContainers. Enfin le **C++ moderne** — lambdas, déplacement, possession — que l'on croit avancé et qui est quotidien : l'ignorer produit du code qui copie sans le savoir et qui fuit sans le dire.

Aucune connaissance préalable n'est supposée. Si vous savez déjà programmer, lisez tout de même les encadrés « Le piège » : ils décrivent des défauts que ce dépôt a réellement payés, pas des curiosités de manuel.

Le C : la matière dont tout est fait

Nkentseu s'écrit en C++, et le C++ *contient* le C. Une adresse mémoire, un tableau, une structure, un octet : tout cela vient du C. Ce chapitre le couvre en entier. Il suppose que vous n'avez jamais programmé, mais il ne vous fera pas perdre de temps : chaque section dit une chose et passe à la suivante.

2.1 Compiler, lier

Un processeur ne comprend que des instructions binaires. On écrit du texte, et le **compilateur** le traduit. Deux étapes, deux familles d'erreurs.

Étape	Entrée → sortie	Erreur typique
Compilation	un .cpp → un .obj	« je ne comprends pas cette ligne »
Édition de liens	les .obj → un .exe	undefined reference to ...

Ce qu'il faut retenir

Une erreur de **compilation** nomme un fichier et une ligne : la faute y est. Une erreur d'**édition de liens** ne nomme qu'un symbole : la ligne est correcte, c'est *ailleurs* qu'un corps de fonction manque. Chercher au mauvais endroit coûte des heures; distinguer les deux messages coûte une seconde.

2.2 Premier programme

Exemple pédagogique

```
1 #include <stdio>
2
3 int main() {
4     printf("Bonjour\n");
5     return 0;
6 }
```

`#include` fait lire un autre fichier — celui qui déclare `printf`. `main` est le point de départ. `\n` est *un* caractère : le passage à la ligne. `return 0` rend le code de sortie : **zéro veut dire succès**, à l'inverse de l'intuition, et tous les outils suivent cette convention.

Dans Nkentseu vous écrirez `nkmain` et non `main` : le dépôt fournit un point d'entrée portable qui devient `WinMain` sur Windows et `android_main` sur Android.

2.3 Variables et types

Une variable est un emplacement mémoire nommé. Le type dit combien d'octets réserver *et comment les lire* : les mêmes 32 bits donnent un entier ou un flottant selon le type. Se tromper de type, c'est lire le bon emplacement de la mauvaise façon.

Nkentseu	C	Octets	Contient
int8 ...int64	char ...long long	1 à 8	entiers signés
uint8 ...uint64	unsigned ...	1 à 8	entiers ≥ 0
float32	float	4	≈ 7 chiffres significatifs
float64	double	8	≈ 16 chiffres
usize	size_t	4 ou 8	une taille en mémoire
—	bool	1	true / false

Ce qu'il faut retenir

int ne fait pas la même taille partout; int32 si. Pour un moteur qui vise huit plateformes, un fichier écrit sur l'une doit se relire sur l'autre. **Écrivez les types du dépôt**, pas ceux du langage.

Le piège

Non initialisée, une variable contient les octets qui traînaient là. Pas zéro. En *Debug* c'est souvent un motif reconnaissable et le programme semble marcher; en *Release* ce sont de vraies valeurs d'un calcul précédent, et le bogue change de forme à chaque exécution. Écrivez `int32 n = 0;`, toujours.

Le piège

Un entier non signé ne descend pas sous zéro : il repasse par le haut. `uint32 n = 0; --n;` donne 4 294 967 295. D'où la boucle infinie classique :

Exemple pédagogique – NE FAIT JAMAIS DE FIN

```
1 for (uint32 i = taille - 1; i >= 0; --i) { ... } // i >= 0 est toujours vrai
```

Et si `taille` vaut 0, `taille - 1` vaut déjà quatre milliards. Pour reculer : un `int32`, ou une boucle en avant.

2.3.1 const, et pourquoi on le met partout

`const` dit « ceci ne changera pas ». Le compilateur le fait respecter.

Exemple pédagogique

```
1 const int32 kMax = 100;           // constante
2 const char *nom = "Rihen";       // pointeur vers des caracteres CONSTANTS
3 char *const p = tampon;          // pointeur CONSTANT vers des caracteres Libres
4 void Lire(const NkPointer &p);    // La fonction promet de ne pas modifier p
```

Ce qu'il faut retenir

Lisez une déclaration **de droite à gauche**. `const char *p` : *p* est un pointeur vers un char constant. `char *const p` : *p* est un pointeur constant vers un char. Le premier protège ce qui est pointé, le second protège le pointeur.

2.4 Opérateurs

Famille	Opérateurs	À savoir
arithmétique	+ - * / %	% = reste, entiers seulement
comparaison	== != < > <= >=	rendent un booléen
logique	&& !	évaluation <i>paresseuse</i>
affectation	= += -= *= /= %=	
incrément	++ --	préfixé ou suffixé
bits	& ^ ~ << >>	agissent sur les bits
ternaire	c ? a : b	un if qui rend une valeur

Le piège

= affecte, == compare. `if (n = 0)` affecte zéro puis teste zéro : toujours faux, et *n* est détruit.

La division entière tronque : `7 / 2` vaut 3. Pour un résultat décimal, un opérande au moins doit l'être : `7.f / 2.f`.

L'évaluation paresseuse est une garantie, pas une optimisation : dans `a && b`, si *a* est faux, *b* n'est pas évalué. C'est ce qui rend sûr `if (p != nullptr && p->champ > 0)`. Inversez les termes et vous dérèferez un pointeur nul.

2.4.1 Les opérateurs de bits, en pratique

Ils servent aux **drapeaux** : plusieurs oui/non dans un seul entier.

Exemple pédagogique

```
1 const uint32 NK_VISIBLE = 1u << 0; // 0001
2 const uint32 NK_ACTIF   = 1u << 1; // 0010
3 const uint32 NK_VERROUIL = 1u << 2; // 0100
4
5 uint32 etat = NK_VISIBLE | NK_ACTIF; // on POSE deux drapeaux
6 if (etat & NK_ACTIF) { ... }         // on TESTE
7 etat &= ~NK_ACTIF;                   // on RETIRE
8 etat ^= NK_VISIBLE;                  // on BASCULE
```

Le piège

& n'est pas &&. `etat & NK_ACTIF` rend un *nombre*; `a && b` rend un *booléen*. Écrire `&&` entre deux drapeaux teste seulement qu'ils sont non nuls — ce qui est vrai en permanence.

2.5 Contrôle de flux

Exemple pédagogique

```
1 if (a) { ... } else if (b) { ... } else { ... }
2
3 for (int32 i = 0; i < n; ++i) { ... } // init ; test avant chaque tour ; apres
4 while (condition) { ... }
5 do { ... } while (condition); // au moins une fois
6
7 switch (v) {
8     case NK_A: ... break; // Le `break` est OBLIGATOIRE
9     case NK_B: ... break;
10    default: ... break;
11 }
12
13 break; // sort de la boucle ou du switch
14 continue; // passe au tour suivant
```

Le piège

Un **case** sans **break** tombe dans le suivant et exécute son code : le programme fait deux choses au lieu d'une.

Mettez toujours les accolades, même pour une instruction. La ligne ajoutée six mois plus tard sous un **if** sans accolades ne fait plus partie du **if** : l'indentation dit une chose, le compilateur en comprend une autre, et l'œil ne voit rien.

2.6 Fonctions

Une **fonction** rend une valeur; une **procédure** rend `void`. Le langage ne les distingue pas, l'intention si — et elle guide le nom.

Exemple pédagogique

```
1 int32 Somme(int32 a, int32 b) { return a + b; } // calcule
2 void Afficher(const char *s) { printf("%s\n", s); } // agit
```

Ce qu'il faut retenir

Les paramètres sont copiés. Une fonction travaille sur sa propre copie et ne peut pas abîmer vos variables par accident. Pour qu'elle en *modifie* une, il faut lui passer son adresse.

2.6.1 Portée et durée de vie

Déclaration	Vit	Visible
dans un bloc	jusqu'à l'accolade fermante	dans le bloc
static dans une fonction	tout le programme	dans la fonction
static au fichier	tout le programme	dans le fichier seul
globale	tout le programme	partout (via extern)

Le piège

Ne rendez jamais l'adresse d'une variable locale. Elle meurt à la sortie de la fonction ; le pointeur rendu désigne une zone réutilisée aussitôt. Le programme ne plante pas tout de suite — il lit des ordures plus tard, ailleurs.

2.7 Pointeurs

Un pointeur contient une **adresse**. Le langage ne vérifie pas qu'elle est valide : c'est à vous.

Exemple pédagogique

```
1  int32 n = 42;
2  int32 *p = &n;    // & PREND L'adresse
3  int32 lu = *p;    // * SUIV L'adresse -> 42
4  *p = 7;           // écrit a travers : n vaut 7
5
6  void Doubler(int32 *v) {
7      if (v == nullptr) { return; } // on vérifie, toujours
8      *v = *v * 2;
9  }
10 Doubler(&n);      // on passe L'ADRESSE : n vaut 14
```

Le piège

Trois façons dont un pointeur tue un programme, et vous les verrez toutes :

1. **nul** — *p quand p vaut nullptr. Plantage immédiat : c'est le cas *heureux*, il est reproductible;
2. **pendant** — il désigne une zone déjà libérée. Le programme continue, lit des ordures, tombe ailleurs et plus tard;
3. **débordement** — écrire au-delà d'un tableau écrase la variable d'à côté. Le symptôme apparaît dans une variable que vous n'avez pas touchée.

Les deux derniers sont la raison d'être de presque tout ce que le C++ ajoute.

2.7.1 Pointeurs de fonction

Une fonction a une adresse, elle aussi. On peut donc la ranger dans une variable — c'est ainsi qu'on branche un comportement sans écrire de switch.

Idiome du depot – une commande d'éditeur

```
1  using NkCommandeFn = void (*)(void *user); // Le TYPE d'une fonction
2
3  void Quitter(void *u) { /* ... */ }
4  NkCommandeFn action = &Quitter;
5  action(nullptr); // on l'appelle
```

Le void *user est le prix d'une signature C simple : il transporte n'importe quoi, et personne ne vérifie quoi. On ne lui passe donc *que* l'objet attendu.

2.8 Tableaux et chaînes

Exemple pédagogique

```
1 int32 scores[4] = {10, 20, 30, 40}; // indices de 0 à 3 ; [4] est DEHORS
2 scores[0] = 15;
3
4 const char *nom = "Rihen"; // octets terminés par '\0'
```

Un tableau est une suite *contiguë*. `scores` vaut l'adresse du premier élément; `scores[2]` veut dire « deux éléments plus loin ».

Le piège

Un tableau ne connaît pas sa taille. Passé à une fonction, il ne reste que l'adresse du premier élément : la longueur est perdue. C'est pourquoi les fonctions C prennent un couple (pointeur, nombre).

Une chaîne C se termine par '\0'. L'oublier fait lire jusqu'au prochain zéro rencontré, parfois très loin.

2.9 Structures, unions, énumérations

Kernel/Runtime/NKEvent/src/NKEvent/NkPointerEvent.h

```
1 struct NkPointer {
2     float32 x = 0.f;
3     float32 y = 0.f;
4     NkPointerPhase phase = NkPointerPhase::NK_POINTER_NONE;
5     uint32 id = 0;
6     bool fromTouch = false;
7 };
```

Code réel du dépôt — celui que vous emploierez au chapitre sur la coquille. Notez que **chaque champ est initialisé à sa déclaration** : une `NkPointer` neuve n'a jamais de contenu indéterminé.

Exemple pédagogique

```
1 NkPointer p;
2 p.x = 100.f; // POINT quand on a l'objet
3 NkPointer *ptr = &p;
4 ptr->y = 50.f; // FLECHE quand on a un pointeur ; == (*ptr).y
```

Une union range plusieurs champs *au même endroit* : un seul est valide à la fois. Elle sert aux événements, où la charge dépend du type — et c'est à vous de savoir lequel est actif.

Une `enum class` nomme un ensemble fermé de valeurs :

Kernel/Runtime/NKEvent/src/NKEvent/NkPointerEvent.h

```
1 enum class NkPointerPhase : uint8 {
2     NK_POINTER_NONE = 0, NK_POINTER_DOWN, NK_POINTER_MOVE,
3     NK_POINTER_UP,      NK_POINTER_CANCEL
4 };
```

Ce qu'il faut retenir

enum class plutôt que enum nu : les valeurs sont *cloisonnées* (on écrit `NkPointerPhase::NK_POINTER_UP`) et ne se convertissent pas silencieusement en entier. Deux énumérations peuvent alors avoir un membre du même nom sans se gêner.

2.10 Conversions

Écriture	Ce qu'elle fait
<code>static_cast<T>(v)</code>	conversion vérifiée à la compilation
<code>reinterpret_cast<T>(v)</code>	« relis ces octets comme un T » — dangereux
<code>const_cast<T>(v)</code>	retire un <code>const</code> — presque toujours une faute
<code>(T)v</code>	la forme C : fait n'importe laquelle des trois, sans le dire

Ce qu'il faut retenir

Employez `static_cast`. La forme C `(T)v` est plus courte et c'est son seul avantage : elle cache *laquelle* des conversions elle applique, et donc à quel point elle est risquée.

2.11 Mémoire : pile et tas

Vos variables vivaient jusqu'ici sur la **pile** : créées à l'entrée d'une fonction, détruites à sa sortie, automatiquement. Rapide, mais petite, et la durée de vie est imposée.

Un objet qui doit survivre à la fonction qui le crée, ou qui est trop gros, vit sur le **tas**. On le demande, on le rend. Oublier de le rendre est une *fuite*.

La règle la plus dure du dépôt

Ni `malloc/free`, ni `new/delete` bruts. Tout passe par `NKMemory` :

Règle du dépôt

```
1 void *bloc = nkentseu::memory::NkAlloc(taille);
2 nkentseu::memory::NkFree(bloc);
```

Et le corollaire, déjà payé plusieurs fois ici : **ce qui est alloué par un allocateur se libère par le même**. Rendre avec le `free()` de la bibliothèque C un tampon alloué par `Nkentseu` corrompt le tas — sous Windows, l'erreur `c0000374`, qui tombe *plus tard*, dans une fonction innocente.

2.12 Le préprocesseur

Il agit *avant* le compilateur : il remplace du texte.

Directive	Effet
#include "x.h"	colle le contenu de x.h ici
#pragma once	ne lire ce fichier qu'une fois
#define NK_X 1	remplace NK_X par 1 partout
#if / #endif	compile ou non selon la plateforme

Le piège

Une macro n'est pas une fonction : c'est un remplacement de texte. #define DOUBLE(x) x * 2 puis DOUBLE(1 + 1) donne 1 + 1 * 2, soit 3. Parenthésez tout : #define DOUBLE(x) ((x) * 2). Mieux : écrivez une vraie fonction, que le compilateur vérifie.

2.13 Compilation séparée

L'en-tête .h déclare; l'implémentation .cpp définit.

Exemple pédagogique

```

1 // NkCompteur.h -- ce que les autres fichiers ont le droit de savoir
2 #pragma once
3 int32 Incrementer(int32 valeur);
4
5 // NkCompteur.cpp -- comment c'est fait
6 #include "NkCompteur.h"
7 int32 Incrementer(int32 valeur) { return valeur + 1; }
```

Déclarer sans définir compile parfaitement — et échoue à l'édition de liens. C'est la deuxième famille d'erreurs de la première page, et vous venez d'avoir le moyen de la provoquer.

Exercice pratique

Écrivez NkVec2.h (une structure à deux float32 et une fonction Longueur) et son NkVec2.cpp.

Faites-la ensuite **échouer exprès**, deux fois : retirez un point-virgule (erreur de compilation), puis retirez le .cpp du projet en gardant l'appel (erreur d'édition de liens). Lisez les deux messages. Savoir les distinguer d'un coup d'œil vous fera gagner plus de temps que n'importe quelle autre astuce de ce cours.

Ce chapitre en bref

Le C tient en peu de choses : des types de taille connue, des fonctions, des pointeurs, et une distinction entre la pile et le tas. Trois idées vous suivront partout. **Un pointeur est une adresse** — il ne dit ni si la chose existe encore, ni combien il y en a. **La compilation et l'édition de liens sont deux étapes** : un symbole déclaré et jamais défini passe la première et échoue à la seconde, ailleurs et plus tard. Et **le préprocesseur ne comprend rien** : il remplace du texte, ce qui explique aussi bien son utilité que ses pièges.

Questions de cours

1. Quelle est la différence entre déclarer et définir une fonction ?
2. Que vaut un pointeur après la libération de ce qu'il désignait ?

3. Pourquoi met-on `const` sur un paramètre qu'on ne modifie pas ?
4. Qu'est-ce qui distingue la pile du tas, du point de vue de la durée de vie ?
5. `struct` et `union` : que partagent-ils, et qu'est-ce qui les sépare ?

Exercice de démonstration

Prouvez qu'un pointeur pendant n'est pas détectable en le lisant.

Écrivez un programme qui alloue un entier, garde son adresse, libère la mémoire, puis *compare* le pointeur à `NULL` et affiche le résultat. Répétez dix fois.

Le résultat attendu, et c'est tout l'enseignement : il n'est jamais `NULL`, et sa valeur ne change pas. Un pointeur libéré est *indistinguishable* d'un pointeur valide par examen.

□ Ne lisez pas ce que le pointeur désigne. Le programme pourrait afficher l'ancienne valeur sans rien signaler, et vous concluriez à tort que c'est inoffensif.

Exercice de logique

Une fonction rend l'adresse d'une variable locale. Le programme compile, tourne, et donne le bon résultat sur votre machine.

1. Expliquez pourquoi il donne pourtant un résultat juste, souvent.
2. Décrivez précisément la situation où il cesse de le donner.
3. Un collègue conclut : « ça marche, donc c'est correct ». Réfutez-le en une phrase, puis dites quel outil ou quelle mesure aurait tranché sans discussion.

Le C++ : ce qu'il ajoute, et ce que Nkentsu en garde

Le C++ ajoute au C de quoi rendre les erreurs *impossibles* plutôt que seulement improbables. Ce chapitre couvre le langage, puis dit — en le mesurant — ce que ce dépôt en emploie et ce qu'il refuse. La seconde partie compte autant que la première : apprendre le C++ d'un livre standard, c'est apprendre des choses qu'il faudra désapprendre ici.

3.1 La norme employée

Norme	Projets	Mesure
C++17	46	cppdialect("C++17")
C++20	2	cppdialect("C++20")

Écrivez du C++17. Les deux projets en C++20 ont une raison particulière; la règle est 17.

3.2 Classes et structures

Une `class` est une `struct` dont les membres sont privés par défaut. C'est *toute* la différence technique.

Exemple pédagogique

```

1  class NkCompteur {
2      public:                                // ce que le monde extérieur voit
3          void Incrementer() { ++mValeur; }
4          int32 Valeur() const { return mValeur; } // `const` : ne modifie rien
5
6      private:                               // ce qui n'appartient qu'à la classe
7          int32 mValeur = 0;
8  };

```

Ce qu'il faut retenir

Convention du dépôt : membres privés préfixés `m` (`mValeur`), méthodes en PascalCase, types préfixés `Nk`. Une méthode qui ne modifie pas l'objet est `const` — ce n'est pas de la politesse : sans lui, on ne peut pas l'appeler sur un objet constant, et la moitié du code devient inaccessible.

Ce qu'il faut retenir

Dans ce dépôt, `struct` sert aux **données nues** (champs publics, aucune invariante à protéger — `NkPointer`, `NkLayoutInfo`) et `class` à ce qui a un **état à défendre** (`NkCanvasApp`, `NkCanvasSplash`). Le compilateur ne fait pas cette distinction; le lecteur, si.

3.3 Construire et détruire : RAI

Un **constructeur** s'exécute à la naissance de l'objet, un **destructeur** à sa mort — automatiquement, même si une erreur survient entre les deux.

Exemple pédagogique

```
1 class NkFichier {
2     public:
3         NkFichier(const char *chemin) { mHandle = Ouvrir(chemin); }
4         ~NkFichier() { if (mHandle) { Fermer(mHandle); } }
5
6     private:
7         void *mHandle = nullptr;
8 };
9
10 void Traiter() {
11     NkFichier f("donnees.txt");
12     // ... on travaille ...
13 } // <- ici, ~NkFichier s'exécute. Le fichier est fermé. Toujours.
```

Ce qu'il faut retenir

C'est **RAI** : *l'acquisition d'une ressource est son initialisation*. On ne peut plus oublier de libérer, parce que ce n'est plus vous qui le faites.

C'est la réponse du C++ aux deux pointeurs mortels du chapitre précédent — le pendant et la fuite. Elle vaut pour tout ce qui se prend et se rend : mémoire, fichier, verrou, contexte GPU.

Le piège

L'ordre de destruction est l'INVERSE de l'ordre de déclaration. Les membres déclarés en dernier meurent en premier.

Ce n'est pas un détail de style. Si un membre A tient un pointeur vers un membre B, alors A doit être déclaré *après* B — sinon B meurt d'abord et A pointe vers un mort pendant sa propre destruction. Le compilateur ne dit rien.

3.4 Références

Une référence est un **autre nom** pour une variable existante. Contrairement au pointeur, elle ne peut être ni nulle ni réaffectée.

Exemple pédagogique

```
1 void Doubler(int32 &v) { v = v * 2; } // pas de `*`, pas de `&` à l'appel
2 int32 n = 21;
3 Doubler(n); // n vaut 42
4
5 void Lire(const NkPointer &p); // référence CONSTANTE : pas de copie,
6 // et la fonction promet de ne rien changer
```

Ce qu'il faut retenir

Quand passer quoi :

Vous voulez	Écrivez	Pourquoi
lire un petit objet	<code>int32 n</code>	copier 4 octets est gratuit
lire un gros objet	<code>const T &</code>	pas de copie, pas de modification
modifier	<code>T &</code>	l'appelant voit le changement
« peut-être rien »	<code>T *</code>	seul un pointeur peut être nul

La dernière ligne est la vraie raison de garder des pointeurs : une référence ne peut pas dire « absent ».

3.5 Surcharge et valeurs par défaut

Exemple pédagogique

```
1 void Dessiner(const NkRect &r);           // surcharge 1
2 void Dessiner(const NkRect &r, const NkColor &c); // surcharge 2
3
4 void Effacer(const NkColor &c = NkColor::Blanc); // valeur par défaut
```

Deux fonctions peuvent porter le même nom si leurs paramètres diffèrent : le compilateur choisit d'après les arguments.

Le piège

Le type de retour ne participe pas au choix. Deux fonctions qui ne diffèrent que par lui ne compilent pas.

Et une surcharge manquante ne se voit pas par son nom. Si vous cherchez `begin()` et trouvez la version `const`, vous conclurez qu'elle existe — alors que la version non-`const` manque. Seule l'édition de liens tranche : le code compile des deux côtés.

3.6 Héritage et polymorphisme

Exemple pédagogique – le patron du depot

```
1 class NkChapitre {                               // La classe de BASE
2     public:
3         virtual ~NkChapitre() = default;         // destructeur VIRTUEL
4         virtual void Dessiner() = 0;              // = 0 : PUREMENT virtuelle
5         virtual void Avancer(float32 dt) { }      // virtuelle, avec un défaut
6 };
7
8 class NkChapitre1 : public NkChapitre { // La classe DERIVEE
9     public:
10        void Dessiner() override { /* ... */ }    // `override` : on REDEFINIT
11 };
```

`virtual` veut dire : « si l'objet réel est d'une classe dérivée, appelle sa version ». C'est le polymorphisme — un seul appel, plusieurs comportements.

Le piège

Un destructeur de classe de base doit être virtuel. Sans lui, détruire un objet dérivé à travers un pointeur de base n'appelle que le destructeur de la base : les membres de la dérivée ne sont jamais libérés. Aucun avertissement, une fuite silencieuse.

Écrivez `override`. Sans ce mot, une faute de frappe dans le nom ou une différence de signature crée une *nouvelle* méthode au lieu d'en redéfinir une. Le code compile, et votre méthode n'est jamais appelée. Avec `override`, le compilateur refuse.

Le piège

`final` interdit la redéfinition, et ce n'est pas un détail. Dans ce dépôt, `NkCanvasGuiApp::OnRender` est `final` : une classe qui en dérive *ne peut pas* dessiner autrement qu'avec la liste d'affichage `NKGui`. Ce seul mot décide de quelle coquille vous devez partir.

3.7 Templates

Un *modèle* écrit du code une fois pour plusieurs types.

Exemple pédagogique

```
1 template <typename T>
2 T Maximum(const T &a, const T &b) { return (a > b) ? a : b; }
3
4 int32 x = Maximum(3, 7);           // T = int32, déduit
5 float32 y = Maximum(1.f, 2.f);    // T = float32
```

C'est ainsi qu'est écrit `NkVector<T>`, et c'est aussi la forme de `NkCanvasApp::Run<T>` : la coquille ne connaît pas votre classe, elle la reçoit en paramètre de modèle.

Le piège

Les erreurs de template sont longues et intimidantes. Lisez la *première* ligne du message : elle nomme la vraie cause. Les vingt suivantes ne font que dérouler la pile d'instanciation. **Et le code d'un template vit dans l'en-tête**, pas dans un `.cpp` : le compilateur doit le voir pour l'instancier avec votre type. Le mettre dans un `.cpp` donne une erreur d'édition de liens.

3.8 Espaces de noms

Exemple pédagogique

```
1 namespace nkentseu {
2     namespace renderer {
3         class NkCanvasApp { ... };
4     }
5 }
6
7 nkentseu::renderer::NkCanvasApp app; // nom complet
8 using namespace nkentseu::renderer; // ou : on ouvre l'espace
9 NkCanvasApp app;
```

Le piège

Deux types peuvent porter le même nom dans deux espaces différents. Ce dépôt en a un cas réel : `NkShaderStage` existe dans `nkentseu` et dans `nkentseu::render`. Non qualifié, le second gagne — et l'erreur ne sort pas dans le fichier fautif. Dans un **en-tête**, qualifiez toujours complètement. Un `using namespace` en en-tête l'impose à tous ceux qui l'incluent.

3.9 Ce que Nkentseu emploie, et ce qu'il refuse

3.9.1 La règle : zéro bibliothèque standard

Nkentseu écrit ses propres conteneurs. `NkVector` au lieu de `std::vector`, `NkString` au lieu de `std::string`, `NkAlloc/NkFree` au lieu de `new/delete`.

Ailleurs	Ici
<code>std::vector<T></code>	<code>NkVector<T></code>
<code>std::string</code>	<code>NkString</code>
<code>new / delete</code>	<code>memory::NkAlloc / NkFree</code>
<code>std::optional<T></code>	<code>NkOptional<T></code>
<code>std::cout</code>	<code>logger.Info(...)</code>
<code>printf</code>	<code>logger.Infof(...)</code>

Ce qu'il faut retenir

La règle est **positive**, pas seulement négative : on n'écrit pas `std::`, et *si la primitive manque, on l'écrit au niveau où elle doit vivre* — dans `Foundation`, pas chez soi. Un utilitaire écrit localement devient le troisième exemplaire d'une chose qui aurait dû être partagée.

3.9.2 Et l'état réel, mesuré

Ce qu'un `grep` vous montrera, et comment le lire

Le 2 septembre 2026, sur `Kernel/Foundation` et `Kernel/Runtime` : **1887** occurrences de `std::`, dont `std::string` (191), `std::vector` (160), `std::cout` (78).

110 fichiers emploient `std::vector` ou `std::string`, et **trois seulement** sont des tests ou des démos. Ce n'est donc pas du code jetable : c'est du code livré.

Ils se concentrent là où le moteur dialogue avec des bibliothèques tierces qui, elles, parlent STL — `NKSL` (`glslang`), `NKRHI` (`Vulkan`), `NKRenderer`.

La règle est donc un cap, pas un état. On la dit franchement pour deux raisons : parce qu'un étudiant qui `grep` le découvrira de toute façon, et parce qu'une règle présentée comme acquise alors qu'elle ne l'est pas cesse d'être respectée. Dans le code *que vous écrirez* — une application, un jeu — elle tient sans exception.

3.9.3 Deux mots qui trompent

Le piège

`std::move` et `std::forward` **n'allouent rien et ne déplacent rien** : ce sont des conversions de type. Les compter comme des « corruptions mémoire » ou des « allocations » est faux. Ce qu'ils signalent, c'est un fichier qui raisonne encore en STL — une surface d'exposition, pas un défaut.

Le piège

Une enveloppe bien nommée endort la méfiance. `env::GetEnvVar()` rend un `const char *` qui vaut `nullptr` quand la variable n'existe pas — exactement comme `getenv`. Mais son nom *maison* suggère un objet de la maison, et l'on écrit alors `NkString v = env::GetEnvVar("X");` sans y penser. Le dépôt a failli planter toutes ses courses GPU sur cette ligne.

Lisez la signature de ce que vous appelez, surtout quand vous remplacez une fonction par une autre sur consigne.

3.10 Ce qu'on ne trouve presque pas ici

Construction	Statut dans le dépôt
exceptions (throw / catch)	rares, aux frontières tierces
<code>dynamic_cast</code> , <code>typeid</code>	rares; on préfère un champ de type
héritage multiple	évit
surcharge d'opérateurs	mesurée : <code>==</code> , <code>+</code> , <code>[]</code> sur les types mathématiques

Ce qu'il faut retenir

Ce ne sont pas des interdits moraux. Un moteur doit tourner sur des plateformes où les exceptions coûtent cher ou sont désactivées, et une hiérarchie profonde rend un défaut d'entrée impossible à situer. Quand vous hésitez : regardez comment le module voisin fait, et faites pareil.

Exercice pratique

Écrivez une classe `NkTampon` qui alloue `n` octets par `NkAlloc` dans son constructeur et les rend par `NkFree` dans son destructeur. Donnez-lui une méthode `Taille() const`. Puis provoquez les deux fautes du chapitre, et regardez ce que dit le compilateur :

- retirez le `const` de `Taille`, et appelez-la sur un `const NkTampon &`;
- faites dériver une classe de `NkTampon`, retirez le `virtual` du destructeur, détruisez l'objet dérivé à travers un pointeur de base. **Le compilateur ne dira rien** — c'est tout le problème. Ajoutez une trace dans chaque destructeur pour voir lequel manque.

Ce chapitre en bref

Le C++ ajoute au C de quoi lier une donnée à ce qui la gère. **RAII** est la plus importante de ces additions : une ressource acquise à la construction et rendue à la destruction ne peut plus fuir, même si l'on sort par un chemin qu'on n'avait pas prévu. Les **références** évitent

la copie sans les dangers du pointeur; le **polymorphisme** permet d'écrire du code qui ignore le type exact qu'il manipule — au prix d'un destructeur virtuel qu'il ne faut jamais oublier. Et Nkentseu *écarte* la bibliothèque standard : les chapitres suivants montrent par quoi.

Questions de cours

1. Qu'est-ce que RAI, et quel problème du C résout-il ?
2. Pourquoi un destructeur de classe de base destinée à l'héritage doit-il être `virtual` ?
3. Quelle différence entre une référence et un pointeur, du point de vue de ce qui est *garanti* ?
4. Que signifie « zéro bibliothèque standard » dans ce dépôt, et l'interdit couvre-t-il les fonctions C ?
5. Citez deux mots du C++ dont le nom trompe sur ce qu'ils font.

Exercice de démonstration

Prouvez que RAI libère même quand on sort par le milieu.

Écrivez une classe qui affiche un message dans son constructeur et un autre dans son destructeur. Employez-la dans une fonction qui contient trois `return` à des endroits différents, puis appelez la fonction de façon à emprunter chacun des trois chemins.

Ce que la démonstration établit : le message de destruction apparaît *trois fois sur trois*, sans qu'aucune ligne de libération ait été écrite. Refaites ensuite la même chose avec un `free` manuel placé avant le premier `return` seulement — et comptez ce qui manque.

Exercice de logique

Une classe de base a un destructeur *non* virtuel. On détruit un objet dérivé à travers un pointeur de base.

1. Décrivez ce qui se passe exactement.
2. Le programme plante-t-il ? Justifiez — et expliquez pourquoi la réponse rend ce défaut particulièrement coûteux.
3. Donnez deux façons de rendre l'erreur impossible : l'une par le destructeur, l'autre par la conception de la hiérarchie.

Templates : écrire une fois pour tous les types

Un *template* — un **modèle** — écrit du code une seule fois, et le compilateur le décline pour chaque type qu'on lui demande. C'est ce qui permet à `NkVector` d'exister sans être réécrit pour les entiers, puis pour les chaînes, puis pour les entités.

Ce chapitre va du modèle de fonction le plus simple jusqu'aux traits de type — la mécanique qui laisse `NkVector` choisir entre copier et déplacer selon ce qu'il transporte. C'est la partie du C++ qui fait peur au début; elle repose en réalité sur trois idées seulement.

4.1 Modèle de fonction

Exemple pédagogique

```
1 template <typename T>
2 T Maximum(const T &a, const T &b) {
3     return (a > b) ? a : b;
4 }
5
6 int32 x = Maximum(3, 7);           // T = int32, DEDUIT des arguments
7 float32 y = Maximum(1.f, 2.f);    // T = float32
8 float32 z = Maximum<float32>(1.f, 2); // T IMPOSE : Le 2 sera converti
```

Le compilateur **déduit** `T` des arguments. Quand il ne peut pas, ou quand la déduction donnerait le mauvais type, on l'impose entre chevrons.

Ce qu'il faut retenir

Un template n'est pas du code : c'est une **recette**. Rien n'est compilé tant que personne ne l'emploie. `Maximum` écrit et jamais appelé ne produit pas une seule instruction — et ses erreurs ne sont pas détectées non plus.

Le piège

La déduction ne convertit pas. `Maximum(1.f, 2)` échoue : le premier argument dit `T = float32`, le second dit `T = int32`, et le compilateur refuse d'arbitrer. Il faut imposer `Maximum<float32>(1.f, 2)`, ou corriger l'appel.

Le message d'erreur parle de « deduced conflicting types » : c'est lui, et il est plus clair qu'il n'en a l'air.

4.2 Modèle de classe

Principe de NkVector, simplifié

```
1  template <typename T>
2  class NkTableau {
3      public:
4          void PushBack(const T &v);
5          T &operator[](usize i);
6          usize Size() const;
7
8      private:
9          T *mDonnees = nullptr;
10         usize mTaille = 0;
11         usize mCapacite = 0;
12 };
13
14 NkTableau<int32> entiers;
15 NkTableau<NkVec2f> points; // une AUTRE classe, engendree du meme modele
```

NkTableau<int32> et NkTableau<NkVec2f> sont deux classes *distinctes*, sans rapport de parenté. Elles viennent de la même recette, c'est tout.

Le piège

Le code d'un template vit dans l'en-tête, jamais dans un .cpp. Le compilateur doit voir le corps pour l'instancier avec votre type; s'il est dans un .cpp qu'il ne lit pas, il croit la déclaration et laisse le corps au lieu — qui ne le trouve nulle part.

Symptôme : undefined reference to NkTableau<int>::PushBack. Le code compile, il ne lie pas. C'est pourquoi NkVector.h fait 2 500 lignes et n'a pas de .cpp.

4.3 Paramètres qui ne sont pas des types

Un modèle peut prendre une *valeur*, connue à la compilation.

Exemple pédagogique

```
1  template <typename T, usize N>
2  class NkTableauFixe {
3      public:
4          usize Size() const { return N; }
5      private:
6          T mDonnees[N]; // taille connue A LA COMPILATION : aucune allocation
7  };
8
9  NkTableauFixe<float32, 16> matrice;
```

Ce qu'il faut retenir

C'est ainsi qu'un tableau de taille fixe garde sa taille — contrairement au tableau C du chapitre précédent, qui la perdait en passant à une fonction. La taille fait partie du *type*.

4.4 Spécialisation

Parfois un type précis demande un traitement à part. On *spécialise*.

Exemple pédagogique

```
1 template <typename T>
2 bool EstVide(const T &v) { return v.Size() == 0; }
3
4 // Spécialisation TOTALE : ce cas-la, et lui seul, se traite autrement.
5 template <>
6 bool EstVide<const char *>(const char *const &s) { return s == nullptr || *s == 0; }
```

Le piège

Une spécialisation qui ne correspond pas exactement est ignorée en silence — le modèle général est employé à sa place, et votre cas particulier n'est jamais pris. Le programme compile et se comporte mal.

Vérifiez la signature au caractère près : `const char *` et `char *` sont deux types différents.

4.5 Modèles variadiques

Un modèle peut prendre un nombre *quelconque* d'arguments.

Principe de EmplaceBack

```
1 template <typename... Args>
2 void EmplaceBack(Args &&...args) {
3     // construit l'élément DIRECTEMENT dans le tableau,
4     // au lieu de le construire puis de le copier
5     new (Emplacement()) T(NkForward<Args>(args)...);
6 }
```

`Args...` est un *paquet* de types; `args...` le paquet de valeurs. Les trois points « dépliant » le paquet à l'endroit où on les écrit.

Ce qu'il faut retenir

C'est la différence entre `PushBack` et `EmplaceBack` : `PushBack` reçoit un objet *déjà construit* et le copie ou le déplace; `EmplaceBack` reçoit les *arguments* et construit l'objet sur place. Une construction au lieu de deux.

4.6 Références universelles et transfert parfait

Exemple pédagogique

```
1 template <typename T>
2 void Relayer(T &&arg) { // `T&&` dans un TEMPLATE : référence universelle
3     Consommer(NkForward<T>(arg)); // conserve : lvalue reste lvalue, rvalue reste rvalue
4 }
```

Le piège

`T&&` ne veut pas dire la même chose selon le contexte. Dans un template avec `T` déduit, c'est une *référence universelle* : elle accepte les deux. Hors template — `NkVector&&` — c'est une vraie référence *rvalue*, qui n'accepte que les temporaires.

Deux notions, une seule écriture. C'est le point du langage qui coûte le plus de malentendus.

Le piège

NkMove et NkForward ne déplacent rien. Ce sont des *conversions de type* : elles disent au compilateur « traite ceci comme un temporaire ». Le déplacement, s'il a lieu, est fait par le constructeur de déplacement de la classe.

Les compter comme des allocations ou des corruptions mémoire est faux. Ce qu'ils signalent, c'est du code qui raisonne en durée de vie — une information, pas un défaut.

4.7 Traits de type

Un *trait* répond à une question sur un type, à la compilation.

Principe employé par NkVector

```
1 if (traits::NkIsTriviallyCopyable<T>::value) {  
2     // un memcpy suffit : le type n'a pas de constructeur à respecter  
3 } else {  
4     // il faut construire chaque élément un par un  
5 }
```

C'est ainsi que NkVector peut recopier mille int32 d'un bloc, et mille NkString un par un — sans que vous ayez à le lui dire.

Ce qu'il faut retenir

Tout se décide à la compilation. Le test ci-dessus ne coûte rien à l'exécution : le compilateur sait déjà quelle branche garder, et supprime l'autre. Un trait n'est pas un if déguisé, c'est un choix fait avant que le programme existe.

4.8 Ce que les templates coûtent

Avantage	Prix
un seul code pour tous les types	le binaire grossit : une copie par type
aucun coût à l'exécution	la compilation ralentit
erreurs vues à la compilation	messages longs et indirects

Le piège

Lisez la PREMIÈRE ligne d'une erreur de template. Elle nomme la vraie cause. Les vingt suivantes déroulent la pile d'instanciation : elles disent *par où* le compilateur est arrivé là, pas ce qui ne va pas.

Le réflexe qui fait gagner le plus de temps : chercher, dans le pavé, la première ligne qui cite *votre* fichier.

Ce chapitre en bref

Un template est une **recette**, pas du code : rien n'existe avant qu'on l'emploie. Son corps vit dans l'en-tête, sinon l'édition de liens échoue. Il se décline par type (typename T) ou par valeur (usize N), se spécialise pour un cas particulier, et se généralise à un nombre quelconque d'arguments. Les traits lui permettent de choisir sa stratégie à la compilation — ce qui rend NkVector rapide sur les types simples sans être faux sur les autres.

Questions de cours

1. Pourquoi le corps d'un template doit-il vivre dans l'en-tête ?
2. Quelle différence entre PushBack et EmplaceBack ?
3. T&& dans un template et NkVector&& hors template : qu'est-ce qui les distingue ?
4. NkMove déplace-t-il quelque chose ? Justifiez.
5. Un template jamais employé peut-il contenir une erreur ? Pourquoi ?

Exercice pratique

Écrivez NkPaire<A, B> : deux champs publics, un constructeur, et une méthode Echanger() qui *rend* une NkPaire<B, A>.
Puis ajoutez un modèle de fonction Fabriquer(a, b) qui construit la paire en déduisant les deux types — de sorte qu'on écrive Fabriquer(3, 1.f) sans chevrons.

Exercice de démonstration

Prouvez, sans lire la documentation, que le corps d'un template doit être dans l'en-tête. Écrivez NkBoite<T> avec une méthode déclarée dans le .h et *définie dans un .cpp*. Employez-la depuis un autre fichier. Recopiez le message d'erreur exact, et dites à quelle étape il survient — compilation ou édition de liens. Déplacez ensuite la définition dans l'en-tête et constatez.

Exercice de logique

NkVector<int32> et NkVector<float32> sont deux classes distinctes engendrées du même modèle.

Une fonction void Afficher(NkVector<int32> &v) peut-elle recevoir un NkVector<float32> ? Et si l'on écrit template <typename T> void Afficher(NkVector<T> &v) ?

Déduisez-en ce que le compilateur engendre réellement, et pourquoi un binaire qui emploie dix types différents dans un NkVector contient dix jeux de code.

C++ moderne : lambdas, déplacement, possession

Le chapitre sur le C++ donnait le langage; celui sur les templates, la généricité. Il reste ce qui sépare un programme qui compile d'un programme qu'on ose modifier : les **lambdas**, la **sémantique de déplacement**, et les objets qui **possèdent** ce qu'ils tiennent.

Ces trois sujets sont ceux qu'on repousse en croyant qu'ils sont avancés. Ils ne le sont pas : ils sont *quotidiens*, et les ignorer produit du code qui copie sans le savoir et qui fuit sans le dire.

5.1 Les lambdas : une fonction écrite là où on s'en sert

Syntaxe

```
1 auto doubler = [](int32 x) { return x * 2; };
2 int32 y = doubler(21); // 42
```

Une lambda est un objet-fonction que le compilateur fabrique pour vous. Les crochets sont la **capture** : ce que la lambda emporte de son entourage.

Capture	Sens
[]	rien — la lambda ne voit que ses paramètres
[x]	une <i>copie</i> de x, figée à la création
[&x]	une <i>référence</i> à x — il doit vivre plus longtemps
[=]	copie de tout ce qui est employé
[&]	référence sur tout ce qui est employé
[this]	l'objet courant, par référence

Ce qu'il faut retenir

Le dépôt s'en sert dès la première ligne d'un programme. Chaque application déclare ses données ainsi :

Applications/NkDames/src/main.cpp

```
1 NKENTSEU_DEFINE_APP_DATA([]() {
2     NkAppData d;
3     d.appName = "NkDames";
4     // ...
5     return d;
6 }());
```

Une lambda *appelée sur place* — notez le () final. Elle sert à construire un objet en plusieurs étapes et à le rendre **constant** : plusieurs affectations, un seul résultat, et pas de

variable temporaire qui traîne dans la portée englobante.

Le piège qui coûte le plus cher

Une capture par référence sur un objet qui meurt avant la lambda est un pointeur pendant, avec un autre nom.

FAUX

```
1 NkFunctionCallback FabriquerRappel() {
2     int32 compteur = 0;
3     return [&compteur]() { return ++compteur; }; // compteur MEURT ici
4 }
```

Le programme compile, et fonctionne souvent — la pile n'a pas encore été réécrite. C'est exactement le défaut du chapitre sur le C, sous un déguisement qui ne ressemble plus du tout à un pointeur.

La règle : une lambda qui *survit* à la portée où elle est écrite — un rappel, un gestionnaire d'événement, une tâche donnée à un pool — capture **par valeur**. [&] ne va qu'avec un usage immédiat.

Ce qu'il faut retenir

[this] capture le *pointeur* sur l'objet, jamais une copie. Une lambda stockée dans un membre et capturant this devient invalide quand l'objet meurt — ce qui arrive plus souvent qu'on ne croit, dans une interface où les panneaux naissent et disparaissent.

5.2 Le déplacement : transférer au lieu de copier

Un vecteur d'un million d'éléments passé par valeur se copie : un million de constructions et une allocation. Or le plus souvent la source ne sert plus après.

Kernel/Foundation/NKCore/src/NKCore/NkTraits.h

```
1 template <typename T> constexpr NkRemoveReference_t<T> &&NkMove(T &&value) noexcept {
2     return static_cast<NkRemoveReference_t<T>> &&>(value);
3 }
```

Ce qu'il faut retenir

NkMove ne déplace rien. C'est une conversion de type : elle dit « traite cette valeur comme temporaire ». Le déplacement effectif est fait par le *constructeur de déplacement* de la classe, qui a le droit de voler les entrailles de la source puisqu'elle est réputée finie. C'est pourquoi compter les NkMove d'un fichier ne mesure aucune allocation : cela mesure une *intention*.

Ce que doit faire un constructeur de déplacement

```
1 NkTampon(NkTampon &&autre) noexcept
2     : mData(autre.mData), mTaille(autre.mTaille) {
3     autre.mData = nullptr; // INDISPENSABLE
4     autre.mTaille = 0;
5 }
```

Le piège

Oublier de vider la source est la faute classique, et elle est silencieuse. Sans les deux dernières lignes, *deux* objets tiennent le même pointeur ; le premier destructeur libère, le second libère à nouveau. Une double libération corrompt le tas — et le plantage arrive plus tard, ailleurs, dans du code innocent.

La source doit rester détruisible, pas utilisable. C'est le contrat exact : après un déplacement, on n'a le droit que de détruire l'objet ou de lui affecter une nouvelle valeur.

Ce qu'il faut retenir

noexcept n'est pas décoratif sur un déplacement. Un conteneur qui s'agrandit ne déplacera ses éléments que si leur déplacement est déclaré `noexcept` ; sinon il les *copie*, pour pouvoir revenir en arrière en cas d'exception.

Un mot-clé oublié, et la réallocation copie au lieu de déplacer — sans qu'aucun message ne le dise, et sans que le résultat soit faux. Seulement lent.

5.3 La possession : qui libère, et quand

Le C++ moderne répond à « qui libère ? » par un *type*. Nkntseu en offre quatre, tous dans `NkMemory` et tous en en-tête seul.

Type	Possession	Quand
<code>NkUniquePtr</code>	un seul propriétaire	le cas par défaut
<code>NkSharedPtr</code>	plusieurs, compteur atomique	durée de vie vraiment partagée
<code>NkWeakPtr</code>	aucune — observe	briser un cycle
<code>NkIntrusivePtr</code>	compteur <i>dans</i> l'objet	entités, composants, ressources

Ce qu'il faut retenir

Commencez toujours par `NkUniquePtr`. Il ne coûte rien de plus qu'un pointeur nu, il dit qui libère, et il libère tout seul. On ne passe à `NkSharedPtr` que si la durée de vie est *réellement* partagée — c'est-à-dire si l'on ne peut pas nommer un propriétaire.

« Je ne sais pas qui doit libérer » n'est pas une raison de partager : c'est le signe qu'une décision de conception n'a pas été prise.

Ce qu'il faut retenir

`NkIntrusivePtr` range le compteur *dans* l'objet, ce qui économise une allocation par rapport à `NkSharedPtr` et son bloc de contrôle séparé. En échange, l'objet doit être conçu pour — on ne peut pas l'appliquer à un type qu'on ne contrôle pas.

L'en-tête le destine explicitement aux « objets qui DOIVENT être référencés : Entity, Component, Resource ». C'est un choix guidé par le nombre d'objets : à cent mille entités, une allocation de moins par entité se compte.

Le cycle

Deux objets qui se tiennent l'un l'autre par `NkSharedPtr` ne meurent **jamais** : chacun maintient le compteur de l'autre à un. Ce n'est pas une fuite banale — aucun outil ne signale une mémoire dont le compteur est légitimement positif.

La parade est `NkWeakPtr` : le lien « retour » observe sans posséder. La règle de conception qui l'accompagne : **dans une relation parent/enfant, le parent possède, l'enfant observe.**

5.4 Deux types qui remplacent des conventions

Kernel/Foundation/NKCore/src/NKCore/NkOptional.h et NkVariant.h

```
1 NkOptional<int32> Chercher(...); // « peut-etre un entier »
2 NkVariant<A, B, C> valeur;      // « exactement un des trois »
```

Ce qu'il faut retenir

NkOptional remplace la **valeur sentinelle**, celle qu'on choisit faute de mieux : `-1`, `0`, `nullptr`, la chaîne vide. Une sentinelle a deux défauts que le type supprime : elle est une *valeur possible* du domaine (le jour où `-1` devient légitime, tout casse), et rien n'oblige l'appelant à la tester.

NkVariant remplace l'union plus un champ de type — même contenu, mais c'est le compilateur qui garantit qu'on lit le bon membre, au lieu d'une convention que chacun doit respecter.

5.5 Le reste du confort, en une page

Écriture	Ce qu'elle apporte
<code>auto x = f();</code>	le type suit sa source quand elle change
<code>for (auto &e : v)</code>	parcours sans indice — donc sans erreur d'indice
<code>constexpr</code>	calculé à la compilation, coût nul à l'exécution
<code>nullptr</code>	un pointeur nul <i>typé</i> , contrairement à <code>0</code>
<code>enum class</code>	pas de conversion implicite en entier
<code>= delete</code>	interdit explicitement une opération
<code>override</code>	le compilateur vérifie que vous redéfinissez vraiment

Le piège

for (auto e : v) copie chaque élément. Sur un vecteur de chaînes, c'est une allocation par tour, et rien ne le dit.

Écrivez `auto &` pour modifier, `const auto &` pour lire. Le `auto` nu ne se justifie que sur des types minuscules — un entier, un flottant.

Ce qu'il faut retenir

override est le mot-clé au meilleur rendement du langage. Sans lui, une signature légèrement différente de celle de la base — un `const` oublié, un type de paramètre voisin — crée une *nouvelle* fonction au lieu d'en redéfinir une. Tout compile, et la méthode de la base continue d'être appelée.

Avec `override`, le compilateur refuse. Une erreur au lieu d'un comportement.

Ce chapitre en bref

Une lambda emporte ce qu'elle capture : par valeur si elle survit à sa portée, par référence seulement pour un usage immédiat. `NkMove` est une conversion, pas un déplacement ; le vrai travail est dans le constructeur de déplacement, qui doit **vider la source** et se déclarer `noexcept`. La possession se dit par un type : `NkUniquePtr` par défaut, `NkSharedPtr` quand la durée de vie est vraiment partagée, `NkWeakPtr` pour briser les cycles, `NkIntrusivePtr` quand les objets se comptent par milliers. Enfin `NkOptional` et `NkVariant` remplacent deux conventions que rien ne faisait respecter.

Questions de cours

1. Quand faut-il capturer par valeur, et quand par référence ?
2. `NkMove` déplace-t-il quelque chose ? Que fait-il exactement ?
3. Que doit faire un constructeur de déplacement à la source, et que se passe-t-il s'il l'oublie ?
4. Pourquoi `noexcept` change-t-il le comportement d'un conteneur qui s'agrandit ?
5. Quand préférer `NkIntrusivePtr` à `NkSharedPtr` ?

Exercice pratique

Écrivez une classe `NkTampon` qui possède un tableau d'octets alloué par `NKMemory`. Donnez-lui les cinq opérations : destructeur, constructeur de copie, affectation par copie, constructeur de déplacement, affectation par déplacement.

Puis remplissez un `NkVector<NkTampon>` de mille éléments, et affichez le nombre de copies et de déplacements réellement effectués — un compteur statique incrémenté dans chaque constructeur suffit.

Retirez ensuite le `noexcept` du constructeur de déplacement, et refaites la mesure.

Exercice de démonstration

Prouvez qu'un cycle de `NkSharedPtr` ne se libère jamais.

Créez deux objets qui se tiennent mutuellement par `NkSharedPtr`, chacun affichant un message dans son destructeur. Sortez de la portée : **aucun message**. Remplacez l'un des deux liens par `NkWeakPtr` : les deux messages apparaissent.

Ce que la démonstration doit établir, et c'est la partie qu'on saute : ne vous contentez pas de constater l'absence de message. Affichez aussi le *compteur de références* des deux objets juste avant la sortie de portée. Sans ce chiffre, « rien ne s'affiche » pourrait aussi vouloir dire que votre destructeur n'affiche pas.

Exercice de logique

Une fonction range des lambdas dans une liste de rappels, appelée plus tard. Certains rappels donnent des résultats absurdes, de façon intermittente.

1. Quelle est la cause la plus probable ? Décrivez le mécanisme précis.
2. Pourquoi le défaut est-il *intermittent* plutôt que systématique ? Votre réponse doit expliquer pourquoi il disparaît souvent sous le débogueur.
3. Un collègue propose de tout capturer par valeur, partout, comme règle générale. Donnez le cas où cette règle est coûteuse, et le cas où elle est *fausse* — car il en existe un.

Ce que cette partie vous laisse

Vous disposez du langage : déclarer, structurer, allouer, hériter, généraliser. Trois idées porteront tout le reste du livre. **RAII** : une ressource est tenue par un objet, et sa libération n'est plus une chose qu'on peut oublier. **La compilation séparée** : ce qui rate à la compilation et ce qui rate au lien sont deux échecs différents, et le second se cherche ailleurs. **La généricité** : un algorithme écrit une fois sert tous les types — et son corps vit dans l'en-tête, sinon rien ne lie.

PARTIE III

Les fondations

Les briques de toute application

Entre le langage et le moteur, il y a une couche que l'on emploie dans *chaque* programme du dépôt — graphique ou non, jeu ou outil en ligne de commande. Ce sont la mémoire, les conteneurs, les mathématiques, la journalisation et les fils d'exécution.

On les place ici, et non parmi les modules, parce qu'ils ne se choisissent pas : l'image, le son et le réseau sont facultatifs ; `NkVector` et `NkString` ne le sont pas. Vous les rencontrerez dans la première ligne du premier chapitre suivant.

NKMemory ouvre la partie, et ce n'est pas un ordre arbitraire : tout le reste alloue à travers lui. C'est aussi le seul chapitre du livre dont une erreur ne se manifeste jamais là où elle a été commise — un tas corrompu plante dans du code innocent, des minutes plus tard.

Deux autres méritent d'être lus même par un programmeur expérimenté. **NKLogger** porte un piège qui ne plante jamais et fausse des messages en silence. **NKThreading** commence par vous conseiller de ne pas vous en servir — et l'argument est sérieux.

NKMemory : qui alloue, qui libère

C'est le chapitre le plus important des fondations, et le seul dont une erreur ne se manifeste jamais là où elle a été commise.

Nkentseu n'emploie ni `new`, ni `delete`, ni `malloc`. Toute la mémoire passe par **NKMemory**. Ce n'est pas une préférence de style : c'est ce qui rend possible le suivi des fuites, les stratégies d'allocation par usage, et le portage sur des plateformes où le tas du système n'est pas celui qu'on croit.

6.1 La règle, et ce qu'elle coûte de violer

Ce qu'il faut retenir

Ce qui a été alloué par NKMemory se libère par NKMemory. Point.

Un bloc obtenu par `NkAlloc` et rendu par `free` — ou par `delete[]` — rend un pointeur à un gestionnaire de tas *qui ne l'a jamais donné*. Sous Windows, cela produit le code `0xC0000374`, « corruption du tas ».

Et voici pourquoi c'est le pire des défauts : **le programme ne plante pas au moment de la faute**. Il plante plus tard, dans une allocation parfaitement innocente, parfois des milliers d'instructions après — et c'est cette allocation-là qu'on accuse.

Trois cas réels du dépôt

1. Le tampon rendu par un encodeur — `NkJPEGCodec::Encode`, `NkPNGCodec::Encode`, `SaveToMemory` — est alloué par `NkAlloc`. Le libérer avec `delete[]` corrompt le tas.
2. `NkImage::Alloc()` suivi de `Free()` puis de `delete img` est une double libération : `Free()` rend déjà l'enveloppe.
3. Appeler `Free()` sur une `NkImage` déclarée **sur la pile** exécute une libération *sur une adresse de pile*. Cela a réellement fait planter une démo du dépôt.

La question à se poser devant toute libération n'est pas « comment libère-t-on ceci ? » mais « qui l'a alloué ? ».

6.2 Les quatre fonctions de base

Kernel/Foundation/NKMemory/src/NKMemory/NkAllocator.h:704-737

```
1 void *NkAlloc(nk_size size, NkAllocator *allocator = nullptr,
2               nk_size alignment = NK_MEMORY_DEFAULT_ALIGNMENT);
3
4 void *NkAllocZero(nk_size count, nk_size size, NkAllocator *allocator = nullptr,
5                  nk_size alignment = NK_MEMORY_DEFAULT_ALIGNMENT);
6
7 void *NkRealloc(void *ptr, nk_size oldSize, nk_size newSize,
8                NkAllocator *allocator = nullptr,
```

```

9         nk_size alignment = NK_MEMORY_DEFAULT_ALIGNMENT);
10
11 void NkFree(void *ptr, NkAllocator *allocator = nullptr);
12 void NkFree(void *ptr, nk_size size, NkAllocator *allocator = nullptr);

```

Ce qu'il faut retenir

Le paramètre `allocator` à `nullptr` signifie « celui par défaut », pas « aucun ». C'est ce qui rend le code ordinaire lisible — `NkAlloc(1024)` suffit — tout en permettant à un sous-système d'imposer le sien.

Et il doit être le MÊME à l'allocation et à la libération. Allouer sur un allocateur nommé et libérer sur celui par défaut est exactement la même faute que mélanger `NkAlloc` et `free`, avec un déguisement de plus.

Ce qu'il faut retenir

`NkRealloc` rend `nullptr` en cas d'échec — et l'ancien pointeur reste valide. C'est écrit dans sa documentation, et c'est la seule façon correcte : écraser votre pointeur par le retour sans le tester perd le bloc d'origine.

FAUX

```
1 p = NkRealloc(p, ancien, neuf); // si echec : p vaut nullptr, et le bloc est perdu
```

6.3 Allouer un objet, pas des octets

`NkAlloc` rend de la mémoire brute : aucun constructeur n'a été appelé. Pour un objet, ce sont les quatre modèles de `NkAllocator` qu'il faut :

Kernel/Foundation/NKMemory/src/NKMemory/NkAllocator.h:239-273

```

1 template <typename T, typename... Args> [[nodiscard]] T *New(Args &&...args);
2 template <typename T> void Delete(T *ptr) noexcept;
3 template <typename T, typename... Args> [[nodiscard]] T *NewArray(SizeType count, Args
   &&...args);
4 template <typename T> void DeleteArray(T *ptr) noexcept;

```

Ce qu'il faut retenir

Chacun fait deux choses, et les fait dans le bon ordre. `New<T>` alloue puis construit; `Delete<T>` détruit puis libère. C'est exactement ce que font `new` et `delete` du langage — mais sur votre allocateur.

Et `NewArray/DeleteArray` vont ensemble : le premier range le nombre d'éléments à côté du tableau, le second le relit pour appeler le bon nombre de destructeurs. Libérer un tableau avec `Delete<T>` n'appelle qu'un seul destructeur et se trompe d'adresse de base.

Le piège

`[[nodiscard]]` sur `New` n'est pas décoratif : ignorer le retour d'une allocation est une fuite garantie, et le compilateur le refuse désormais. C'est le genre de garde qui coûte un mot et travaille pour toujours.

6.4 Neuf allocateurs, et pourquoi il en faut plusieurs

Un tas généraliste doit répondre à n'importe quelle demande dans n'importe quel ordre. C'est un problème difficile — donc lent. Or la plupart des programmes n'ont pas ce besoin : ils allouent *par motif*.

Allocateur	Motif qu'il exploite	Ce qu'il abandonne
NkMallocAllocator	aucun — le tas du système	rien, mais rien de gagné
NkNewAllocator	aucun — new/delete	idem
NkVirtualAllocator	très gros blocs, pages OS	la granularité fine
NkLinearAllocator	« tout meurt en même temps »	la libération individuelle
NkArenaAllocator	idem, par zones	idem
NkStackAllocator	dernier alloué, premier libéré	l'ordre libre
NkPoolAllocator	objets de <i>même taille</i>	les tailles variables
NkFreeListAllocator	tailles variables, réemploi	la simplicité
NkBuddyAllocator	tailles en puissances de deux	jusqu'à 50 % de place

Ce qu'il faut retenir

Chaque ligne de la colonne du milieu est une ***hypothèse sur la durée de vie***, et c'est elle qu'on achète. Un allocateur rapide n'est pas rapide en soi : il est rapide *parce qu'il refuse de répondre à des questions que vous ne poserez pas*.

Le corollaire est immédiat : **choisir un allocateur, c'est déclarer un motif d'usage**. Si le motif est faux, l'allocateur est plus lent que le tas ordinaire, ou il échoue.

6.4.1 Le cas le plus utile : l'allocateur linéaire

Kernel/Foundation/NKMemory/src/NKMemory/NkAllocator.h:393-398

```
1 // Tres rapide pour Les allocations temporaires frame-based.
2 // Ne supporte pas la deallocation individuelle - Reset() Libere tout.
```

Ce qu'il faut retenir

Un allocateur linéaire n'est qu'un **curseur qui avance**. Allouer, c'est ajouter la taille au curseur et rendre l'adresse précédente : quelques instructions, aucune recherche, aucune fragmentation possible.

Deallocate n'y fait presque rien — l'en-tête le dit : il n'optimise que le cas où l'on rend le *dernier* bloc. La libération réelle est `Reset()`, qui remet le curseur à zéro et rend tout d'un coup.

Le patron d'emploi : un allocateur linéaire par image. Tout ce qui est temporaire y est alloué ; à la fin de l'image, un `Reset()` et le coût de libération de milliers d'objets tombe à une affectation.

Le piège

Rien de ce qui est alloué sur un allocateur linéaire ne doit survivre à son `Reset()`. Garder un pointeur d'une image sur l'autre donne une adresse qui pointe désormais sur *autre chose* — pas sur du vide, ce qui serait détectable, mais sur des données valides appartenant

nant à quelqu'un d'autre.

C'est le pointeur pendant sous sa forme la plus perverse : la mémoire est lisible, le programme ne plante pas, et les valeurs sont fausses.

Ce qu'il faut retenir

Capacity(), Used() et Available() sont là pour être *lus*. Un allocateur linéaire qui débordé ne s'agrandit pas : il échoue. La bonne pratique est de journaliser le pic d'utilisation en développement, et de dimensionner sur la mesure — jamais sur une estimation.

6.5 Savoir ce qu'on a alloué

Outil	Ce qu'il apporte
NkTracker	détection de fuites en $O(1)$, par table de hachage
NkTag	étiqueter une allocation — « textures », « audio »...
NkProfiler	compteurs, pics, statistiques par allocateur
NkGc	un ramasse-miettes <i>marquage et balayage</i> , pour qui le veut

Ce qu'il faut retenir

Le suivi n'est possible que parce que tout passe par NKMemory. C'est le retour sur investissement de la règle du début de chapitre : un seul `new` échappé, et le bloc correspondant devient invisible au traqueur.

Autrement dit, la discipline n'est pas une contrainte *en échange de rien* : elle est ce qui achète l'instrument.

Le piège

Un traqueur de fuites signale ce qui n'a pas été libéré à la fin du programme. Il ne voit pas une mémoire qui grossit indéfiniment mais qui finit par être rendue — et c'est pourtant le défaut qui tue une application longue.

Pour celui-là, l'instrument est le *pic* d'utilisation dans le temps, pas le solde à la fermeture. Deux mesures différentes, deux défauts différents.

6.6 Ce que vous écrirez vraiment

En pratique, dans du code d'application, vous n'appellerez presque jamais `NkAlloc` :

Besoin	Ce qu'on emploie
une suite d'éléments	<code>NkVector</code> — il alloue pour vous
du texte	<code>NkString</code>
un objet qui vit tant qu'on le tient	<code>NkUniquePtr</code>
un tampon brut, une image, un fichier	<code>NkAlloc/NkFree</code>

Ce qu'il faut retenir

Les conteneurs et les pointeurs intelligents **sont** la façon normale d'employer NKMemory. Ils allouent à travers lui, et ils libèrent tout seuls.

Ce chapitre n'est donc pas un mode d'emploi quotidien : c'est ce qu'il faut avoir lu pour comprendre ce que le conteneur fait à votre place — et pour savoir quoi regarder le jour où le tas se corrompt.

Ce chapitre en bref

Toute la mémoire de Nkntseu passe par NKMemory, et **ce qui a été alloué par lui se libère par lui** : mélanger les gestionnaires corrompt le tas, et le plantage arrive plus tard, ailleurs, dans du code innocent. NkAlloc rend des octets, New<T> rend un objet construit — et NewArray va avec DeleteArray, jamais avec Delete. Les neuf allocateurs n'existent pas pour le choix : chacun *achète de la vitesse en refusant de répondre à une question*, et le linéaire — allouer sans jamais libérer individuellement, Reset() à la fin de l'image — est celui qui rapporte le plus. Enfin, le suivi des fuites n'est possible que parce que la règle est tenue partout.

Questions de cours

1. Que se passe-t-il si l'on libère avec free un bloc obtenu par NkAlloc — et *quand* cela se voit-il ?
2. Quelle différence entre NkAlloc et New<T> ?
3. Pourquoi NewArray exige-t-il DeleteArray ?
4. Que fait Deallocate sur un allocateur linéaire, et comment y libère-t-on réellement ?
5. Pourquoi un traqueur de fuites ne voit-il pas une mémoire qui grossit puis est rendue ?

Exercice pratique

Écrivez un NkLinearAllocator de 64 Kio et servez-vous-en pour allouer, à chaque image d'une petite application, un tableau temporaire dont la taille dépend de ce qui est à l'écran. Trois exigences :

1. afficher Used() et Available() en clair, à chaque image ;
2. appeler Reset() à la fin de l'image, et *nulle part ailleurs* ;
3. faire délibérément déborder l'allocateur, et voir ce qui se passe. Notez ce que l'allocateur rend, et ce que votre code en fait — c'est la deuxième partie qui compte.

Exercice de démonstration

Prouvez qu'un pointeur survivant à un Reset() donne des valeurs fausses sans planter. Allouez une structure sur l'allocateur linéaire à l'image n , remplissez-la avec des valeurs que vous reconnaîtrez, et **gardez le pointeur**. À l'image $n + 1$, après le Reset() et après avoir alloué autre chose, relisez la structure et affichez son contenu.

Ce que la démonstration doit établir, et c'est le point : non pas que le programme plante — il ne plantera pas — mais que les valeurs relues sont *plausibles et fausses*. Affichez-les à côté des valeurs d'origine.

□ Ne concluez rien d'une seule exécution où les valeurs n'auraient pas changé : cela signifie seulement que rien n'a encore réécrit cette zone. Allouez davantage à l'image $n + 1$ et

recommencez.

Exercice de logique

Une application plante avec `0xC0000374` au bout de dix minutes, toujours dans une allocation différente, jamais au même endroit.

1. Expliquez pourquoi le point de plantage ne désigne pas le coupable.
2. Énumérez au moins quatre gestes du code susceptibles de produire ce symptôme, et rangez-les du plus au moins probable.
3. Vous soupçonnez un module précis. Décrivez comment vous le *disculperiez* — pas comment vous le confirmeriez. Expliquez pourquoi c'est le bon sens de l'enquête ici, alors que la démarche inverse serait naturelle.

NKContainers : ranger des données sans la bibliothèque standard

Vous savez maintenant ce qu'est un template. NKContainers en est la démonstration : cinquante-deux en-têtes qui donnent à Nkentseu ses tableaux, ses chaînes, ses tables et ses arbres — sans une ligne de bibliothèque standard.

Ce chapitre montre ceux dont vous vous servirez tous les jours, dit ce que chacun coûte, et nomme les pièges que le dépôt a déjà payés.

7.1 Pourquoi ne pas employer la bibliothèque standard

Trois raisons, et aucune n'est idéologique.

Raison	Conséquence concrète
Plateformes contraintes	certaines n'ont pas de STL utilisable
Maîtrise de l'allocation	tout passe par NKMemory, donc se mesure
Comportement identique partout	la STL varie d'un compilateur à l'autre

Ce qu'il faut retenir

La règle est **positive** : on n'écrit pas `std::`, et *si la primitive manque, on l'écrit au niveau où elle doit vivre* — dans Foundation, pas chez soi. Un utilitaire écrit localement devient le troisième exemplaire d'une chose qui aurait dû être partagée.

7.2 Le tableau dynamique : `NkVector<T>`

C'est le conteneur que vous emploierez neuf fois sur dix.

Kernel/Foundation/NKContainers/src/NKContainers/Sequential/NkVector.h

```

1  NkVector<int32> v;
2  v.PushBack(10);
3  v.PushBack(20);
4  v.EmplaceBack(30);           // construit SUR PLACE, pas de copie
5
6  const usize n = v.Size();    // combien d'elements
7  const usize c = v.Capacity(); // combien AVANT la prochaine reallocation
8  int32 premier = v[0];
9  v.Reserve(100);              // reserve d'un coup, evite N reallocations
10 v.Clear();                   // vide, GARDE la capacite

```

Méthode	Effet	Coût
PushBack(v)	ajoute à la fin	amorti constant
EmplaceBack(args...)	construit à la fin	amorti constant
operator[]	accès direct	constant
Insert(pos, v)	insère au milieu	linéaire
Erase(pos)	retire	linéaire
Find(v)	cherche	linéaire
Reserve(n)	réserve	une allocation

Ce qu'il faut retenir

Size n'est pas Capacity. La taille est le nombre d'éléments présents; la capacité, la place réservée. Quand la taille atteint la capacité, le tableau réalloue — il demande un bloc plus grand et recopie tout.

D'où Reserve : si vous savez que vous allez ajouter mille éléments, une réservation remplace une dizaine de réallocations. Ce n'est pas une micro-optimisation, c'est une allocation contre dix.

Le piège

Toute réallocation invalide les pointeurs et les itérateurs. Un pointeur pris sur `v[0]`, puis un `PushBack` qui déclenche une réallocation : le pointeur désigne l'ancien bloc, déjà libéré.

Exemple pédagogique – FAUX

```
1 int32 *p = &v[0];
2 v.PushBack(42);    // peut REALLOUER
3 *p = 7;           // pointeur pendant : on écrit dans le vide
```

C'est le *pointeur pendant* du chapitre sur le C, sous sa forme la plus courante. Prenez l'indice, pas l'adresse.

Le piège

Clear ne libère pas la mémoire, il remet la taille à zéro. Un vecteur qui a contenu un million d'éléments occupe toujours autant après un `Clear`. Pour rendre la mémoire : `ShrinkToFit`.

7.3 Les chaînes : NkString

Kernel/Foundation/NKContainers/src/NKContainers/String/NkString.h

```
1 NkString s = "Bonjour";
2 s.Append(" tout le monde");
3 const char *brut = s.Data();           // pour Les API qui veulent du C
4 const usize n = s.Size();
5 bool ok = s.StartsWith("Bon");
6 NkString morceau = s.SubStr(8);        // à partir de l'indice 8
7
8 NkString msg = NkString::Format("%d entite(s) en %0.2f ms", 42, 1.5f);
```

Le piège

NkString::Format est de la famille printf : ses marqueurs sont %d, %s, %f. Le journal, lui, a *deux* familles — `logger.Infof` en printf, et `logger.Info` en marqueurs indexés {0}. Mélanger les deux n'échoue pas : le marqueur s'affiche littéralement, le message sort faux et ressemble à un message.

Le piège

Une chaîne peut venir de nulle part. `env::GetEnvVar` rend un `const char *` qui vaut `nullptr` quand la variable n'existe pas. Écrire `NkString v = env::GetEnvVar("X");` construit alors une chaîne depuis un pointeur nul. Ici le constructeur garde par `if (str)` et rend une chaîne vide — mais «absente» et «définie à vide» deviennent indistinguables. Si la différence compte, testez le `const char *` *avant* de le convertir.

7.4 Les tables

Conteneur	Ordonné ?	Recherche
<code>NkUnorderedMap<K, V></code>	non	constante en moyenne
<code>NkMap<K, V></code>	oui, par clé	logarithmique
<code>NkUnorderedSet<T></code>	non	constante en moyenne
<code>NkSet<T></code>	oui	logarithmique

Ce qu'il faut retenir

Prenez la version *non ordonnée* par défaut : elle est plus rapide. Ne prenez l'ordonnée que si vous devez **parcourir dans l'ordre des clés** — c'est la seule chose qu'elle apporte, et elle se paie.

7.5 Le reste, et à quoi ça sert

Conteneur	Quand
<code>NkArray<T,N></code>	taille fixe connue à la compilation, zéro allocation
<code>NkSpan<T></code>	<i>vue</i> sur des données qu'on ne possède pas
<code>NkOptional<T></code>	« une valeur, ou rien » — sans pointeur nul
<code>NkResult<T,E></code>	« une valeur, ou une erreur »
<code>NkPair</code> , <code>NkTuple</code>	deux valeurs, ou n
<code>NkVariant</code>	une valeur parmi plusieurs types
<code>NkFunction</code>	ranger un appel dans une variable
<code>NkRingBuffer<T></code>	file de taille fixe qui tourne (audio, journaux)
<code>NkQueue</code> , <code>NkStack</code> , <code>NkDeque</code>	file, pile, file double
<code>NkPriorityQueue</code>	le plus urgent en premier
<code>NkList</code> , <code>NkDoubleList</code>	listes chaînées
<code>NkQuadTree</code> , <code>NkOctree</code>	partition de l'espace, 2D et 3D
<code>NkBTree</code> , <code>NkTrie</code> , <code>NkGraph</code>	arbres et graphes
<code>NkSort</code>	le tri du dépôt
<code>NkUTF8</code> , <code>NkUTF16</code> , <code>NkUTF32</code>	encodages de texte
<code>NkBase64</code>	encodage binaire vers texte

Ce qu'il faut retenir

`NkSpan<T>` ne possède rien : c'est un pointeur et une longueur. Il résout le piège du chapitre sur le C — « un tableau ne connaît pas sa taille » — en transportant les deux ensemble. Mais il ne prolonge pas la vie de ce qu'il désigne : un span sur un vecteur détruit est un pointeur pendant.

Le piège

`NkOptional` n'est pas un pointeur nullable, et c'est mieux. `HasValue()` dit s'il y a quelque chose ; `Value()` le donne. Appeler `Value()` sur un optional vide est une faute — alors que déréférencer un pointeur nul est un plantage. La différence : l'un se teste, et la signature vous oblige à y penser.

Ce chapitre en bref

`NkVector` pour presque tout, `NkString` pour le texte, `NkUnorderedMap` pour associer. `Size` n'est pas `Capacity`, et toute réallocation invalide les pointeurs. `Clear` ne rend pas la mémoire. `NkSpan` transporte un tableau *avec* sa taille, mais ne le possède pas. `NkOptional` remplace le pointeur nullable par quelque chose que la signature oblige à tester.

Questions de cours

1. Quelle différence entre `Size()` et `Capacity()` ?
2. Après `Clear()`, la mémoire est-elle rendue ? Comment la rendre ?
3. Pourquoi `Reserve` peut-il changer les performances d'un facteur important ?

4. Quand prendre `NkMap` plutôt que `NkUnorderedMap` ?
5. `NkSpan` possède-t-il ses données ? Qu'est-ce que cela implique ?

Exercice pratique

Écrivez une fonction qui reçoit un `NkVector<NkString>` et rend un `NkUnorderedMap<NkString, int32>` comptant les occurrences de chaque chaîne. Puis mesurez : construisez un vecteur de 100 000 chaînes, comptez avec et sans `Reserve` sur la table, et comparez les durées avec `NkChrono`.

Exercice de démonstration

Prouvez qu'une réallocation invalide les pointeurs. Prenez un `NkVector<int32>`, réservez exactement 2 éléments, gardez `&v[0]`, puis ajoutez jusqu'à dépasser la capacité. Affichez la *valeur de l'adresse* de `&v[0]` avant et après. Deux adresses différentes prouvent la réallocation. **N'écrivez pas à travers l'ancien pointeur** pour le « vérifier » : le programme pourrait ne rien faire d'anormal, et vous concluriez à tort que c'est sûr.

Exercice de logique

`PushBack` est dit « à coût amorti constant » alors qu'une réallocation recopie tous les éléments. Supposez que la capacité *double* à chaque réallocation. Comptez le nombre total de recopies pour insérer n éléments en partant d'une capacité de 1. Montrez que ce total est proportionnel à n , et non à n^2 — puis expliquez en une phrase pourquoi doubler, plutôt qu'ajouter une place, change la nature du coût.

NKMath : vecteurs, matrices, angles

Tout ce qui bouge à l'écran passe par NKMath. Une position, une taille, une rotation, une couleur, un rectangle : ce sont ses types. Ce chapitre les couvre, puis nomme les pièges du calcul à virgule flottante — ceux qui font qu'un programme juste sur le papier donne un résultat faux à l'exécution.

8.1 Les vecteurs

Suffixe	Type	Alias
f	float32	NkVec2f, NkVec3f, NkVec4f
d	float64	NkVec2d, NkVec3d, NkVec4d
i	int32	NkVec2i, NkVec3i, NkVec4i
u	uint32	NkVec2u, NkVec3u, NkVec4u

Exemple pédagogique

```

1 NkVec2f position(100.f, 50.f);
2 NkVec2f vitesse(1.f, 0.f);
3 position = position + vitesse * dt;    // Les operateurs sont surcharges
4
5 const math::NkVec2u taille = fenetre.GetSize(); // une TAILLE, non signee
```

Ce qu'il faut retenir

Le suffixe n'est pas décoratif. NkVec2u pour une taille d'écran — elle ne peut pas être négative. NkVec2i pour une position en pixels — elle peut l'être. NkVec2f pour une position dans le monde. Choisir le mauvais suffixe compile et donne une soustraction qui repasse par le haut.

8.2 Matrices et quaternions

NkMat2, NkMat3, NkMat4 — et leurs variantes f et d. Une matrice 4×4 transporte une transformation complète : translation, rotation, échelle.

NkQuat représente une rotation sans les blocages de cardan que produisent trois angles d'Euler enchaînés.

Ce qu'il faut retenir

Décision d'architecture du dépôt : les transformations se stockent en **translation, rotation et échelle séparées**, jamais en matrice unique — et **jamais de cisaillement**. Une matrice compose les trois de façon irréversible : on ne peut plus retrouver la rotation seule pour l'animer.

8.3 Angles, aléatoire, géométrie

En-tête	Contenu
NkAngle.h	angles typés — degrés et radians ne se confondent plus
NkEulerAngle.h	les trois angles, quand on en a besoin
NkRandom.h	générateur reproductible à graine égale
NkRectangle.h	NkRect2f, NkRect2i
NkSegment.h	segments
NkColor.h	couleurs
NkRange.h	intervalles
NkSIMD.h	calcul vectoriel accéléré

Ce qu'il faut retenir

Un générateur reproductible à graine égale n'est pas un défaut : c'est la condition d'un **banc**. Les trois jeux du dépôt tirent leurs dés avec NkRandom et une graine fixée — c'est ce qui permet à un banc de rejouer exactement la même partie et de vérifier qu'elle se termine.

8.4 Les fonctions

Kernel/Foundation/NKMath/src/NKMath/NkFunctions.h

```
1 math::NkSin(x)      math::NkCos(x)      math::NkTan(x)
2 math::NkSqrt(x)     math::NkAbs(x)      math::NkPow(x, y)
3 math::NkMin(a, b)   math::NkMax(a, b)   math::NkClamp(v, lo, hi)
4 math::NkLerp(a, b, t)
5 math::NkRound(x)    math::NkFloor(x)    math::NkCeil(x)
```

Le piège

On emploie NKMath, jamais `<cmath>`. `std::sqrt` et `math::NkSqrt` donnent le même nombre aujourd'hui ; mais la règle du dépôt est qu'aucun `std::` n'entre, et NKMath porte en plus les variantes SIMD et les types du moteur.

Ce qu'il faut retenir

`NkClamp(v, lo, hi)` borne une valeur ; `NkLerp(a, b, t)` interpole. Ces deux-là suffisent à écrire la plupart des animations : une valeur qui va de `a` vers `b` en fonction d'un `t` entre 0 et 1, bornée pour ne pas dépasser.

8.5 Les constantes

NK_PI_F pour les `float32`, NK_PI_D pour les `float64`.

Le piège

Il n'existe pas de `NK_PI` tout court. Le dépôt a payé cette absence : un module avait défini sa propre macro locale, perdue lors d'une extraction. Employez le suffixe qui correspond à votre type — et si vous cherchez `NK_PI` et ne le trouvez pas, ce n'est pas que π manque.

8.6 La virgule flottante, et ce qu'elle ne sait pas faire

Le piège

`==` sur des flottants est presque toujours une faute. $0.1f + 0.2f$ ne vaut pas $0.3f$: ces nombres n'ont pas de représentation exacte en binaire. Comparez à une tolérance près :

Idiome du depot

```
1 if (math::NkAbs(a - b) < 1e-4f) { /* egaux, a la tolerance pres */ }
```

Et la tolérance est un CHOIX, pas une constante universelle. Le dépôt emploie $1e-4f$ pour fusionner des sommets de maillage : trop grande, deux sommets distincts se confondent ; trop petite, deux sommets identiques restent séparés. Le bon epsilon dépend de l'échelle de vos données.

Le piège

L'addition flottante n'est pas associative. $(a+b)+c$ peut différer de $a+(b+c)$. C'est pourquoi deux exécutions d'un même calcul GPU peuvent donner des résultats légèrement différents si l'ordre de réduction varie — sans qu'aucun code ait changé. Conséquence pratique : **ne concluez pas d'un écart minuscule entre deux mesures qu'un défaut existe**, avant d'avoir mesuré le bruit de la mesure elle-même.

Ce chapitre en bref

Les types portent leur précision dans leur suffixe : `f`, `d`, `i`, `u` — et le choix a des conséquences, pas seulement un style. Les transformations se stockent en `T + R + S` séparées, jamais en matrice unique. `NkClamp` et `NkLerp` suffisent à la plupart des animations. On n'emploie pas `<cmath>`. Et sur les flottants : jamais `==`, une tolérance choisie, et l'addition n'est pas associative.

Questions de cours

1. Pourquoi `NkVec2u` pour une taille d'écran et `NkVec2i` pour une position ?
2. Pourquoi le dépôt refuse-t-il de stocker une transformation en matrice unique ?
3. Existe-t-il un `NK_PI` ? Que faire si l'on ne le trouve pas ?
4. Pourquoi `a == b` est-il dangereux sur des `float32` ?
5. En quoi un générateur aléatoire reproductible sert-il un banc ?

Exercice pratique

Écrivez une fonction `NkVec2f Osciller(float32 t, NkVec2f centre, float32 rayon)` qui rend un point tournant autour du centre.

Puis animez-la dans une application `NkCanvasApp` : le point doit décrire un cercle, et sa vitesse doit être *indépendante* de la cadence d’affichage. Indice : c’est `deltaTime` qui décide, pas le nombre d’images.

Exercice de démonstration

Prouvez que $0.1f + 0.2f \neq 0.3f$.
Affichez la différence avec dix décimales. Puis trouvez, par essais, la plus petite tolérance qui déclare ces deux nombres égaux. Enfin, refaites l’essai avec des `float64` : la tolérance nécessaire change-t-elle ? De combien ?
Concluez sur ce que « choisir un epsilon » veut dire.

Exercice de logique

Un objet se déplace de `vitesse * deltaTime` à chaque image.
Un joueur a 60 images par seconde, un autre 144. Après une seconde, ont-ils parcouru la même distance ? Démontrez-le.
Puis supposez que le code s’écrive `position += vitesse` sans `deltaTime` : que devient la réponse, et pourquoi ce défaut ne se voit-il jamais sur la machine du développeur ?

NKLogger : dire ce qui se passe

Un programme graphique n'a pas de console. Quand il se comporte mal, la seule trace qui reste est celle qu'il a écrite lui-même. Ce chapitre montre comment l'écrire — et surtout comment ne pas la rendre fausse, ce qui arrive plus souvent qu'on ne croit.

9.1 Les six niveaux

Niveau	Pour quoi	Qui le lit
Trace	le détail fin d'une boucle	vous, en cherchant
Debug	un état intermédiaire	vous, en développant
Info	une étape franchie	tout le monde
Warn	anormal, mais on continue	tout le monde
Error	la fonction a échoué	tout le monde
Fatal	on ne peut pas continuer	tout le monde

Ce qu'il faut retenir

Le niveau n'est pas une nuance de politesse : c'est ce qui décide de *ce qu'on voit* quand on filtre. Un message important écrit en Trace disparaît le jour où l'on relève le filtre — et c'est précisément le jour où l'on cherche quelque chose.

9.2 □ Deux familles de messages, et on ne les mélange pas

C'est le piège le plus coûteux du module, et il ne plante jamais.

Les deux familles

```

1 // FAMILLE 1 -- suffixe `f`, style printf. Le `\\n` final est EXPLICITE.
2 logger.Infof("[nkdamers] %d coups legaux en %.2f ms\\n", n, duree);
3
4 // FAMILLE 2 -- sans suffixe, marqueurs INDEXES {i}. Pas de `\\n` à écrire.
5 logger.Info("[nkdamers] {0} coups legaux en {1} ms", n, duree);
```

Le piège

Un marqueur `{0}` passé à `Infof` n'est pas interprété : il s'affiche littéralement. Un `%s` passé à `Info` fait de même.

Dans les deux cas **rien ne plante, rien n'avertit** : le message sort, faux, et ressemble à un message. On lit alors une trace qui ment, et l'on cherche le défaut ailleurs.

Choisissez la famille *avant* d'écrire la chaîne, et n'en changez pas en cours de ligne.

Le piège des accolades, et il a coûté trois jours

Un texte qui contient des accolades ne passe pas par un formateur à marqueurs. La famille 2 prend toute accolade pour un marqueur : un bloc de code, un JSON, une règle CSS y perdent leur corps — *en silence*.

Ce qui arrive réellement

```
1 Entree : "uniform BlocA {\n    vec4 va;\n} uA;"
2 Sortie : "uniform BlocA 0 uA;"
```

Et le second membre compte autant que le premier : **le texte amputé reste valide** pour l'étape suivante. La panne sort donc à l'autre bout de la chaîne, des heures plus tard, sous la forme d'une valeur qui n'arrive pas. Pour ces textes-là : Infof (famille 1), ou une écriture directe. Jamais la famille 2.

9.3 Où le journal atterrit

Fichier	Contenu
logs/app_<date>_<heure>_<pid>.log	un par lancement, conservé
logs/app.log	la course <i>courante</i> seule, tronquée au lancement

Le piège

Une application de ce dépôt n'écrit pas sur la sortie standard. Elle écrit par NKLogger dans logs/, *relativement au répertoire courant* — jamais à côté du binaire, jamais sur la console en Release.

Conséquence : capturer la sortie standard pour prouver qu'une démo tourne mesure **le bon programme sur le mauvais canal**, et fait conclure « démo muette » sur du code sain. Le diagnostic qui tranche en trois secondes :

Commande

```
1 find . -name "*.log" -mmin -30
```

Ce qu'il faut retenir

Un journal par lancement, et c'est ce qui rend une mesure lisible. Avant, app.log accumulait indéfiniment : un fichier réel portait six lancements sur deux jours, avec vingt-six heures entre deux lignes consécutives et aucun marqueur. On attribuait à une course les lignes de la précédente.

Les vingt derniers journaux sont conservés, puis purgés au démarrage. NKENTSEU_LOG_KEEP change le nombre; 0 désactive la purge. Un *compte* et non un âge : un âge ne borne rien — six courses tiennent dans vingt minutes.

9.4 Ce qu'un bon message contient

Ce qu'il faut retenir

Un message utile porte un chiffre et sa provenance.

Inutile	Utile
« erreur de chargement »	« [nimage] hero.png : 0 octet lu sur 4096 attendus »
« ça marche »	« [banc] 16 cas sur 16 : VERT »
« lent »	« [rendu] 42,3 ms/image, budget 16,6 »

Le préfixe entre crochets dit *qui parle*. Sans lui, une trace de cent lignes ne se démêle plus.

Ce chapitre en bref

Six niveaux, et le niveau décide de ce qu'on verra en filtrant. **Deux familles de messages** — Infof en printf, Info en marqueurs {0} — qu'on ne mélange jamais, parce que l'erreur ne plante pas : elle produit un message faux. Aucun texte à accolades ne passe par la famille 2. Le journal va dans logs/, un fichier par lancement, jamais sur la sortie standard. Et un message utile porte un chiffre.

Questions de cours

1. Quelles sont les deux familles de messages, et comment les reconnaît-on ?
2. Que se passe-t-il si l'on passe {0} à Infof ?
3. Pourquoi capturer la sortie standard ne montre-t-il rien ?
4. Pourquoi un journal par lancement plutôt qu'un fichier qui s'accumule ?
5. Pourquoi la rétention se compte-t-elle en *nombre* et non en *âge* ?

Exercice pratique

Instrumentez une petite application : au démarrage, journalisez la taille de la fenêtre, le backend graphique retenu et la zone sûre. Toutes les cent images, journalisez la durée moyenne d'une image.

Puis lancez-la, ouvrez le fichier produit dans logs/, et vérifiez que vous pouvez répondre à cette question sans relancer : *combien d'images par seconde a-t-elle tenu ?*

Exercice de démonstration

Prouvez que mélanger les familles produit un message faux sans rien casser. Écrivez la même information deux fois, une fois avec le mauvais formateur :

A tester

```
1 logger.Infof("valeur = {0}\n", 42);
2 logger.Info("valeur = %d", 42);
```

Recopiez les deux lignes telles qu'elles sortent du journal. Puis faites de même avec un texte contenant une accolade, et montrez ce qui disparaît.

Exercice de logique

Un journal en mode ajout contient six lancements. Une ligne dit [rendu] 42,3 ms/image. Quelles informations vous manquent pour savoir si cette ligne concerne la course que vous venez de lancer? Énumérez-les.

Puis dites laquelle, à elle seule, suffirait — et pourquoi un fichier par lancement les rend toutes inutiles.

NKThreading : faire deux choses à la fois

Un fil d'exécution — un *thread* — permet à un programme de faire plusieurs choses en même temps. C'est la partie du métier où les défauts sont les plus difficiles à trouver : ils ne se reproduisent pas, disparaissent sous le débogueur, et changent de forme d'une machine à l'autre.

Ce chapitre donne le vocabulaire, l'API de Nkntseu, et surtout la discipline qui évite d'en avoir besoin.

Ce qu'il faut retenir

La meilleure façon de gagner du temps avec les fils est de ne pas en employer. Un programme à un seul fil est lent mais juste ; un programme à plusieurs fils mal synchronisé est rapide et faux — et il vous le dira six mois plus tard, chez un utilisateur, une fois sur mille.
N'en ajoutez que lorsqu'une *mesure* montre qu'il le faut.

10.1 Ce qu'est une course, et pourquoi c'est si dur

Exemple pédagogique – FAUX

```
1 int32 compteur = 0;
2 // deux fils executent ceci en meme temps :
3 ++compteur;
```

`++compteur` n'est pas une instruction : c'est *lire, ajouter un, écrire*. Deux fils peuvent lire la même valeur, ajouter un chacun, et écrire le même résultat. Deux incréments, un seul effet.

Le piège

Une course ne se reproduit pas à la demande. Elle dépend de l'ordre exact où le système donne la main, qui change à chaque exécution. Elle disparaît souvent sous le débogueur, parce qu'il ralentit tout et change cet ordre.
C'est pourquoi on ne *teste* pas l'absence de course : on la rend impossible par construction.

10.2 Les fils

Kernel/System/NKThreading/src/NKThreading/NkThread.h

```
1 NkThread t([]() {
2     // ... Le travail, dans un AUTRE fil ...
3 });
4 t.SetName("calcul");           // Le nom apparait dans Le debogueur : prenez-en un
5 t.Join();                     // on ATTEND qu'il finisse
```


Méthode	Effet
Join()	attend la fin du fil
Detach()	le laisse vivre seul — on ne l'attendra plus
SetName(n)	nomme le fil pour le débogueur
SetAffinity(c)	l'épingle sur un cœur
SetPriority(p)	change sa priorité

Le piège

Un fil détaché qui touche des données détruites est un plantage aléatoire. Detach veut dire «je ne l'attendrai jamais» : si l'objet qu'il manipule meurt avant lui, il écrit dans le vide.

Nommez vos fils. Un débogueur qui affiche «Thread 14892» ne dit rien; «calcul» dit tout. C'est une ligne, et elle se rembourse au premier incident.

10.3 Protéger une donnée : mutex et verrou à portée

Kernel/System/NKThreading/src/NKThreading/NkScopedLock.h

```

1 NkMutex mutex;
2 int32 compteur = 0;
3
4 void Incrementer() {
5     NkScopedLock<NkMutex> verrou(mutex);    // prend le verrou ICI
6     ++compteur;                             // un seul fil a la fois
7 }                                           // le rend a l'accolade -- TOUJOURS

```

Ce qu'il faut retenir

NkScopedLock est du **RAII** appliqué au verrouillage : il prend le verrou à sa construction et le rend à sa destruction. On ne peut plus oublier de déverrouiller, même en sortant par un return au milieu.

C'est exactement le mécanisme du chapitre C++, appliqué à une autre ressource.

L'étreinte fatale

Deux fils, deux verrous, pris dans un ordre *opposé* : chacun attend celui que l'autre tient. Le programme ne plante pas — il s'arrête, pour toujours.

La parade est une convention, pas un outil : quand plusieurs verrous sont nécessaires, on les prend **toujours dans le même ordre**, partout dans le programme. Écrivez cet ordre en commentaire à côté de leur déclaration.

Le piège

Un verrou trop large annule le bénéfice des fils. Verrouiller pendant tout un calcul de trois secondes fait attendre les autres fils trois secondes : on a payé la complexité du parallélisme pour obtenir un programme séquentiel.

Verrouillez *le temps de toucher la donnée*, pas le temps de calculer.

10.4 Le reste de la boîte à outils

Outil	Quand
NkMutex	une donnée partagée, un fil à la fois
NkRecursiveMutex	la même fonction peut se rappeler elle-même
NkSharedMutex	plusieurs lecteurs <i>ou</i> un écrivain
NkReaderWriterLock	même idée, autre interface
NkSpinLock	attente très courte, sans passer par le système
NkConditionVariable	« attends qu'une condition devienne vraie »
NkSemaphore	au plus n fils à la fois
NkBarrier	tout le monde s'attend à un point de rendez-vous
NkLatch	attendre que n choses soient faites, une fois
NkThreadPool	un ensemble de fils qui consomment des tâches
NkFuture, NkPromise	un résultat qui arrivera plus tard
NkThreadLocal	une variable propre à chaque fil

Ce qu'il faut retenir

Pour du travail découppable, prenez le NkThreadPool plutôt que de créer des fils vous-même. Créer un fil coûte cher ; le pool les crée une fois et leur donne des tâches (Enqueue). Créer un fil par tâche est le défaut le plus fréquent du débutant : le programme passe son temps à créer et détruire des fils au lieu de calculer.

Le piège

NkSharedMutex n'est un gain que si les lectures dominant LARGEMENT. Il coûte plus cher qu'un mutex simple à chaque prise ; s'il y a autant d'écritures que de lectures, il est plus lent. C'est une optimisation à *mesurer*, pas à supposer.

10.5 Ce que le rendu impose

Le piège

Le contexte graphique appartient au fil qui l'a créé. Dessiner depuis un autre fil ne marche pas — selon le backend, c'est un plantage, un écran noir, ou pire : cela fonctionne sur votre machine et pas sur celle du voisin.

La règle du dépôt : **le calcul se parallélise, le dessin non.** Un fil prépare des données, le fil principal les dessine.

Ce qu'il faut retenir

NKGui range son contexte courant dans une variable **propre à chaque fil** (`thread_local`) plutôt que dans un singleton global. Deux fils peuvent donc avoir chacun leur contexte, sans se gêner — mais chacun ne voit que le sien.

Ce chapitre en bref

Une course vient de ce qu'une opération qui semble unique ne l'est pas, et elle ne se reproduit pas à la demande : on la rend impossible, on ne la teste pas. `NkScopedLock` applique RAI au verrouillage — on ne peut plus oublier de déverrouiller. Plusieurs verrous se prennent **toujours dans le même ordre**, sinon étreinte fatale. Un verrou se tient le temps de toucher la donnée, pas le temps de calculer. Pour du travail découppable : le pool, pas des fils créés à la main. Et le dessin reste sur le fil principal.

Questions de cours

1. Pourquoi `++compteur` n'est-il pas sûr entre deux fils ?
2. Qu'apporte `NkScopedLock` qu'un verrouillage manuel n'apporte pas ?
3. Comment évite-t-on une étreinte fatale ?
4. Pourquoi préférer un `NkThreadPool` à des fils créés à la demande ?
5. Peut-on dessiner depuis un fil secondaire ? Justifiez.

Exercice pratique

Écrivez un programme qui somme un tableau d'un million d'entiers : d'abord sur un fil, puis sur quatre fils qui traitent chacun un quart et dont les résultats partiels sont additionnés à la fin.

Mesurez les deux avec `NkChrono`, et répétez chaque mesure trois fois. Le gain est-il de quatre ? Si non, proposez au moins deux explications — sans conclure entre elles.

Exercice de démonstration

Provoquez une course, puis prouvez qu'elle a eu lieu.

Deux fils incrémentent un même compteur un million de fois chacun, sans verrou. Affichez le total : il doit être inférieur à deux millions. Répétez dix fois et notez les dix valeurs.

Ce que la démonstration doit établir, et c'est le point : non pas que le total est faux, mais qu'il est *différent à chaque exécution*. Ajoutez ensuite le `NkScopedLock` et refaites les dix mesures.

Exercice de logique

Deux fils, deux verrous *A* et *B*. Le premier prend *A* puis *B* ; le second prend *B* puis *A*.

Décrivez l'enchaînement exact qui bloque le programme. Puis démontrez que si les deux prennent toujours *A* avant *B*, le blocage devient *impossible* — pas improbable, impossible. Enfin : avec trois verrous, la même règle suffit-elle ? Justifiez.

Ce que cette partie vous laisse

Vous savez allouer (tout passe par `NKMemory`, et ce qu'il a donné c'est lui qui le reprend — sinon le tas se corrompt et le plantage arrive ailleurs), ranger des données (`NkVector`, `NkString`, `NkSpan` — et qu'une réallocation invalide les pointeurs), calculer (vecteurs suffixés, jamais `==` sur des flottants, transformations en `T + R + S` séparées), tracer (deux familles de messages qu'on ne mélange pas), et paralléliser sans le regretter (RAI appliqué au verrouillage, un ordre de prise unique, et le dessin qui reste sur le fil principal). Tout ce qui suit s'appuie là-dessus sans le réexpliquer.

PARTIE IV

Le moteur

Dessiner, et faire une application

Voici le sujet annoncé : **NkCanvas**, le rendu 2D, et **NKGui**, l'interface. Vous ouvrirez une fenêtre, dessinerez des formes, afficherez du texte, réagirez à la souris — puis vous découvrirez qu'écrire tout cela à la main est justement ce qu'il ne faut pas faire.

Le dernier chapitre présente la **coquille** : **NkCanvasApp** et **NkCanvasGuiApp**, qui tiennent la fenêtre, la boucle, le cycle de vie mobile et la zone sûre à votre place. L'ordre est voulu. On montre d'abord le mécanisme nu, parce qu'une abstraction qu'on adopte sans avoir vu ce qu'elle remplace devient une boîte noire qu'on n'ose pas ouvrir le jour où elle se comporte mal.

C'est la partie la plus longue du livre, et celle dont dépendent toutes les applications de la dernière.

Le décor : la pile complète

Avant d'écrire une seule ligne de code, il faut savoir *qui fait quoi*. Ce chapitre pose la carte du terrain. Il se lit de bas en haut : de la fenêtre que le système d'exploitation nous donne, jusqu'au bouton sur lequel l'utilisateur clique. À la fin, vous saurez nommer chaque étage, dire de quoi il dépend, et — c'est le point le plus important du chapitre — vous comprendrez pourquoi l'étage du haut ne connaît pas l'étage du bas.

11.1 Vue d'ensemble

Vue d'ensemble — synthèse (voir Documentation, section « Raccordement au reste du moteur »)

```

1  Application (NKGuiDemo, NKCode, ...NK3DModeler)
2      |   cree une NkWindow, pompe les NKEvent -> remplit ctx.input
3      |   declare ses widgets entre BeginFrame / EndFrame
4      v
5  NKGui (NkGuiContext -> NkGuiDrawList : vtx + idx + cmds)
6      |   ne connaît AUCUN GPU
7      v
8  Backend (au choix)
9      | - NkGuiCanvasBackend (header-only, dans NKCanvas/UI/) -> NkIRenderer2D
10     | - NkGuiRHIBackend (Integrations/NKGui/) -> NKRHI
11     v
12  NKCanvas (NkBatchRenderer2D -> SubmitBatches) | NKRHI
13     v
14  GPU

```

Cinq étages, donc, mais qui ne s'empilent pas comme on l'imaginerait. Prenons-les un par un.

11.2 Étage 1 — la fenêtre : NkWindow

NkWindow est l'abstraction de fenêtre native. C'est le seul module de la pile qui parle au système d'exploitation : HWND sous Windows, Window X11 ou surface Wayland sous Linux, NSWindow sous macOS.

Son usage se réduit à trois gestes :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:243-251

```

1  NkWindow window;
2  NkWindowConfig cfg;
3  cfg.title = "NKGui - Demo Phase 2 (Coeur : drawlist + interaction)";
4  cfg.width = 1100;
5  cfg.height = 800;
6  cfg.centered = true;
7  cfg.resizable = true;
8  if (!window.Create(cfg))
9      return -1;

```

puis, dans la boucle, `window.IsOpen()` pour savoir si elle vit encore, et `window.SetCursor(...)` pour changer le pointeur. Le reste — taille courante, position, état maximisé — s'interroge par des accesseurs (`GetSize()` renvoie un `math::NkVec2u`).

Une option mérite d'être signalée dès maintenant parce qu'elle explique l'apparence des applications du dépôt :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:234

```
1      wc.frame = false;           // SANS bordure OS -> barre de titre custom (VSCode)
```

Avec `frame = false`, le système ne dessine plus ni barre de titre ni bordure : l'application les dessine elle-même, comme le fait Visual Studio Code. C'est ce que fait `NKCode`. Notre application du chapitre 4 gardera la décoration native, qui est plus simple.

X11 définit None

Sous Linux, `NKWindow` tire `<X11/Xlib.h>`, qui contient `#define None 0L`. Or plusieurs énumérations de `NKGui` ont un membre nommé `None`. Le préprocesseur le transforme alors en `0L = 0` et le compilateur signale « expected identifier » sur des lignes parfaitement correctes, très loin de la vraie cause. La parade est en tête de l'en-tête parapluie de `NKGui` :

Kernel/Runtime/NKGui/src/NKGui/NKGui.h:18-27 (abrégé)

```
1  // Retire Les macros X11 (None, Bool, Status...) AVANT toute declaration de
2  // NKGui. Sur Linux, NKWindow tire <X11/Xlib.h>, dont le `#define None 0L`
3  // transforme ensuite `None = 0` - membre de plusieurs enumerations de NkGuiTypes
4  // - en `0L = 0`. Le compilateur signale alors « expected identifier » sur des
5  // lignes parfaitement correctes, tres loin de la vraie cause.
6  #include "NKPlatform/NkX11Clean.h"
```

La leçon générale : **incluez toujours `NKGui/NKGui.h` et jamais un en-tête interne de `NKGui`**. L'ordre d'inclusion de l'en-tête parapluie n'est pas arbitraire, c'est l'ordre des dépendances.

11.3 Étape 2 — les événements : `NKEvent`

`NKEvent` transforme les messages du système en **événements typés** : `NkWindowCloseEvent`, `NkMouseMoveEvent`, `NkMouseButtonPressEvent`, `NkKeyPressEvent`, `NkTextInputEvent`...

Le modèle est celui des *callbacks* enregistrés une fois pour toutes, puis d'une file qu'on vide à chaque image :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:326-331 (extrait)

```
1      auto &events = NkEvents();
2      bool running = true;
3      events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
4      });
5      events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
6          ctx.input.mousePos = {static_cast<float32>(e->GetX()),
7                                static_cast<float32>(e->GetY())};
8      });
```

et, dans la boucle :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:438-440

```
1     while (NkEvent *ev = NkEvents().PollEvent()) {  
2         (void)ev;  
3     }
```

Ce `while` déroutant mérite un mot : **il ne fait rien du résultat**. Le travail a déjà été accompli par les *callbacks* enregistrés plus haut ; `PollEvent` sert uniquement à *pomper* la file jusqu'à ce qu'elle soit vide, ce qui déclenche au passage l'appel de tous les *callbacks*. On retrouve exactement le même idiome dans la boucle principale de `NKEditorKit` (`Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:314`).

Une distinction fondamentale, qu'on retrouvera au chapitre 4 :

- `NkTextInputEvent` porte le **texte** tapé, déjà décodé par le système en *codepoint* Unicode — claviers AZERTY et méthodes de saisie asiatiques comprises. C'est lui qui alimente les champs de saisie ;
- `NkKeyPressEvent` / `NkKeyReleaseEvent` portent les **touches** : flèches, Suppr, Entrée, F2, Ctrl+quelque chose. Ce sont elles qui alimentent la navigation.

Confondre les deux est un classique : on branche les flèches sur le texte, et plus rien ne fonctionne. Le commentaire du pont de `NKEditorKit` le dit :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:367-370

```
1 // Mappe une touche OS d'edition vers l'etat ENFONCE NKGui (press/release ->  
2 // repetition au maintien geree par NKGui). Sans ce pont, aucune navigation  
3 // clavier ni edition de texte dans les panneaux.
```

11.4 Étage 3 — le GPU. Attention, il y a deux abstractions

C'est ici que le débutant se perd, et c'est normal : le moteur contient **deux** abstractions GPU distinctes, qui ne se superposent pas.

11.4.1 NKRHI — l'abstraction GPU bas niveau

`Kernel/Runtime/NKRHI` est l'abstraction « moderne » du GPU : périphérique (`NkIDevice`), tampons de commandes, passes de rendu, *pipelines*, descripteurs. Son arborescence de backends parle d'elle-même : `DirectX11`, `DirectX12`, `Metal`, `OpenGL`, `Software`, `Vulkan`. C'est sur elle que repose le rendu 3D (`NKRenderer`) et, par ricochet, les applications qui font de la 3D.

11.4.2 NKCanvas — sa propre abstraction, pour la 2D

`NKCanvas` ne passe pas par `NKRHI`. Il possède ses propres contextes graphiques et ses propres renderers 2D. La preuve tient en une ligne de son fichier de build :

Kernel/Runtime/NKCanvas/NKCanvas.jenga:65

```
1     _canvasDeps = [ "NKWindow", "NKFont", "NKImage", "NKStream", "NKTime", "NKGlad",  
                    "NKThreading" ]
```


Pas de NKRHI dans la liste. L'arborescence `Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/` contient DirectX/ (DX11 et DX12), Metal/, OpenGL/, Software/ et Vulkan/ : ce sont les implémentations *de NKCanvas*, pas celles de NKRHI.

Il y a donc, dans le dépôt, deux familles de backends portant les mêmes noms d'API graphique et ne se connaissant pas. Cette duplication est assumée et même documentée à l'endroit où elle pose problème :

`Engine/NKEditorKit/src/NKEditorKit/NkIEditorRenderer.h:28-31`

```
1 // Choix d'API graphique NEUTRE (decouple de NKCANVAS ET NKRHI : Leurs enums
2 // NkGraphicsApi se dupliquent dans Le namespace nkentseu et ne peuvent
3 // cohabiter dans un meme TU). Chaque impl mappe vers son propre enum.
4 enum class NkEditorGfxApi : uint8 { Auto = 0, OpenGL, Vulkan, DX11, DX12, Software };
```

Lisez bien : les deux énumérations `NkGraphicsApi` — celle de `NKCanvas` et celle de `NKRHI` — *ne peuvent pas cohabiter dans une même unité de traduction*. D'où le besoin d'un troisième énuméré, neutre, quand une couche doit pouvoir parler aux deux.

Une fenêtre = une pile

C'est la doctrine du dépôt, énoncée mot pour mot :

`Integrations/NKGui/NkEditorRHIRenderer.h:8-13 (extrait)`

```
1 * toute application moteur (NkAnimaEditor, Noguee, ...) qui veut la coquille
2 * NkEditorShell rendue sur NKRHI/NKRenderer (et PAS NKCanvas – regle
3 * « une fenetre = une pile ») injecte cette classe via
4 * NkEditorShellConfig::renderer.
```

Une fenêtre est rendue **soit** par `NKCanvas`, **soit** par `NKRHI` — jamais les deux. Le choix se fait une fois, à la création. Pour ce cours, ce sera toujours `NKCanvas`.

11.4.3 Les cinq backends 2D de NKCanvas, et leur état réel

Les cinq dossiers de backends ne fournissent pas tous un renderer 2D, et ceux qui le fournissent ne sont pas tous validés à l'exécution. Voici l'état, tel qu'il ressort des sources et des feuilles de route :

Backend	Renderer 2D	État à l'exécution
OpenGL	<code>NkOpenGLRenderer2D</code> (926 l.)	le seul validé
Vulkan	<code>NkVulkanRenderer2D</code> (1524 l.)	le code le plus complet; plantages signalés
DX12	<code>NkDX12Renderer2D</code> (1320 l.)	plantages signalés
DX11	<code>NkDX11Renderer2D</code> (764 l.)	écran vide/noir signalé
Software	<code>NkSoftwareRenderer2D</code> (674 l.)	écran vide/noir signalé
Metal	aucun	non implémenté

La ligne « Metal » n'est pas une supposition, elle est écrite dans la fabrique :

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DFactory.cpp:86-89`

```
1 case NkGraphicsApi::NK_GFX_API_METAL:
2     NK_R2D_FACTORY_ERR("Metal 2D renderer not yet implemented");
```

```
3     return nullptr;
```

Quant aux autres verdicts, ils viennent de [Kernel/Runtime/NKCanvas/ROADMAP.md](#) (lignes 185-200 et 601-607), à la date du 30 mai 2026 : seul OpenGL était alors validé à l'exécution sur la démo Pong. **Ces états sont datés et méritent d'être revérifiés** avant de tirer une conclusion définitive : le dépôt bouge. Mais ils expliquent pourquoi, quand une application « ne montre rien », la première chose à essayer est de changer de backend.

Le choix, lui, est explicite. Soit on nomme l'API :

[Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:253-262](#)

```
1     NkContextDesc desc;
2     desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
3     if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
4     #if defined(NKENTSEU_PLATFORM_WINDOWS)
5         desc.api = NkGraphicsApi::NK_GFX_API_DX11;
6     #else
7         desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
8     #endif
9     }
```

soit on laisse la fabrique essayer une liste, par `NkContextFactory::CreateWithFallback(window, preferenceOrder, count)`. L'ordre conseillé documenté en [.../Factory/NkContextFactory.h:46](#) est {DX12, DX11, Metal, Vulkan, OpenGL, Software}.

11.5 Étage 4 — NkCanvas, le rendu 2D

NkCanvas est décrit dans [Guides/05-NKCanvas.md](#) comme « la couche de rendu 2D conviviale de Nkntseu, l'équivalent de la partie *Graphics* de SFML ». Il fournit :

- un **contexte graphique** (`NkIGraphicsContext`) créé par une fabrique selon l'API choisie;
- une **cible de rendu** (`NkRenderTarget`), dont la spécialisation `NkRenderWindow` est celle qu'on utilise 99 % du temps : elle possède le cycle d'image;
- un **render 2D** (`NkIRenderer2D`, façade `NkRenderer2D`) qui offre les primitives : lignes, rectangles, cercles, triangles, et le point d'entrée bas niveau `DrawVertices`;
- des **ressources** : `NkTexture`, `NkFont`, `NkSprite`, `NkText`, `NkShader`;
- des **formes persistantes** façon SFML : `NkRectangleShape`, `NkCircleShape`, `NkLineShape`, `NkConvexShape`.

Le chapitre 2 lui est entièrement consacré. Retenez pour l'instant une seule chose : NkCanvas dessine des **triangles**. Tout le reste — cercles, lignes épaisses, texte — est fabriqué à partir de triangles par le module lui-même.

11.6 Étage 5 — NKGui, les widgets

NKGui est le framework d'interface : boutons, cases à cocher, sliders, champs de saisie, arbres, tables, menus, fenêtres flottantes, docking. Environ 7 780 lignes réparties sur 13 fichiers source, dont [Kernel/Runtime/NKGui/src/NKGui/Widgets/NKGuiWidgets.cpp](#) qui pèse à lui seul 4 973 lignes.

Son arborescence est petite et lisible — c'est par là qu'il faut commencer :

Kernel/Runtime/NKGui/ — arborescence

```
1 Kernel/Runtime/NKGui/
2   NKGui.jenga · ARCHITECTURE.md · README.md · ROADMAP.md
3   src/NKGui/
4     NKGui.h                <- en-tete PARAPLUIE (la seule chose a inclure)
5     NKGuiExport.h -> NKGuiApi.h <- macros d'export (defaut : statique)
6     Core/
7       NKGuiTypes.h        (333 l.) types de base : NKGuiId, enums, themes de flags
8       NKGuiInput.h        (145 l.) etat d'entree par frame (header-only, tout inline)
9       NKGuiDrawList.h/.cpp (101/342 l.) la LISTE DE COMMANDES de dessin
10      NKGuiFont.h/.cpp    ( 81/158 l.) police + atlas (wrapper NKFont)
11      NKGuiContext.h/.cpp (551/573 l.) LE contexte : etat complet de l'UI
12      Widgets/
13      NKGuiWidgets.h/.cpp (405/4973 l.) TOUS les widgets
```

11.6.1 Le point capital : NKGui ne connaît aucun GPU

Voici la ligne la plus importante de ce chapitre :

Kernel/Runtime/NKGui/NKGui.jenga:22-27

```
1 nkentseudependson(
2   ["NKPlatform", "NKCore", "NKMemory", "NKMath", "NKThreading",
3    "NKLogger", "NKContainers", "NKEvent", "NKFont", "NKImage"],
4   selfexport="NKGui",
5   extra_includes=["src"],
6 )
7 files(["src/**/*.cpp"])
```

Relisez la liste. Il n'y a ni **NKCanvas**, ni **NKRHI**, ni même **NKWindow**. NKGui ne connaît que les mathématiques, les conteneurs, la mémoire, les polices, les images et les événements. Il ne sait pas ce qu'est une texture GPU, ni un *shader*, ni une fenêtre.

Ce qu'il faut retenir

NKGui ne dépend ni de NKCanvas ni de NKRHI. Il ne dessine rien : il *produit une liste de commandes* — des sommets, des indices, des commandes de dessin — et laisse quelqu'un d'autre les envoyer au GPU. Ce quelqu'un s'appelle un **backend**, et le mariage se fait par un **pont**.

11.6.2 Ce que NKGui produit : NKGuiDrawList

La sortie de NKGui est purement géométrique :

Kernel/Runtime/NKGui/src/NKGui/Core/NKGuiDrawList.h:23-46 (abrégé)

```
1 struct NKGuiVertex {
2     NKVec2 pos;
3     NKVec2 uv;    ///< (0,0) = couleur unie (pas de texture)
4     uint32 col;
5 };
6
7 enum class NKGuiDrawCmdType : uint8 {
8     Triangles,          ///< triangles unis
9     TexturedTriangles   ///< triangles textures (texte/atlas/image)
10 };
```

```

11
12 struct NkGuiDrawCmd {
13     NkGuiDrawCmdType type = NkGuiDrawCmdType::Triangles;
14     uint32 idxOffset = 0;
15     uint32 idxCount = 0;
16     uint32 texId = 0;
17     NkRect clipRect = {0.f, 0.f, 1.0e9f, 1.0e9f}; // 1e9 = « pas de clip »
18 };

```

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.h:48-56 (abrégé)

```

1 struct NkGuiDrawList {
2     NkVector<NkGuiVertex> vtx;
3     NkVector<uint32> idx;
4     NkVector<NkGuiDrawCmd> cmds;
5     NkRect clipStack[32] = {};
6     int32 clipDepth = 0;
7     float32 thickScale = 1.f; ///< echelle DPI des epaisseurs (preservee par Reset)
8 };

```

Trois tableaux, donc : les sommets, les indices, et les commandes. Une *commande* décrit une tranche d'indices à dessiner avec une texture donnée et un rectangle de découpe donné. Notez le `texId` : c'est un simple entier, pas un pointeur vers une texture GPU. NKGui manipule des *identifiants*, à charge du backend de savoir à quoi ils correspondent.

Notez aussi la sentinelle `1e9` pour « pas de découpe ». Le backend la reconnaît par un test tolérant (`w < 1e8`), ce qui évite toute comparaison exacte de flottants.

11.7 Le pont : marier NKGui et NkCanvas

Puisque NKGui ne connaît pas NkCanvas, et que NkCanvas ne connaît pas NKGui, il faut un tiers qui connaisse les deux. Le dépôt en fournit deux, selon le public.

11.7.1 NkGuiCanvasBackend — le pont vers NkCanvas

Il vit dans `Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h` et il est **header-only**. Ce détail est délibéré et sa justification est en tête de fichier :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:2-9

```

1 // NkGuiCanvasBackend.h — rend un nkgui::NkGuiDrawList via NKCanvas (NkIRenderer2D).
2 // Backend RÉUTILISABLE (Lib) : Le cœur NKGui reste render-agnostique ; ce pont
3 // traduit ses draw-lists en appels NkIRenderer2D. Gère l'atlas de police
4 // (gray8 -> RGBA blanc+alpha) et les images RGBA pour les commandes texturées.
5 //
6 // HEADER-ONLY : NKCanvas ne le compile pas (pas de .cpp) ; seuls les consommateurs
7 // (qui dépendent déjà de NKGui ET NKCanvas) l'incluent. Modelé sur NkUICanvasBackend.

```

Comprenez bien l'astuce. Ce fichier inclut `"NKGui/NKGui.h"`, donc il dépend de NKGui. Mais il vit dans NKCanvas, dont le `files([...])` ne prend que les `.cpp` : **personne ne le compile dans NKCanvas**. Il n'est compilé que dans l'application qui l'inclut — application qui, elle, dépend bien des deux modules. Résultat : NKCanvas n'acquiert aucune dépendance vers NKGui, et le pont existe quand même.

Son API tient en quatre méthodes :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:34, 41, 94, 120

```
1 bool Init(nkentseu::renderer::NkIRenderer2D *renderer);
2 bool UploadFontGray8(uint32 texId, const uint8 *gray, int32 w, int32 h);
3 bool UploadImageRGBA(uint32 texId, const uint8 *rgba, int32 w, int32 h);
4 void Submit(const nkentseu::nkgui::NkGuiDrawList &dl, uint32 fbW, uint32 fbH);
```

C'est tout. Init lui donne le renderer 2D; les deux Upload enregistrent une texture sous un identifiant (un atlas de police, une image); Submit traduit une liste de commandes en appels NkIRenderer2D. Le chapitre 4 montrera la séquence complète.

11.7.2 NkGuiRHIBackend — le pont vers NKRHI

L'alternative vit dans `Integrations/NKGui/NkGuiRHIBackend.h`, compilée dans une bibliothèque statique NkGuiIntegration. Son API est le miroir de la précédente, avec un tampon de commandes en plus :

Integrations/NKGui/NkGuiRHIBackend.h:45-59

```
1 bool Init(NkIDevice* device, NkRenderPassHandle renderPass, NkGraphicsApi api);
2 void Destroy();
3 void Submit(NkICommandBuffer* cmd, const NkGuiDrawList& dl, uint32 fbW, uint32 fbH);
4 bool UploadTextureRGBA8(uint32 texId, const uint8* data, int32 width, int32 height);
5 bool UploadTextureGray8(uint32 texId, const uint8* data, int32 width, int32 height);
6 bool RegisterTexture(uint32 texId, NkTextureHandle texture);
7 bool HasTexture(uint32 texId) const noexcept;
```

La doctrine de répartition est écrite dans le fichier de build de l'intégration :

Integrations/NKGui/NKGuiIntegration.jenga:5-8

```
1 Rend une UI NKGui (nkgui::NkGuiDrawList) via NKRHI/NKRenderer, sans NKCanvas.
2 Sert aux applications 2D/3D (app d'animation, moteur de jeu). NKCanvas reste
3 le backend de l'IDE (NKCode).
```

Un point de conception intéressant de ce second pont : il enregistre aussi des textures *externes* (RegisterTexture). C'est ainsi qu'une application 3D affiche le rendu de sa scène dans un panneau de son interface : elle rend la scène dans une texture hors écran, l'enregistre sous un texId, et NKGui la dessine comme une image ordinaire.

11.7.3 Pourquoi cette séparation ?

On pourrait trouver la construction alambiquée. Elle rapporte trois choses concrètes :

1. **On peut changer de moteur de rendu sans toucher un seul widget.** L'IDE utilise NkCanvas, l'application d'animation utilise NKRHI, et le code des boutons est le même. Mieux : NKEditorKit rend cette substitution explicite, avec une interface NkIEditorRenderer injectable (`Engine/NKEditorKit/src/NKEditorKit/NkIEditorRenderer.h:7-15`).
2. **NKGui est testable sans GPU.** Une liste de commandes est une structure de données : on peut la produire, l'inspecter et la comparer sans jamais ouvrir de fenêtre.
3. **Les dépendances restent en arbre, pas en graphe.** NKGui ne connaît pas NkCanvas; NkCanvas ne connaît pas NKGui; l'application connaît les deux. Aucun cycle.

Il y a un prix, et il faut le connaître : **c'est à l'application de faire le raccordement**. Personne ne le fera à votre place. Si vous oubliez d'appeler `Submit`, rien ne s'affiche et aucun message d'erreur n'apparaît. Si vous oubliez d'uploader l'atlas de police, tous les textes sont invisibles et aucun message d'erreur n'apparaît non plus. Le chapitre 4 revient longuement sur ces silences.

11.8 Le chemin complet d'un clic, de bout en bout

Récapitulons en suivant un seul clic de souris, de l'appui jusqu'au pixel qui change de couleur.

1. L'utilisateur appuie. Le système envoie un message ; `NKWindow` le reçoit, `NKEvent` le transforme en `NkMouseButtonPressEvent`.
2. La boucle appelle `PollEvent()` ; le *callback* enregistré écrit `ctx.input.mouseDown[0] = true`.
3. L'application appelle `ctx.BeginFrame(dt)`. `NKGui` en déduit la *transition* : `mouseClicked[0] = mouseDown[0] && !mousePrev[0]`.
4. L'application appelle `Button(ctx, "OK")`. Le widget calcule son rectangle, demande son identité, teste le survol, voit le clic, et retourne `true` — au *relâchement*, comme nous le verrons au chapitre 3. Au passage, il écrit deux rectangles et six sommets dans la liste de commandes.
5. L'application appelle `ctx.EndFrame()`. `NKGui` fusionne les listes des fenêtres par ordre de profondeur, ferme les popups, remet à zéro la molette et le texte consommés.
6. L'application appelle `target->Clear()`, puis soumet les deux listes au pont : `ctx.dl` d'abord, `ctx.dlOverlay` ensuite, chacune par `backend.Submit(liste, w, h)`. Le pont convertit chaque sommet, résout chaque `texId` en `NkTexture*`, applique chaque rectangle de découpe et appelle `DrawVertices`.
7. `NkCanvas` accumule tout dans ses tampons, regroupe par texture, et à la fermeture de l'image envoie un appel de dessin par groupe.
8. L'application appelle `target->Display()` : l'image est présentée.

L'invariant d'ordre

événements → `ctx.BeginFrame(dt)` → widgets → `ctx.EndFrame()` → `Clear` → `Submit` ×2 → `Display`.

Cet ordre n'est pas négociable et le chapitre 3 expliquera pourquoi `BeginFrame` doit venir *après* le pompage des événements.

11.9 Deux mots sur les macros d'export

En lisant les en-têtes, vous rencontrerez partout des macros du genre `NKENTSEU_NKGUI_API` :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h (forme générale)

```
1 NKENTSEU_NKGUI_API void Text(NkGuiContext &ctx, const char *s) noexcept;
```

Par défaut, le module est construit en **statique** et cette macro est **vide** (`Kernel/Runtime/NKGui/src/NKGui/NkGuiApi.h:32-40`). Quand vous lisez une signature, effacez-la mentalement : il faut lire `void Text(NkGuiContext &ctx, const char *s) noexcept`. Il existe deux variantes : `NKENTSEU_NKGUI_CLASS_EXPORT` (classe entière) et `NKENTSEU_NKGUI_API_INLINE` (fonction inline exportée). Dans la suite de ce cours, nous les omettons pour la lisibilité.

Ce chapitre en bref

Cinq étages, et une règle : chacun ignore celui du dessus. **NKWindow** donne une fenêtre, **NKEvent** des événements typés, **NKCanvas** un GPU abstrait — distinct de NKRHI, et les deux ne se mélangent pas dans une même fenêtre. **NkCanvas** dessine, **NKGui** produit une liste de commandes et *ne connaît aucun GPU* : c'est un pont qui traduit sa `NkGuiDrawList`. Cette ignorance est ce qui permet à la même interface de sortir sur `NkCanvas` ou sur `NKRHI` sans changer une ligne.

11.10 Exercices

1 — Vérifier la carte soi-même

Ouvrez les trois fichiers de build suivants et relevez, pour chacun, la liste des modules dont il dépend : `Kernel/Runtime/NKGui/NKGui.jenga`, `Kernel/Runtime/NKCanvas/NKCanvas.jenga`, `Engine/NKEditorKit/NKEditorKit.jenga`. Dessinez le graphe. Vérifiez qu'il n'y a aucun cycle, et identifiez le seul module qui connaît à la fois `NKGui` et `NKCanvas`.

2 — Trouver le pont

Ouvrez `Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h` et comptez : combien de méthodes publiques ? Combien de lignes au total ? Comparez avec les 4 973 lignes de `NkGuiWidgets.cpp`. Que vous dit ce rapport sur la quantité de travail nécessaire pour porter `NKGui` vers un nouveau moteur de rendu ?

3 — Changer de backend à chaud

Reprenez la démo `NkCanvasDemo` du dossier `Sandbox`, qui accepte un argument de ligne de commande :

```
NkCanvasDemo.exe --backend=opengl
NkCanvasDemo.exe --backend=dx11
NkCanvasDemo.exe --backend=sw
```

Lancez-la avec chacun des backends et notez ce que vous observez. Recoupez avec le tableau d'état de la section 1.4 et avec `logs/app.log`. Ce petit exercice vous évitera plus tard de chercher un bug dans votre code alors que le problème est dans le choix du backend.

Questions de cours

1. Citez les cinq étages de la pile et ce que chacun apporte.
2. Pourquoi `NKCanvas` et `NKRenderer` sont-ils exclusifs dans une même fenêtre ?
3. Que produit `NKGui`, et qu'est-ce qu'il ne fait *pas* ?
4. À quoi sert un « pont » comme `NkGuiCanvasBackend` ?
5. Décrivez le trajet d'un clic, du système jusqu'au widget.

NkCanvas : dessiner en deux dimensions

Nous descendons maintenant d'un étage pour examiner NkCanvas en profondeur. C'est le module qui, en dernier ressort, fait apparaître des pixels. Tout ce que fera NKGui au chapitre suivant finira ici.

Le plan de ce chapitre : d'abord *quelle est la forme de l'API* (deux niveaux, une cible de rendu, un repère); ensuite *ce qu'on peut dessiner* et surtout *ce qu'on ne peut pas*; puis *comment ça arrive au GPU* — c'est-à-dire le *batching*, qui est le vrai sujet du module; enfin un programme complet qui tourne.

12.1 Ce que NkCanvas est, en une phrase

Guides/05-NkCanvas.md:11-19 (extrait)

```
1 **NkCanvas** est la couche de **rendu 2D conviviale** de Nkntseu. C'est
2 l'équivalent de la partie *Graphics* de **SFML** : ... multi-backend ...
```

La comparaison avec SFML est juste et vous fera gagner du temps si vous connaissez cette bibliothèque : on retrouve la cible de rendu qui ouvre et ferme l'image, les formes persistantes qu'on positionne et qu'on dessine, le *drawable*, la vue. Ce qui change : le multi-backend, c'est-à-dire cinq API graphiques possibles derrière une seule et même interface.

Environ 23 400 lignes réparties sur 101 fichiers, organisées ainsi :

Dossier	Contenu
Core/	NkIGraphicsContext, NkContextDesc, NkGraphicsApi
Factory/	NkContextFactory — choisit et crée le contexte GPU
Backend/	OpenGL/, Vulkan/, DirectX/, Metal/, Software/
Renderer/Core/	NkIRenderer2D, NkRenderer2D, NkVertexArray, NkTransform
Renderer/Batch/	NkBatchRenderer2D — <i>le batcher</i> , base commune
Renderer/Resources/	NkTexture, NkFont, NkSprite/NkText, NkShader
Renderer/Shapes/	NkRectangleShape, NkCircleShape, NkLineShape...
Renderer/Targets/	NkRenderTarget, NkRenderWindow, NkRenderTexture
UI/	NkGuiCanvasBackend.h — le pont vu au chapitre 1
Compute/	NkIComputeContext — GPGPU, hors sujet ici

Tous les types vivent dans l'espace de noms nkntseu::renderer.

12.2 Les deux niveaux d'API

12.2.1 NkIRenderer2D : l'interface

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkIRenderer2D.h` déclare l'interface de rendu 2D. Il y a une implémentation par API graphique, mais elles héritent toutes d'une classe intermédiaire, `NkBatchRenderer2D`, qui fait l'essentiel du travail (section 2.9).

12.2.2 NkRenderer2D : la façade

`.../Renderer/Core/NkRenderer2D.h` est la classe concrète que manipule le code applicatif. Tout est transféré *inline* vers l'interface. Elle n'ajoute presque rien — sauf une surcharge de commodité — et son intérêt est ailleurs : elle documente la propriété.

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2D.h:31-35`

```
1  /// PROPRIETE
2  ///   NkRenderer2D ne POSSEDE pas son backend NkIRenderer2D* — Le NkRenderTarget
3  ///   qui le contient est propriétaire (typiquement via memory::NkUniquePtr).
4  ///   Le NkRenderer2D est juste une projection. La durée de vie suit celle du
5  ///   NkRenderTarget englobant.
```

avec, en clair, dans les membres privés (lignes 314-315) : `NkIRenderer2D *mBackend{nullptr};`
`///< non-owning et NkRenderTarget *mTarget{nullptr}; ///< non-owning.`

La façade meurt avec sa cible

Un `NkRenderer2D` récupéré par `target.GetRenderer2D()` n'est qu'une vue sur le renderer possédé par la cible. Le conserver au-delà de la durée de vie de la `NkRenderWindow` — dans un membre de classe, par exemple — donne un objet qui pointe vers de la mémoire libérée. C'est le même raisonnement, et le même risque, que pour `ctx.font` au chapitre 4.

12.3 La cible de rendu : NkRenderWindow

En pratique, on ne manipule ni le contexte graphique ni le renderer directement : on crée une `NkRenderWindow`, qui fait les deux et possède le cycle d'image.

`Applications/Sandbox/src/DemoNkentseu/NKCanvas/NKCanvasDemo.cpp:69-77`

```
1  // — 2. Cible de rendu NKCanvas (la voie haut-niveau, comme Pong) —————
2  NkContextDesc desc;
3  desc.api = ParseBackend(state.args);
4  NkRenderWindow target(window, desc);
5  if (!target.IsValid()) {
6      logger.Error("[nkcanvas] NkRenderWindow init FAILED");
7      window.Close();
8      return -2;
9  }
```

Une ligne de commentaire, en tête de cette même démo, résume la doctrine d'usage — il faut la retenir :

`Applications/Sandbox/src/DemoNkentseu/NKCanvas/NKCanvasDemo.cpp:6-7`

```
1  // NkRenderWindow possède le cycle de frame -> Clear() ouvre+efface, on dessine,
2  // Display() termine+présente (pas de Begin/End/SetView/SetViewport à la main).
```

Voici ce que ces deux méthodes font réellement :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Targets/NkRenderWindow.cpp:145-171 (abrégé)

```
1 void NkRenderWindow::Clear(const NkColor2D &color) {
2     if (!mRenderer) return;
3     // Pousser la couleur de clear au contexte AVANT Begin() : sur Vulkan/
4     // Software, BeginFrame() ouvre le render pass / clear le back-buffer avec
5     // cette couleur (sinon ils utilisent un gris en dur -> fond incorrect).
6     // No-op sur OpenGL/DX (ils clearent via le renderer->Clear ci-dessous).
7     if (mContext)
8         mContext->SetClearColor(color.r / 255.f, color.g / 255.f,
9                                   color.b / 255.f, color.a / 255.f);
10    // Begin() ouvre la frame si pas déjà ouverte (idempotent par convention
11    // du backend) ; Clear() est appellable en milieu de frame.
12    if (!mFrameOpen) { mRenderer->Begin(); mFrameOpen = true; }
13    mRenderer->Clear(color);
14 }
15
16 void NkRenderWindow::Display() {
17     if (mRenderer && mFrameOpen) { mRenderer->End(); mFrameOpen = false; }
18     if (mContext) mContext->Present();
19 }
```

Autrement dit : Clear ouvre l'image (si elle ne l'est pas déjà) et l'efface; Display la ferme — ce qui vide les tampons accumulés vers le GPU — puis la présente à l'écran. Vous n'appellerez jamais Begin() ni End() vous-même.

Ne pas ré-initialiser le renderer

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Targets/NkRenderWindow.cpp:95-98

```
1 // NB : NE PAS appeler mRenderer->Initialize(mContext) ici !
2 // NkRenderer2DFactory::Create l'a déjà fait (cf. NkRenderer2DFactory.cpp:83).
3 // Double-init -> NkOpenGLRenderer2D::Initialize Log "Already initialized"
4 // ERR et corrompt l'état (rectangles creux observés en runtime).
```

Symptôme mémorable : des « rectangles creux » — les formes apparaissent en contour, vides. Si vous voyez ça, cherchez une double initialisation ou une matrice dégénérée (section 2.8).

12.3.1 Le redimensionnement

OnResize(w, h) recrée la chaîne d'échange (*swapchain*) et recalcule la vue par défaut. Deux subtilités :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Targets/NkRenderWindow.cpp:353-358 (extrait)

```
1 // Si une frame est en cours, on la termine avant la recreation
2 // (sinon submit sur swapchain détruite -> UB sur Vulkan/DX12).
3 if (mFrameOpen && mRenderer) { mRenderer->End(); mFrameOpen = false; }
```

et, du côté du batcher, OnResize recalcule la vue *par défaut* mais **préserve une vue personnalisée** posée par SetView (*../Renderer/Batch/NkBatchRenderer2D.cpp:169-196*).

Détecter le resize sur la fenêtre, pas sur la swapchain

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:444-448

```
1 // Synchroniser la SWAPCHAIN à la taille de la FENÊTRE. target->GetSize()
2 // renvoie la taille de la swapchain (qui ne change QU'À OnResize) – s'en
3 // servir pour détecter le resize était faux : la swapchain restait figée à
4 // sa taille de création (rendu basse-résolution étiré). On pilote OnResize
5 // depuis la taille fenêtre (inclut la 1re frame → sync initiale).
```

Le code correct compare `target->GetWindow().GetSize()` — la fenêtre — à la dernière taille connue, et appelle `OnResize` si elle a changé. Se servir de `target->GetSize()` pour détecter le changement est une boucle logique : cette valeur ne change qu'après `OnResize`. Le symptôme est très reconnaissable : l'application semble marcher, mais tout est flou et étiré dès qu'on agrandit la fenêtre.

12.4 Le repère : Y vers le bas

L'origine est en haut à gauche et l'axe Y descend. C'est la convention de toutes les interfaces graphiques, et elle est confirmée dans le batcher :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:183 (commentaire)

```
1 // Vue par défaut = écran plein-cadre (origine haut-gauche, Y-down)
```

La projection orthographique correspondante est construite ici :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.cpp:85-87

```
1 const float32 a = 2.f / size.x;
2 const float32 b = -2.f / size.y; // Y-down (positive Y goes down in screen space)
```

Le signe moins sur `b` est toute l'histoire : c'est lui qui retourne l'axe.

Quelques lignes plus haut, une note sur la convention mémoire des matrices, qui est un piège classique dès qu'on vise plusieurs API :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.cpp:81-84

```
1 // Column-major output (compatible with glUniformMatrix4fv with transpose=GL_FALSE,
2 // and with HLSL cbuffer when uploaded as-is since HLSL is row-major – we compensate
3 // by transposing at upload in the DX backends)
```

12.4.1 La caméra 2D

`NkView2D` (`.../Renderer/Core/NkRenderer2DTypes.h:41-47`) porte un `center`, une `size` et une `rotation`. On la pose par `SetView`. Deux conversions utiles existent pour passer du monde à l'écran et réciproquement : `MapPixelToCoords(NkVec2i)` et `MapCoordsToPixel(NkVec2f)` (`.../Renderer/Batch/NkBatchRenderer2D.cpp:538-550`).

Tant que vous dessinez une interface, vous n'y toucherez pas : la vue par défaut est exactement l'écran en pixels, ce qui est ce qu'on veut.

12.5 Les primitives disponibles

Toutes sont déclarées dans `NkIRenderer2D.h` et **toutes sont implémentées** dans `NkBatchRenderer2D`, donc disponibles sur les cinq backends actifs.

Signature (abrégée)	Ligne	Réalisation
<code>Draw(const NkSprite &)</code>	135	quad texturé
<code>Draw(const NkText &)</code>	138	délègue à <code>NkText::Draw</code>
<code>DrawPoint(pos, color, size)</code>	141	quad <code>size×size</code>
<code>DrawLine(a, b, color, thickness)</code>	143	quad orienté
<code>DrawRect(rect, color, outline, outlineColor)</code>	146	rempli + contour
<code>DrawFilledRect(rect, color)</code>	149	un quad
<code>DrawCircle(center, radius, color, segments, ...)</code>	151	polygone
<code>DrawFilledCircle(center, radius, color, segments)</code>	155	éventail de triangles
<code>DrawTriangle(a, b, c, ...)</code>	158	contour
<code>DrawFilledTriangle(a, b, c, color)</code>	161	un triangle
<code>DrawVertices(verts, n, indices, m, texture)</code>	192	le point d'entrée bas niveau

La dernière ligne est celle qui compte : `DrawVertices` est la primitive *universelle*, celle qu'utilise le pont `NKGui`, et à travers laquelle transite absolument toute l'interface d'une application.

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkIRenderer2D.h:192-193

```
1 virtual void DrawVertices(const NkVertex2D *vertices, uint32 vertexCount,
2                           const uint32 *indices, uint32 indexCount,
3                           const NkTexture *texture = nullptr) = 0;
```

12.5.1 Le sommet

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.h:73-77

```
1 struct NkVertex2D { float32 x, y; float32 u, v; uint8 r, g, b, a; };
```

Vingt octets : deux flottants de position, deux de coordonnées de texture, quatre octets de couleur. C'est délibérément compact — on en pousse des dizaines de milliers par image.

12.5.2 Les primitives composites, réalisées dans l'en-tête

Trois fonctions sont implémentées *par défaut* directement dans `NkIRenderer2D.h`, donc sans que le moindre backend ait à s'en occuper :

- `DrawRectOutline(rect, color, thickness)` — quatre `DrawFilledRect` (lignes 167-175);
- `DrawCircleOutline(center, radius, color, thickness, segments)` — un anneau en segments de `DrawLine` (lignes 178-189);
- `DrawTexturedRect(rect, texture, color, uv)` — construit un quad `NkVertex2D[4]` et appelle `DrawVertices` (lignes 199-233).

C'est un patron de conception qu'on retrouvera : **on n'ajoute une méthode virtuelle que si elle a besoin du backend**. Tout ce qui se compose à partir des primitives existantes s'écrit une fois, dans l'interface.

12.5.3 Les types de primitives, et l'absence de QUADS

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.h:60-70

```
1 // Pas de QUADS — Les quads sont decomposes en 2 triangles par Le batcher
2 // (compatible cross-API : Vulkan/DX/Metal n'ont pas de QUADS natif)
3 enum class NkPrimitiveType : uint8 {
4     NK_POINTS = 0, NK_LINES = 1, NK_LINE_STRIP = 2,
5     NK_TRIANGLES = 3, NK_TRIANGLE_STRIP = 4, NK_TRIANGLE_FAN = 5,
6 };
```

Le commentaire est une bonne illustration de ce que « multi-backend » veut dire en pratique : l'API commune ne peut offrir que l'intersection de ce que les API sous-jacentes savent faire. OpenGL avait des quads; Vulkan, Direct3D et Metal n'en ont pas. Donc l'abstraction n'en a pas non plus, et le batcher découpe.

12.6 Ce que NkCanvas n'a PAS

Cette section est courte mais elle vous fera gagner des heures. Voici ce que vous chercherez en vain :

- **Pas de DrawText.** Il n'existe aucune méthode qui prenne une chaîne et l'affiche. Le texte passe obligatoirement par le *drawable* NkText — Draw(const NkText &) — ou par le patron SFML target.Draw(drawable, states).
- **Pas de DrawImage.** Même chose : on passe par NkSprite, ou par DrawTexturedRect, ou par DrawVertices avec une texture.
- **Pas de dégradé.** Aucune primitive DrawGradient*. Le seul moyen d'obtenir un dégradé est de donner des couleurs différentes aux sommets d'un même triangle et de laisser le GPU interpoler — donc via DrawVertices ou NkVertexArray.
- **Pas de coins arrondis.** Aucun DrawRoundedRect.

Ce qu'il faut retenir

Ces quatre absences ne sont pas des oublis : ce sont des choix de niveau. NkCanvas dessine des *formes géométriques*. Le dégradé, les coins arrondis et le texte mis en page sont des affaires de *présentation*, et c'est NKGui qui les fabrique — en triangulant lui-même des arcs de coin, en donnant quatre couleurs à un quad, en émettant un quad par glyphe. Nous verrons tout cela au chapitre 3.

12.7 Les couleurs

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.h:12

```
1 using NkColor2D = math::NkColor;
```

`math::NkColor` (`Kernel/Foundation/NKMath/src/NKMath/NkColor.h:1046`) est une classe **RGBA sur 8 bits**, quatre octets en tout (`uint8 r, g, b, a`). On l'écrit en agrégat : `NkColor2D{18, 20, 28, 255}`.

Il existe un pendant flottant, `NkColorF` (composantes dans $[0, 1]$, 16 octets), qui porte tout l'arsenal colorimétrique : HSL, HSV, LAB, LCH, XYZ, OKLab, OKLch, CMYK, hexadécimal, luminance WCAG, DeltaE et DeltaE2000, modes de fusion *Multiply*, *Screen*, *Overlay*, *SoftLight*, *HardLight*. Utile pour calculer une palette; inutile pour dessiner. Le rendu 2D n'emploie que les quatre `uint8`.

12.8 Les textures

`.../Renderer/Resources/NkTexture.h:36` — une ressource GPU, ni copiable ni déplaçable.

12.8.1 Créer et remplir

Méthode	Rôle
<code>Create(renderer, w, h, fillColor)</code>	texture vide d'une couleur
<code>LoadFromFile(renderer, path)</code>	depuis un fichier image
<code>LoadFromImage(renderer, image, area)</code>	depuis un <code>NkImage</code> déjà décodé
<code>LoadFromMemory(...)</code>	depuis un tampon d'octets
<code>Update(pixels, destX, destY)</code>	mise à jour <i>partielle</i>
<code>SetFilter / SetWrap / GenerateMipmap</code>	paramètres d'échantillonnage
<code>GetTexCoords(const NkRect2i &)</code>	sous-rectangle en pixels → UV dans $[0, 1]^2$

Les filtres et modes de répétition sont deux petites énumérations : `NkTextureFilter{NK_NEAREST, NK_LINEAR}` et `NkTextureWrap{NK_CLAMP, NK_REPEAT, NK_MIRROR_REPEAT}` (`NkTexture.h:22-33`).

SetFilter ne fait rien partout

Sur DX12 et Vulkan, il n'y a pas de *sampler* par texture : le *sampler* est global et immuable, donc `SetFilter` et `SetWrap` sont des *no-op*. C'est écrit dans les « pièges connus » du module (`Kernel/Runtime/NKCanvas/USAGE.md:482-488`), avec deux autres limites de la même liste : `NkRenderTexture` n'est un vrai tampon hors-écran que sur OpenGL (ailleurs c'est un *stub*), et `NkShader::Compile()` ne fonctionne que sur OpenGL — il retourne *false* ailleurs.

12.8.2 La texture blanche de 1 pixel

Voici une idée simple et instructive :

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.h:121`

```
1 static NkTexture *NkTexture::GetWhiteTexture(NkIRenderer2D &);
```

Cette texture d'un pixel blanc est utilisée par *toutes* les primitives *non texturées* : `DrawLine`, `DrawFilledRect`, `DrawFilledCircle`, `DrawFilledTriangle` (`.../Renderer/Batch/NkBatchRenderer2D.cpp:394, 401, 426, 479`). Pourquoi? Pour que **tout passe par le même *shader* et le même *pipeline***. Il n'y a pas un chemin « uni » et un chemin « texturé » : il y a un seul chemin, texturé, et le uni est simplement du blanc multiplié par la couleur du sommet.

Ce qu'il faut retenir

«Tout est texturé; le uni, c'est juste du blanc.» Cette astuce supprime un changement d'état GPU par forme, et surtout permet de regrouper dans un même appel de dessin des rectangles pleins et du texte — à condition qu'ils partagent la même texture, ce qui n'est pas le cas ici, mais nous verrons que le principe se généralise.

12.8.3 Les pixels conservés côté CPU

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:109-111

```
1 // (=> GetCPUPixels()==NULL). Le rasterizer Software echantillonne
2 // directement mCPUPixels ; sans cette copie -> textures/texte
3 // INVISIBLES en Software. Fix bug 2026-06-05.
```

Chaque texture garde donc une copie de ses pixels en mémoire centrale (`mCPUPixels`). C'est du gaspillage sur GPU, mais c'est la seule façon de faire fonctionner le rasteriseur logiciel. À connaître si vous vous demandez pourquoi une texture consomme deux fois sa taille.

12.9 Les polices et le texte

`NkFont`, dans `NkCanvas`, est un **enveloppeur GPU** au-dessus du module externe `NKFont`, qui fait la rasterisation (celle-ci a été sortie de `NkCanvas` le 28 mai 2026, cf. [.../Renderer/Resources/NkFont.h:56-63](#)).

- **Chargement** : `NkFont::LoadFromFile(NkIRenderer2D &renderer, const char *path)` ou `LoadFromMemory(...)`. L'implémentation lit les octets et *conserve la copie en mémoire* (`mFontData`, [NkFont.cpp:65-81](#)) : la fonte doit rester disponible pour rasteriser de nouvelles tailles.
- **Atlas** : une *page* par taille de caractère demandée (`struct Page`, [NkFont.h:102-107](#)), créée paresseusement par `GetOrCreatePage` ([NkFont.cpp:86-128](#)). Chaque page construit un atlas puis l'envoie dans une `NkTexture`.
- **Glyphes** : `GetGlyph(codepoint, characterSize, bold)` renvoie un `NkGlyph = textureRect` (pixels dans l'atlas), `bounds` (décalage et taille), `advance` (avance horizontale).

Deux limites à connaître, écrites dans le code :

- **Le crénage n'est pas exposé**. `GetKerning` retourne toujours `0.f` ([NkFont.cpp:163-167](#)) : « Le module `NKFont` n'expose pas le kerning par paire (intègre dans l'advance des glyphes) ».
- **Le faux-gras n'existe pas**. Le paramètre `bold` est ignoré ([NkFont.cpp:133](#), paramètre nommé `/*bold*/`) : « charger une fonte bold dediee si necessaire ».

12.9.1 Comment un texte devient des triangles

Chaque glyphe devient un quad de quatre `NkVertex2D` ([.../Renderer/Resources/NkSprite.cpp:213-228](#)). Puis `NkText::Draw` applique la transformation — rotation, échelle, origine — **côté CPU**, sommet par sommet ([NkSprite.cpp:290-316](#)), et soumet le tout d'un coup :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkSprite.cpp:318

```
1 renderer.DrawVertices(tverts.Data(), tverts.Size(),
2                       indices.Data(), indices.Size(), atlas);
```

Résultat : **un seul appel de dessin par texte**, quelle que soit sa longueur. C'est la logique du *batching*, appliquée localement.

Une fonction reste non implémentée et il vaut mieux le savoir avant de compter dessus :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkSprite.cpp:332-334

```
1 return 0; // TODO: binary search on glyph bounds
```

(NkText::FindCharacterPos — convertir un index de caractère en position à l'écran).

12.10 Les formes persistantes

Pour du contenu qui ne change pas à chaque image, NkCanvas offre des objets persistants façon SFML. Ils héritent tous de NkShape : public NkTransformable, public NkDrawable (../Renderer/Shapes/NkShape.h:40), qui factorise le remplissage (triangulation en éventail), le contour et une texture optionnelle.

Classe	API propre
NkRectangleShape	explicit NkRectangleShape(NkVec2f size), SetSize/GetSize
NkCircleShape	explicit NkCircleShape(float32 r, uint32 segments = 32)
NkLineShape	NkLineShape(NkVec2f a, NkVec2f b, float32 thickness = 1.f)
NkConvexShape	SetPointCount(n), SetPoint(i, p)

Limite documentée : la triangulation en éventail n'est valable que pour des polygones **convexes** (NkShape.h:21-26, NkConvexShape.h:19-23). Un polygone concave dessiné ainsi produira des recouvrements aberrants.

Le tampon temporaire détruit avant l'envoi

C'est le meilleur exemple pédagogique du dépôt et il mérite d'être lu deux fois :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Shapes/NkShape.cpp:62-71

```
1 // On triangule en fan COTE NkShape (1 triangle = (p0, p_i, p_{i+1}))
2 // et on emet directement en NK_TRIANGLES. Pourquoi pas NK_TRIANGLE_FAN
3 // (n vertices, plus economique) ? Car NkRenderWindow::Draw raw fait
4 // une expansion TRIANGLE_FAN -> TRIANGLES dans un buffer scratch
5 // local au switch ; si le backend batch lazyement ce buffer (et ne
6 // recopie pas immediatement), le scratch est detruit avant flush ->
7 // donnees garbage / artefacts (visible : « rectangles creux »).
8 // En emettant TRIANGLES direct, le buffer `v` de NkShape reste vivant
9 // pendant tout l'appel target.Draw, et NkRenderWindow se contente
10 // d'un identity-indices submit deterministe.
```

La leçon dépasse largement ce cas : **dès qu'un système accumule des données pour les envoyer plus tard, tout tampon local devient suspect**. Vous retrouverez exactement ce raisonnement au chapitre 3 (une surcouche différée ne doit jamais détenir de pointeur vers la pile de l'appelant) et au chapitre 4 (la police doit survivre à toute la boucle).

Une matrice dégénérée aplatit tout

Corrigé depuis, mais très instructif :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkTransformable.h:161-172

```
1  /// a01 = -sy*sin = -sys      <- fix 2026-05-30 : etait -syc (FAUX, degenerait)
2  /// a11 = sy*cos = sys       <- fix 2026-05-30 : etait sys (FAUX, degenerait)
3  ///
4  /// Bug d'origine : avec rotation=0, scale=(1,1) on obtenait sys=0
5  /// et syc=1, et le code mettait m01=-syc=-1, m11=sys=0 -> matrice
6  /// degeneratee transformant tout en ligne Y=ty. Conséquence :
7  /// paddles NkRectangleShape invisibles (vertices alignés en ligne),
8  /// meme symptome sur ball/dashes/circles. Les scores marchent car
9  /// DrawFilledRect ne passe PAS par NkTransformable.
```

Notez le détail final, qui est la clé du diagnostic : *ce qui marchait encore* indiquait précisément le chemin de code fautif. Quand une partie de l’affichage disparaît et pas une autre, cherchez ce qui les distingue.

12.11 Le batching : le vrai sujet du module

Nous arrivons au cœur de NkCanvas. Fichier central : `Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp` (552 lignes), classe de base de *tous* les backends actifs.

L’idée : au lieu d’envoyer chaque forme au GPU au moment où on la dessine — ce qui coûterait un appel de pilote par rectangle — on **accumule** tout dans des tableaux en mémoire centrale, et on n’envoie qu’à la fin, en gros paquets.

12.11.1 L’accumulation

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.h:157-159

```
1  NkVector<NkVertex2D> mVertices;
2  NkVector<uint32>      mIndices;
3  NkVector<NkBatchGroup> mGroups;
```

Un NkBatchGroup est un futur appel de dessin :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.h:23-28 (abrégé)

```
1  struct NkBatchGroup {
2      const NkTexture *texture;
3      NkBlendMode blendMode;
4      uint32 indexStart;
5      uint32 indexCount;
6  };
```

Et voici la règle qui gouverne la performance de toute votre interface :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:207-229 (principe)

```
1  // EnsureGroup(tex, blend) ferme le groupe courant et en ouvre un nouveau
2  // DES QUE la texture ou le mode de fusion change (et incremente mStats.textureSwap).
```

La règle d'or du 2D

Un appel de dessin par groupe; un nouveau groupe à chaque changement de texture ou de mode de fusion. Donc : moins on change de texture, moins il y a d'appels de dessin. C'est la raison d'être des *atlas* — une seule texture qui contient tous les glyphes, ou toutes les icônes — et c'est pourquoi alterner « une icône, un texte, une icône, un texte » coûte quatre appels là où « deux icônes puis deux textes » n'en coûte que deux.

12.11.2 La capacité, et l'histoire qui va avec

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.h:33-40

```
1 // Maximum vertices / indices before an automatic flush.
2 // 262144 (~5 Mo GPU) : une UI dense (IDE plein écran : arbre + onglets +
3 // icônes) dépasse 65536 sommets ENTRE DEUX CLIPS — La draw-list NkGui
4 // arrive alors en UNE commande plus grosse que l'ancien buffer, et
5 // l'upload écrivait HORS du buffer GPU (crash memcpy). Les 3 backends
6 // (DX11/DX12/GL) créent leurs buffers avec ces constantes.
7 static constexpr uint32 kMaxVertices = 262144;
8 static constexpr uint32 kMaxIndices = kMaxVertices * 6 / 4;
```

Deux cent soixante-deux mille sommets, environ 5 Mo côté GPU. Le commentaire raconte pourquoi : une interface dense produit, *entre deux changements de découpe*, une seule commande de plus de 65 536 sommets. L'ancien tampon faisait exactement cette taille; l'envoi écrivait donc au-delà, et le programme plantait dans un memcpy.

Quand l'accumulation risque de déborder, un vidage est déclenché automatiquement :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:235-237

```
1 if (mVertices.Size() + 4 > kMaxVertices || mIndices.Size() + 6 > kMaxIndices) {
2     Flush();
3 }
```

12.11.3 La soumission : Flush()

Flush() fait quatre choses (NkBatchRenderer2D.cpp:56-98) : fermer le dernier groupe et compacter les groupes vides; appliquer le découpage courant; vérifier qu'on ne dépasse pas la capacité; envoyer.

Le troisième point est une garde défensive exemplaire :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:80-88

```
1 // GARDE-FOU : ne JAMAIS soumettre plus que la capacité des buffers GPU
2 // (l'upload écrivait hors buffer -> crash). Troncature (indices en
3 // multiple de 3) : dégradation visuelle plutôt que corruption mémoire.
4 uint32 vSub = mVertices.Size(), iSub = mIndices.Size();
5 if (vSub > kMaxVertices) vSub = kMaxVertices;
6 if (iSub > kMaxIndices) iSub = kMaxIndices - (kMaxIndices % 3);
7 SubmitBatches(mGroups.Data(), validCount, mVertices.Data(), vSub, mIndices.Data(), iSub);
```

Ce qu'il faut retenir

« Dégradation visuelle plutôt que corruption mémoire. » Cette phrase est un principe de conception à part entière : quand on ne peut pas satisfaire une demande, on rend un résultat incomplet mais *sûr*, jamais un résultat qui détruit la mémoire. Notez aussi le % 3 : tronquer des indices de triangles à un multiple de trois, faute de quoi le dernier triangle serait incomplet.

12.11.4 Le vrai appel de dessin

SubmitBatches est la seule méthode virtuelle pure que chaque backend doit fournir. Voici la version OpenGL, la plus lisible :

Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/OpenGL/NkOpenGLRenderer2D.cpp:591-613

```
1 void NkOpenGLRenderer2D::SubmitBatches(const NkBatchGroup *groups, uint32 groupCount,
2                                       const NkVertex2D *verts, uint32 vCount,
3                                       const uint32 *idx, uint32 iCount) {
4     glBindVertexArray((GLuint)mVAO);
5     glBindBuffer(GL_ARRAY_BUFFER, (GLuint)mVBO);
6     glBufferSubData(GL_ARRAY_BUFFER, 0, (GLsizeiptr)(vCount * sizeof(NkVertex2D)), verts);
7     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, (GLuint)mEBO);
8     glBufferSubData(GL_ELEMENT_ARRAY_BUFFER, 0, (GLsizeiptr)(iCount * sizeof(uint32)), idx);
9     // Issue one draw call per group
10    for (uint32 i = 0; i < groupCount; ++i) {
11        const auto &g = groups[i];
12        ApplyBlendMode(g.blendMode);
13        BindTexture(g.texture);
14        glDrawElements(GL_TRIANGLES, (GLsizei)g.indexCount, GL_UNSIGNED_INT,
15                      (void *) (uintptr_t)(g.indexStart * sizeof(uint32)));
16    }
17 }
```

Tout le lot est envoyé en **un seul** glBufferSubData par vidage, puis **un** glDrawElements par groupe. Voilà, très concrètement, ce que « batching » veut dire.

Begin() n'est pas ré-entrant

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:27-31

```
1 bool NkBatchRenderer2D::Begin() {
2     if (mInFrame) {
3         logger.Warnf("[NkBatch] Begin() called twice");
4         return false;
5     }
6 }
```

Si ce message apparaît dans `logs/app.log`, c'est que quelque chose ouvre une image sans la fermer. En pratique, cela signifie qu'on a appelé Begin() à la main alors que NkRenderWindow::Clear() le fait déjà.

12.12 Le découpage (scissor) et sa pile

Le découpage restreint le rendu à un rectangle. C'est indispensable pour les panneaux défilables : un élément qui dépasse doit être coupé net, pas dessiné par-dessus le voisin.

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NklRenderer2D.h:105-121

```
1 virtual void SetClip(const NkRect2i &rectPixels) { (void)rectPixels; }
2 virtual void PopClip() {}
3 virtual void ResetClip() {}
4 virtual bool HasClip() const { return false; }
5 virtual NkRect2i GetClip() const { return NkRect2i{}; }
```

Le contrat est écrit juste au-dessus :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NklRenderer2D.h:98-104

```
1 // Pile : SetClip empile Le rect (intersecte avec Le clip courant) ; PopClip
2 // depile. ResetClip vide la pile. Implementation backend : glScissor (GL),
3 // VkRect2D dynamique (Vulkan), RSetScissorRects (DX11/12), clamp CPU (Software).
```

Deux propriétés à retenir : **c'est une pile**, et **l'empilement intersecte**. Un enfant ne peut donc jamais dessiner en dehors de son parent, quoi qu'il demande.

12.12.1 Changer de découpe force un vidage

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:120-126

```
1 void NkBatchRenderer2D::SetClip(const NkRect2i &rect) {
2     Flush(); // committe la geometrie en cours avec Le clip actuel
3     const NkRect2i clip = mHasClip ? NkIntersectClip(mClipRect, rect) : rect;
4     mClipStack.PushBack(clip);
5     mClipRect = clip;
6     mHasClip = true;
7 }
```

C'est logique — le *scissor* est un état GPU, pas un attribut de sommet : tout ce qui est accumulé avant le changement doit partir avec l'ancien rectangle. Mais c'est aussi une conséquence de performance directe :

Ce qu'il faut retenir

Chaque **SetClip** et chaque **PopClip** coûte un vidage, donc au moins un appel de dessin. Une interface très découpée — un **PushClipRect** par ligne de liste, par exemple — multiplie les appels de dessin. Le bon réflexe est de poser *une* découpe pour toute une zone, puis d'écarter soi-même les éléments hors vue avant de les émettre (nous verrons ce « culling manuel » au chapitre 3).

12.12.2 Le retournement d'axe sous OpenGL

Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/OpenGL/NkOpenGLRenderer2D.cpp:532-546 (extrait)

```
1 // Clip en pixels, origine haut-gauche -> glScissor a l'origine bas-gauche :
2 // on inverse Y avec la hauteur de la surface (= mViewport.height, viewport
3 // plein ecran a top=0).
4 const int32 y = mViewport.height - rect.y - h; // flip Y
5 glEnable(GL_SCISSOR_TEST);
6 glScissor((GLint)x, (GLint)y, (GLsizei)w, (GLsizei)h);
```

OpenGL place son origine en bas à gauche. Le module a choisi la convention « haut-gauche » pour toute son API et compense dans le seul backend concerné. C'est exactement le bon endroit pour faire ce genre de correction : *une fois, au plus près de l'API*, et jamais dans le code applicatif.

L'état OpenGL survit d'une image à l'autre

Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/OpenGL/NkOpenGLRenderer2D.cpp:524-526

```
1 // Chaque frame démarre sans scissor (sinon un Clear serait clipé par le
2 // scissor laissé par la frame précédente, l'état GL étant persistant)
```

Symptôme, si on l'oublie : l'écran ne s'efface que partiellement, et une zone garde l'image de la frame précédente comme une traînée. Les machines à états globales — OpenGL en est l'archétype — demandent qu'on remette explicitement tout à zéro au début de chaque image.

12.13 La surface réellement utilisée en pratique

Nous venons de parcourir une API large : des dizaines de méthodes, des formes, des sprites, des vues, des shaders. Voici maintenant un fait qui remet tout en perspective.

L'IDE NKCode — l'application la plus grosse du dépôt, plus de trente mille lignes d'interface — n'utilise de NkCanvas que **sept méthodes** :

Méthode	Ligne	Appelée par
bool Begin()	59	NkRenderWindow::Clear
void End()	60	NkRenderWindow::Display
void Clear(const NkColor2D &)	73	le fond de l'image
void OnResize(uint32, uint32)	89	redimensionnement
void SetClip(const NkRect2i &)	105	découpage
void PopClip()	109	dépile le découpage
void DrawVertices(...)	192	tout le dessin

plus, côté ressources, NkTexture::Create, NkTexture::Update et NkTexture::SetFilter.

Voilà tout. NKCode ne dessine ni sprite, ni forme, ni texte NkCanvas : il envoie des triangles pré-calculés par NKGui. Une recherche exhaustive des `#include "NKCanvas/..."` dans tout `Applications/NKCode/src/` ne renvoie **aucun** résultat — le module n'apparaît que dans les dépendances d'édition de liens.

Ce qu'il faut retenir

Pour écrire une interface, la surface utile de NkCanvas se réduit à : Clear / Display au niveau de la fenêtre, SetClip / PopClip pour découper, DrawVertices pour dessiner, et NkTexture pour les atlas. Le reste — formes, sprites, vues, shaders — sert aux jeux et aux démos, pas aux interfaces.

12.14 Un programme complet

Assemblons. Le programme ci-dessous est la démo NkCanvas du dépôt, prise telle quelle : une fenêtre, une balle qui rebondit, un rectangle qui pulse, un triangle qui tourne, un éventail

de lignes. Aucun NKGui, aucune police.

12.14.1 L'ossature

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:11-33 (extrait)

```
1  #include "NKWindow/NKWindow.h"
2  #include "NKWindow/NKMain.h"
3  #include "NKWindow/Core/NKWindowConfig.h"
4  #include "NKEvent/NKWindowEvent.h"
5  #include "NKTime/NKTime.h"
6  #include "NKLogger/NKLog.h"
7  #include "NKMATH/NKColor.h"
8  #include "NKMATH/NKMath.h" // math::NkCos / NkSin
9
10 #include "NKCanvas/Core/NKContextDesc.h"
11 #include "NKCanvas/Core/NKGraphicsApi.h"
12 #include "NKCanvas/Renderer/Targets/NKRenderWindow.h"
13 #include "NKCanvas/Renderer/Core/NKRenderer2D.h"
14
15 using namespace nkentseu;
16 using namespace nkentseu::renderer;
17
18 NKENTSEU_DEFINE_APP_DATA([]() {
19     NkAppData d{};
20     d.appName = "NkCanvas Demo";
21     d.appVersion = "1.0.0";
22     return d;
23 })());
```

12.14.2 Fenêtre, cible, état

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:57-93 (abrégé)

```
1  int nkmain(const NkEntryState &state) {
2      // — 1. Fenetre —————
3      NkWindowConfig cfg;
4      cfg.title = "NkCanvas Demo (NkCanvas seul, style SFML/SDL)";
5      cfg.width = 900;
6      cfg.height = 600;
7      cfg.centered = true;
8      cfg.resizable = true;
9      NkWindow window;
10     if (!window.Create(cfg)) {
11         logger.Error("[nkcanvas] window failed");
12         return -1;
13     }
14
15     // — 2. Cible de rendu NKCanvas —————
16     NkContextDesc desc;
17     desc.api = ParseBackend(state.args);
18     NkRenderWindow target(window, desc);
19     if (!target.IsValid()) { window.Close(); return -2; }
20     logger.Info("[nkcanvas] backend = %s", NkGraphicsApiName(desc.api));
21
22     // — 3. Etat de la scene —————
23     bool running = true;
24     auto &events = NkEvents();
```

```

25     events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
    });
26
27     NkClock clock;
28     uint32 lastW = 0, lastH = 0;
29     float32 t = 0.f;
30
31     math::NkVec2f ball{220.f, 200.f};
32     math::NkVec2f vel{260.f, 215.f}; // pixels / seconde
33     const float32 R = 42.f;

```

12.14.3 La boucle

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:96-118 (abrégé)

```

1     while (running && window.IsOpen()) {
2         float32 dt = clock.Tick().delta;
3         if (dt > 0.1f)
4             dt = 1.0f / 60.0f;
5         t += dt;
6         while (NkEvent *ev = events.PollEvent()) {
7             (void)ev;
8         }
9         if (!running)
10            break;
11
12         // Resize : la cible suit la fenetre (vue par defaut = ecran) -> pas de clip.
13         const math::NkVec2u sz = target.GetSize();
14         if (sz.x != lastW || sz.y != lastH) {
15             if (lastW != 0 && sz.x > 0 && sz.y > 0)
16                 target.OnResize(sz.x, sz.y);
17             lastW = sz.x;
18             lastH = sz.y;
19         }
20         const float32 W = static_cast<float32>(sz.x), H = static_cast<float32>(sz.y);

```

12.14.4 Le rendu

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:139-170

```

1     // — 5. Rendu : NkRenderer2D direct (Clear -> formes -> Display) —————
2     target.Clear(NkColor2D{18, 20, 28, 255});
3     NkRenderer2D &r = target.GetRenderer2D();
4
5     // Rectangle "pulsant" au centre (remplissage + contour).
6     const float32 hp = 60.f + 28.f * math::NkSin(t * 2.f);
7     const NkRect2f box{cx - hp, cy - hp, hp * 2.f, hp * 2.f};
8     r.DrawFilledRect(box, NkColor2D{52, 84, 150, 255});
9     r.DrawRectOutline(box, NkColor2D{140, 180, 255, 255}, 2.f);
10
11     // Triangle qui tourne autour du centre.
12     const float32 tr = 95.f;
13     const math::NkVec2f p0{cx + tr * math::NkCos(t), cy + tr * math::NkSin(t)};
14     const math::NkVec2f p1{cx + tr * math::NkCos(t + 2.094f), cy + tr * math::NkSin(t +
15         2.094f)};
16     const math::NkVec2f p2{cx + tr * math::NkCos(t + 4.188f), cy + tr * math::NkSin(t +
17         4.188f)};

```

```

16     r.DrawFilledTriangle(p0, p1, p2, NkColor2D{255, 180, 60, 200});
17
18     // Eventail de Lignes depuis le coin haut-gauche.
19     for (int32 i = 0; i < 7; ++i) {
20         const float32 a = t * 0.6f + static_cast<float32>(i) * 0.22f;
21         r.DrawLine(math::NkVec2f{0.f, 0.f},
22                 math::NkVec2f{110.f + 90.f * math::NkCos(a), 110.f + 90.f *
math::NkSin(a)}},
23                 NkColor2D{80, 200, 160, 180}, 1.5f);
24     }
25
26     // La balle, par-dessus (remplissage + contour clair).
27     r.DrawFilledCircle(ball, R, NkColor2D{255, 110, 80, 255}, 48);
28     r.DrawCircleOutline(ball, R, NkColor2D{255, 220, 200, 255}, 2.f, 48);
29
30     target.Display();
31 }

```

12.14.5 Ce que ce programme raconte

Quelques observations qui valent pour toute application NkCanvas :

- **L'ordre de dessin est l'ordre de recouvrement.** La balle est dessinée en dernier, elle passe donc devant. Il n'y a ni tampon de profondeur, ni tri : ce qui est émis après recouvre.
- **Le temps est explicite.** `clock.Tick().delta` donne le temps écoulé depuis l'image précédente; toute animation se calcule en « unités par seconde » multipliées par `dt`. Notez la borne `if (dt > 0.1f) dt = 1/60`, qui empêche un bond énorme quand la fenêtre a été bloquée (déplacement, mise en veille).
- **Aucune ressource n'est allouée dans la boucle.** La position de la balle est une variable locale à `nkmain`; il n'y a ni `new`, ni conteneur créé par image.
- **On ne touche jamais à `Begin/End`.** `Clear` et `Display` suffisent.

12.15 Récapitulatif des pièges du chapitre

1. Ne jamais appeler `Initialize` sur un `render` déjà initialisé par la fabrique (*rectangles creux*).
2. Détecter le redimensionnement sur la taille de la *fenêtre*, jamais sur celle de la *swapchain* (rendu basse résolution étiré).
3. Terminer l'image avant de recréer la *swapchain* (comportement indéfini sur Vulkan et DX12).
4. Ne jamais conserver un tampon local vers des données que le batcher lira plus tard (données parasites, formes creuses).
5. Chaque `SetClip` / `PopClip` coûte un vidage : découper largement, et filtrer soi-même le hors-vue.
6. Sous OpenGL, l'état est persistant entre les images : le *scissor* doit être explicitement désactivé au début de chaque image.
7. `SetFilter` / `SetWrap` sont sans effet sur DX12 et Vulkan; `NkShader::Compile` ne fonctionne que sur OpenGL; `NkRenderTexture` n'est réel que sur OpenGL.
8. `NkFont::GetKerning` retourne toujours zéro et le paramètre `bold` est ignoré.

Ce chapitre en bref

NkCanvas dessine des triangles, et rien d'autre : toute forme — rectangle, cercle, texte — finit en sommets. Le repère a **Y vers le bas**, comme l'écran et non comme les mathématiques. Le sujet réel du module est le **regroupement** : les dessins s'accumulent dans un tampon et ne partent au GPU qu'au Flush, ce qui fait du *nombre de vidages* la seule mesure de performance qui compte. Changer de découpe force un vidage — donc un découpage par élément coûte un appel de dessin par élément.

12.16 Exercices

1 — Une horloge

Partez de NkCanvasDemo. Remplacez la scène par une horloge analogique : un DrawCircleOutline pour le cadran, douze DrawLine courts pour les graduations, trois DrawLine d'épaisseurs différentes pour les aiguilles. Faites tourner les aiguilles avec t. Contrainte : *aucun* allocation dans la boucle.

2 — Compter les appels de dessin

Le batcher tient des statistiques (mStats, dont textureSwap). Trouvez comment y accéder depuis l'application, puis affichez leur valeur dans le journal une fois par seconde. Dessinez ensuite cent rectangles pleins : combien d'appels de dessin ? Ajoutez maintenant, *entre chaque rectangle*, une forme utilisant une autre texture. Que devient le compte ? Concluez.

3 — Un dégradé sans primitive de dégradé

NkCanvas n'a pas de DrawGradient. Construisez-en un : remplissez un tableau de quatre NkVertex2D formant un rectangle, avec quatre couleurs différentes, un tableau de six indices, et appelez DrawVertices avec texture = nullptr. Vérifiez que le GPU interpole. Puis expliquez, en vous appuyant sur la section « la texture blanche de 1 pixel », pourquoi passer nullptr fonctionne.

4 — Le découpage, et son coût

Dessinez une grille de 20×20 petits carrés. Version A : un seul SetClip autour de toute la grille. Version B : un SetClip et un PopClip autour de *chaque* carré. Comparez les statistiques du batcher et le temps par image. Vous devriez mesurer, à l'écran, la règle énoncée à la section 2.10.

Questions de cours

1. Combien de sommets pour un rectangle plein, et pourquoi ?
2. Dans quel sens va l'axe Y, et quelle conséquence pratique ?
3. Qu'est-ce qui déclenche un vidage du tampon ?
4. À quoi sert la texture blanche d'un pixel ?
5. Pourquoi n'y a-t-il pas de primitive « quadrilatère » ?

Exercice de logique

Une interface dessine 500 icônes, chacune dans son propre rectangle de découpe.

1. Combien d'appels de dessin, au minimum ? Justifiez.
2. Proposez deux façons de réduire ce nombre, et dites ce que chacune coûte en contre-partie.
3. L'une de vos deux solutions change le résultat à l'écran dans un cas précis. Lequel ?
Ne concluez pas qu'elle est mauvaise : dites à quelle condition elle reste correcte.

NKGui : l'interface immédiate

Voici le cœur du cours. NKGui est le framework qui vous donnera des boutons, des cases à cocher, des champs de saisie, des arbres, des tables, des menus et des fenêtres ancrables. Mais avant la liste des widgets, il faut comprendre *comment il pense* — parce qu'il ne pense pas comme la plupart des frameworks d'interface, et que tout le reste en découle.

13.1 L'idée : on ne crée rien, on redéclare tout

13.1.1 Le mode immédiat

Dans un framework classique, un bouton est un *objet*. On le crée une fois, on lui donne un texte, une position, on lui branche un gestionnaire de clic, et il vit jusqu'à ce qu'on le détruise. L'interface est un arbre d'objets qu'on modifie.

Dans NKGui, il n'y a **aucun objet bouton**. Il y a un *appel de fonction*, qu'on refait à chaque image :

Forme canonique d'une frame NKGui

```
1 ctx.BeginFrame(dt);
2     Text(ctx, "Bonjour");
3     if (Button(ctx, "Cliquez-moi")) {
4         /* reaction au clic */
5     }
6 ctx.EndFrame();
```

Le bouton n'existe que le temps de l'appel. À l'image suivante, on le redéclare. S'il ne faut plus l'afficher, on ne l'appelle pas — il n'y a rien à détruire, rien à cacher, rien à retirer d'un parent.

Ce qu'il faut retenir

NKGui est une interface immédiate. Les widgets ne sont pas des objets qu'on crée, mais des appels qu'on refait à chaque image. Ce qui survit d'une image à l'autre, ce n'est pas le widget : c'est un *état*, rangé dans le contexte et retrouvé par *identifiant*. Toute la mécanique du chapitre découle de cette phrase.

13.1.2 Ce que ce choix vous épargne

Le gain principal est qu'il n'y a **aucune synchronisation à écrire**. Considérez une case à cocher qui reflète un booléen de votre application. Dans un modèle à objets, il faut : écrire le booléen vers la case quand il change, écrire la case vers le booléen quand l'utilisateur clique, et se souvenir de ne pas boucler. Dans NKGui :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:45

```
1 NKENTSEU_NKGUI_API bool Checkbox(NkGuiContext &ctx, const char *label, bool &value) noexcept;
```

La valeur est passée *par référence*. Le widget la lit pour se dessiner et l'écrit si l'utilisateur clique. Il n'y a plus qu'une seule copie de la vérité, et c'est la vôtre. Ce point est fondamental et mérite d'être formulé explicitement :

Qui possède quoi

Les **valeurs métier** — le booléen d'une case, le flottant d'un *slider*, le tampon de caractères d'un champ de saisie — appartiennent à **votre application**. NKGui ne les stocke pas. NKGui ne stocke que l'**état d'interaction** : ce nœud d'arbre est-il ouvert ? où est le curseur de saisie ? quelle est la largeur des colonnes de cette table ? quel onglet est actif ? Ces informations-là n'ont pas de sens pour votre modèle de données, et c'est pour cela que le framework s'en charge.

13.1.3 Une note d'honnêteté sur la documentation du module

La documentation interne de NKGui annonce deux paradigmes :

Kernel/Runtime/NKGui/ARCHITECTURE.md:5-6

```
1 Deux paradigmes : immédiat (façon ImGui) et retenu (façon Qt/Unity).
```

Le mode retenu n'existe pas dans le code. Il est placé en « Phase 6 » ([ARCHITECTURE.md:191](#)) et l'arborescence réelle du module ne contient ni dossier Retained/, ni Layout/, ni Window/, ni Dock/ : tout ce qui est implémenté tient dans Core/ et Widgets/.

Symétriquement, [README.md:14](#) et [ARCHITECTURE.md:185](#) annoncent une « Phase 1 — squelette », alors que [NkGuiWidgets.cpp](#) fait 4 973 lignes et couvre les tables, le docking, le sélecteur de couleur et les menus imbriqués. **Les fichiers d'état sont en retard sur le code. Fiez-vous au code.** Ce cours ne documente que ce qui existe vraiment.

13.2 L'identité d'un widget

Puisqu'un widget n'est pas un objet, comment le framework sait-il, d'une image à l'autre, qu'il s'agit du *même* bouton ? Par son identifiant.

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiTypes.h:26-46 (abrégé)

```
1 using NkGuiId = uint32;  
2 static constexpr NkGuiId NKGUI_ID_NONE = 0;  
3  
4 NKENTSEU_NKGUI_API_INLINE NkGuiId NkGuiHashStr(const char *s, NkGuiId seed = 2166136261u)  
5     noexcept {  
6     NkGuiId h = seed;  
7     while (s && *s) { h ^= static_cast<uint8>(*s++); h *= 16777619u; }  
8     return h ? h : 1u; // jamais 0 (0 = « aucun »)  
9 }  
10 NKENTSEU_NKGUI_API_INLINE NkGuiId NkGuiHashPtr(const void *p, NkGuiId seed = 2166136261u)  
11     noexcept { ... }
```

C'est un **hachage FNV-1a sur 32 bits** du libellé. Deux constantes, une boucle : le décalage initial 2166136261 et le multiplicateur 16777619, qui sont les valeurs canoniques de FNV-1a 32 bits.

Notez le détail final : `return h ? h : 1u`. La valeur zéro est *réservée* — elle signifie « aucun widget » (`NKGUI_ID_NONE`). Si le hachage tombe sur zéro, on renvoie 1. Un identifiant valide n'est donc jamais nul, ce qui permet d'utiliser zéro comme sentinelle sans ambiguïté.

13.2.1 La pile d'identifiants

Deux boutons « Supprimer » dans deux panneaux différents auraient le même libellé, donc le même hachage, donc la même identité — et cliquer sur l'un activerait l'autre. La parade est une *pile de portées* :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:144-164

```
1 void NkGuiContext::PushId(const char *s) noexcept {
2     const NkGuiId seed = idDepth > 0 ? idStack[idDepth - 1] : 2166136261u;
3     if (idDepth < 32) idStack[idDepth++] = NkGuiHashStr(s, seed);
4 }
5 void NkGuiContext::PopId() noexcept { if (idDepth > 0) --idDepth; }
6 NkGuiId NkGuiContext::GetId(const char *s) const noexcept {
7     const NkGuiId seed = idDepth > 0 ? idStack[idDepth - 1] : 2166136261u;
8     return NkGuiHashStr(s, seed);
9 }
```

Le mécanisme est élégant : **la graine de chaque niveau est l'identifiant du niveau précédent**. L'identifiant final dépend donc du *chemin complet*, pas du seul libellé. Il existe une surcharge `PushId(const void *)` qui hache une adresse — pratique pour donner une identité distincte à chaque élément d'une liste.

Idiome — écrit pour ce cours, d'après NkGuiContext.h:448-451

```
1 for (int32 i = 0; i < itemCount; ++i) {
2     ctx.PushId(&items[i]); // ou ctx.PushId(items[i].name.CStr())
3     if (Button(ctx, "Supprimer")) { /* supprime items[i] */ }
4     ctx.PopId();
5 }
```

Un identifiant qui change pendant l'édition

Plusieurs widgets acceptent un libellé *modifiable* — renommage d'un fichier dans un arbre, d'un onglet, d'une cellule. Si l'identité était calculée sur ce libellé, elle changerait à chaque frappe, et le framework croirait à chaque caractère qu'il s'agit d'un widget différent : le curseur de saisie sauterait, le focus se perdrait.

C'est pourquoi toutes les variantes `*Editable` exigent un `idStr` **stable**, distinct du libellé affiché (`NkGuiWidgets.h:198`, :209-210, :221-222). La règle générale : **l'identité ne se calcule jamais sur une donnée qui bouge**.

13.3 Le contexte : NkGuiContext

Tout l'état de l'interface vit dans une seule structure, `Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:131`. C'est un struct volumineux — plus de cent champs. Voici la carte, par famille :

Famille	Champs clés	Lignes
Vue / échelle	viewW, viewH, scale (DPI)	132-134
Thème	theme, syntax	135-136
Entrée	input	137
Dessin	d1, d1Overlay	138-139
Mise en page	layout	140
Polices	font, codeFont (non possédés)	141-142
Popups	popupStack[8], popupRects[8], popupDepth	147-152
Occlusion	occlRects, occlLayers, curInputLayer	175-182
Fenêtres	winDL[32], winMeta[32], winZ[32]	228-245
Docking	dockNodes, dockRoot, dockSpaceId	248-259
Presse-papiers	clipboardGetFn, clipboardSetFn	278-292
Interaction	hotId, hotIdPrev, activeId	317-321
Piles	idStack[32], disabledStack[16]	324-388
Saisie	inputId, inputCaret, inputAnchor	333-338
Stockage	openNodes, tabBarSel, scrollVals...	358-425
Hook de style	styleFn, styleUser	429-431

13.3.1 Cycle de vie

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:437-443

```

1      // — Cycle de vie —————
2      bool Init(int32 width, int32 height) noexcept;
3      void Shutdown() noexcept;
4
5      // — Cycle de frame —————
6      // BeginFrame : APRÈS que l'app ait posé l'input brut (events). Calcule
7      // les transitions, réinitialise hotId + la draw list.
8      void BeginFrame(float32 dt) noexcept;
```

13.3.2 Le contexte courant

Le contexte est **explicite** : toutes les fonctions de widget le prennent en premier paramètre. Il n'y a pas de singleton. Mais un « contexte courant » propre à chaque fil d'exécution existe, pour le code qui n'a pas le contexte sous la main :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:14-25

```

1  namespace {
2      // Contexte courant per-thread (non-singleton, comme NkUIContext).
3      thread_local NkGuiContext *gCurrentContext = nullptr;
4  }
5  void SetCurrentContext(NkGuiContext *ctx) noexcept { gCurrentContext = ctx; }
6  NkGuiContext *GetCurrentContext() noexcept { return gCurrentContext; }
```

Une fois ctx.Init(...) passé, appelez SetCurrentContext(&ctx); avant de rendre la main, appelez SetCurrentContext(nullptr). Rien de plus.

13.3.3 Le thème

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:28-49

```
1 struct NKENTSEU_NKGUI_CLASS_EXPORT NkGuiTheme {
2     NkColor bgPrimary   = {26, 29, 36, 255};
3     NkColor panel       = {38, 42, 52, 255};
4     NkColor header      = {46, 51, 63, 255};
5     NkColor button      = {58, 64, 80, 255};
6     NkColor buttonHover = {78, 92, 120, 255};
7     NkColor buttonActive= {96, 150, 230, 255};
8     NkColor border      = {86, 94, 112, 255};
9     NkColor text        = {230, 232, 240, 255};
10    NkColor textDisabled= {120, 124, 134, 255};
11    NkColor selection    = {64, 110, 200, 235};
12    NkColor accent       = {96, 165, 250, 255};
13    NkColor track        = {30, 34, 43, 255};
14    NkColor tabBar       = {22, 25, 31, 255};
15    NkColor tab          = {40, 45, 56, 255};
16    NkColor tabHover     = {58, 64, 80, 255};
17    NkColor tabActive    = {52, 58, 72, 255};
18    float32 rounding    = 5.f;
19    float32 framePadX   = 10.f;
20    float32 framePadY   = 6.f;
21 };
```

Ce sont des couleurs *nommées par usage*, lues directement (`ctx.theme.button`). Il n’y a pas de rôles sémantiques abstraits. Pour changer l’apparence, on écrase les champs après `Init` — c’est ce que fait le shell de l’éditeur ([Engine/NKEditorKit/src/NKEditorKit/NKEditorShell.cpp:155-179](#), qui y écrit une palette «GitHub Dark»).

Un second thème, `NkGuiSyntax` (`NkGuiContext.h:53-66`), porte les couleurs de coloration syntaxique : `keyword`, `type`, `string`, `comment`, `number`, `preproc`, `heading`, `mdcode`, `function`, `constant`, `oper`. Il sert à l’éditeur de code, et rien ne vous empêche de vous en servir pour autre chose.

Un idiome courant dans le dépôt, pour savoir si l’on est en thème clair ou sombre sans le stocker :

Engine/NKEditorKit/src/NKEditorKit/NKEditorScrollbar.h:29

```
1 const bool light = ((int32)th.bgPrimary.r + th.bgPrimary.g + th.bgPrimary.b) > 384;
```

13.4 L’entrée : `NkGuiInput`

`Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiInput.h` — 145 lignes, entièrement dans l’entête, tout `inline`. C’est la structure que *votre application remplit*, et c’est la frontière entre le monde des événements et le monde des widgets.

13.4.1 Ce que l'application pose

Champ	Sens
mousePos	position du pointeur
mouseDown[3]	0 = gauche, 1 = droit, 2 = milieu
wheel, wheelH	molette verticale et horizontale (cumulées)
ctrlDown, shiftDown, altDown	modificateurs
PushChar(cp)	un caractère saisi (<i>codepoint</i> Unicode)
SetKey(NkGuiKey, bool)	une touche enfoncée ou relâchée
SetDoubleClick(button)	double-clic détecté par le système
wantCopy, wantCut, wantPaste, wantSelectAll	raccourcis d'édition

13.4.2 Ce que NKGui calcule

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiInput.h:103-135 (abrégé)

```
1 void NewFrame() noexcept {
2     for (int32 i = 0; i < 3; ++i) {
3         mouseClicked[i] = mouseDown[i] && !mousePrev[i];
4         mouseReleased[i] = !mouseDown[i] && mousePrev[i];
5         ...
6         // Double-clic : (a) détection interne (2e clic < 0.40 s) OU
7         // (b) injection OS via SetDoubleClick (consommée puis remise à 0).
8         ...
9     }
10    for (int32 i = 0; i < KeyCount; ++i) {
11        keyInit[i] = keyDown[i] && !keyPrev[i];
12        ...
13    }
14 }
```

Les champs calculés — `mouseClicked`, `mouseReleased`, `mouseDoubleClicked`, `mouseDownDur`, `keyInit`, `keyDur` — sont ce que les widgets consultent. Vous ne les écrivez jamais; vous ne posez que l'état *brut*.

13.4.3 La répétition au maintien

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiInput.h:87-94 (signature)

```
1 int32 KeyPressedRepeat(NkGuiKey k, float32 delay = 0.30f, float32 rate = 0.04f) const
   noexcept;
```

Cette fonction renvoie le *nombre* de déclenchements franchis entre deux durées d'appui. C'est ce qui fait qu'une flèche maintenue déplace le curseur en rafale après un court délai, comme dans n'importe quel éditeur de texte. Elle sert aussi aux boutons à rafale.

13.4.4 Les touches suivies

NKGui ne suit pas tout le clavier. Il maintient une liste explicite (`NkGuiTypes.h:75-129`) : Left, Right, Up, Down, Home, End, Backspace, Delete, Enter, Escape, Tab, F2, F5, plusieurs lettres, Space,

F8, F12, etc.

Ce qu'il faut retenir

La saisie de texte passe par `chars[]` — des *codepoints* Unicode poussés par `PushChar` — et **jamais** par les touches. Les touches servent à la navigation et aux commandes. Confondre les deux donne une application où l'on ne peut pas taper d'accents, ou bien où les flèches écrivent des caractères.

13.5 Le cycle d'une image

13.5.1 BeginFrame

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:39-75

```
1 void NkGuiContext::BeginFrame(float32 dt) noexcept {
2     input.dt = dt;
3     input.NewFrame();      // transitions clic/relâche
4     time += dt;           // blink du caret
5     hotIdPrev = hotId;    // le survol résolu de la frame précédente
6     hotId = NKGUI_ID_NONE; // re-calculé par les widgets (greedy)
7     interact = NkGuiInteract::None;
8     lastItemHovered = false;
9     wantCursor = NkGuiCursor::Arrow;
10    idDepth = 0;
11    disabledDepth = 0;
12    inputClickConsumed = false;
13    curPopupLevel = -1;    // le dessin reprend sur la couche principale
14    // Routeur d'occlusion : la liste écrite la frame PRECEDENTE devient la
15    // liste LUE (stable toute la frame, comme hotIdPrev) ; on repart à zero
16    // pour l'écriture de cette frame.
17    occlCount = occlCountNew;
18    for (int32 i = 0; i < occlCountNew; ++i) { occlRects[i] = occlRectsNew[i]; occlLayers[i]
19    = occlLayersNew[i]; }
20    occlCountNew = 0;
21    curInputLayer = 0;
22    winCount = 0;         // pool de fenêtres ré-attribué cette frame
23    curWindow = -1; curWindowId = NKGUI_ID_NONE; curWindowDocked = false;
24    containerDepth = 0;   // pile de conteneurs ré-attribuée
25    overlayDepth = 0;
26    for (uint32 i = 0; i < windowMeta.Size(); ++i) {
27        windowMeta[i].hostRendered = false;
28        windowMeta[i].frameDL = -1;
29        windowMeta[i].dockDL = -2;
30    }
31    dl.Reset();
32    dlOverlay.Reset();
33 }
```

Lisez cette fonction comme une remise à zéro sélective : **tout ce qui est recalculé chaque image est effacé** (les listes de dessin, les piles, le survol courant, le *pool* de fenêtres), et **tout ce qui doit survivre est préservé** (les états persistants indexés par identifiant, la position des fenêtres, l'arbre de docking).

Deux lignes méritent qu'on s'y arrête, parce qu'elles portent le même mécanisme : `hotIdPrev = hotId` et le recopiage des rectangles d'occlusion. Dans les deux cas, **ce qui a été écrit à l'image précédente devient ce qu'on lit à l'image courante**. Nous verrons pourquoi à la section 3.6.

13.5.2 EndFrame

EndFrame (NkGuiContext.cpp:77-142) fait cinq choses :

1. **Fusionne les listes de dessin des fenêtres dans d1, dans l'ordre de profondeur** (tri par insertion, lignes 80-94). C'est ce qui donne un recouvrement correct entre fenêtres.
2. **Détermine la fenêtre survolée** pour l'image suivante (hoveredWindowId, lignes 96-106) : celle qui est le plus en avant sous le curseur.
3. **Ferme la chaîne de popups** : Échap ferme le niveau le plus profond ; un clic hors de tous les popups et hors de leur ancre ferme tout (lignes 113-124).
4. **Libère activeId si le bouton est relâché** — la garde anti-gel, ci-dessous.
5. **Consomme la molette et le texte** (lignes 139-141) : input.wheel et input.wheelH repassent à zéro, puis input.ClearPerFrameText().

L'anti-gel d'activeId

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:125-132

```
1 // ANTI-GEL : aucun glissement légitime ne conserve activeId bouton RELÂCHÉ.
2 // Si Le widget détenant activeId a disparu (hôte redevenu flottant, onglet
3 // caché, fenêtre fermé...e) il ne libère jamais activeId et l'occlusion bloque
4 // TOUTE interaction. Souris haute + activeId encore posé => on libère d'office.
5 if (!input.mouseDown[0] && activeId != NKGUI_ID_NONE) {
6     activeId = NKGUI_ID_NONE;
7     movingWindowId = NKGUI_ID_NONE;
8 }
```

Retenez le symptôme, car il est spectaculaire et déroutant : **plus rien ne répond**. Aucun bouton ne réagit, aucun survol ne s'allume, l'application semble figée alors qu'elle continue de tourner à soixante images par seconde. La cause est presque toujours un activeId resté posé par un widget qui a disparu. Cette garde le libère d'office, mais si vous écrivez votre propre mécanisme de glissement (comme la barre de défilement de NKEditorKit), c'est à vous de remettre ctx.activeId = 0 au relâchement.

13.5.3 L'invariant d'ordre

Ce qu'il faut retenir

événements → ctx.BeginFrame(dt) → widgets → ctx.EndFrame() → backend.Submit(...)

BeginFrame appelle input.NewFrame() en interne. Si vous pompez les événements *après* BeginFrame, les transitions clic/relâche portent sur l'état de l'image *précédente* : vos clics arrivent avec une image de retard, ou pas du tout.

L'ordre ne se copie pas d'une bibliothèque à l'autre

Événements *puis* BeginFrame : c'est l'ordre de NKGui, et il tient à une raison précise — c'est BeginFrame qui appelle input.NewFrame(). D'autres bibliothèques d'interface immédiate attendent l'ordre *inverse*, parce qu'elles font ce travail ailleurs.

Un ordre correct dans une bibliothèque est donc un ordre faux dans une autre, et l'erreur ne se voit pas : rien ne plante. Les clics arrivent avec une image de retard, ou pas

du tout — et l'on cherche le défaut dans le widget.
Si vous transposez du code venu d'ailleurs — d'une autre bibliothèque, ou du module NKUI déprécié qui traîne encore dans l'arbre — c'est la première chose à vérifier, avant même que le code compile.

13.6 Comment un widget conserve son état

Réponse : il **ne conserve rien lui-même**. L'état vit dans le contexte, indexé par identifiant, dans des tableaux parallèles clé/valeur.

État persistant	Stockage	Lignes
Nœuds d'arbre ouverts	openNodes	358
Onglet sélectionné	tabBarKeys + tabBarSel	359-360
Défilement d'une zone	scrollKeys + scrollVals	377-378
Focus/ancre de liste	selKeys + selFocusStore	363-365
Largeurs de colonnes	tblKeys + tblWidths	419-420
Teinte du sélecteur couleur	pickerKeys + pickerHSV	424-425
Fenêtres (pos, taille, z)	windowMeta	236
Arbre de dock	dockNodes	248

Le mécanisme est volontairement simple :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:336-355 (abrégé)

```
1 bool NkGuiContext::IsNodeOpen(NkGuiId id) const noexcept {
2     for (uint32 i = 0; i < openNodes.Size(); ++i)
3         if (openNodes[i] == id) return true;
4     return false;
5 }
6 void NkGuiContext::SetNodeOpen(NkGuiId id, bool open) noexcept { ... /* swap-remove */ }
```

Recherche linéaire, pas de table de hachage. C'est assumé : ces listes restent petites — quelques dizaines de nœuds ouverts au plus.

13.7 Comment un widget reçoit les événements

Nous arrivons au mécanisme le plus subtil de NKGui. Prenez le temps.

13.7.1 Le problème

En mode immédiat, quand on traite le bouton numéro trois, **on ne connaît pas encore les rectangles des widgets qui viendront après**. Or ce sont eux qui seront dessinés par-dessus. Comment savoir si le pointeur est réellement sur le bouton trois, ou sur une fenêtre qui le recouvre et qu'on n'a pas encore déclarée ?

Un framework à objets répondrait facilement : il a l'arbre complet, il fait un test de haut en bas. Le mode immédiat n'a pas cet arbre.

13.7.2 La solution : glouton, avec une image de retard

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:491-528

```
1  bool NkGuiContext::ItemHoverable(const NkRect &r, NkGuiId id) noexcept {
2      // Routeur d'occlusion UNIFIE : un widget n'est jamais survolable si une
3      // surface flottante d'une couche SUPERIEURE (modal, palette, popover
4      // declares via PushOcclusion) recouvre Le pointeur – quel que soit
5      // l'ordre de dessin des panneaux.
6      if (!PointReachable(input.mousePosition)) return false;
7      // Désactivé : aucune interaction.
8      if (IsDisabled()) return false;
9      // Un popup ouvert capture Le pointeur : un widget ne réagit pas si le
10     // pointeur est au-dessus d'un popup PLUS PROFOND que le niveau courant
11     // (couche principale = -1 ; un item de menu ne capture pas sous son
12     // sous-menu déployé).
13     for (int32 i = curPopupLevel + 1; i < popupDepth; ++i)
14         if (NkGuiRectContains(popupRects[i], input.mousePosition)) return false;
15     // Occlusion par fenêtre : hors popup, un widget ne réagit que si SA fenêtre
16     // est celle survolée au-dessus (curWindowId). Bloque le fond ET les fenêtres
17     // recouvertes. (hoveredWindowId/curWindowId = NONE pour le fond hors fenêtre.)
18     if (curPopupLevel < 0 && hoveredWindowId != NKGUI_ID_NONE && hoveredWindowId !=
        curWindowId)
19         return false;
20     // Hors du CLIP courant (zone défilable, panneau) : pas d'interaction –
21     // un item scrollé hors-vue ne doit pas capturer le pointeur.
22     if (!NkGuiRectContains(DL().CurrentClip(), input.mousePosition)) return false;
23     // Bloqué si un AUTRE widget capture le pointeur.
24     if (activeId != NKGUI_ID_NONE && activeId != id) return false;
25     if (!NkGuiRectContains(r, input.mousePosition)) return false;
26     // Greedy : Le DERNIER widget soumis sous le pointeur écrase hotId →
27     // celui dessiné par-dessus gagne. On ne déclare « survolé » QUE le
28     // front-most de la frame précédente (hotIdPrev) ; le widget masqué
29     // dessous, lui, met à jour hotId mais retourne false → ne capture pas.
30     hotId = id;
31     return hotIdPrev == id;
32 }
```

Chaque `return false` est une leçon, et l'ordre des tests est celui du moins cher au plus cher. Mais l'essentiel est dans les deux dernières lignes.

Glouton (*greedy*) : tout widget qui se trouve sous le pointeur écrase `hotId`. Comme les widgets sont déclarés dans l'ordre de dessin, le dernier qui écrit est celui qui est visuellement au-dessus. À la fin de l'image, `hotId` contient donc bien l'identifiant du widget de premier plan.

Une image de retard : mais on ne peut pas exploiter cette information pendant l'image en cours — elle n'est complète qu'à la fin. Alors on la reporte : `BeginFrame` copie `hotId` dans `hotIdPrev`, et `ItemHoverable` ne déclare « survolé » que le widget dont l'identifiant correspond à `hotIdPrev`. Le survol effectif est donc celui *résolu à l'image précédente*.

Glouton plus une image de retard

Chaque widget sous le pointeur écrit `hotId` (le dernier gagne, donc celui du dessus); le résultat sert à l'image *suivante*, via `hotIdPrev`. Un widget masqué met bien à jour `hotId` mais retourne `false` : il ne capture rien.

Conséquence pratique et **contre-intuitive** : ce qui est dessiné plus tard capte le clic. L'ordre de peinture définit la priorité d'interaction. La démo l'écrit noir sur blanc :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:806-808

```
1 // Soumise APRÈS Les boutons → au-dessus. ItemHoverable résout Le z-ordre :  
2 // au-dessus d'un bouton, c'est la boîte qui capture, pas Le bouton dessous.
```

Le prix à payer est un décalage d'une image sur le survol. À soixante images par seconde, il est invisible. Mais il explique un comportement qu'on remarque parfois : le tout premier clic sur un élément jamais survolé auparavant peut ne pas prendre. Nous verrons à la section 3.12 que les menus contournent ce point par un test géométrique direct.

13.7.3 ButtonBehavior : la sémantique du clic

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:541-543

```
1 bool ButtonBehavior(NkGuiId id, const NkRect &r, NkGuiButtonFlags flags =  
    NkGuiButtonFlags::None,  
2 float32 repeatDelayOverride = -1.f, float32 repeatRateOverride = -1.f,  
3 bool *outHovered = nullptr, bool *outHeld = nullptr) noexcept;
```

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:530-570 (extraits)

```
1 const bool hovered = ItemHoverable(r, id); // respecte Le z-ordre...  
2  
3 if (hovered && input.mouseClicked[0]) {  
4     activeId = id;  
5     if (repeat) pressed = true; // rafale : déclenche dès l'appui  
6 }  
7 const bool held = (activeId == id);  
8 if (held) {  
9     interact = NkGuiInteract::EditWidget;  
10    ...  
11    if (input.mouseReleased[0]) {  
12        // Sans Repeat : clic validé au relâchement DANS Le rect.  
13        if (!repeat && NkGuiRectContains(r, input.mousePos)) pressed = true;  
14        activeId = NKGUI_ID_NONE;  
15    }  
16 } else if (hovered) {  
17     interact = NkGuiInteract::HoverWidget;  
18 }...  
19  
20 lastItemHovered = hovered; // pour IsItemHovered() / SetTooltip
```

Ce qu'il faut retenir

Un clic se valide au **RELÂCHEMENT**, à l'intérieur du rectangle. C'est la convention de tous les systèmes de fenêtrage : on peut appuyer sur un bouton, glisser en dehors, relâcher, et rien ne se produit — l'action est annulée. Avec le drapeau Repeat, en revanche, le déclenchement se fait dès l'appui, puis en rafale.

13.7.4 La machine à états d'interaction

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiTypes.h:52-61

```
1 // — Machine à états d'interaction (Le coeur du fix UX) —————
2 // À chaque frame, UN SEUL mode actif et explicite. Zones de préhension
3 // disjointes et priorisées (resize > move > contenu), chacune avec son
4 // curseur et son affordance. Voir ARCHITECTURE.md §4.
5 enum class NkGuiInteract : uint8 {
6     None = 0,
7     HoverWidget, ///< survol d'un widget
8     EditWidget,  ///< édition active (slider/...input)
9     MoveWindow,  ///< déplacement d'une fenêtre (barre de titre / onglet)
10    ResizeWindow, ///< redimensionnement (bord/coin – cf. edge)
11    DragSplitter, ///< glissement d'un séparateur de dock
12    DragTab,      ///< glissement d'un onglet (réordre / détache)
13    DockTarget    ///< visée d'une cible de dock (boussole)
14 };
```

La règle de conception associée mérite d'être citée, parce qu'elle décrit un défaut d'ergonomie que beaucoup d'interfaces ont :

Kernel/Runtime/NKGui/ARCHITECTURE.md:121-124

```
1 les zones de préhension sont disjointes et priorisées (bord/coin de resize >
2 barre de titre/onglet pour move > contenu), chacune avec son curseur
3 (NkWindow::SetCursor, déjà en place) et son affordance visuelle.→
4 Plus jamais « je voulais redimensionner et ça a docké ».
```

13.8 Le routeur d'occlusion : gérer vos propres surfaces flottantes

NKGui sait gérer ses propres popups. Mais dès que votre application dessine *elle-même* une surface flottante — une boîte de dialogue modale, une palette de commandes, un aperçu au survol — le framework ne peut pas deviner qu'elle est là. Sans précaution, les clics la traversent.

Le problème est décrit précisément dans l'en-tête :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:163-182

```
1 // — ROUTEUR D'OCCCLUSION UNIFIÉ (surfaces flottantes « maison ») ———
2 // PROBLÈME DE FOND résolu ici : Les overlays applicatifs (modals, palettes,
3 // popovers, ...peek) faisaient des hit-tests BRUTS (rect + mouseClicked) qui
4 // ignoraient ce qui est dessiné AU-DESSUS d'eux → traversées de clics,
5 // boutons inertes, consommations manuelles fragiles dupliquées partout.
6 // PRINCIPE : chaque surface flottante déclare son rect + sa couche CHAQUE
7 // frame (PushOcclusion) pendant son dessin ; tous les hit-tests passent par
8 // InputHits/ClickIn qui refusent un point recouvert par une couche PLUS
9 // HAUTE que celle en cours de dessin (curInputLayer). Comme hotIdPrev, la
10 // liste lue est celle de la FRAME PRÉCÉDENTE (stable pour toute la frame,
11 // indépendante de l'ordre de dessin des panneaux). Couches conseillées :
12 // 0 fond/panneaux • 50 menus/palettes/popovers • 100 modals • 200 debug.
```

13.8.1 Les quatre couches

Couche	Contenu
0	fond, panneaux, contenu ordinaire
50	menus, palettes, <i>popovers</i>
100	boîtes de dialogue modales
200	superpositions de débogage

Ce ne sont que des entiers : rien n'interdit d'en intercaler d'autres. Ce qui compte est la relation d'ordre.

13.8.2 L'API

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:185-223 (abrégé)

```
1 void PushOcclusion(const NkRect &r, int32 layer) noexcept; // déclarer une surface
2 bool PointReachable(const NkVec2 &p) const noexcept; // point atteignable ?
3 bool InputHits(const NkRect &r) const noexcept; // hit-test UNIFIE
4 bool ClickIn(const NkRect &r) const noexcept; // clic gauche unifie
5
6 // RAI : fixe la couche d'input pendant le dessin d'une surface flottante.
7 struct NkInputLayerScope {
8     NkGuiContext *c; int32 saved;
9     NkInputLayerScope(NkGuiContext &ctx, int32 layer) : c(&ctx), saved(ctx.curInputLayer)
10     {
11         ctx.curInputLayer = layer;
12     }
13     ~NkInputLayerScope() { c->curInputLayer = saved; }
```

Et la directive, écrite dans le fichier même (:202-203) :

Ce qu'il faut retenir

« Hit-test UNIFIÉ : souris dans *r* ET non recouverte par une couche supérieure. À utiliser PARTOUT à la place de `NkGuiRectContains(mousePos)`. »

Donc : `ctx.InputHits(r)` et `ctx.ClickIn(r)`, jamais un test géométrique brut, sauf pour du chrome de très bas niveau.

13.8.3 L'usage type

Applications/NKCode/src/NKCode/Shell/NkAppCommands.h:137-164 (condensé, idiome des modales)

```
1  const NkRect box = { (W-pw)*0.5f, (Vh-ph)*0.5f, pw, ph };
2
3  ctx.PushOcclusion(box, 100);                                // routeur d'occlusion, couche
   100
4  NkGuiContext::NkInputLayerScope _layer(ctx, 100);          // RAI : mes hit-tests passent
5
6  // MODALITE GLOBALE : popupDepth > 0 est LE signal verifie par les points
7  // d'interaction des panneaux – sans lui les clics TRAVERSENT vers l'editeur.
8  if (ctx.popupDepth == 0) ctx.popupDepth = 1;
9  ctx.popupRects[0] = box;
10 ctx.popupAnchor = box;
```

Le blocage protège ce qui est dessous, jamais ce qui est au-dessus

Cette phrase vient d'un rapport de défaut du dépôt ([Kernel/Runtime/NKGui/ROADMAP.md](#), « Défaut 1 ») et c'est la meilleure formulation du principe :

« ma propre correction du point 3 : en armant le blocage, j'ai neutralisé la liste elle-même, peinte après. D'où la règle qui manquait, à inscrire dans le socle : **le blocage protège ce qui est dessous, jamais ce qui est au-dessus.** »

et la directive qui suit : « Dans un socle correct, l'ordre de peinture définit la priorité d'interaction, sans rien à armer. Ce qui est peint plus tard capte le clic en premier ; le reste en découle. »

Quatre bugs de la même famille sont recensés dans ce document : un menu d'en-tête dont les clics étaient inopérants et qui ne se refermait pas, un panneau qui laissait passer ses clics, une liste déroulante dans laquelle « je ne peux choisir aucun format ». Tous avaient la même cause : un test de survol brut qui ignorait ce qui était peint au-dessus.

Les tests négatifs

Applications/NKCode/src/NKCode/Shell/Panels.h:3461-3465

```
1  // « Clic en dehors du champ -> valider » : test NEGATIF, donc on
2  // exige d'abord que le clic ATTEIGNE notre couche
3  // (PointReachable). Sinon un clic destine a une surface
4  // flottante posee au-dessus (modale, menu) validait le
5  // renommage au passage.
6  else if (mRenameArmed && ctx.input.mouseClicked[0] && !ctx.input.mouseDoubleClicked[0]
   &&
7      ctx.PointReachable(ctx.input.mousePos) &&
8      !NkGuiRectContains(mRenameRect, ctx.input.mousePos)) {
```

Un test *positif* (« le clic est-il dans mon rectangle ? ») échoue naturellement quand quelque chose recouvre. Un test *négatif* (« le clic est-il *en dehors* de mon rectangle ? ») réussit au contraire dans ce cas — et déclenche à tort. Chaque fois que vous écrivez « clic en dehors □ fermer ou valider », commencez par exiger `ctx.PointReachable(mousePos)`.

Ne jamais écraser `ctx.input.mousePos`

C'est probablement le piège le plus coûteux du dépôt, et le commentaire qui le documente est le plus important sur le sujet de l'entrée :

Applications/NKCode/src/NKCode/Shell/NkExplorer.h:89-98 (extrait)

```
1 // /\ NE PAS toucher mousePos : il n'est mis à jour QUE sur un événement
2 // de DÉPLACEMENT (jamais re-sondé) -> L'écraser Le GÈLE pour toute
3 // l'app à la frame suivante (clic sans bouger = position figée). On
4 // neutralise donc SEULEMENT les clics/molette/frappes.
```

La neutralisation correcte est *sélective* :

Applications/NKCode/src/NKCode/Shell/NkExplorer.h:99-107

```
1 if (mPeekOpen || mDelMenu.open) {
2     for (int32 b = 0; b < 3; ++b) {
3         ctx.input.mouseClicked[b] = false;
4         ctx.input.mouseDown[b] = false;
5         ctx.input.mouseDoubleClicked[b] = false;
6     }
7     ctx.input.wheel = ctx.input.wheelH = 0.f;
8     ctx.input.charCount = 0;
9 }
```

Un shell peut se permettre de téléporter la souris hors de l'écran, parce qu'il restaure l'intégralité de input quelques lignes plus loin. Un panneau, non : ce qu'il écrase reste écrasé.

13.9 La liste de dessin

13.9.1 Choisir la bonne couche : ctx.DL()

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:299-303

```
1 NkGuiDrawList &DL() noexcept {
2     if (curPopupLevel >= 0 || overlayDepth > 0)
3         return dloOverlay; // popup / couche overlay forcée
4     return curWindow >= 0 ? winDL[curWindow] : dl;
5 }
```

Trois destinations possibles : la liste de superposition (si l'on dessine un popup), la liste propre à la fenêtre courante, ou la liste principale.

Ce qu'il faut retenir

On écrit toujours dans **ctx.DL()**, jamais dans **ctx.dl** en dur. Un panneau qui écrirait directement dans **ctx.dl** se dessinerait *sous* tout le reste et *sans* découpe — parce qu'il aurait court-circuité à la fois le *pool* de fenêtres et la pile de clip.

Chaque fenêtre a sa propre liste, ce qui rend le tri par profondeur possible :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:225-232

```
1 // — Fenêtres flottantes (Begin/End) —————
2 // Chaque fenêtre dessine dans SA draw-List (pool `winDL`), fusionnées dans
3 // `dl` triées par z-order à EndFrame → recouvrement correct + passage devant.
4 static constexpr int32 WinMax = 32;
```

```
5 NkGuiDrawList winDL[WinMax];
```

13.9.2 Les primitives

Toutes sont implémentées dans `NkGuiDrawList.cpp`.

Primitive	Décl.	Remarque
<code>AddRectFilled(r, col, rounding)</code>	.h :66	coins arrondis inclus
<code>AddRect(r, col, thickness)</code>	.h :67	contour
<code>AddRectFilledMultiColor(r, tl, tr, br, bl)</code>	.h :69	dégradé bilinéaire
<code>AddImage(texId, r, uv0, uv1, tint)</code>	.h :73	quad texturé
<code>AddLine(a, b, col, thickness)</code>	.h :75	
<code>AddTriangleFilled(a, b, c, col)</code>	.h :76	
<code>AddTriangleMultiColor(a, b, c, ca, cb, cc)</code>	.h :78	dégradé barycentrique
<code>AddCircleFilled(center, r, col, segs)</code>	.h :80	segs = 0 → auto (12...128)
<code>AddText(face, texId, baseline, s, col, maxWidth, skew)</code>	.h :87	
<code>AddTextRange(face, texId, baseline, begin, end, col)</code>	.h :91	sous-chaîne

Il n'y a pas de `AddPolyline`, `AddBezier`, `AddArc`, ni `AddCircle` (contour non plein), contrairement à ce qu'annonce [ARCHITECTURE.md:101-102](#).

Rappelez-vous du chapitre 2 : `NkCanvas` n'a ni dégradé ni coins arrondis. Voici donc où ils sont fabriqués. Le rectangle arrondi est quatre arcs de coin, quatre segments chacun, triangulés en éventail depuis le centre du rectangle (`NkGuiDrawList.cpp:114-141`). Le dégradé est simplement quatre couleurs de sommets différentes.

Une bordure n'est pas quatre lignes

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:205-208

```
1 // Bordure = 4 rectangles pleins tracés STRICTEMENT à l'intérieur du rect.
2 // (AddLine centrerait l'épaisseur sur l'arête → la moitié extérieure sort
3 // du rect et est rognée par le clip - scissor exclusif à droite/bas → bord
4 // droit invisible/aminci.) Ici chaque bord tient dans [r.x, r.x+r.w] etc.
```

Symptôme : les bordures droite et basse de vos panneaux sont plus fines que les autres, ou disparaissent complètement. Cause : une épaisseur centrée sur l'arête, dont la moitié extérieure tombe hors de la zone de découpe.

13.9.3 Le découpage

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:34-55 (abrégé)

```
1 NkRect NkGuiDrawList::CurrentClip() const noexcept {
2     return clipDepth > 0 ? clipStack[clipDepth - 1] : NkRect{0.f, 0.f, 1.0e9f, 1.0e9f};
3 }
4 void NkGuiDrawList::PushClipRect(const NkRect &r, bool intersect) noexcept { ... }
5 void NkGuiDrawList::PopClipRect() noexcept { if (clipDepth > 0) --clipDepth; }
```

Pile de 32 niveaux, intersection avec le parent par défaut. Et la règle de découpage des commandes :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:57-75 (extrait)

```
1  if (need) {
2      NkGuiDrawCmd c;
3      c.texId = texId;
4      c.clipRect = clip;
5      c.idxOffset = static_cast<uint32>(idx.Size());
6      c.idxCount = 0;
7      c.type = texId ? NkGuiDrawCmdType::TexturedTriangles : NkGuiDrawCmdType::Triangles;
8      cmds.PushBack(c);
9  }
```

Une nouvelle commande est ouverte **dès que la texture ou le rectangle de découpe change**. Puisque, côté NkCanvas, un changement de découpe force un vidage (chapitre 2), on voit la chaîne de causalité complète : *beaucoup de PushClipRect* → *beaucoup de commandes* → *beaucoup d'appels de dessin*.

Le double filtrage

Le découpage garantit la *correction* visuelle ; il ne dispense pas d'éviter le travail inutile. L'idiome systématique du dépôt combine les deux :

Applications/NKCode/src/NKCode/Shell/NkAppCommands.h:344-354 (abrégé)

```
1  u.dl->PushClipRect(body, true);
2  float32 y = body.y - d->helpScroll;
3  for (int32 i = 0; i < N; ++i) {
4      if (y + rowH >= body.y && y <= body.y + body.h && SC[i].k[0]) { // culling manuel
5          u.Text(body.x, y + u.s(4), SC[i].k, nkcode::NkCol::primary);
6          u.Text(body.x + keyW, y + u.s(4), SC[i].a, nkcode::NkCol::foreground);
7      }
8      y += SC[i].k[0] ? rowH : u.s(10.f);
9  }
10 u.dl->PopClipRect();
```

13.9.4 Le texte, et le piège du flou

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:226-235

```
1  // — ALIGNEMENT D'UN GLYPHE SUR LA GRILLE DE PIXELS —————
2  // Le curseur de texte accumule des avances FRACTIONNAIRES. Sans arrondi,
3  // seul le PREMIER glyphe d'une chaîne tombe sur un pixel entier ; tous les
4  // suivants dérivent et échantillonnent l'atlas ENTRE deux texels, ce qui
5  // rend le texte uniformément flou. Le défaut est d'autant plus marqué que le
6  // corps est petit : l'erreur vaut un demi-texel CONSTANT, soit 4 % de la
7  // hauteur d'un caractère à 13 px et 3,3 % à 15 px.
```

La solution tient en une fonction d'arrondi, mais sa *subtilité* est dans ce qu'elle n'arrondit pas :

Kernel/Runtime/NKGui/src/NKGui/Core/NKGuiDrawList.cpp:258-266

```
1 // On arrondit la POSITION du quad, jamais L'AVANCE : `x` continue
2 // d'accumuler la valeur exacte, donc la largeur totale de la chaîne
3 // est inchangée et MeasureWidth reste d'accord avec le rendu. ...[]
4 // La LARGEUR est reportée telle quelle : arrondir les deux bords
5 // séparément étirerait le glyphe d'un pixel et le rééchantillonnerait,
6 // ce qu'on cherche précisément à éviter.
```

Trois décisions, trois raisons : on arrondit la position (sinon flou), on n'arrondit pas l'avance (sinon la mesure ne correspond plus au rendu), on ne touche pas à la largeur (sinon on rééchantillonne, donc on refloute). C'est un excellent exemple de ce qu'est une correction *bien* faite.

13.9.5 La formule de la ligne de base

AddText prend une **ligne de base**, pas un coin haut-gauche. Les deux conversions canoniques sont :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NKGuiWidgets.cpp:111-118

```
1 float32 TextAt(NKGuiContext &ctx, const NkVec2 &topLeft, const char *s, const NkColor &col)
2     noexcept {
3     if (!ctx.font || !ctx.font->Valid() || !s) return 0.f;
4     // topLeft = coin haut-gauche → ligne de base = y + ascender.
5     const float32 baseY = topLeft.y + ctx.font->Ascent();
6     ctx.DL().AddText(ctx.font->Face(), ctx.font->TexId(), {topLeft.x, baseY}, s, col);
7     return ctx.font->MeasureWidth(s);
8 }
```

Kernel/Runtime/NKGui/src/NKGui/Widgets/NKGuiWidgets.cpp:125-133

```
1 static void DrawCenteredLabel(NKGuiContext &ctx, const NkRect &r, const char *label, const
2     NkColor &col) {
3     const float32 tw = ctx.font->MeasureWidth(label);
4     const float32 tx = r.x + (r.w - tw) * 0.5f;
5     const float32 baseY = r.y + (r.h - ctx.font->LineHeight()) * 0.5f + ctx.font->Ascent();
6     ctx.DL().AddText(ctx.font->Face(), ctx.font->TexId(), {tx, baseY}, label, col, r.w - 6.f);
7 }
```

Deux formules à mémoriser

- Aligné en haut : `baseline.y = y + font->Ascent();`
- Centré dans un rectangle : `baseline.y = r.y + (r.h - font->LineHeight()) * 0.5f + font->Ascent();`

Et la garde qui doit précéder tout appel à AddText ou MeasureWidth : `if (!ctx.font || !ctx.font->Valid()) return;`

Attention : `NKGuiFont::LineHeight()` renvoie `face->lineAdvance`, et **non** `ascent + descent`.

13.10 La mise en page

13.10.1 Le modèle : un curseur qui descend

NkGuiLayout ([NkGuiContext.h:71-95](#)) est un curseur, comme celui d'une machine à écrire : region (la zone disponible), cursor (où l'on est), lineStartX, curLineH, prevItem, maxX/maxY, plus les espacements (padding = 10, itemSpacingX = 8, itemSpacingY = 6).

Le mode le plus simple, le mode vertical, tient en douze lignes :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:303-317

```
1  if (w <= 0.f) w = ContentWidth();
2  const NkRect rect = {layout.cursor.x, layout.cursor.y, w, h};
3  layout.prevItem = rect;
4  if (rect.x + rect.w > layout.maxX) layout.maxX = rect.x + rect.w; // Largeur contenu
5  if (h > layout.curLineH) layout.curLineH = h;
6  layout.cursor.x = layout.lineStartX;
7  layout.cursor.y += layout.curLineH + layout.itemSpacingY;
8  if (layout.cursor.y > layout.maxY) layout.maxY = layout.cursor.y;
9  layout.curLineH = 0.f;
10 return rect;
```

13.10.2 Les sept modes de flux

Le champ layout.flow sélectionne le comportement de NextItemRect ([NkGuiContext.cpp:194-318](#)) :

flow	Mode	Comportement
0	Vertical (défaut)	un élément par ligne, le curseur descend; $w \leq 0$ = remplir la largeur
1	HBox	le curseur avance en X, pas de retour à la ligne; $w \leq 0 \rightarrow 120$ px
2	Grid	colonnes régulières, retour à la ligne tous les gridCols
3	Stack	enfants superposés, ancrés dans une boîte (9 ancrés)
4	Flow	comme HBox mais passe à la ligne quand ça déborde (étiquettes, barres d'outils)
5	Row (flex)	rangée horizontale, largeurs pré-calculées, hauteur étirée
6	Column (flex)	colonne verticale, hauteurs pré-calculées, largeur étirée

On ne règle pas flow à la main : on ouvre un conteneur.

13.10.3 Les helpers de curseur

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:453-466

```
1 void BeginLayout(const NkRect &region) noexcept; // ouvre une region de contenu
2 NkRect NextItemRect(float32 w, float32 h) noexcept; // w<=0 = remplir la largeur
3 void SameLine(float32 spacingX = -1.f) noexcept; // item suivant a droite du precedent
4 void Spacing(float32 px = -1.f) noexcept; // saut vertical
5 float32 ContentWidth() const noexcept; // Largeur restante au curseur
```

```

6 float32 AvailHeight() const noexcept;           // hauteur restante sous le curseur
7 float32 ItemHeight() const noexcept;           // hauteur standard d'un widget
8 float32 S(float32 px) const noexcept;          // px Logiques -> px écran (DPI)
9 void Indent(float32 w) noexcept;               // decale le début de ligne (arbres)

```

ItemHeight() vaut simplement lineHeight + 2 * theme.framePadY, avec un repli de 16 px si aucune police n'est chargée (NkGuiContext.cpp:189-192).

AvailHeight() vaut un milliard dans une zone défilable

Applications/NKCode/src/NKCode/Shell/Panels.h:676-685 (extrait)

```

1 // AvailHeight()/ContentWidth() = taille du CONTENU (scrollable, ~1e9), PAS
2 // la taille visible -> on borne par le rect de CLIP (zone visible du dock)
3 // sinon viewH gigantesque (pas de scrollbar, barre H hors écran).
4 const NkRect clip = ctx.DL().CurrentClip();
5 NkRect r = {ctx.layout.cursor.x, ctx.layout.cursor.y, ctx.ContentWidth(),
6             ctx.AvailHeight()};
7 if (r.x + r.w > clip.x + clip.w) r.w = clip.x + clip.w - r.x;
8 if (r.y + r.h > clip.y + clip.h) r.h = clip.y + clip.h - r.y;

```

Règle : CurrentClip() donne le VISIBLE; ContentWidth() et AvailHeight() donnent le CONTENU — qui, dans une zone défilable, est volontairement gigantesque. Il faut toujours intersecter les deux. Symptôme si on l'oublie : plus de barre de défilement, et une barre horizontale placée à un million de pixels du bord.

13.10.4 Les conteneurs

Ils sauvegardent le layout parent, le remplacent, puis le restaurent en avançant le parent du bloc consommé (NkGuiWidgets.cpp:1322-1365). Ils sont **composables** — un HBox dans une cellule de grille, par exemple.

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:146-189 (signatures)

```

1 void BeginVBox(NkGuiContext &ctx, float32 gap = -1.f) noexcept; void EndVBox(NkGuiContext &) noexcept;
2 void BeginHBox(NkGuiContext &ctx, float32 gap = -1.f) noexcept; void EndHBox(NkGuiContext &) noexcept;
3 void BeginGrid(NkGuiContext &ctx, int32 columns, float32 gap = -1.f) noexcept; void EndGrid(NkGuiContext &) noexcept;
4 void BeginGroup(NkGuiContext &ctx) noexcept; void EndGroup(NkGuiContext &) noexcept;
5 void BeginFlow(NkGuiContext &ctx, float32 gap = -1.f) noexcept; void EndFlow(NkGuiContext &) noexcept;
6 void BeginRow(NkGuiContext &ctx, float32 height, const float32 *sizes, int32 count, float32 gap = -1.f) noexcept;
7 void EndRow(NkGuiContext &ctx) noexcept;
8 void BeginColumn(NkGuiContext &ctx, float32 width, const float32 *sizes, int32 count, float32 totalHeight = -1.f, float32 gap = -1.f) noexcept;
9 void EndColumn(NkGuiContext &ctx) noexcept;
10 void BeginStack(NkGuiContext &ctx, float32 width, float32 height) noexcept;
11 void StackAnchor(NkGuiContext &ctx, int32 anchor) noexcept; // 0=HG 1=HC 2=HD 3=CG 4=C 5=CD 6=BG 7=BC 8=BD
12 void EndStack(NkGuiContext &ctx) noexcept;
13 void Spacer(NkGuiContext &ctx, float32 w, float32 h) noexcept;
14 void SpringRight(NkGuiContext &ctx, float32 width) noexcept;
15 bool Splitter(NkGuiContext &ctx, const char *idStr, const NkRect &handle, bool vertical,

```

```

17         float32 *value, float32 minV, float32 maxV) noexcept;
18 void SetUiScale(NkGuiContext &ctx, float32 s) noexcept;
19 float32 Scaled(NkGuiContext &ctx, float32 px) noexcept;
20 void PushOverlay(NkGuiContext &ctx) noexcept; void PopOverlay(NkGuiContext &) noexcept;

```

Le flex à poids mérite une note :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:159-160

```

1 `sizes[i] > 0` = px fixes ; `sizes[i] < 0` = poids (-1 = poids 1) qui se
2 partagent l'espace RESTANT. Single-pass exact (total connu).

```

13.10.5 Le DPI

SetUiScale(ctx, s) multiplie les marges, l'arrondi et les espacements. Mais attention : l'application doit recharger la police à tailleBase * scale, faute de quoi la mise en page grossit et le texte reste petit. La démo le fait ainsi :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:461-466

```

1     if (pendingScale > 0.f) { // F9 : appliquer la nouvelle echelle DPI
2         SetUiScale(ctx, pendingScale);
3         if (fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f * pendingScale))
4             backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
5             fontPtr->atlasH);
6         pendingScale = 0.f;
7     }

```

Notez où ce code est placé : **avant** BeginFrame. Recharger un atlas au milieu d'une image laisserait des commandes de dessin pointant vers l'ancienne texture.

13.11 Le catalogue des widgets

Voici l'inventaire de ce qui existe réellement. Toutes les signatures viennent de `Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h`; la macro `NKENTSEU_NKGUI_API` est omise. Toutes ont une définition dans le `.cpp` — cela a été vérifié fonction par fonction.

13.11.1 Texte et boutons

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:16-44, 140

```

1 void PanelBackground(NkGuiContext &ctx, const NkRect &r) noexcept; //
   .cpp:106
2 float32 TextAt(NkGuiContext &ctx, const NkVec2 &topLeft, const char *s) noexcept; //
   .cpp:120
3 float32 TextAt(NkGuiContext &ctx, const NkVec2 &topLeft, const char *s, const NkColor &col)
   noexcept;
4 void Text(NkGuiContext &ctx, const char *s) noexcept; //
   .cpp:166
5 void TextWrapped(NkGuiContext &ctx, const char *text, float32 wrapWidth = -1.f) noexcept;
6 bool Button(NkGuiContext &ctx, const char *label, const NkRect &r) noexcept; //
   rect explicite
7 bool Button(NkGuiContext &ctx, const char *label) noexcept; //
   rect auto

```

```

8  bool ButtonEx(NkGuiContext &ctx, const char *label, const NkRect &r, NkGuiButtonFlags
    flags,
9      float32 repeatDelay = -1.f, float32 repeatRate = -1.f) noexcept;
10 bool RepeatButton(NkGuiContext &ctx, const char *label, const NkRect &r,
11     float32 repeatDelay = -1.f, float32 repeatRate = -1.f) noexcept;
12 void Separator(NkGuiContext &ctx) noexcept;

```

Le widget le plus simple du framework, en entier — il résume tout ce que nous avons vu :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:166-172

```

1  void Text(NkGuiContext &ctx, const char *s) noexcept {
2      const float32 lh = (ctx.font && ctx.font->Valid()) ? ctx.font->LineHeight() : 16.f;
3      const float32 w  = (ctx.font && ctx.font->Valid()) ? ctx.font->MeasureWidth(s) : 0.f;
4      const NkRect r = ctx.NextItemRect(w, lh);
5      TextAt(ctx, {r.x, r.y}, s, ctx.IsDisabled() ? ctx.theme.textDisabled : ctx.theme.text);
6  }

```

Mesurer, demander un rectangle au layout, dessiner. Quatre lignes, et la garde sur la police est présente deux fois avec son repli chiffré.

13.11.2 Cases à cocher

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:45-64

```

1  bool Checkbox(NkGuiContext &ctx, const char *label, bool &value) noexcept;
2  bool CheckboxTristate(NkGuiContext &ctx, const char *label, NkGuiCheck &state) noexcept;
3  bool CheckBox3(NkGuiContext &ctx, const char *idStr, NkGuiCheck &state) noexcept; //
    compact, sans libelle
4  void NkGuiTreeCascade(NkGuiCheck *states, const int32 *parent, int32 count, int32 node,
5      NkGuiCheck value) noexcept;
6  void NkGuiTreeRecomputeMixed(NkGuiCheck *states, const int32 *parent, int32 count) noexcept;

```

Les deux dernières fonctions illustrent un choix de conception intéressant :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:55-58

```

1  NKGui ne stocke PAS la hiérarchie (façon ImGui) : l'app possède `states[]`
2  (1 NkGuiCheck par œnœud) + `parent[]` (index du parent, -1 = racine). Ces
3  utilitaires agissent sur ces tableaux quand l'app le décide (ex. Alt+clic).

```

13.11.3 Valeurs numériques

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:65, 112-122

```

1  bool SliderFloat(NkGuiContext &ctx, const char *label, float32 &value, float32 vmin, float32
    vmax) noexcept;
2  bool DragFloat (NkGuiContext &ctx, const char *label, float32 &v, float32 speed = 0.1f,
3      float32 vmin = -1.0e30f, float32 vmax = 1.0e30f,
4      NkGuiDragDir dir = NkGuiDragDir::Horizontal) noexcept;
5  bool DragInt   (NkGuiContext &ctx, const char *label, int32 &v, float32 speed = 0.25f,
6      int32 vmin = -2147483640, int32 vmax = 2147483640,
7      NkGuiDragDir dir = NkGuiDragDir::Horizontal) noexcept;
8  bool InputFloat(NkGuiContext &ctx, const char *label, float32 &v, float32 step = 1.f, ...)
    noexcept;

```



```
9 bool InputInt (NkGuiContext &ctx, const char *label, int32 &v, int32 step = 1, ...) noexcept;
```

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:106-111

```
1 DragFloat/DragInt : GLISSER sur le champ pour changer la valeur (vitesse
2 `speed`/pixel) + DOUBLE-CLIC pour saisir au clavier. InputFloat/InputInt : idem
3 + boutons -/+ (pas `step`). ...[] Pendant le survol/glisser, le widget pose
4 `ctx.wantCursor` ⇨( ou ⇩) que l'app peut appliquer.
```

Ce dernier point est à retenir : le widget *demande* un curseur, il ne le pose pas. C'est à votre boucle d'appliquer `ctx.wantCursor` en appelant `window.SetCursor(...)`. Nous le ferons au chapitre 4.

13.11.4 Saisie de texte

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:126-138

```
1 bool InputText(NkGuiContext &ctx, const char *label, char *buf, int32 bufSize) noexcept;
2 bool InputTextEx(NkGuiContext &ctx, const char *label, char *buf, int32 bufSize,
3                 NkGuiInputFlags flags, int32 maxChars = -1) noexcept;
4 bool InputTextMultiline(NkGuiContext &ctx, const char *idStr, char *buf, int32 bufSize,
5                         const NkRect &rect, NkGuiInputFlags flags = NkGuiInputFlags::None,
6                         int32 maxChars = -1, bool wrap = false) noexcept;
```

Drapeaux disponibles (`NkGuiTypes.h:212-220`) : Password, CharsDecimal, CharsHex, Uppercase, NoBlank, ReadOnly.

Deux distinctions à ne pas rater

- `maxChars` compte en **codepoints**, `bufSize` borne les **octets**. Un caractère accentué occupe deux octets.
- `InputText` renvoie *true* à la **pression d'Entrée**; `InputTextMultiline` renvoie *true* si **le texte a changé**. Ce n'est pas la même sémantique, et confondre les deux donne soit une validation qui n'arrive jamais, soit une action déclenchée à chaque frappe.

13.11.5 Graphiques et couleur

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:71-93

```
1 void ProgressBar(NkGuiContext &ctx, float32 fraction, const char *overlay = nullptr) noexcept;
2 void PlotLines(NkGuiContext &ctx, const char *label, const float32 *values, int32 count,
3               float32 minV = 0.f, float32 maxV = 0.f, float32 height = 0.f) noexcept;
4 void PlotHistogram(NkGuiContext &ctx, const char *label, const float32 *values, int32 count,
5                   float32 minV = 0.f, float32 maxV = 0.f, float32 height = 0.f) noexcept;
6
7 bool ColorButton(NkGuiContext &ctx, const char *idStr, const float32 *col, float32 w = 0.f,
8                 float32 h = 0.f, NkGuiColorFlags flags = NkGuiColorFlags::None) noexcept;
9 bool ColorPicker4(NkGuiContext &ctx, const char *label, float32 *col,
10                  NkGuiColorFlags flags = NkGuiColorFlags::None) noexcept;
11 bool ColorEdit4(NkGuiContext &ctx, const char *label, float32 *col,
12                 NkGuiColorFlags flags = NkGuiColorFlags::None) noexcept;
```

NkGuiColorFlags ([NkGuiTypes.h:151-159](#)) : NoAlpha, Wheel (roue et triangle SV), NoInputs, Disc, Hexagon, Honeycomb. Détail de conception noté dans l'en-tête (:87-88) : « La TEINTE est préservée même au noir/blanc (HSV mémorisé par id) » — d'où les tableaux pickerKeys/pickerHSV du contexte.

13.11.6 Images

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:99-105

```
1 void Image(NkGuiContext &ctx, uint32 texId, float32 w, float32 h,
2           NkColor tint = NkColor{255,255,255,255},
3           NkVec2 uv0 = NkVec2{0.f,0.f}, NkVec2 uv1 = NkVec2{1.f,1.f}) noexcept;
4 bool ImageButton(NkGuiContext &ctx, const char *idStr, uint32 texId, float32 w, float32 h,
5                 NkColor tint = NkColor{255,255,255,255},
6                 NkVec2 uv0 = NkVec2{0.f,0.f}, NkVec2 uv1 = NkVec2{1.f,1.f}) noexcept;
```

Le texId est l'identifiant côté *backend* : au préalable, votre application doit avoir enregistré l'image sous cet identifiant par backend.UploadImageRGBA(texId, ...) (chapitre 4). La teinte multiplie l'échantillon : blanc donne l'image telle quelle, une couleur donne une icône monochrome recolorée.

13.11.7 Arbres, sélection, onglets

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:191-239

```
1 bool CollapsingHeader(NkGuiContext &ctx, const char *label) noexcept;
2 bool TreeNode(NkGuiContext &ctx, const char *label) noexcept;
3 void TreePop(NkGuiContext &ctx) noexcept;
4 bool TreeNodeEditable(NkGuiContext &ctx, const char *idStr, char *label, int32 bufSize,
5                      bool allowRename = true) noexcept;
6 bool Selectable(NkGuiContext &ctx, const char *label, bool selected) noexcept;
7 bool SelectableEditable(NkGuiContext &ctx, const char *idStr, char *label, int32 bufSize,
8                        bool selected, bool allowRename = true) noexcept;
9 bool SelectItem(NkGuiContext &ctx, const char *label) noexcept;
10 bool SelectItemEditable(NkGuiContext &ctx, const char *idStr, char *label, int32 bufSize,
11                       bool allowRename = true) noexcept;
12 int32 TabBar(NkGuiContext &ctx, const char *id, const char *const *labels, int32 count)
13             noexcept;
14 int32 TabBarEx(NkGuiContext &ctx, const char *id, const char *const *labels, int32 count,
15               const bool *enabled) noexcept;
16 int32 TabBarEditable(NkGuiContext &ctx, const char *id, char *const *labels, int32 count,
17                    int32 labelBufSize, const bool *enabled = nullptr,
18                    const bool *allowRename = nullptr) noexcept;
```

Les variantes *Editable sont une signature de NKGui : le renommage en place — par double-clic ou Maj+Entrée — est intégré au socle, pas laissé à l'application. Elles prennent toutes un idStr stable, pour la raison vue à la section 3.2.

13.11.8 Zones défilables et tables

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:315-364

```
1 bool BeginChild(NkGuiContext &ctx, const char *idStr, const NkRect &rect, bool border = true,
2               bool horizontal = false) noexcept;
3 void EndChild(NkGuiContext &ctx) noexcept;
4 bool BeginListBox(NkGuiContext &ctx, const char *idStr, const NkRect &rect) noexcept;
5 void EndListBox(NkGuiContext &ctx) noexcept;
6
7 bool BeginTable(NkGuiContext &ctx, const char *idStr, int32 columns,
8               NkGuiTableFlags flags = NkGuiTableFlags::Borders) noexcept;
9 void TableSetupColumn(NkGuiContext &ctx, const char *label, float32 width = 0.f) noexcept;
10 void TableHeadersRow(NkGuiContext &ctx) noexcept;
11 void TableNextRow(NkGuiContext &ctx, float32 minHeight = 0.f) noexcept;
12 bool TableNextColumn(NkGuiContext &ctx) noexcept;
13 bool TableSetColumnIndex(NkGuiContext &ctx, int32 n) noexcept;
14 void EndTable(NkGuiContext &ctx) noexcept;
15 bool TableGetSortColumn(NkGuiContext &ctx, int32 *outCol, bool *outAscending) noexcept;
16 bool TableCellText(NkGuiContext &ctx, const char *idStr, char *buf, int32 bufSize,
17                  bool editable = false) noexcept;
```

L’idiome complet est donné en commentaire dans l’en-tête :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:323-334

```
1 if (BeginTable(ctx, "t", 3, Borders | RowBg | Resizable)) {
2     TableSetupColumn(ctx, "Nom", 0); // 0 = colonne étirable
3     TableSetupColumn(ctx, "Type", 90); // largeur fixe px
4     TableSetupColumn(ctx, "Taille", 70);
5     TableHeadersRow(ctx);
6     for ...() { TableNextRow(ctx);
7                 TableNextColumn(ctx); Text(ctx, nom);
8                 TableNextColumn(ctx); Text(ctx, type);
9                 TableNextColumn(ctx); Text(ctx, taille); }
10    EndTable(ctx);
11 }
```

NkGuiTableFlags (NkGuiTypes.h:132-140): Borders, RowBg, Resizable, Header, ResizableOuter, Sortable. Limites : NkGuiTableMaxCols = 16, et une seule table active à la fois — « pas d’imbrication v1 » (NkGuiContext.h:397).

13.11.9 Popups, combos, menus, infobulles

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:373-402

```
1 bool BeginCombo(NkGuiContext &ctx, const char *label, const char *preview, int32 itemCount,
2               float32 heightOverride = 0.f) noexcept;
3 void EndCombo(NkGuiContext &ctx) noexcept;
4 void EndPopup(NkGuiContext &ctx) noexcept;
5
6 bool BeginMenuBar(NkGuiContext &ctx, const NkRect &rect) noexcept;
7 void EndMenuBar(NkGuiContext &ctx) noexcept;
8 bool BeginMenu(NkGuiContext &ctx, const char *label) noexcept;
9 void EndMenu(NkGuiContext &ctx) noexcept;
10 bool MenuItem(NkGuiContext &ctx, const char *label, const char *shortcut = nullptr,
11              bool enabled = true) noexcept;
12 bool BeginPopupMenu(NkGuiContext &ctx, const char *idStr) noexcept;
```

```

13 void EndPopupMenu(NkGuiContext &ctx) noexcept;
14
15 void SetTooltip(NkGuiContext &ctx, const char *text) noexcept;

```

Usage de l'infobulle, documenté en :400-401:if (ctx.IsItemHovered()) SetTooltip(ctx, "...");. Pour un menu contextuel, l'application appelle ctx.OpenPopupAt(ctx.GetId(idStr), mousePos) au clic droit, puis dessine entre BeginPopupMenu et EndPopupMenu.

Voici, en passant, comment un menu contourne le décalage d'une image :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:4806-4812

```

1 // Ouverture au PRESS via test geometrique direct : ne depend PAS du
2 // survol resolu (hotIdPrev) de la frame precedente. Sinon un clic sur
3 // un titre jamais survole avant n'ouvrirait pas le menu au 1er coup.
4 // On N'utilise PAS `clicked` (relachement) ici : sinon press ouvrirait
5 // et release refermerait aussitot (double bascule).

```

13.11.10 Le contexte utilisé comme un widget

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:448-543 (sélection)

```

1 void PushId(const char *s); void PushId(const void *p); void PopId();
2 NkGuiId GetId(const char *s) const;
3 void BeginDisabled(bool disabled = true); void EndDisabled(); bool IsDisabled() const;
4 void BeginSelectList(const char *id, bool *mask, int32 count, NkGuiSelectFlags flags);
5 void ApplySelectClick(int32 idx); void EndSelectList();
6 void OpenPopup(NkGuiId); void OpenPopupAt(NkGuiId, NkVec2); void ClosePopup();
7 bool IsPopupOpen(NkGuiId) const;
8 bool IsHovered(const NkRect &r) const; bool IsItemHovered() const;
9 bool ItemHoverable(const NkRect &r, NkGuiId id);
10 NkString GetClipboard() const; void SetClipboard(const char *t);

```

13.12 Panneaux, fenêtres, docking

13.12.1 BeginPanel / EndPanel

Le conteneur le plus simple. Son code entier tient sur une page et vaut d'être lu en entier — il utilise presque tout ce que nous avons vu :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:2047-2078

```

1 bool BeginPanel(NkGuiContext &ctx, const char *title, const NkRect &r) noexcept {
2     ctx.DL().AddRectFilled(r, ctx.theme.panel, ctx.theme.rounding); // fond
3     float32 top = r.y;
4     if (title && *title) {
5         const float32 th = ctx.ItemHeight();
6         ctx.DL().AddRectFilled({r.x, r.y, r.w, th}, ctx.theme.header, ctx.theme.rounding);
7         ctx.DL().AddRectFilled({r.x, r.y + th - 1.f, r.w, 1.f}, ctx.theme.border); //
        séparateur
8         if (ctx.font && ctx.font->Valid()) {
9             ctx.DL().AddText(ctx.font->Face(), ctx.font->TexId(),
10                             {r.x + 10.f, r.y + (th - ctx.font->LineHeight()) * 0.5f +
11                             ctx.font->Ascent()},
12                             title, ctx.theme.text);
12     }

```

```

13     top = r.y + th;
14 }
15 // Bordure EN DERNIER → entoure tout le panneau (en-tête inclus), n'est
16 // plus recouverte par le fond de la barre de titre.
17 ctx.DL().AddRect(r, ctx.theme.border, 1.f);
18 // Le contenu (sous l'en-tête) est une ZONE DÉFILABLE : si les widgets
19 // dépassent la hauteur visible, une scrollbar apparaît ...automatiquement
20 const NkRect content = {r.x, top, r.w, r.y + r.h - top};
21 const NkGuiId id = ctx.GetId((title && *title) ? title : "##panel");
22 return BeginScrollFrame(ctx, id, content, false);
23 }
24
25 void EndPanel(NkGuiContext &ctx) noexcept { EndScrollFrame(ctx); }

```

Trois choses à en retirer : la formule de la ligne de base est exactement celle de la section 3.7; la bordure est dessinée *en dernier* pour ne pas être recouverte; et le contenu est automatiquement une zone défilable.

13.12.2 Begin / EndWindow

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:253-263

```

1 // Appeler End UNIQUEMENT si Begin retourne true (convention NKGui, comme
2 // BeginPanel).
3 // if (Begin(ctx, "Inspecteur", &open)) { .....widgets EndWindow(ctx); }
4 void SetNextWindowPos(NkGuiContext &ctx, float32 x, float32 y) noexcept;
5 void SetNextWindowSize(NkGuiContext &ctx, float32 w, float32 h) noexcept;
6 bool Begin(NkGuiContext &ctx, const char *title, bool *open = nullptr,
7            NkGuiWindowFlags flags = NkGuiWindowFlags::None) noexcept;
8 void EndWindow(NkGuiContext &ctx) noexcept;

```

La convention Begin/End de NKGui

On appelle **End*** uniquement si **Begin*** a retourné **true**. C'est vrai pour **Begin**, **BeginPanel**, **BeginCombo**, **BeginTable**, **BeginChild**, **BeginMenu**, **BeginPopupMenu**. Les conteneurs de layout (**BeginVBox**, **BeginHBox**...) ne retournent rien : ceux-là s'apparentent toujours.

NkGuiWindowFlags (**NkGuiTypes.h:245-252**) : **NoResize**, **NoMove**, **NoCollapse**, **NoTitleBar**, **NoClose**.

À la création, une fenêtre reçoit un décalage en cascade pour ne pas se superposer aux précédentes :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:2286-2288

```

1 const float32 off = static_cast<float32>(ctx.windowMeta.Size() % 8) * 28.f;
2 nm.rect = {90.f + off, 80.f + off, 320.f, 210.f};

```

SetNextWindowPos et **SetNextWindowSize** ne s'appliquent qu'à la création — c'est la sémantique «la première fois seulement» — et sont consommés aussitôt (**ctx.hasNextPos** = **ctx.hasNextSize** = **false**;, **cpp:2293**). Vous ne pouvez donc pas vous en servir pour forcer une position à chaque image; c'est délibéré, sans quoi l'utilisateur ne pourrait jamais déplacer la fenêtre.

Interactions avant dessin

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:2396-2397

```
1 // — PHASE 1 : INTERACTIONS — mettent à jour `wr` AVANT Le dessin (sinon le
2 //   contenu, dessiné à la nouvelle position, « court après » Le chrome). —
```

Si vous écrivez vous-même une surface déplaçable, traitez le glissement *avant* de dessiner, sinon le contenu accuse une image de retard sur le cadre et l'ensemble semble élastique.

13.12.3 Le docking

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:271-306

```
1 void DockSpace(NkGuiContext &ctx, const char *idStr, const NkRect &rect) noexcept;
2 void DockBuilderDock(NkGuiContext &ctx, const char *windowTitle, int32 zone) noexcept;
3 void DockBuilderDockTab(NkGuiContext &ctx, const char *windowTitle, const char *targetTitle)
   noexcept;
4 bool DockFocusWindow(NkGuiContext &ctx, const char *windowTitle) noexcept;
5 bool DockIsWindowDocked(NkGuiContext &ctx, const char *windowTitle) noexcept;
6 int32 DockWindowNode(NkGuiContext &ctx, const char *windowTitle) noexcept;
7 void DockDetachWindow(NkGuiContext &ctx, const char *windowTitle) noexcept;
8 void DockWindowHideSingleTab(NkGuiContext &ctx, const char *windowTitle, bool hide) noexcept;
9 void DockSpaceOverViewport(NkGuiContext &ctx, float32 topMargin = 0.f) noexcept;
10 void DockWindowIntoWindow(NkGuiContext &ctx, const char *hostTitle, const char *winTitle)
   noexcept;
11 int32 DockTabAddRequest(NkGuiContext &ctx) noexcept;
12 void DockAddTab(NkGuiContext &ctx, const char *windowTitle, int32 node) noexcept;
```

Les zones de DockBuilderDock (NkGuiWidgets.h:273) : «0 = onglet, 1 = gauche, 2 = droite, 3 = haut, 4 = bas (sur la 1re feuille)».

DockSpace avant les Begin

«À appeler AVANT les Begin des fenêtres dockables» (NkGuiWidgets.h:266-267). L'espace de docking calcule les rectangles que les fenêtres ancrées viendront occuper; si les fenêtres sont déclarées d'abord, elles utilisent les rectangles de l'image précédente.

L'arbre de dock est un arbre binaire de nœuds (NkGuiDockNode, NkGuiTypes.h:288-299) : kind (0 = vide, 1 = division, 2 = feuille), vertical, ratio, child0/child1/parent, windows[8], activeTab, rect. Huit onglets au maximum par feuille.

13.13 Le hook de style

Dernier mécanisme, et le plus élégant : re-styler entièrement l'interface sans toucher à la logique des widgets.

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiTypes.h:301-325

```
1 // — Hook de style : override de DESSIN par widget —————
2 // L'app enregistre `ctx.styleFn` ; pour chaque élément visuel, NKGui l'appelle
3 // AVANT Le rendu par défaut. Si Le callback retourne true, il a dessiné lui-même
4 // (Le défaut est sauté) → re-skin TOTAL sans toucher la logique du widget.
5 enum class NkGuiStyleKind : uint8 {
6     Button = 0, FrameBg, CheckMark, Header, Selectable, DockTarget, Count
```

```

7  };
8
9  struct NkGuiStyleItem {
10      NkGuiStyleKind kind = NkGuiStyleKind::Button;
11      NkRect rect = {0.f, 0.f, 0.f, 0.f};
12      NkGuiId id = NKGUI_ID_NONE;
13      const char *label = nullptr;
14      bool hovered = false;
15      bool active = false;    ///< pressé / en cours d'édition / cible visée
16      bool selected = false;
17      bool disabled = false;
18      int32 value = 0;        ///< donnée par type (DockTarget :
                             0=onglet, 1=G, 2=D, 3=H, 4=B, 5=bord)
19  };

```

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:429-431

```

1  using NkGuiStyleFn = bool (*)(NkGuiContext &, const NkGuiStyleItem &, void *);
2  NkGuiStyleFn styleFn = nullptr;
3  void *styleUser = nullptr;

```

Le contrat est simple : votre fonction reçoit la description de l'élément à dessiner; si elle retourne true, elle a dessiné et NKGui saute son rendu par défaut; si elle retourne false, le rendu par défaut a lieu. Vous pouvez donc ne surcharger que les boutons et laisser le reste inchangé. La démo en donne un exemple complet, DemoStyle ([Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:193-238](#), branché ligne 271).

Ce qu'il faut retenir

Pour changer les *couleurs*, on écrase ctx.theme. Pour changer la *forme* — un bouton en biseau, une case à cocher dessinée comme un interrupteur — on branche ctx.styleFn. On ne modifie jamais NkGuiWidgets.cpp.

13.14 Les limites dures

Toutes ces constantes sont des tailles de tableaux fixes, décidées à la compilation. Les dépasser ne plante pas : le framework ignore silencieusement ce qui excède.

Limite	Valeur	Fichier :ligne
Popups imbriqués	8	NkGuiContext.h:147
Surfaces d'occlusion	16	NkGuiContext.h:175
Fenêtres	32	NkGuiContext.h:228
Zones défilables imbriquées	8	NkGuiContext.h:379
Conteneurs de layout	32	NkGuiContext.h:385
Pile d'identifiants	32	NkGuiContext.h:324
Pile « désactivé »	16	NkGuiContext.h:328
Colonnes de table	16	NkGuiContext.h:120
Onglets par feuille de dock	8	NkGuiTypes.h:295
Caractères saisis par image	32	NkGuiInput.h:53
Pile de découpe (draw-list)	32	NkGuiDrawList.h:52

13.15 Récapitulatif des idiomes

1. Récupérer la liste de dessin par `ctx.DL()`, jamais `ctx.dl` en dur.
2. Convertir un coin haut-gauche en ligne de base par `y + font->Ascent()`; centrer par `r.y + (r.h - font->LineHeight()) * 0.5f + font->Ascent()`.
3. Toujours vérifier `if (!ctx.font || !ctx.font->Valid())` avant `AddText` ou `MeasureWidth`, avec un repli chiffré.
4. `PushClipRect(zone, true) ...PopClipRect()` — toujours appariés, toujours avec intersection.
5. Ajouter un filtrage manuel du hors-vue en plus de la découpe.
6. Molette : `if (Hit(zone) && ctx.input.wheel != 0.f) { scroll -= wheel * pas; ctx.input.wheel = 0.f; }` puis bornage.
7. Glissement : poser `ctx.activeId = monId` à l'appui, agir tant que `ctx.activeId == monId && mouseDown[0]`, remettre `ctx.activeId = 0` au relâchement.
8. Préférer `ctx.InputHits(r)` et `ctx.ClickIn(r)` à un test géométrique brut.
9. Surface flottante : `ctx.PushOcclusion(rect, couche) + NkInputLayerScope`.
10. Toute taille en dur passe par `ctx.S(px)`.
11. Identifiant stable, distinct du libellé affiché; `PushId/PopId` autour des éléments d'une liste.
12. Lire les couleurs dans `ctx.theme` et `ctx.syntax`, jamais de littéral RGBA dans le contenu.
13. Ne jamais recharger un atlas de police pendant une image : le différer avant `BeginFrame`.
14. Différer toute mutation d'état lourde (ouverture de fichier, reconstruction d'arbre, accès disque) hors du rendu : lever un drapeau, agir à l'image suivante.

Ce chapitre en bref

En mode immédiat, un widget n'existe pas : il est *redessiné* à chaque image, et ce qui doit survivre vit dans vos propres variables. L'identité d'un widget vient de son **identifiant**, pas de sa position — deux widgets qui partagent un identifiant partagent leur état, et le symptôme n'y ressemble pas. Le **roulage de l'entrée** est ce qui sépare NKGui d'un simple dessin : le z-ordre décide qui reçoit un clic, et une modale doit *consommer* ce qu'elle recouvre, faute de quoi le clic traverse.

13.16 Exercices

1 — Le compteur

Écrivez, dans la boucle de la démo, un petit panneau contenant : un Text affichant une valeur entière, un bouton « + » et un bouton « - ». La valeur est une variable locale de votre boucle. Vérifiez que le compteur ne s'incrémente qu'au *relâchement* du bouton, et qu'appuyer puis glisser en dehors avant de relâcher n'incrémente pas. Remplacez ensuite Button par RepeatButton et observez la différence.

2 — Le piège des identifiants

Affichez une liste de cinq éléments, chacun suivi d'un bouton « Supprimer ». Version A : sans `PushId`. Version B : avec `PushId(&items[i])` autour de chaque ligne. Observez ce qui se passe en version A quand vous survolez et cliquez. Expliquez le comportement à partir de `GetId` et de `ItemHoverable`.

3 — Comprendre le z-ordre

Dessinez deux boutons qui se chevauchent partiellement, le second déclaré après le premier. Cliquez dans la zone de recouvrement : lequel réagit ? Inversez l'ordre de déclaration et recommencez. Ajoutez ensuite un `logger.Info` qui affiche `ctx.hotId` et `ctx.hotIdPrev` à chaque image, et observez le décalage d'une image en déplaçant lentement la souris d'un bouton à l'autre.

4 — Une modale correcte

Écrivez une boîte de dialogue modale maison : un rectangle centré dessiné dans `ctx.dlOverlay` (via `PushOverlay/PopOverlay`), avec un fond assombri sur toute la fenêtre et deux boutons « Valider » et « Annuler ». Appliquez les gestes de la section 3.6 : `PushOcclusion(box, 100)`, `NkInputLayerScope`, et la fermeture au clic en dehors — en n'oubliant pas d'exiger `PointReachable` pour ce test négatif. Vérifiez qu'un clic sur la modale n'atteint pas les widgets qui sont dessous.

Questions de cours

1. Que veut dire « mode immédiat », et où vit l'état d'un widget ?
2. D'où vient l'identité d'un widget ? Que se passe-t-il si deux widgets la partagent ?
3. Qui décide quel widget reçoit un clic quand deux se recouvrent ?
4. Pourquoi une modale doit-elle consommer l'entrée qu'elle recouvre ?
5. Pourquoi `NKGui` n'ouvre-t-il jamais de fenêtre lui-même ?

Construire une application, de zéro à l'exécutable

Nous avons vu la pile (chapitre 1), le rendu (chapitre 2) et les widgets (chapitre 3). Il reste à tout assembler. Ce chapitre construit une application complète, dans l'ordre, en expliquant à chaque étape *pourquoi* elle est là et *ce qui casse* si on l'oublie.

À la fin, vous aurez un programme qui compile, se lance, affiche une interface et répond à la souris et au clavier.

14.1 Vue d'ensemble du squelette

Avant le détail, voici la carte. Une application NKGui sur NkCanvas se construit en sept étapes, puis boucle.

Les sept étapes de l'initialisation

```

1 1. FENETRE           NkWindow + NkWindowConfig
2 2. CONTEXTE GPU      NkContextDesc (choix de l'API graphique)
3 3. CIBLE DE RENDU    NkRenderWindow (possede le cycle de frame)
4 4. CONTEXTE UI       NkGuiContext::Init + SetCurrentContext
5 5. BACKEND (PONT)    NkGuiCanvasBackend::Init(target->GetRenderer())
6 6. POLICE            NkGuiFont::LoadEmbedded PUIS backend.UploadFontGray8
7 7. ENTREE            callbacks NKEvent -> ctx.input
8
9 BOUCLE :
10  evenements -> [resize] -> ctx.BeginFrame(dt) -> widgets -> ctx.EndFrame()
11  -> target->Clear() -> backend.Submit(ctx.dl) -> backend.Submit(ctx.dlOverlay)
12  -> target->Display()
```

L'ordre n'est pas indifférent. Le pont a besoin du renderer, donc de la cible. L'upload de l'atlas a besoin du pont *et* de la police chargée. Les *callbacks* ont besoin du contexte, puisqu'ils écrivent dedans.

14.2 Étape 1 — la fenêtre

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:243-251

```

1  NkWindow window;
2  NkWindowConfig cfg;
3  cfg.title = "NKGui - Demo Phase 2 (Coeur : drawlist + interaction)";
4  cfg.width = 1100;
5  cfg.height = 800;
6  cfg.centered = true;
7  cfg.resizable = true;
8  if (!window.Create(cfg))
9      return -1;
```

Rien de subtil, mais notez le test de retour : Create renvoie `false` si la fenêtre n'a pas pu être créée, et il n'y a pas d'exception à attraper. Chaque étape de cette initialisation suit le même modèle — retour booléen ou méthode `IsValid()` — et chacune doit être testée.

Les en-têtes nécessaires :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:7-11 (extrait)

```
1 #include "NKWindow/NKWindow.h"
2 #include "NKWindow/NKMain.h"
3 #include "NKEvent/NkWindowEvent.h"
4 #include "NKEvent/NkMouseEvent.h"
5 #include "NKEvent/NkKeyboardEvent.h"
```

14.3 Étape 2 et 3 — contexte GPU et cible de rendu

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:253-265

```
1     NkContextDesc desc;
2     desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
3     if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
4 #if defined(NKENTSEU_PLATFORM_WINDOWS)
5         desc.api = NkGraphicsApi::NK_GFX_API_DX11;
6 #else
7         desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
8 #endif
9     }
10    auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
11    if (!target || !target->IsValid())
12        return -1;
```

Trois observations :

- `NK_GFX_API_AUTO` est résolu ici, à la main, par une directive de préprocesseur. On aurait pu utiliser `NkContextFactory::CreateWithFallback` et laisser le moteur essayer une liste ordonnée; la démo préfère être explicite.
- La cible est créée par `memory::NkMakeUnique` et vivra jusqu'à la fin de `nkmain`. Elle possède le contexte graphique *et* le renderer 2D.
- Le double test `!target || !target->IsValid()` couvre les deux modes d'échec : l'allocation, et l'initialisation GPU.

Les en-têtes correspondants :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:12-16 (extrait)

```
1 #include "NKCanvas/Core/NkContextDesc.h"
2 #include "NKCanvas/Core/NkGraphicsApi.h"
3 #include "NKCanvas/Renderer/Targets/NkRenderWindow.h"
4 #include "NKTime/NkClock.h"
5 #include "NKMemory/NkUniquePtr.h"
```

Ce qu'il faut retenir

Si votre fenêtre reste noire alors que le programme tourne, changez d'API graphique avant de chercher dans votre code. Rappel du chapitre 1 : seul OpenGL était validé à l'exécution au 30 mai 2026 selon [Kernel/Runtime/NKCanvas/ROADMAP.md](#), et DX11 comme le rasteriseur logiciel affichaient un écran vide. Le journal ([logs/app.log](#)) vous dit quel backend a réellement été retenu.

14.4 Étape 4 — le contexte NKGui

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:266-272

```
1 auto ctxPtr = memory::NkMakeUnique<NKGuiContext>();
2 if (!ctxPtr)
3     return -1;
4 NKGuiContext &ctx = *ctxPtr;
5 ctx.Init(static_cast<int32>(cfg.width), static_cast<int32>(cfg.height));
6 ctx.styleFn = &DemoStyle; // hook de style (override de dessin par widget)
7 SetCurrentContext(&ctx);
```

Le contexte est volumineux — plus de cent champs et trente-deux listes de dessin de fenêtres — ce qui explique qu'on l'alloue plutôt que de le poser sur la pile. On prend ensuite une référence, ce qui rend tout le code des widgets lisible.

`Init(width, height)` pose la taille de vue initiale et prépare les structures. `SetCurrentContext` installe le contexte courant du fil d'exécution; la ligne `ctx.styleFn` est facultative.

C'est ici, juste après `Init`, que vous personnaliserez le thème :

Personnalisation du thème — écrit pour ce cours, d'après `NkEditorShell.cpp:155-179`

```
1 ctx.Init(1100, 800);
2 ctx.theme.bgPrimary = {13, 17, 23, 255};
3 ctx.theme.panel     = {22, 27, 34, 255};
4 ctx.theme.accent     = {88, 166, 255, 255};
5 ctx.theme.rounding   = 6.f;
6 SetCurrentContext(&ctx);
```

14.5 Étape 5 — le pont vers NKCanvas

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:274-276

```
1 renderer::NKGuiCanvasBackend backend;
2 if (!backend.Init(target->GetRenderer()))
3     return -1;
```

Deux lignes, et c'est tout le mariage entre les deux modules. L'en-tête à inclure est celui qui vit dans `NKCanvas` :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:25

```
1 #include "NKCanvas/UI/NKGuiCanvasBackend.h" // backend NKGui->NKCanvas (lib réutilisable)
```

Rappelez-vous du chapitre 1 : ce fichier est *header-only*. Il n'est compilé nulle part ailleurs que dans votre propre unité de traduction. C'est pour cela que votre .jenga doit dépendre à la fois de NKGui et de NKCanvas.

backend est déclaré sur la pile de nkmain. Son destructeur libère les textures qu'il a créées :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:24-32

```
1 ~NkGuiCanvasBackend() {
2     auto &alloc = nkentseu::memory::NkGetDefaultAllocator();
3     for (uint32 i = 0; i < mFonts.Size(); ++i) if (mFonts[i].tex)
4         alloc.Delete(mFonts[i].tex);
5     for (uint32 i = 0; i < mImages.Size(); ++i) if (mImages[i].tex)
6         alloc.Delete(mImages[i].tex);
7 }
```

L'ordre de destruction

Le pont détruit des textures GPU, donc il doit mourir *avant* le renderer qui les a créées. Comme backend est déclaré *après* target dans nkmain, il est détruit *avant* — les objets locaux se détruisent dans l'ordre inverse de leur déclaration. Cet ordre est correct par construction; inverser les deux déclarations produirait une libération de textures sur un renderer déjà mort.

14.6 Étape 6 — la police, et l'upload de son atlas

C'est l'étape où l'on se trompe le plus souvent, parce qu'elle est en deux temps et que rien ne signale l'oubli du second.

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:281-290

```
1 auto fontPtr = memory::NkMakeUnique<NkGuiFont>();
2 if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f)) {
3     fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
4 }
5 ctx.font = fontPtr.Get();
6 if (fontPtr->Valid()) {
7     backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
8     fontPtr->atlasH);
9 }
```

14.6.1 Ce que fait LoadEmbedded

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:19-40 (abrégé)

```
1 struct NKENTSEU_NKGUI_CLASS_EXPORT NkGuiFont {
2     NkFontAtlas atlas;          ///< possède la texture + les glyphes
3     NkFont *face = nullptr;     ///< face produite (détenue par l'atlas)
4     uint32 texId = 0x4E4B4654u; ///< 'NKFT' - id stable pour le backend
5     uint8 *pixels = nullptr;    ///< atlas alpha8 (détenu par l'atlas)
6     int32 atlasW = 0;
7     int32 atlasH = 0;
8     bool dirty = false;        ///< à (ré)uploader côté backend
9
10    NkGuiFont() = default;
11    NkGuiFont(const NkGuiFont &) = delete;           // NON COPIABLE
```

```

12     NkGuiFont &operator=(const NkGuiFont &) = delete;
13
14     bool LoadEmbedded(NkEmbeddedFontId id, float32 sizePx, bool extFallback = true)
        noexcept;
15     bool LoadFromFile(const char *path, float32 sizePx, bool extFallback = true) noexcept;

```

L'appel rasterise les glyphes dans un atlas et laisse pixels pointer sur un bitmap en niveaux de gris (un octet par pixel, l'alpha). **Cet atlas est en mémoire centrale ; le GPU ne le connaît pas encore.**

Les polices embarquées disponibles (`Kernel/Runtime/NKFont/src/NKFont/Embedded/NkFontEmbedded.h:49-67`) : ProggyClean, ProggyTiny (bitmap monospace), DroidSans, Karla, Roboto (sans-serif), Cousine, SourceCodePro (monospace vectorielles), DroidSerif, DejaVuSansMono.

14.6.2 Ce que fait UploadFontGray8

Le pont convertit l'atlas en RGBA — blanc partout, l'alpha portant la forme du glyphe — et crée la texture GPU :

`Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:47-55`

```

1  const usize n = (usize)w * (usize)h;
2  mExpand.Resize(n * 4u);
3  uint8 *d = mExpand.Data();
4  for (usize i = 0; i < n; ++i) {
5      d[i*4+0] = 255u; d[i*4+1] = 255u; d[i*4+2] = 255u; d[i*4+3] = gray[i];
6  }

```

C'est là que le `texId` prend son sens : le pont range la texture dans une table indexée par cet identifiant, et `Submit` la retrouvera quand une commande de dessin portera le même. La constante par défaut est `0x4E4B4654` — les quatre lettres «NKFT». Elle n'est **jamais nulle**, car zéro est réservé à la texture blanche interne côté RHI.

Oublier l'upload : du texte parfaitement invisible

Si vous chargez la police et posez `ctx.font` mais oubliez `UploadFontGray8`, il ne se passe *rien* de visible et *aucun* message n'apparaît. Les widgets se dessinent (leurs rectangles sont là), le layout est correct — mais chaque commande de texte référence un `texId` que le pont ne connaît pas, et elle est silencieusement ignorée.

Symptôme : des boutons vides, des cases à cocher sans étiquette, un panneau au titre absent. Diagnostic : vérifiez que `fontPtr->Valid()` est vrai, et que l'upload est bien appelé après le chargement.

La police ne doit jamais mourir avant la boucle

`Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:141-142 (sens)`

```

1  NkGuiFont *font = nullptr;    ///< pointeur BRUT, NON POSSEDE
2  NkGuiFont *codeFont = nullptr; ///< idem

```

`ctx.font = fontPtr.Get()` donne au contexte un pointeur *brut*. Le contexte ne possède rien. Si l'objet `NkGuiFont` est détruit — parce qu'il était local à une fonction d'initialisation, par exemple — le contexte pointe sur de la mémoire libérée et la première mesure de texte plante.

La règle : l'objet police doit vivre au moins aussi longtemps que la boucle. La démo le tient dans un `NkUniquePtr` déclaré dans `nkmain`, à côté de tout le reste.

Recharger un atlas à une autre taille

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:65-66

```
1 // (Re)créer la texture si elle n'existe pas OU si la taille change (rechargement
2 // de police à une autre taille = zoom / DPI). Sinon Update() déborderait l'ancienne
   taille.
```

Le pont gère ce cas pour vous. Mais **vous** devez veiller à ne jamais recharger une police au milieu d'une image : les commandes déjà émises référenceraient un atlas remplacé, et leurs coordonnées de texture ne correspondraient plus. Faites-le avant `BeginFrame`, comme la démo le fait pour la touche F9.

14.6.3 Les polices de repli

Pour les alphabets que la police principale ne couvre pas :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:71-78

```
1 // Polices de REPLI EXTERNES (fichiers .ttf charges au runtime) : tout glyphe
2 // absent des polices principales (Inter/DejaVu) y est cherché. Rôles :
3 //   broad : large couverture (Latin étendu, grec, cyrillique, hébreu, arabe, symboles)
4 //   cjk   : ideogrammes (chinois/japonais/coreen) - volumineux, opt-in
5 //   emoji : emoji monochrome
6 // A poser par L'APPLICATION (ex. NKCode, depuis son dossier data/fonts) AVANT
7 // de charger les polices. Un chemin nullptr/vidé = rôle désactivé.
8 void NkSetFallbackFontPaths(const char *broad, const char *cjk, const char *emoji) noexcept;
```

Avant de charger la police, donc. C'est un invariant d'ordre : l'appel place des chemins dans des variables globales que la construction de l'atlas consultera.

Pour une police monospace destinée au code, on désactive au contraire les replis (`extFallback = false`) : « atlas minuscule => reconstruction RAPIDE au zoom » (`NkGuiFont.h:34-35`).

14.7 Étape 7 — brancher l'entrée

Les *callbacks* écrivent directement dans `ctx.input`. Voici l'ensemble minimal, puis les compléments.

14.7.1 Souris et fermeture

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:326-343 (extrait)

```
1 auto &events = NkEvents();
2 bool running = true;
3 events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
   });
4
5 events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
```

```

6      ctx.input.mousePos = {static_cast<float32>(e->GetX()),
static_cast<float32>(e->GetY())};
7  });
8  events.AddEventCallback<NkMouseButtonPressEvent>([&](NkMouseButtonPressEvent *e) {
9      if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = true;
10     if (e->GetButton() == NkMouseButton::NK_MB_RIGHT) ctx.input.mouseDown[1] = true;
11     ctx.input.ctrlDown = e->GetModifiers().ctrl;
12     ctx.input.shiftDown = e->GetModifiers().shift;
13     ctx.input.altDown = e->GetModifiers().alt;
14 });
15 events.AddEventCallback<NkMouseButtonReleaseEvent>([&](NkMouseButtonReleaseEvent *e) {
16     if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = false;
17     if (e->GetButton() == NkMouseButton::NK_MB_RIGHT) ctx.input.mouseDown[1] = false;
18 });

```

Indices des boutons : 0 = gauche, 1 = droit, 2 = milieu.

14.7.2 La molette

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:344-352 (extrait)

```

1      events.AddEventCallback<NkMouseWheelVerticalEvent>([&](NkMouseWheelVerticalEvent *e) {
2          ctx.input.wheel += static_cast<float32>(e->GetDeltaY()); // >0 = haut ; consommé en
EndFrame
3          const auto m = e->GetModifiers(); // modificateurs au moment
de la molette
4          ctx.input.ctrlDown = m.ctrl; ctx.input.shiftDown = m.shift; ctx.input.altDown = m.alt;
5      });
6      events.AddEventCallback<NkMouseWheelHorizontalEvent>([&](NkMouseWheelHorizontalEvent *e) {
7          ctx.input.wheelH += static_cast<float32>(e->GetDeltaX());
8      });

```

Notez le += et non le = : plusieurs crans peuvent arriver dans la même image, ils s'accumulent. EndFrame remet le compteur à zéro. Notez aussi la relecture des modificateurs : sans elle, le Ctrl+molette (zoom) ne fonctionnerait pas de façon fiable.

14.7.3 Le double-clic

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:353-359 (extrait)

```

1      // Double-clic détecté par L'OS : NKeyEvent L'émet comme événement DÉDIÉ (le 2e
2      // clic n'arrive PAS comme un mouseDown normal) → on l'injecte explicitement.
3      events.AddEventCallback<NkMouseDoubleClickEvent>([&](NkMouseDoubleClickEvent *e) {
4          ctx.input.mousePos = {static_cast<float32>(e->GetX()),
static_cast<float32>(e->GetY())};
5          if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.SetDoubleClick(0);
6      });

```

Sans ce *callback*, le second clic d'un double-clic n'existe tout simplement pas pour NKGui : le système l'a converti en événement dédié. Les renommages en place et la saisie au clavier dans les DragFloat, qui reposent sur le double-clic, ne fonctionneraient pas.

14.7.4 Le texte et les touches

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:360-380 (extrait)

```
1 // Saisie texte + touches d'édition (pour InputText). On pose l'état ENFONCÉ
2 // (press/release) pour permettre la répétition au maintien.
3 events.AddEventCallback<NkTextInputEvent>([&](NkTextInputEvent *e) {
4     ctx.input.PushChar(e->GetCodepoint()); });
5
6 auto setKey = [&](NkKey k, bool down) {
7     switch (k) {
8         case NkKey::NK_LEFT:   ctx.input.SetKey(NkGuiKey::Left, down);   break;
9         case NkKey::NK_RIGHT:  ctx.input.SetKey(NkGuiKey::Right, down);  break;
10        case NkKey::NK_UP:      ctx.input.SetKey(NkGuiKey::Up, down);      break;
11        case NkKey::NK_DOWN:    ctx.input.SetKey(NkGuiKey::Down, down);    break;
12        case NkKey::NK_HOME:    ctx.input.SetKey(NkGuiKey::Home, down);    break;
13        case NkKey::NK_END:     ctx.input.SetKey(NkGuiKey::End, down);     break;
14        case NkKey::NK_BACK:    ctx.input.SetKey(NkGuiKey::Backspace, down); break;
15        case NkKey::NK_DELETE:  ctx.input.SetKey(NkGuiKey::Delete, down);  break;
16        case NkKey::NK_ENTER:   ctx.input.SetKey(NkGuiKey::Enter, down);   break;
17        case NkKey::NK_ESCAPE:  ctx.input.SetKey(NkGuiKey::Escape, down);  break;
18        default: break;
19    }
20 };
21 events.AddEventCallback<NkKeyPressEvent>([&](NkKeyPressEvent *e) { setKey(e->GetKey(),
22     true); ... });
23 events.AddEventCallback<NkKeyReleaseEvent>([&](NkKeyReleaseEvent *e){ setKey(e->GetKey(),
24     false); ... });
```

Deux points essentiels, déjà annoncés au chapitre 1 :

- on pose l'état **enfoncé/relâché**, pas un « appui » ponctuel. C'est ce qui permet à NKGui de calculer les durées et donc la répétition au maintien;
- le **texte** passe par PushChar et les **touches** par SetKey. Ce sont deux canaux séparés.

Il reste à brancher les raccourcis d'édition, sur le modèle du shell de l'éditeur :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:318-326 (extrait)

```
1 if (e->GetModifiers().ctrl) { // raccourcis copier/couper/coller/tout-sélectionner
2     if (k == NkKey::NK_C) mUI.input.wantCopy = true;
3     else if (k == NkKey::NK_X) mUI.input.wantCut = true;
4     else if (k == NkKey::NK_V) mUI.input.wantPaste = true;
5     else if (k == NkKey::NK_A) mUI.input.wantSelectAll = true;
```

14.7.5 Le presse-papiers

NKGui ne connaît pas la fenêtre, donc il ne peut pas accéder au presse-papiers du système. On lui donne deux pointeurs de fonction (ctx.clipboardGetFn, ctx.clipboardSetFn, plus un pointeur utilisateur ctx.clipboardUser); c'est ce que fait le shell en [NkEditorShell.cpp:185-190](#). Sans cela, copier-coller dans un champ de saisie ne fait rien.

14.8 La boucle principale

14.8.1 Le temps

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:431-436

```
1 while (running && window.IsOpen()) {
2     float32 dt = clock.Tick().delta;
3     if (dt <= 0.f)
4         dt = 1.f / 60.f;
5     if (dt > 0.1f)
6         dt = 0.1f;
```

Les deux bornes ont chacune leur raison : $dt \leq 0$ arrive à la toute première image (l'horloge n'a pas encore de référence); $dt > 0.1$ arrive après un blocage — déplacement de fenêtre, mise en veille, point d'arrêt du débogueur. Sans la borne haute, tout ce qui dépend du temps ferait un bond : le curseur de saisie clignoterait plusieurs fois d'un coup, les répétitions au maintien se déclencheraient en rafale.

14.8.2 Les événements — avant tout le reste

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:438-442

```
1 while (NkEvent *ev = NkEvents().PollEvent()) {
2     (void)ev;
3 }
4 if (!running)
5     break;
```

Le `if (!running) break;` évite de dessiner une image entière sur une fenêtre qu'on vient de fermer.

14.8.3 Le redimensionnement

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:444-458

```
1 // Synchroniser la SWAPCHAIN à la taille de la FENÊTRE. target->GetSize()
2 // renvoie la taille de la swapchain (qui ne change QU'À OnResize) - s'en
3 // servir pour détecter le resize était faux : la swapchain restait figée à
4 // sa taille de création (rendu basse-résolution étiré). On pilote OnResize
5 // depuis la taille fenêtre (inclut la 1re frame → sync initiale).
6 const math::NkVec2u wsz = target->GetWindow().GetSize();
7 if (wsz.x > 0 && wsz.y > 0 && (wsz.x != lastW || wsz.y != lastH)) {
8     target->OnResize(wsz.x, wsz.y);
9     lastW = wsz.x;
10    lastH = wsz.y;
11 }
12 const math::NkVec2u sz = target->GetSize(); // = wsz après OnResize
13 if (sz.x > 0 && sz.y > 0) {
14     ctx.viewW = static_cast<int32>(sz.x);
15     ctx.viewH = static_cast<int32>(sz.y);
16 }
```

`lastW` et `lastH` sont initialisés à zéro, ce qui garantit que la première image déclenche une synchronisation. Et les tests > 0 évitent de recréer une chaîne d'échange de taille nulle quand la fenêtre est réduite.

Fenêtre minimisée

Une fenêtre en cours de réduction glisse un rectangle intermédiaire — environ 160×28 sous Windows — entre le test « minimisée ? » et la lecture de taille. Le shell de l'éditeur documente une garde qui refuse toute taille sous 32 pixels :

Integrations/NKGui/NkEditorRHIRenderer.h:123-130 (extrait)

```
1 // GARDE ANTI-MINIMISATION : une fenetre en cours de reduction
2 // glisse son rect placeholder (~160x28 sous Windows) entre le
3 // test « minimisee ? » de la boucle principale et la lecture
4 // de taille -- la course est reelle (defaut 4.3, reproduite).
5 // Sous 32 px, les cibles divisees du rendu (bloom /32) tombent
6 // a zero et CreateTexture2D echoue en E_INVALIDARG -> mort a
7 // la restauration. On refuse net : la vraie taille arrivera
8 // avec le retour de la fenetre.
```

14.8.4 L'image d'interface

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:468-481 (extrait)

```
1     ctx.BeginFrame(dt);
2     NkGuiDrawList &d1 = ctx.dl;
3     const float32 W = static_cast<float32>(ctx.viewW);
4     const float32 H = static_cast<float32>(ctx.viewH);
5
6     // Fond + barre d'en-tête + TITRE (texte NKFont).
7     d1.AddRectFilled({0.f, 0.f, W, H}, ctx.theme.bgPrimary);
8     d1.AddRectFilled({0.f, 0.f, W, 40.f}, ctx.theme.header);
9     d1.AddRectFilled({0.f, 38.f, W, 2.f}, ctx.theme.accent);
10    TextAt(ctx, {16.f, 11.f}, "NKGui - Phase 3 : layout + widgets (NKFont)");
```

Le fond est dessiné en premier — tout le reste passera par-dessus. Ici la démo écrit dans `ctx.d1` directement, ce qui est légitime parce qu'aucune fenêtre n'est ouverte à ce moment : nous sommes bien sur la couche principale. Dès qu'on est à l'intérieur d'un `Begin`, il faut passer par `ctx.DL()` (chapitre 3).

Puis les widgets, avec une région de layout ouverte :

Squelette de widgets — écrit pour ce cours

```
1     ctx.BeginLayout({20.f, 60.f, 400.f, H - 80.f});
2     Text(ctx, "Bonjour NKGui");
3     if (Button(ctx, "Cliquez-moi")) {
4         logger.Info("clac !");
5     }
6     static bool visible = true;
7     Checkbox(ctx, "Afficher le panneau", visible);
8     static float32 zoom = 1.f;
9     SliderFloat(ctx, "Zoom", zoom, 0.5f, 4.f);
```

14.8.5 Le curseur

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:1020-1041

```
1      // Curseur souhaité par NKGui → curseur OS (OPTIONNEL : L'app choisit d'appliquer).
2      // Ex. DragFloat pose ↕ / ⇅ pendant Le survol/glisser. SetCursor est persistant.
3      {
4          NkWindow::NkCursorType c = NkWindow::NkCursorType::Arrow;
5          switch (ctx.wantCursor) {
6              case NkGuiCursor::Text:
7                  c = NkWindow::NkCursorType::TextInput;
8                  break;
9              case NkGuiCursor::Hand:
10                 c = NkWindow::NkCursorType::Hand;
11                 break;
12              case NkGuiCursor::ResizeEW:
13                 c = NkWindow::NkCursorType::ResizeWE;
14                 break;
15              case NkGuiCursor::ResizeNS:
16                 c = NkWindow::NkCursorType::ResizeNS;
17                 break;
18              default:
19                 break;
20          }
21          window.SetCursor(c);
22      }
```

Rappel du chapitre 3 : NKGui *demande* un curseur en posant `ctx.wantCursor`, il ne le pose pas — il ne connaît pas la fenêtre. Ce bloc est le traducteur, et il est facultatif : sans lui, tout fonctionne, mais le pointeur ne change jamais de forme au-dessus d'un champ de saisie ou d'un séparateur. Notez qu'il est placé *avant* `EndFrame`, parce que `wantCursor` est réinitialisé par `BeginFrame` et rempli par les widgets.

14.8.6 La présentation

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:1043-1048

```
1      ctx.EndFrame();
2
3      target->Clear();
4      backend.Submit(dl, sz.x, sz.y); // couche principale
5      backend.Submit(ctx.dlOverlay, sz.x, sz.y); // couche popups/overlay (au-dessus)
6      target->Display(); // Capture d'ecran (F12) : APRES Display() – La frame presentee.
      Self-capture
```

DEUX Submit, jamais un seul

C'est le piège le plus coûteux de tout ce cours, parce qu'il est absolument silencieux. NKGui produit **deux** listes de dessin :

- `ctx.dl` — le contenu ordinaire, y compris les fenêtres fusionnées par ordre de profondeur;
- `ctx.dlOverlay` — les popups, les menus déroulants, les sous-menus, les combos, les infobulles, et tout ce que vous dessinez entre `PushOverlay` et `PopOverlay`.

Si vous n'appellez `Submit` que sur la première, **tout fonctionne sauf les surfaces flottantes**. Le menu s'ouvre — l'état est correct, les clics sont capturés, la logique tourne — mais rien

ne s'affiche. Vous cliquez dans le vide sur des éléments invisibles.

Et il n'y a *aucun* message d'erreur : le framework a fait son travail, le pont aussi; c'est simplement qu'on ne lui a jamais donné la seconde liste.

L'ordre compte aussi : ctx.dl d'abord, ctx.dlOverlay ensuite. Inversé, les popups passeraient *sous* le contenu.

Le shell de l'éditeur fait exactement la même chose, à travers son interface de renderer :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:725-732

```
1 mUI.EndFrame();
2
3 mWindow.SetCursor(MapCursor(mUI.wantCursor));
4
5 mRenderer->BeginFrame();
6 mRenderer->SubmitDrawList(mUI.dl, sz.x, sz.y);
7 mRenderer->SubmitDrawList(mUI.dlOverlay, sz.x, sz.y);
8 mRenderer->EndFrame();
```

14.8.7 Ce que fait Submit, en détail

Pour comprendre ce qui peut mal tourner, voici la traduction complète :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:120-195 (condensé)

```
1 void Submit(const NkGuiDrawList &dl, uint32 fbW, uint32 fbH) {
2     if (!mRenderer || dl.vtx.Size() == 0 || dl.idx.Size() == 0) return;
3
4     // 1) conversion des sommets NkGui -> NkVertex2D (couleur dépaquetée)
5     mScratch.Resize(dl.vtx.Size());
6     for (uint32 i = 0; i < dl.vtx.Size(); ++i) {
7         const nkgui::NkGuiVertex &s = dl.vtx[i];
8         renderer::NkVertex2D &d = mScratch[i];
9         d.x = s.pos.x; d.y = s.pos.y; d.u = s.uv.x; d.v = s.uv.y;
10        d.r = (uint8)(s.col & 0xFF);
11        d.g = (uint8)((s.col >> 8) & 0xFF);
12        d.b = (uint8)((s.col >> 16) & 0xFF);
13        d.a = (uint8)((s.col >> 24) & 0xFF);
14    }
15
16    // 2) une passe par commande
17    for (uint32 ci = 0; ci < dl.cmds.Size(); ++ci) {
18        const nkgui::NkGuiDrawCmd &dc = dl.cmds[ci];
19        if (dc.idxCount == 0u) continue;
20
21        // 2a) clip : Le rect « infini » (1e9) signifie « pas de clip »
22        const bool hasClip = (dc.clipRect.w < 1.0e8f && dc.clipRect.h < 1.0e8f);
23        if (hasClip) {
24            ... clamp a [0, fbW] x [0, fbH] ...
25            if (x1 <= x0 || y1 <= y0) continue; // clip vide -> on saute
26            mRenderer->SetClip(math::NkRect2i...{});
27        }
28
29        // 2b) résolution de la texture : d'abord les atlas de police, sinon les images
30        renderer::NkTexture *tex = nullptr;
31        if (dc.type == nkgui::NkGuiDrawCmdType::TexturedTriangles) { ... }
32
33        // 2c) sous-ensemble de sommets + indices REBASÉS
34        uint32 lo = 0xFFFFFFFFu, hi = 0u;
```

```

35     for (uint32 k = 0; k < dc.idxCount; ++k) { const uint32 v = dl.idx[dc.idxOffset+k];
36                                             if (v<lo) lo=v; if (v>hi) hi=v; }
37     mIdxTmp.Resize(dc.idxCount);
38     for (uint32 k = 0; k < dc.idxCount; ++k) mIdxTmp[k] = dl.idx[dc.idxOffset+k] - lo;
39     mRenderer->DrawVertices(mScratch.Data() + lo, hi - lo + 1u, mIdxTmp.Data(),
    dc.idxCount, tex);
40
41     if (hasClip) mRenderer->PopClip();
42 }
43 }

```

L'étape (2c) mérite l'attention, parce qu'elle corrige un défaut qui coûtait un plantage :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:176-178

```

1 // Ne soumet que le SOUS-ENSEMBLE de vertices reference par cette
2 // commande (indices rebases). Indispensable : passer tout le buffer
3 // depasse kMaxVertices (65536) des qu'un draw list est gros -> crash.

```

Le pont calcule l'indice minimal et maximal utilisés par la commande, n'envoie que cette tranche de sommets, et *décale tous les indices* d'autant. Sans cela, chaque commande enverrait tout le tampon.

(Le commentaire mentionne 65 536; la constante actuelle est 262 144 — voir chapitre 2, section 2.10. Le commentaire garde la trace du dimensionnement d'origine, et c'est la même famille de bug qui a motivé l'agrandissement.)

14.9 Le programme complet

Rassemblons tout dans un fichier minimal mais fonctionnel. Ce squelette est reconstitué fidèlement depuis `Applications/NKGuiDemo/src/NKGuiDemo/main.cpp` (lignes 240 à 1067), réduit au strict nécessaire.

Squelette complet — reconstitué depuis `Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:240-1067`

```

1 #include "NKWindow/NKWindow.h"
2 #include "NKWindow/NKMain.h"
3 #include "NKEvent/NkWindowEvent.h"
4 #include "NKEvent/NkMouseEvent.h"
5 #include "NKEvent/NkKeyboardEvent.h"
6 #include "NKCanvas/Core/NkContextDesc.h"
7 #include "NKCanvas/Core/NkGraphicsApi.h"
8 #include "NKCanvas/Renderer/Targets/NkRenderWindow.h"
9 #include "NKTime/NkClock.h"
10 #include "NKMemory/NkUniquePtr.h"
11 #include "NKLogger/NkLog.h"
12 #include "NKGui/NKGui.h"
13 #include "NKCanvas/UI/NkGuiCanvasBackend.h"
14
15 using namespace nkentseu;
16 using namespace nkentseu::nkgui;
17 using namespace nkentseu::renderer;
18
19 NKENTSEU_DEFINE_APP_DATA([]() {
20     NkAppData d{};
21     d.appName = "MonApp";
22     d.appVersion = "0.1.0";
23     return d;

```

```

24  }());
25
26  int nkmain(const NkEntryState &state) {
27      (void)state;
28
29      // — 1) FENETRE OS (NkWindow) —————
30      NkWindow window;
31      NkWindowConfig cfg;
32      cfg.title = "Mon app NKGui";
33      cfg.width = 1100; cfg.height = 800;
34      cfg.centered = true; cfg.resizable = true;
35      if (!window.Create(cfg)) return -1;
36
37      // — 2) CONTEXTE GRAPHIQUE + CIBLE DE RENDU (NkCanvas) —————
38      NkContextDesc desc;
39      desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
40      if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
41  #if defined(NKENTSEU_PLATFORM_WINDOWS)
42      desc.api = NkGraphicsApi::NK_GFX_API_DX11;
43  #else
44      desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
45  #endif
46      }
47      auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
48      if (!target || !target->IsValid()) return -1;
49
50      // — 3) CONTEXTE NKGui —————
51      auto ctxPtr = memory::NkMakeUnique<NkGuiContext>();
52      if (!ctxPtr) return -1;
53      NkGuiContext &ctx = *ctxPtr;
54      ctx.Init(static_cast<int32>(cfg.width), static_cast<int32>(cfg.height));
55      SetCurrentContext(&ctx);
56
57      // — 4) BACKEND (NKGui -> NKCanvas) —————
58      renderer::NkGuiCanvasBackend backend;
59      if (!backend.Init(target->GetRenderer())) return -1;
60
61      // — 5) POLICE : charger PUIS uploader L'atlas —————
62      auto fontPtr = memory::NkMakeUnique<NkGuiFont>(); // DOIT survivre a la boucle
63      if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f)) {
64          fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
65      }
66      ctx.font = fontPtr.Get(); // pointeur brut NON possede
67      if (fontPtr->Valid()) {
68          backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
fontPtr->atlasH);
69      }
70
71      // — 6) GLUE D'ENTREE (callbacks NKEvent -> ctx.input) —————
72      auto &events = NkEvents();
73      bool running = true;
74      events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
});
75      events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
76          ctx.input.mousePos = {static_cast<float32>(e->GetX()),
static_cast<float32>(e->GetY())};
77      });
78      events.AddEventCallback<NkMouseButtonPressEvent>([&](NkMouseButtonPressEvent *e) {
79          if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = true;
80      });
81      events.AddEventCallback<NkMouseButtonReleaseEvent>([&](NkMouseButtonReleaseEvent *e) {
82          if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = false;

```

```

83     });
84     events.AddEventCallback<NkMouseWheelVerticalEvent>([&](NkMouseWheelVerticalEvent *e) {
85         ctx.input.wheel += static_cast<float32>(e->GetDeltaY());
86     });
87     events.AddEventCallback<NkTextInputEvent>([&](NkTextInputEvent *e) {
88         ctx.input.PushChar(e->GetCodepoint());
89     });
90
91     NkClock clock;
92     uint32 lastW = 0, lastH = 0;
93     bool showPanel = true;
94     float32 zoom = 1.f;
95     char nom[64] = "sans titre";
96
97     // — 7) BOUCLE PRINCIPALE —————
98     while (running && window.IsOpen()) {
99         float32 dt = clock.Tick().delta;
100         if (dt <= 0.f) dt = 1.f / 60.f;
101         if (dt > 0.1f) dt = 0.1f; // borne anti-saut
102
103         // a) EVENEMENTS — AVANT BeginFrame (BeginFrame appelle input.NewFrame())
104         while (NkEvent *ev = NkEvents().PollEvent()) { (void)ev; }
105         if (!running) break;
106
107         // b) Synchroniser La SWAPCHAIN sur La taille de La FENETRE
108         const math::NkVec2u wsz = target->GetWindow().GetSize();
109         if (wsz.x > 0 && wsz.y > 0 && (wsz.x != lastW || wsz.y != lastH)) {
110             target->OnResize(wsz.x, wsz.y);
111             lastW = wsz.x; lastH = wsz.y;
112         }
113         const math::NkVec2u sz = target->GetSize();
114         if (sz.x > 0 && sz.y > 0) { ctx.viewW = (int32)sz.x; ctx.viewH = (int32)sz.y; }
115
116         // c) DEBUT DE FRAME UI
117         ctx.BeginFrame(dt);
118         const float32 W = (float32)ctx.viewW;
119         const float32 H = (float32)ctx.viewH;
120         ctx.dl.AddRectFilled({0.f, 0.f, W, H}, ctx.theme.bgPrimary);
121
122         // d) LES WIDGETS
123         if (BeginPanel(ctx, "Reglages", {20.f, 20.f, 340.f, H - 40.f})) {
124             Text(ctx, "Bonjour NKGui");
125             Separator(ctx);
126             Checkbox(ctx, "Afficher l'aperçu", showPanel);
127             SliderFloat(ctx, "Zoom", zoom, 0.5f, 4.f);
128             InputText(ctx, "Nom", nom, sizeof(nom));
129             if (Button(ctx, "Reinitialiser")) {
130                 zoom = 1.f;
131                 showPanel = true;
132             }
133             EndPanel(ctx);
134         }
135
136         // e) FIN DE FRAME UI
137         ctx.EndFrame();
138
139         // f) PRESENTATION : Clear ouvre la frame, Display la ferme et presente
140         target->Clear();
141         backend.Submit(ctx.dl,          sz.x, sz.y); // couche principale
142         backend.Submit(ctx.dlOverlay, sz.x, sz.y); // popups/overlay AU-DESSUS
143         target->Display();
144     }

```



```

145
146     SetCurrentContext(nullptr);
147     ctx.Shutdown();
148     return 0;
149 }

```

Ce qu'il faut retenir

Deux cents lignes, dont la moitié est du branchement d'entrée. Tout le reste de votre application tiendra entre `BeginFrame` et `EndFrame`.

14.10 Ajouter une image

Les icônes et les images suivent exactement le chemin de la police : on charge des pixels, on les envoie sous un identifiant, on dessine.

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:292-321 (abrégé)

```

1     uint32 imgTexId = 0u;
2     int32 imgW = 0, imgH = 0;
3     {
4         static const char *kCandidates[] = {
5             "Applications/Mou/assets/brand/rihen-logo.png",
6             "Resources/Icons/ContentBrowser/FileIcon.png",
7             "../Applications/Mou/assets/brand/rihen-logo.png",
8         };
9         NKImage img;
10        bool ok = false;
11        for (const char *p : kCandidates)
12            if (img.Load(p, 4)) { ok = true; break; }
13        if (ok && img.Width() > 0 && img.Height() > 0 && img.Pixels()) {
14            imgW = img.Width();
15            imgH = img.Height();
16            static NkVector<uint8> rgba;
17            rgba.Resize(static_cast<usize>(imgW) * imgH * 4u);
18            for (int32 y = 0; y < imgH; ++y) {
19                const uint8 *src = img.RowPtr(y);
20                uint8 *dst = rgba.Data() + static_cast<usize>(y) * imgW * 4u;
21                for (int32 x = 0; x < imgW * 4; ++x)
22                    dst[x] = src[x];
23            }
24            imgTexId = 0x494D4731u; // 'IMG1'
25            backend.UploadImageRGBA(imgTexId, rgba.Data(), imgW, imgH);
26            logger.Info("Image chargée : {0}x{1}", imgW, imgH);
27        } else {
28            logger.Info("Image demo introuvable (cwd) -> section image vide");
29        }
30    }

```

Puis, dans la boucle, `Image(ctx, imgTexId, 128.f, 128.f)` ou `ctx.DL().AddImage(imgTexId, rect, {0,0}, {1,1}, tint)`.

Trois choses à noter :

- **La liste de chemins candidats**, avec des chemins relatifs à la racine du dépôt *et* des chemins remontants. C'est la parade au piège du répertoire courant vu au chapitre 0. Le dépôt formalise cette approche : « Candidats RELATIFS AU CWD (dev, lancement depuis la racine du repo) PUIS relatifs à l'EXECUTABLE » ([Applications/NKCode/src/NKCode/Shell/NkAppFonts.h:18-21](#)).

- **L'échec est journalisé, pas fatal.** L'application se lance quand même, avec une section image vide.
- **Le choix du texId.** Ici 0x494D4731, soit « IMG1 ». Il doit être distinct de celui de toute police (0x4E4B4654 pour la première), car le pont résout **les atlas de police d'abord, les images ensuite** (`NkGuiCanvasBackend.h:161-174`). Le shell de l'éditeur va plus loin et réserve des plages entières :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.h:392

```
1 uint32 mNextTexId = 0x4E4B0100u; // ids de textures app (Logos/icones) – distincts des
    polices
```

Ce qu'il faut retenir

`texId == 0` est **réservé** : c'est la texture blanche interne côté RHI (`Integrations/NKGui/NkGuiRHIBackend.h:40`), et un `AddImage` avec `texId == 0` est purement ignoré (`NkGuiDrawList.cpp:160-161`). Si votre image ne s'affiche pas, la première chose à vérifier est que son identifiant n'est pas resté à zéro parce que le chargement a échoué.

14.11 Le fichier de build

Il reste à déclarer la cible pour Jenga. Créez `Applications/MonApp/MonApp.jenga` :

Applications/MonApp/MonApp.jenga — écrit pour ce cours, calqué sur Applications/NKGuiDemo/NKGuiDemo.jenga:41-75

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """MonApp – application de demonstration NKGui sur NKCanvas."""
4
5 from Jenga import *
6 from jengaconfig import *
7
8 with project("MonApp"):
9     windowedapp()
10     language("C++")
11     cppdialect("C++17")
12     location(".")
13
14     files(["src/**/*.cpp"])
15
16     nkentseudependson(
17         ["NKGui", "NKCanvas", "NKWindow", "NKEvent", "NKGlad", "NKFont", "NKImage",
18          "NKStream", "NKFileSystem", "NKLogger", "NKMath", "NKTime",
19          "NKContainers", "NKMemory", "NKCore", "NKPlatform", "NKThreading"],
20         extra_includes=["src", "src/MonApp", "%{NKGlad.location}/include"],
21     )
22
23     objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
24     targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
25
26     apppublisher("Rihen Universe")
27     appversion("0.1.0")
28     licensefile("../LICENSE")
29
30     with filter("system:Windows"):
31         windowedapp()
32         usetoolchain(TC_WINDOWS)
```

```

33     defines(["WIN32_LEAN_AND_MEAN", "_UNICODE", "UNICODE"])
34     links(["user32", "gdi32", "opengl32", "dwmapi", "shell32",
35           "uuid", "ole32", "dxguid",
36           "d3d11", "d3d12", "dxgi", "d3dcompiler"])

```

14.11.1 Pourquoi chaque dépendance est là

Module	Pourquoi
NKGui	les widgets et le contexte
NKCanvas	le rendu, la cible, et l'en-tête du pont
NKWindow	la fenêtre et le point d'entrée nkmain
NKEvent	les événements typés
NKFont	la rasterisation des glyphes
NKImage	le décodage des images
NKGlad	le chargeur de fonctions OpenGL
NKTime	NkClock et le <i>delta time</i>
NKLogger	l'objet logger
NKMath, NKContainers, NKMemory, NKCore, NKPlatform	les fondations
NKFileSystem, NKStream, NKThreading	tirés par les modules ci-dessus

Oublier NKCanvas ou NKGui

Le pont `NkGuiCanvasBackend.h` est *header-only*. Il inclut `"NKGui/NKGui.h"` et appelle des méthodes de `NkIRenderer2D`. Si votre `.jenga` ne déclare qu'un seul des deux modules, l'erreur ne se produit pas à la compilation de `NKCanvas` — qui ne compile pas ce fichier — mais *chez vous*, sous forme d'en-têtes introuvables ou de symboles non résolus.

14.11.2 Enregistrer la cible et compiler

Selon l'organisation du dépôt, la nouvelle cible doit être connue du fichier racine `Nkentseu.jenga` — vérifiez comment les autres applications y sont listées avant de lancer la compilation.

Compilation et lancement — depuis la racine du dépôt

```

1 cd D:\Projets\2026\Nkentseu\Nkentseu
2 jenga build --target MonApp --config Release
3 .\Build\Bin\Release-Windows\MonApp\MonApp.exe

```

Et la configuration de mise au point, qui garde symboles et assertions :

Compilation en Debug

```

1 jenga build --target MonApp --config Debug
2 .\Build\Bin\Debug-Windows\MonApp\MonApp.exe

```

Lancer depuis la racine

Toujours depuis `D:/Projets/2026/Nkentseu/Nkentseu`, pas depuis le dossier de l'exécutable. Les chemins de ressources — polices de repli, images, icônes — sont écrits relativement à la racine du dépôt pendant le développement. Et le journal, `logs/app.log`, est écrit relativement au répertoire courant : lancer d'ailleurs le disperse.

14.12 Diagnostiquer les silences

Cette section est un aide-mémoire. Elle liste ce qui, dans une application NKGui, échoue *sans message d'erreur*.

Symptôme	Cause la plus probable
Fenêtre noire, l'application tourne	Backend graphique non fonctionnel; essayer OpenGL, lire <code>logs/app.log</code>
Widgets visibles, aucun texte	<code>UploadFontGray8</code> manquant, ou <code>fontPtr->Valid()</code> faux
Menus et combos invisibles mais cliquables	Le second Submit sur <code>ctx.dlOverlay</code> manque
Popups sous le contenu	Les deux Submit sont dans le mauvais ordre
Les clics arrivent avec une image de retard, ou jamais	Les événements sont pompés <i>après</i> <code>BeginFrame</code>
Plus rien ne réagit, l'application tourne	<code>activeId</code> figé : un glissement maison n'a pas remis <code>ctx.activeId = 0</code>
La souris semble gelée après une modale	Un panneau a écrasé <code>ctx.input.mousePos</code>
Rendu flou et étiré après agrandissement	Le redimensionnement est détecté sur la swap-chain au lieu de la fenêtre
Pas de barre de défilement, barre horizontale hors écran	<code>AvailHeight()</code> utilisé sans intersection avec <code>CurrentClip()</code>
Image absente	<code>texId</code> resté à zéro (chargement échoué) ou en collision avec une plage de police
Bords droit et bas des panneaux amincis	Une bordure dessinée à l' <code>AddLine</code> au lieu de quatre rectangles internes
Texte flou	Position de glyphe non alignée sur la grille de pixels
Deux boutons réagissent ensemble	Même libellé, même portée : il manque <code>PushId/PopId</code>

14.13 Pour aller plus loin

Ce que ce chapitre n'a pas fait, et que vous savez maintenant aborder seul :

- **Des fenêtres flottantes et du docking.** Ajoutez `DockSpaceOverViewport(ctx)` avant vos `Begin`, puis déclarez chaque panneau entre `Begin` et `EndWindow`. La démo le fait en `main.cpp:888-1017`.

- **Le HiDPI.** `SetUiScale(ctx, 1.5f)` plus un rechargement de la police à `18.f * 1.5f`, avant `BeginFrame` ([main.cpp:461-466](#)).
- **La capture d'écran**, après `Display()` : `target->Capture("capture.png")` ([main.cpp:1049-1061](#)).
- **La coquille d'éditeur.** Si votre application ressemble à un IDE — barre de titre maison, palette de commandes, préférences, panneaux ancrables — n'écrivez pas tout cela : [Engine/NKEditorKit](#) le fournit, et vous n'avez plus qu'à écrire des panneaux dérivant de `NkEditorPanel`.

Ce chapitre en bref

Une application complète tient en quatre gestes répétés à chaque image : recueillir les événements, les donner à `NKGui`, décrire l'interface, soumettre le résultat au rendu. Rien de plus n'est nécessaire — et rien de moins ne suffit, car sauter l'un des quatre produit une interface qui s'affiche mais ne réagit pas, ou qui réagit sans s'afficher. C'est cette ossature que la coquille du chapitre suivant écrit à votre place, et vous venez de voir exactement ce qu'elle remplace.

14.14 Exercices

1 — Faire tourner le squelette

Créez `Applications/MonApp/` avec son `.jenga` et son `src/MonApp/main.cpp`, recopiez le squelette de la section 4.9, compilez, lancez. Vérifiez que le panneau s'affiche avec son titre, que la case à cocher répond et que le *slider* se déplace. Puis, **volontairement**, supprimez la ligne `backend.Submit(ctx.dlOverlay, ...)`, ajoutez un `BeginCombo` au panneau, et constatez par vous-même ce que « échoue sans message » veut dire.

2 — Un vrai clavier

Complétez le branchement de l'entrée : double-clic, molette horizontale, touches d'édition (`setKey`), raccourcis `wantCopy/wantCut/wantPaste/wantSelectAll`, et le presse-papiers via `ctx.clipboardGetFn / ctx.clipboardSetFn`. Vérifiez ensuite, dans un `InputText`, que les flèches déplacent le curseur, que `Maj+flèche` sélectionne, que `Ctrl+C` et `Ctrl+V` fonctionnent, et qu'un accent s'écrit correctement.

3 — Une image et son identifiant

Chargez une image PNG avec `NkImage`, envoyez-la par `backend.UploadImageRGBA` sous un identifiant de votre choix, affichez-la avec le widget `Image`. Puis attribuez-lui *délibérément* le même identifiant que la police (`0x4E4B4654`) et observez ce qui s'affiche. Expliquez le résultat à partir de l'ordre de résolution du pont : atlas de police d'abord, images ensuite.

4 — Une application complète

Écrivez un petit gestionnaire de tâches : une table à trois colonnes (fait, titre, priorité), un champ de saisie et un bouton pour ajouter une ligne, une case à cocher par ligne, et un bouton « Supprimer les tâches terminées ». Les données vivent dans un `NkVector` de votre propre structure. Contraintes : un `PushId` par ligne, un identifiant stable pour la table, et **aucune** modification du tableau pendant le parcours — levez un drapeau et agissez après `EndTable`, conformément au dernier idiome du chapitre 3.

Questions de cours

1. Quels sont les quatre gestes d'une image, dans l'ordre ?
2. Que se passe-t-il si l'on oublie de transmettre les événements à NKGui ?
3. Où doit vivre l'état de votre application, et pourquoi pas dans le widget ?
4. Pourquoi faut-il recréer les ressources dépendantes de la taille lors d'un redimensionnement ?
5. Quelle est la dernière opération d'une image, et que se passe-t-il si on l'oublie ?

Exercice de démonstration

Prouvez que l'ordre des quatre gestes compte.

Partez de votre application qui fonctionne. Déplacez la transmission des événements *après* la description de l'interface, et relancez. Décrivez précisément ce que vous observez — et notez surtout si l'interface a l'air cassée ou seulement *en retard*.

Ce que la démonstration établit : le défaut ne se présente pas comme une erreur mais comme une latence d'une image. C'est la forme la plus difficile à repérer, parce qu'on l'attribue au matériel.

Exercice de logique

Un bouton de votre interface « ne répond qu'une fois sur deux ».

1. Énumérez au moins quatre causes possibles, en distinguant celles de l'entrée, celles de l'identité du widget, et celles de la boucle.
2. Pour chacune, donnez une mesure qui la distinguerait des autres — une mesure dont le résultat *diffère* selon la cause.
3. Vous n'avez le droit qu'à *une* mesure. Laquelle choisissez-vous, et qu'est-ce qu'elle ne pourra pas trancher ?

La coquille d'application : ce que vous n'écrivez plus

Le chapitre précédent vous a fait écrire une application *à la main* : créer la fenêtre, ouvrir le contexte graphique, tenir la boucle, écouter les événements, présenter l'image. C'était la bonne façon de comprendre ce qui se passe — et c'est la seule façon d'apprendre ce qu'une coquille vous cache.

Ce chapitre montre la coquille. Depuis `NkCanvasApp`, cette plomberie devient quelques méthodes à remplir. Surtout, la coquille apporte trois services qu'une application écrite à la main *oublie presque toujours* — et l'oubli ne se voit qu'une fois le programme entre les mains de quelqu'un d'autre.

Ce qu'il faut retenir

Une coquille ne vous fait pas gagner des lignes : elle vous fait gagner les lignes que *vous n'auriez pas écrites*.

Mesure du dépôt, le 2 septembre 2026 : **quatorze** applications créent elles-mêmes leur fenêtre de rendu. Sur ces quatorze, **trois** gèrent le cycle de vie mobile. Les onze autres — dont `Sandbox`, `NkRef`, `NKGuiDemo`, `NkVideoPlayer` et `NkAudioPlayer` — se ferment quand le téléphone se verrouille, et personne ne l'a remarqué parce que personne ne les lance sur un téléphone.

Ce n'est pas de la négligence : c'est que *rien ne rappelle d'écrire ce qu'on ne voit pas manquer*. C'est précisément ce qu'une coquille existe pour donner.

15.1 Le plus petit programme complet

Applications/NkDames/src/main.cpp

```
1 #include "Dames/NkDamesJeu.h"
2 #include "NKWindow/Core/NkEntry.h"
3 #include "NKWindow/NKMain.h"
4
5 NKENTSEU_DEFINE_APP_DATA([]() {
6     nkentseu::NkAppData d{};
7     d.appName = "NkDames";
8     d.appVersion = "1.0.0";
9     return d;
10 })();
11
12 int nkmain(const nkentseu::NkEntryState &state) {
13     return nkentseu::renderer::NkCanvasApp::Run<nkentseu::jeux::dames::NkDamesJeu>(state);
14 }
```

Trois choses seulement, et elles se lisent dans l'ordre.

`NKENTSEU_DEFINE_APP_DATA` déclare le nom et la version de l'application. `nkmain` est le point d'entrée portable : c'est `NKWindow/NKMain.h` qui fournit derrière lui le `winMain` de Windows et

l'android_main d'Android. NkCanvasApp::Run<T> construit votre classe et lui rend la main.

Le piège

La macro attend un NkAppData, **pas une chaîne** — son nom laisse croire le contraire. Écrivez NKENTSEU_DEFINE_APP_DATA("MonJeu") et le compilateur répond no viable overloaded '=', sans jamais nommer la cause. Le lambda ci-dessus est le patron employé par toutes les applications du dépôt.

15.2 Les méthodes que vous remplissez

Kernel/Runtime/NkCanvas/src/NkCanvas/App/NkCanvasApp.h (extraits)

```
1 virtual NkOptional<int> OnCommandLine(const NkVector<NkString> &args);
2 virtual bool OnInit();
3 virtual void OnUpdate(float32 deltaTime);
4 virtual void OnRender(NkRenderWindow &target);
5 virtual bool OnEvent(const NkEvent &event); // true = consomme
6 virtual bool OnPointer(const NkPointer &pointer);
7 virtual void OnLayout(const NkLayoutInfo &layout);
8 virtual bool OnCloseRequested(); // false = refuse la fermeture
9 virtual void OnPause();
10 virtual void OnResume();
11 virtual void OnShutdown();
```

Aucune n'est obligatoire : elles ont toutes un corps par défaut. On remplit ce dont on a besoin.

Méthode	Quand elle est appelée
OnCommandLine	avant toute fenêtre — peut refuser de démarrer
OnInit	la fenêtre et le GPU existent
OnUpdate	une fois par image, avec le pas de temps
OnRender	une fois par image, avec la cible de rendu
OnPointer	souris <i>ou</i> doigt, unifiés
OnLayout	taille, densité et zone sûre ont changé
OnPause, OnResume	l'application part en arrière-plan, revient

Le piège

OnCommandLine rend un NkOptional<int>. Rendre une valeur *arrête l'application avec ce code*, sans ouvrir de fenêtre — c'est ce qui permet à une option de banc de répondre en mode automatique. Rendre un NkOptional vide laisse le programme démarrer.

15.3 Les trois services que la coquille rend

15.3.1 La boucle cède la main, et sur le Web c'est vital

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasApp.cpp (CadencerFrame)

```
1  if (mConfig.imagesParSeconde <= 0) {
2      NkChrono::YieldThread();
3      return;
4  }
5  const float64 budgetMs = 1000.0 / static_cast<float64>(mConfig.imagesParSeconde);
6  const float64 ecoleMs = chrono.Elapsed().milliseconds;
7  if (ecoleMs < budgetMs) {
8      NkChrono::Sleep(budgetMs - ecoleMs);
9  } else {
10     NkChrono::YieldThread();
11 }
```

Sous Emscripten, `NkChrono::Sleep` appelle `emscripten_sleep(ms)` et `YieldThread` appelle `emscripten_sleep(0)`. Les deux rendent la main au navigateur.

Le piège

Les deux branches cèdent, et c'est tout l'intérêt. Une version « ne dormir que si on est en avance » gèle l'onglet dès la première image lente — c'est-à-dire tout le temps sur le Web, où une image dépasse presque toujours son budget. Le `else` n'est pas une optimisation : c'est lui qui rend la main.

Le mécanisme existait avant la coquille, et il était juste. Ce qui manquait, c'était *l'appel*.

15.3.2 La zone sûre, mesurée et non devinée

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasApp.h (NkLayoutInfo)

```
1  struct NkLayoutInfo {
2      uint32 width = 0;
3      uint32 height = 0;
4      float32 density = 1.f; // facteur d'echelle de l'ecran (DPI)
5      NkSafeAreaInsets safeArea;
6  };
```

`safeArea` porte quatre marges — `top`, `bottom`, `left`, `right` — en pixels. Elles décrivent la partie de l'écran qui n'est masquée ni par l'encoche, ni par les coins arrondis, ni par l'indicateur de geste du bas.

Ce qu'il faut retenir

La zone sûre est une **ancree**, pas une marge : elle change à la rotation, à l'ouverture du clavier, en écran partagé, et d'un appareil à l'autre. On la demande à la plateforme à l'exécution ; une marge écrite en dur est fautive sur le modèle suivant.

Et c'est un choix *par élément* : les fonds, les images de couverture et les listes qui défilent vont jusqu'au bord ; le texte et les boutons restent dans la zone sûre. Un fond qui s'arrête à la zone sûre laisse des bandes disgracieuses ; un bouton qui déborde sous l'indicateur de geste est **inatteignable**.

15.3.3 Le pointeur : souris et doigt unifiés

Kernel/Runtime/NKEvent/src/NKEvent/NkPointerEvent.h

```
1 enum class NkPointerPhase : uint8 {
2     NK_POINTER_NONE = 0,    // l'événement lu ne concerne pas le pointeur
3     NK_POINTER_DOWN,        // clic gauche enfonce, ou doigt pose
4     NK_POINTER_MOVE,        // déplacement
5     NK_POINTER_UP,          // clic relache, ou doigt leve
6     NK_POINTER_CANCEL       // le système a repris la main (appel entrant, geste OS)
7 };
8
9 struct NkPointer {
10     float32 x = 0.f;
11     float32 y = 0.f;
12     NkPointerPhase phase = NkPointerPhase::NK_POINTER_NONE;
13     uint32 id = 0;           // 0 = souris, ou premier doigt
14     bool fromTouch = false;  // true = doigt
15
16     bool IsValid() const noexcept; // phase != NK_POINTER_NONE
17 };
```

Un clic de souris et un appui du doigt arrivent dans la même méthode, avec la même forme. `fromTouch` dit lequel c'était, pour les rares cas où la distinction compte — une infobulle au survol n'a pas de sens au doigt.

Le piège

x et y sont en pixels *client* : le repère de la zone de dessin, origine en haut à gauche, hors bordure de fenêtre. C'est le même repère qu'une cible de rendu. N'y mêlez jamais des coordonnées écran — le dépôt a payé une journée entière de recherche sur une sélection morte, pour avoir comparé des pixels de fenêtre à des pixels de viseur.

Le piège

Agissez au relâchement, pas à l'appui. `NK_POINTER_UP` permet à l'utilisateur de glisser hors d'un bouton pour annuler son geste — comportement attendu partout, et gratuit ici. Un bouton qui déclenche à l'appui ne pardonne pas l'erreur de visée, et sur un téléphone on vise mal.

Et n'oubliez pas `NK_POINTER_CANCEL` : un appel entrant ou un geste système interrompt la saisie. Le traiter comme un relâchement laisse un objet « collé » au doigt qui n'est plus là.

15.4 Deux coquilles, et il faut choisir

`NkCanvasGuiApp` dérive de `NkCanvasApp` et y ajoute `NKGui` : le contexte, les polices, le backend de dessin. Mais elle *prend la place* du rendu.

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasGuiApp.h

```
1 void OnUpdate(float32 deltaTime) final {
2     mLastDelta = deltaTime;
3     OnTick(deltaTime);
4 }
5
6 void OnRender(NkRenderWindow &target) final {
```

```

7   const math::NkVec2u size = target.GetSize();
8   mGuiContext.BeginFrame(mLastDelta);
9   OnDraw(mGuiContext.dl);
10  mGuiContext.EndFrame();
11  mGuiBackend.Submit(mGuiContext.dl, size.x, size.y);
12  mGuiBackend.Submit(mGuiContext.dlOverlay, size.x, size.y);
13 }

```

Les deux méthodes sont `final`. Une classe qui dérive de `NkCanvasGuiApp` ne peut donc **pas** redéfinir `OnRender` : elle remplit `OnTick` et `OnDraw` à la place, et elle dessine avec une `NkGuiDrawList`, pas avec un `NkRenderer2D`.

Vous dérivez de	Vous dessinez avec	Vous remplissez
rien (à la main)	<code>NkRenderer2D</code>	tout
<code>NkCanvasApp</code>	<code>NkRenderer2D</code>	<code>OnUpdate</code> , <code>OnRender</code>
<code>NkCanvasGuiApp</code>	<code>NkGuiDrawList</code>	<code>OnTick</code> , <code>OnDraw</code>

Ce qu'il faut retenir

Comment choisir. Vous dessinez des formes, des sprites, un monde de jeu : `NkCanvasApp` et `NkRenderer2D`. Vous dessinez une interface — boutons, listes, panneaux : `NkCanvasGuiApp` et la liste d'affichage. Vous voulez comprendre ce qui se passe dessous, ou vous avez un besoin qu'aucune des deux ne couvre : à la main, comme au chapitre précédent.

Ce ne sont pas trois niveaux de qualité : ce sont trois outils pour trois travaux.

15.5 L'écran d'ouverture, offert

Applications/NkLudo/src/Ludo/NkLudoJeu.cpp

```

1  bool NkLudoJeu::OnGuiInit() {
2      mSplash.PoserJeu("Ludo");
3      Rejouer();
4      return true;
5  }
6
7  void NkLudoJeu::OnTick(float32 deltaTime) {
8      if (mSplash.Avancer(deltaTime)) {
9          return; // L'ouverture tient l'ecran
10     }
11     /* ... Le jeu ... */
12 }

```

`NkCanvasSplash` peint deux volets — la marque Rihen, puis le moteur et le nom du jeu — avec le logo officiel embarqué, rasterisé depuis un SVG de 514 octets. `PoserJeu` est la *seul* réglage : les couleurs, les durées et la mise en page sont les mêmes partout, délibérément.

Le piège

`Avancer` rend `true` tant que l'ouverture tient l'écran, et il faut **s'arrêter là**. Sans ce retour, le jeu tournerait *derrière* l'écran de marque : une IA jouerait ses premiers coups pendant les quatre secondes d'ouverture, et le joueur trouverait la partie déjà entamée en arrivant.

Et l'ouverture doit **manger l'appui** qui la saute. Sans cela, le même appui passe le splash et active le bouton qui se trouve dessous — l'utilisateur lance une partie qu'il n'a pas choisie.

Exercice pratique

Reprenez l'application du chapitre précédent, écrite à la main, et portez-la sur NkCanvasApp. Comptez les lignes avant et après.

Puis répondez à deux questions, en les *mesurant* plutôt qu'en les devinant : votre version manuelle gérait-elle OnPause ? Et cédait-elle la main à chaque image ? Compilez-la pour le Web et ouvrez-la dans un navigateur — vous aurez la réponse en une seconde.

Ce chapitre en bref

La coquille tient la fenêtre, la boucle, le cycle de vie mobile, la zone sûre et le pointeur unifié — c'est-à-dire tout le chapitre précédent. Vous ne remplissez que OnGuiInit, OnTick et le dessin. OnRender est final : c'est la seule chose qu'elle vous interdit, et c'est ce qui garantit que les quatre gestes restent dans le bon ordre. Deux coquilles existent, et le choix se fait sur une seule question : avez-vous besoin de NKGui, ou seulement de dessiner ?

Questions de cours

1. Citez trois services que la coquille rend et que vous n'écrivez plus.
2. Pourquoi OnRender est-il final ?
3. Qu'est-ce que la zone sûre, et pourquoi ne se code-t-elle pas en dur ?
4. Sur le Web, que se passe-t-il si la boucle ne cède pas la main ?
5. Quelle question départage NkCanvasApp et NkCanvasGuiApp ?

Exercice de démonstration

Prouvez que la zone sûre est *mesurée* et non devinée.

Affichez, à chaque image, les quatre marges de NkLayoutInfo en clair sur l'écran. Lancez sur ordinateur : elles doivent être nulles ou presque. Lancez sur téléphone, puis faites pivoter l'appareil.

Ce que la démonstration établit : les valeurs *changent* à la rotation. Une marge codée en dur aurait été juste dans une orientation et fautive dans l'autre — et le défaut ne se voit que sur l'appareil, là où il coûte le plus cher.

Exercice de logique

Votre écran d'ouverture dure quatre secondes et se saute d'un appui.

1. Pourquoi la méthode qui l'anime doit-elle rendre « vrai, je tiens l'écran » plutôt que de simplement dessiner ?
2. Décrivez ce qui arrive si elle ne le fait pas, dans un jeu où une IA joue.
3. Pourquoi l'ouverture doit-elle *consommer* l'appui qui la saute ? Décrivez le comportement exact si elle ne le fait pas, du point de vue de l'utilisateur — et non du code.

Ce que cette partie vous laisse

Vous savez dessiner — formes, textures, texte — et construire une interface en mode immédiat, où rien n'est conservé d'une image à l'autre et où tout état qui doit survivre vit *ailleurs* que dans le widget. Vous savez aussi que la coquille existe, ce qu'elle fait pour vous, et la seule chose qu'elle vous interdit : redéfinir `OnRender`, qui est `final`. Toutes les applications de la dernière partie tiennent dans ce que vous venez de lire.

PARTIE V

Les modules

Images, polices, son, vidéo, réseau, caméra

Les modules de cette partie sont **facultatifs**, et c'est leur trait commun : une application peut être complète sans en toucher un seul. On les prend quand on en a besoin, dans l'ordre qu'on veut — ces chapitres ne dépendent pas les uns des autres.

Chacun est écrit *depuis zéro* dans ce dépôt : les décodeurs d'images, les polices, les codecs audio et vidéo, la pile réseau. Ce n'est pas une prouesse qu'on affiche, c'est une contrainte qui a des conséquences pratiques — des capacités qui existent ailleurs et manquent ici, des comportements qui n'appartiennent qu'à nous. Les chapitres le disent quand c'est le cas.

Pour le réseau, la démarche est celle annoncée dans la partie sur les langages : on explique d'abord la façon standard, puis l'alternative Nkentsu et pourquoi elle diffère.

Le dernier chapitre, la **caméra**, est celui qui renverse le plus d'habitudes : avec un fichier vous choisissez, avec une caméra vous subissez. Il montre aussi, code à l'appui, ce qu'on fait d'un réglage qu'on ne peut pas honorer — on le retire, plutôt que de le laisser accepter des valeurs qu'il ignore.

NKImage : charger, fabriquer, afficher des images

Jusqu'ici, tout ce que nous avons affiché était fabriqué par le moteur : rectangles, cercles, atlas de police. Ce chapitre ouvre la porte du monde extérieur. NKImage est le module qui transforme un fichier .png posé sur un disque en un tableau d'octets que le GPU sait consommer — et, dans l'autre sens, un tableau d'octets en fichier.

C'est le premier des cinq modules d'intégration, et ce n'est pas un hasard : c'est le plus simple, le plus stable, et surtout *les autres en dépendent*. NKMedia s'en sert pour son codec MJPEG (qui n'est rien d'autre que le codec JPEG de NKImage appliqué image par image), et NKAudio dépend de NKMedia. L'ordre d'apprentissage est donc imposé par les dépendances :

Ordre imposé par les dépendances (voir les .jenga de chaque module)

```

1 NKImage  └─> NKMedia ──> NKAudio
2              (NKMedia depend de NKImage : codec MJPEG = codec JPEG de NKImage)
3              (NKAudio depend de NKMedia : decodeur Opus)
4              └─> NKFont   (independant : ne depend ni de NKImage ni de NkCanvas)
5
6 NKNetwork (totalement independant – peut venir n'importe ou)
```

16.1 Ce qu'est NKImage, et ce qu'il n'est pas

16.1.1 Aucune dépendance externe

La première chose à regarder, comme toujours, c'est le fichier de build :

Kernel/Runtime/NKImage/NKImage.jenga:25-29

```

1 nkentseudependson(
2   ["NKPlatform", "NKCore", "NKMemory", "NKMath", "NKContainers", "NKLogger",
3   "NKThreading", "NKFileSystem", "NKStream"],
4   selfexport="NKImage",
5   extra_includes=["src"],
6 )
```

Que des modules de base. Pas de libpng, pas de libjpeg, pas de stb_image. Le fichier de build le dit en toutes lettres (NKImage.jenga:6) : « Bibliothèque self-contained sans dépendance externe ». Chaque codec — PNG, JPEG, BMP, TGA, HDR, EXR, PPM, QOI, GIF, ICO, WebP, SVG — est écrit dans le dépôt, y compris le deflate du PNG.

Cela a une conséquence pratique immédiate : **il n'y a rien à installer**. Si le moteur compile, NKImage décode.

16.1.2 NKImage ne connaît aucun GPU

Relisez la liste des dépendances : pas de NKCanvas, pas de NKRHI, pas de NKWindow. NKImage produit et consomme des *octets en mémoire vive*. Il ne sait pas ce qu'est une texture.

Ce qu'il faut retenir

NKImage produit des octets CPU ; le renderer produit un identifiant GPU. Le point de couture entre les deux est toujours le même : un pointeur `const uint8*`, une largeur, une hauteur. Tout le reste de ce chapitre découle de cette phrase.

16.1.3 L'arborescence

Kernel/Runtime/NKImage/ — arborescence

```
1 NKImage/
2   NKImage.jenga
3   src/NKImage/
4     NKImage.h                <- en-tete PARAPLUIE (tout inclure d'un coup)
5     Core/
6       NKImage.h              (1218 l.)    <- LE conteneur + NKImageStream + NkDeflate
7       NKImage.cpp            (3034 l.)
8       NKImageExport.h        ( 58 l.)    <- macros NKENTSEU_IMAGE_API / NKIMG_INLINE
9     Codecs/
10      PNG/  JPEG/  BMP/  TGA/  HDR/  EXR/
11      PPM/  QOI/  GIF/  ICO/  WEBP/  SVG/
```

Une seule classe compte pour ce chapitre : `NkImage`, dans `Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h`. Les codecs sont appelables directement, mais on n'en a presque jamais besoin : `NkImage` choisit le bon selon le contenu du fichier.

L'exemple en tête de `NKImage.h` est périmé

Le commentaire de documentation en tête de l'en-tête parapluie (`Kernel/Runtime/NKImage/src/NKImage/NKImage.h:12-33`) contient un exemple `@code` qui utilise `NkSVGRenderer::RenderFromFile`, `NkSVGDOM` et `NkSVGDOMBuilder`. Ces trois classes n'existent plus. Le pipeline SVG a été remplacé par un codec unique, et le même fichier le dit vingt lignes plus bas :

Kernel/Runtime/NKImage/src/NKImage/NKImage.h:53-54

```
1 // SVG : NkSVGDOM/Renderer ont ete remplaces par NkSVGCodec (codec unique).
```

Ne recopiez jamais cet exemple : il ne compile pas. C'est le premier cas d'une règle que nous retrouverons dans chacun des cinq chapitres qui suivent : **quand un commentaire d'en-tête contredit le `.cpp`, c'est le `.cpp` qui a raison.**

16.2 Les deux vocabulaires de formats

Avant toute chose, il faut distinguer deux choses que le débutant confond systématiquement : le format des *pixels en mémoire* et le format du *fichier sur le disque*.

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:67-75

```
1 enum class NkImagePixelFormat : uint8 {
2     NK_UNKNOWN = 0, NK_GRAY8 = 1, NK_GRAY_A16 = 2,
3     NK_RGB24 = 3, NK_RGBA32 = 4, NK_RGBA128F = 5, NK_RGB96F = 6,
4 };
```

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:127-142

```
1 enum class NkImageFormat : uint8 { // format de FICHIER
2     NK_UNKNOWN=0, NK_PNG, NK_JPEG, NK_BMP, NK_TGA, NK_HDR,
3     NK_PPM, NK_PGM, NK_PBM, NK_QOI, NK_GIF, NK_ICO, NK_SVG, NK_EXR,
4 };
```

Le premier décrit comment un pixel est rangé en RAM : un octet de gris (NK_GRAY8), trois octets (NK_RGB24), quatre (NK_RGBA32), ou quatre flottants 32 bits (NK_RGBA128F, pour le HDR). Le second décrit l'enveloppe du fichier. Un .png peut contenir du NK_GRAY8 comme du NK_RGBA32; un .hdr contient forcément du flottant.

Deux fonctions constexpr font le pont :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:78, 98

```
1 constexpr int32 ChannelsOf(NkImagePixelFormat f) noexcept;
2 constexpr int32 BytesPerPixelOf(NkImagePixelFormat f) noexcept;
```

Enfin, le troisième énuméré du module gouverne le rééchantillonnage :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:157-162

```
1 enum class NkResizeFilter : uint8 {
2     NK_NEAREST, NK_BILINEAR, NK_BICUBIC, NK_LANCZOS3,
3 };
```

Retenez ces quatre noms, nous y reviendrons : deux d'entre eux mentent.

16.3 Deux familles d'API, et pourquoi il faut choisir

C'est le point structurant du module, et la source de la moitié des plantages des débutants. NkImage propose deux façons complètement différentes de faire la même chose. Le commentaire de tête du fichier les nomme lui-même :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:7-36 (abrégé)

```
1 1. API STATIQUE -> retourne `NkImage*` (ownership explicite, heap-alloue)
2 2. API INSTANCE -> retourne `bool` (opere sur *this, aucune allocation visible)
```

16.3.1 L'API instance : l'objet vit sur la pile

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:232, 242, 262, 282

```
1 bool Create(uint32 width, uint32 height, math::NkColor color,
2             int32 desiredChannels = 4) noexcept;
3 bool LoadFromFile(const char* path) override;
4 bool Load(const char* path, int32 desiredChannels = 0) noexcept;
5 bool LoadFromMemory(const void* data, usize size, int32 desiredChannels) noexcept;
```

Toutes ces méthodes renvoient un bool et travaillent sur *this. On déclare une NkImage sur la pile, on l'utilise, et le destructeur libère les pixels. Il n'y a rien à écrire pour libérer.

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:400-407 (abrégé)

```
1     NkImage img;
2     bool ok = false;
3     for (const char* p : kCandidates)
4         if (img.Load(p, 4)) {
5             ok = true;
6             break;
7         }
8     if (ok && img.Width() > 0 && img.Height() > 0 && img.Pixels()) {
```

16.3.2 L'API statique : vous possédez le pointeur

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:308, 317, 792, 804, 815

```
1 static NkImage* Create(uint32 w, uint32 h, int32 desiredChannels = 0, uint32 color = 0)
   noexcept;
2 static NkImage* Create(uint32 w, uint32 h, NkImagePixelFormat fmt, uint32 color = 0) noexcept;
3 static NkImage* Alloc(int32 w, int32 h, NkImagePixelFormat fmt) noexcept;
4 static NkImage* Wrap(uint8* pixels, int32 w, int32 h, NkImagePixelFormat fmt, int32 stride=0)
   noexcept;
5 static NkImage* ConvertToTexture(const NkImage& hdrImage, float exposure=1.0f, float
   gamma=2.2f) noexcept;
```

Chacune renvoie un NkImage* alloué sur le tas. Vous devez appeler ->Free() dessus. Et ce n'est pas tout : **plusieurs méthodes membres renvoient elles aussi une image neuve**, avec la même obligation :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:396, 404, 475, 485, 523

```
1 NkImage* Convert(NkImagePixelFormat newFmt) const noexcept;
2 NkImage* Resize(int32 nw, int32 nh, NkResizeFilter f = NkResizeFilter::NK_BILINEAR) const
   noexcept;
3 NkImage* Crop(int32 x, int32 y, int32 w, int32 h) const noexcept;
4 NkImage* Copy() const noexcept;
5 NkImage* CopyAs(NkImagePixelFormat fmt) const noexcept;
```

C'est le piège le plus courant du module : on charge sur la pile — bien —, puis on appelle Resize — et on oublie que le résultat est un pointeur qu'il faut libérer.

La règle en une phrase

Si la fonction renvoie `boo1`, il n'y a rien à libérer. Si elle renvoie `NkImage*`, vous devez appeler `->Free()`. Aucune exception dans tout le module.

16.3.3 Copie interdite, déplacement autorisé

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:202-215

```
1 NkImage(const NkImage&) = delete; // copie INTERDITE
2 NkImage& operator=(const NkImage&) = delete;
3 NkImage(NkImage&& other) noexcept;
4 NkImage& operator=(NkImage&& other) noexcept;
```

La raison est écrite juste au-dessus ([Core/NkImage.h:178-180](#)) : « La copie par valeur est désactivée (= `delete`) pour éviter les doubles-*free* accidentels. Utiliser `Copy()` (*deep clone*) ou le *move constructor*. » Vous ne pouvez donc pas ranger une `NkImage` dans un `NkVector` par copie; utilisez le déplacement, ou stockez des pointeurs.

16.4 Charger une image

16.4.1 La signature qui compte

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:262

```
1 bool Load(const char *path, int32 desiredChannels = 0) noexcept;
```

Le second paramètre est le seul qui demande réflexion. 0 signifie « garde le nombre de canaux du fichier » : un PNG en niveaux de gris donnera une image `NK_GRAY8`, un PNG avec transparence donnera du `NK_RGBA32`. C'est économe, mais cela veut dire que *vous ne savez pas* ce que vous obtenez avant d'interroger `Format()`.

Passer 4 force la conversion en `NK_RGBA32`. C'est ce que fait la démo, et c'est ce que vous ferez presque toujours dès qu'il s'agit d'envoyer l'image au GPU : les backends veulent du RGBA, et faire la conversion à la lecture évite d'écrire une deuxième passe.

16.4.2 Les accesseurs

Une fois l'image chargée, tout se lit par des accesseurs `inline` ([Core/NkImage.h:527-585](#)) :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:527-585 (liste)

```
1 Pixels() // uint8* : Le premier octet de la première ligne
2 Width() Height()
3 Channels() // 1, 2, 3 ou 4
4 BytesPP() // octets par pixel
5 Stride() // octets par LIGNE <- Lire la section suivante
6 Format() // NkImagePixelFormat courant
7 SourceFormat() // NkImageFormat du fichier d'origine
8 IsValid() IsHDR() TotalBytes()
9 RowPtr(y) // uint8* sur la ligne y
```

16.4.3 Le stride : la cause numéro un des images « penchées »

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:182-184

```
1 STRIDE :
2 Le stride (bytes par ligne) est aligne sur 4 octets : stride = (w*bpp+3)&~3.
3 Utiliser RowPtr(y) pour acceder a la ligne y de facon portable.
```

Traduisons. Pour une image RGB24 de 5 pixels de large, une ligne fait $5 \times 3 = 15$ octets — mais le module en réserve 16, pour que chaque ligne commence à une adresse multiple de 4. Il y a donc **un octet de bourrage invisible à la fin de chaque ligne**.

Conséquence : `Pixels()` n'est pas un tableau contigu de `width * height * bpp` octets. Un `memcpy` global qui suppose le contraire produit une image décalée d'un pixel de plus à chaque ligne — l'effet « penché » caractéristique. La parade est de recopier ligne par ligne :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:410-419

```
1 static NkVector<uint8> rgba;
2 rgba.Resize(static_cast<usize>(imgW) * imgH * 4u);
3 for (int32 y = 0; y < imgH; ++y) {
4     const uint8 *src = img.RowPtr(y);
5     uint8 *dst = rgba.Data() + static_cast<usize>(y) * imgW * 4u;
6     for (int32 x = 0; x < imgW * 4; ++x)
7         dst[x] = src[x];
8 }
```

Quand le stride ne se voit pas

En RGBA32, `bpp = 4`, donc `w*4` est *toujours* multiple de 4 et `stride == w*4`. Un `memcpy` global marchera. Puis un jour vous chargerez un JPEG en trois canaux, et tout se penchera — dans un code que vous n'avez pas touché. **Utilisez `RowPtr(y)` dès la première ligne que vous écrivez**, même quand ce n'est pas nécessaire ce jour-là.

16.5 Fabriquer une image de toutes pièces

16.5.1 Le squelette canonique de l'API statique

L'application `NKImageCodecTest` est le meilleur exemple court du dépôt. Elle alloue une image, écrit les pixels à la main, et la sauvegarde dans neuf formats :

Applications/NKImageCodecTest/src/main.cpp:15-27 (abrégé)

```
1 NkImage *img = NkImage::Alloc(W, H, NkImagePixelFormat::NK_RGB24);
2 if (!img) {
3     printf("[ERREUR] Alloc\n");
4     return 1;
5 }
6 nk_uint8 *db = img->Pixels();
7 const int st = img->Stride();
8 for (int y = 0; y < H; ++y) {
9     for (int x = 0; x < W; ++x) {
10         /* ... remplissage motif ... */
11         nk_uint8 *p = db + (nk_size)y * st + x * 3;
12         p[0] = r; p[1] = g; p[2] = b;
13     }
```

```
14     }
```

Remarquez l'arithmétique : $db + y * st + x * 3$. C'est `Stride()` qui sert de pas vertical, jamais `Width() * 3`. Puis :

Applications/NKImageCodecTest/src/main.cpp:70-78 (abrégé)

```
1     for (const Fmt &f : formats) {
2         const bool ok = img->Save(f.file, 100);
3         printf("  %-12s : %s\n", f.file, ok ? "ecrit" : "EHEC SAVE");
4     }
5     img->Free();
```

Alloc → Pixels() + Stride() → Save → Free. Voilà le squelette complet.

16.5.2 Créer une image remplie d'une couleur

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:232

```
1 bool Create(uint32 width, uint32 height, math::NkColor color,
2             int32 desiredChannels = 4) noexcept;
```

Cette signature a une histoire, et le dépôt l'a documentée sur les lieux du crime :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:72-75

```
1 // NkImage::Create(w, h, NkColor color, int32 channels=4) : La COULEUR
2 // d'abord, Les canaux ensuite. (Avant : args inverses -> channels=0
3 // pour un fill transparent -> Create echouait -> texture jamais creee.)
```

Couleur d'abord, canaux ensuite

Il existe aussi une *Create statique* dont le troisième paramètre est le nombre de canaux et le quatrième la couleur — l'ordre inverse ([Core/NkImage.h:308](#)). Les deux compilent. L'une des deux ne fait pas ce que vous croyez. Vérifiez la famille d'API que vous employez : `bool` ou `NkImage*`.

16.5.3 Dessiner dans une image (sur le processeur)

`NkImage` embarque un petit rasteriseur logiciel, entièrement `inline` dans l'en-tête ([Core/NkImage.h:587-765](#)) — vous pouvez donc lire son code sans quitter la déclaration :

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:587-765 (liste)

```
1 SetPixel GetPixel BlendPixel Fill
2 DrawHLine DrawVLine DrawLine // Bresenham
3 DrawRect FillRect
4 DrawCircle FillCircle
5 DrawEllipse FillEllipse
```

Toutes prennent une `math::NkColor`. Un exemple complet, écrit pour ce cours :

Exemple écrit pour le cours (API : Core/NkImage.h:232, 587-765, 349)

```
1 #include "NkImage/Core/NkImage.h"
2 #include "NkMath/NkColor.h"
3 using namespace nkentseu;
4
5 NkImage canvas;
6 canvas.Create(640, 480, math::NkColor(20, 20, 28, 255), 4);
7 canvas.FillRect(40, 40, 200, 120, math::NkColor(255, 90, 0, 255));
8 canvas.DrawCircle(320, 240, 100, math::NkColor(0, 245, 255, 255));
9 canvas.DrawLine(0, 0, 639, 479, math::NkColor(255, 255, 255, 255));
10 canvas.SavePNG("dessin.png");
```

Deux limites, énoncées par le module lui-même :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:587-590

```
1  Formats LDR 8-bit (RGBA/RGB/RG/R). No-op si image invalide, hors
2  bornes, ou HDR. Couleur = math::NkColor (= NkColor2D). SetPixel ECRASE
3  (pas de blending) ; BlendPixel fait un src-over alpha.
```

«No-op si hors bornes» : dessiner à côté de l'image ne plante pas et ne prévient pas. Et sur une image HDR, ces méthodes ne font strictement rien — silencieusement.

16.6 Écrire sur le disque

16.6.1 Le dispatch par extension

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:348-360

```
1 bool Save(const char* path, int32 quality = 90) const noexcept; // dispatch par extension
2 bool SavePNG (const char* path) const noexcept;
3 bool SaveJPEG(const char* path, int32 quality = 90) const noexcept;
4 bool SaveBMP (const char* path) const noexcept;
5 bool SaveTGA (const char* path) const noexcept;
6 bool SavePPM (const char* path) const noexcept;
7 bool SaveHDR (const char* path) const noexcept;
8 bool SaveEXR (const char* path) const noexcept; // EXR scanline FLOAT, compression NONE
9 bool SaveQOI (const char* path) const noexcept;
10 bool SaveGIF (const char* path) const noexcept; // palette 256 (median-cut) + LZW
11 bool SaveWebP(const char* path, bool lossless = true, int32 quality = 90) const noexcept;
12 bool SaveSVG (const char* path) const noexcept;
```

Save("photo.png") appelle SavePNG; Save("photo.jpg", 80) appelle SaveJPEG avec une qualité de 80. Le dispatch se lit dans [Core/NkImage.cpp:2010-2030](#).

16.6.2 Les deux dernières lignes sont des mensonges

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.cpp:2119-2134

```
1 /**
2  * SaveWebP - non implémenté.
3  * Nécessiterait Libwebp ou une implémentation custom VP8/VP8L.
4  */
5 bool NkImage::SaveWebP(const char *, bool, int32) const noexcept {
```

```

6     return false;
7 }
8
9 /**
10  * SaveSVG — non implémenté.
11  * L'encodage raster → SVG (vectorisation) n'est pas dans le scope.
12  */
13 bool NkImage::SaveSVG(const char *) const noexcept {
14     return false;
15 }

```

SaveWebP et SaveSVG renvoient toujours false

Ces deux méthodes sont déclarées, exportées, documentées — et vides. Elles ne plantent pas : elles renvoient false sans rien écrire. Si vous ne testez pas la valeur de retour, votre programme fera comme si tout allait bien.

Corollaire moins évident : Save("sortie.webp") échoue aussi, parce que l'extension webp n'est pas dans le dispatch de Save() ([Core/NkImage.cpp:2010-2030](#)). Le fichier de test du dépôt le prouve malgré lui : NkImageCodecTest liste ic_out.webp dans ses formats attendus, et cette ligne affiche ECHEC SAVE à chaque exécution.

Nuance importante : le codec WebP, lui, existe et fonctionne (NkWebPCodec::Encode, [Codecs/WEBP/NkWebPCodec.h:32](#), chemin VP8L sans perte). C'est uniquement la méthode de commodité NkImage::SaveWebP qui est vide. Si vous avez vraiment besoin de WebP en écriture, appelez le codec directement.

16.6.3 Encoder en mémoire

Parfois on ne veut pas de fichier : on veut les octets, pour les envoyer sur le réseau ou les ranger dans une base.

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:369-373

```

1 bool EncodePNG (uint8*& out,  uint& size) const noexcept;
2 bool EncodeJPEG(uint8*& out,  uint& size, int32 quality = 90) const noexcept;
3 bool EncodeBMP (uint8*& out,  uint& size) const noexcept;
4 bool EncodeTGA (uint8*& out,  uint& size) const noexcept;
5 bool EncodeQOI (uint8*& out,  uint& size) const noexcept;

```

Le paramètre out est une *référence de pointeur* : la fonction alloue le tampon et vous le rend. La question devient donc : qui le libère, et avec quoi ?

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:31-36 (abrégé)

```

1  REGLE DE MEMOIRE :
2  Les buffers alloués via l'API statique DOIVENT être libérés avec img->Free().
3  Les buffers encodés (EncodePNG, EncodeJPEG, ...) DOIVENT être libérés avec
4  nkentseu::memory::NkFree(ptr).
5  [...] l'allocateur custom NKMemory n'est pas compatible avec le heap CRT
6  standard (crash c0000374 sur Windows en cas de mélange).

```

c0000374 est le code d'un tas Windows corrompu. Le plantage n'arrive d'ailleurs pas au moment de la libération fautive, mais bien plus tard, dans une allocation parfaitement innocente — d'où la difficulté à le diagnostiquer.

Exemple écrit pour le cours (API : Core/NkImage.h:369, NKMemory)

```
1 uint8 *buf = nullptr;
2 usize n = 0;
3 if (src.EncodePNG(buf, n)) {
4     /* ... envoyer buf/n sur le reseau, dans une base, ... */
5     memory::NkFree(buf);          // et JAMAIS free() ni delete[]
6 }
```

16.7 Transformer une image

16.7.1 Redimensionner

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:404

```
1 NkImage* Resize(int32 nw, int32 nh, NkResizeFilter f = NkResizeFilter::NK_BILINEAR) const
    noexcept;
```

Rappel : cela renvoie un NkImage*. L'usage complet :

Exemple écrit pour le cours (API : Core/NkImage.h:262, 404, 349, 775)

```
1 NkImage src;
2 if (!src.Load("photo.jpg", 4))          // 4 = force RGBA32
3     return 1;
4
5 NkImage *thumb = src.Resize(256, 256, NkResizeFilter::NK_BILINEAR);
6 if (!thumb) return 1;
7
8 thumb->Save("photo_thumb.png");          // format deduit de l'extension
9 thumb->Free();                            // OBLIGATOIRE (tas)
10 // ~NkImage() libere src : rien a ecrire.
```

NK_BICUBIC et NK_LANCZOS3 ne font pas ce qu'ils disent

Les quatre filtres compilent. Deux seulement existent.

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.cpp:2901-2903

```
1 // — Bilineaire (défaut pour NK_BILINEAR, NK_BICUBIC, NK_LANCZOS3) —
2 //
3 // Note : NK_BICUBIC et NK_LANCZOS3 ne sont pas encore implémentés ici.
```

Et la documentation de la méthode le répète (Core/NkImage.cpp:3009) : «NK_BICUBIC, NK_LANCZOS3 : *fallback* bilinéaire (non encore implémentés).»

Demander du Lanczos ne plante pas, ne journalise rien, et vous rend du bilinéaire. C'est la pire catégorie de bug : celui qui produit un résultat plausible. Si la qualité de vos vignettes compte, sachez que vous avez du bilinéaire, quoi que dise votre code.

16.7.2 Recadrer, convertir, recopier

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:378-388, 415-464, 505-514

```
1 void FlipVertical() noexcept;
2 void FlipHorizontal() noexcept;
3 void PremultiplyAlpha() noexcept; // RGBA32 seulement
4 void Blit(const NkImage& src, int32 dstX, int32 dstY) noexcept;
5 bool BlitRegion(const NkImage& src, const math::NkIntRect& srcRegion,
6               const math::NkIntRect& dstRegion) noexcept;
7 bool BlitRegion(const NkImage& src, const math::NkIntRect& srcRegion,
8               const math::NkIntRect& dstRegion, NkResizeFilter filter) noexcept;
9 bool Copy(const NkImage& src, int32 dstX, int32 dstY,
10          const math::NkIntRect& area, bool clip = true) noexcept;
11 bool CopyTo(NkImage& dst) const noexcept;
```

Notez le jeu de noms : `Copy(...)` avec arguments est une méthode *instance* qui renvoie `bool` et copie *dans* `*this`; `Copy()` sans argument renvoie un `NkImage*` neuf. Deux méthodes homonymes, deux régimes de mémoire opposés. C'est exactement pourquoi la règle « regardez le type de retour » n'est pas une coquetterie.

16.7.3 Le cas HDR

Une image `.hdr` ou `.exr` contient des flottants dont les valeurs dépassent 1,0 — c'est tout l'intérêt. La convertir naïvement en 8 bits détruit cette information :

Applications/NkImageDemo/src/Demo/ViewerApp.cpp:369-372

```
1 NkImage::Load(.hdr, 4) ferait un Convert lineaire-clamp qui
2 crame toutes les valeurs > 1.0. On passe par [le codec HDR]
3 pour préserver le float, puis ConvertToTexture(exposure,gamma).
```

La bonne séquence est donc : décoder en flottant (le codec HDR le fait), puis appliquer un *tone mapping* explicite avec `ConvertToTexture(image, exposure, gamma)` ([Core/NkImage.h:815](#)), qui rend une image 8 bits affichable.

16.8 Le cas particulier du GIF animé

Le codec GIF a une API à part, parce qu'un GIF peut contenir plusieurs images :

Kernel/Runtime/NkImage/src/NkImage/Codecs/GIF/NkGIFCodec.h:36-53

```
1 struct NkGIFFrame {
2     NkImage *image = nullptr; ///< Frame RGBA32, taille canvas global
3     uint32 delayMs = 0;      ///< Duree d'affichage avant frame suivante
4     uint16 left = 0;         ///< Position originale (info, deja composee)
5     uint16 top = 0;
6     uint8 disposal = 0;      ///< 0=undef,1=keep,2=clear,3=restore (info)
7 };
8 struct NkGIFAnimation {
9     uint32 width = 0; uint32 height = 0; uint32 frameCount = 0;
10    NkGIFFrame *frames = nullptr; ///< Array de frameCount entries
11    uint16 loopCount = 0;          ///< 0 = infini (NETSCAPE2.0)
12 };
```

Le mot important est « déjà composée » : vous n'avez pas à gérer vous-même les modes de *disposal* du format GIF, chaque `frames[i].image` est une image RGBA32 complète, à la taille du canevas global. Les champs `left`, `top` et `disposal` ne sont là que pour information.

Exemple écrit pour le cours (API : `Codecs/GIF/NkGIFCodec.h:64, 67`)

```
1 #include "NKImage/Codecs/GIF/NkGIFCodec.h"
2
3 // data/bytes = contenu brut du .gif lu en memoire
4 NkGIFAnimation *anim = NkGIFCodec::DecodeAnimation(data, bytes);
5 if (anim) {
6     for (uint32 i = 0; i < anim->frameCount; ++i) {
7         NkImage *f = anim->frames[i].image;    // RGBA32, deja composee
8         uint32 d = anim->frames[i].delayMs;    // duree d'affichage
9         /* ... uploader f ... */
10    }
11    NkGIFCodec::FreeAnimation(anim);    // Libere le struct ET toutes les NkImage*
12 }
```

Un seul appel à `FreeAnimation` libère tout. N'appellez pas `Free()` sur les images individuelles : elles appartiennent à l'animation.

16.9 Libérer correctement : les quatre cas

Récapitulons, parce que c'est ici que les débutants perdent leurs soirées.

Origine de l'image	Libération	Exemple
<code>NkImage img;</code> sur la pile	rien (destructeur)	<code>img.Load(...)</code>
Fabrique statique / <code>Resize</code>	<code>img->Free()</code>	<code>NkImage::Alloc(...)</code>
Tampon <code>Encode*</code>	<code>memory::NkFree(buf)</code>	<code>EncodePNG(buf,n)</code>
<code>NkGIFCodec::DecodeAnimation</code>	<code>FreeAnimation(anim)</code>	animation entière

Et une cinquième méthode, qui n'appartient à aucune de ces lignes :

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:769-783

```
1 Libere les pixels (si owning) ET libere le struct NkImage lui-meme via nkFree.
2 A utiliser UNIQUEMENT sur les images creees via les fabriques statiques
3 (Alloc, Wrap, Create statique, Copy, CopyAs, Convert, Resize, Crop, ...).
4 Ne jamais appeler Free() sur une image allouee sur la stack.
5
6 [NKIResource] Unload() : libere les pixels (si owning) et remet *this dans
7 l'etat « image vide » SANS liberer le struct (contrairement a Free()).
```

`Unload()` sert quand vous voulez réutiliser une `NkImage` de pile pour charger autre chose sans attendre la fin de la portée. `Free()` sur une image de pile, en revanche, tente de libérer une adresse qui n'a jamais été allouée : plantage garanti.

Wrap() ne possède rien

Kernel/Runtime/NKImage/src/NKImage/Core/NKImage.h:795-800

- 1 Cree une vue non-owning sur un buffer pixel externe.
- 2 Le buffer n'est PAS libere par le destructeur ni par Free().
- 3 L'appelant reste responsable de la duree de vie du buffer.

Wrap est très pratique : il permet d'appliquer les méthodes de NKImage (dessin, Save, Encode) à des pixels que vous possédez déjà, sans copie. Mais la NKImage obtenue est une *vue* : si le tampon d'origine meurt avant elle, toute lecture est une lecture après libération.

16.10 Afficher une image dans une interface NKGui

Nous y voilà. C'est la partie que vous attendez, et elle tient en trois gestes : charger, uploader, déclarer un widget.

16.10.1 Le widget

Kernel/Runtime/NKGui/src/NKGui/Widgets/NKGuiWidgets.h:96-105

```
1 // — Image / Icône (quad texturé) —————
2 // Dessine la texture `texId` (id backend) à la taille w×h. `tint` multiplie
3 // (blanc = telle quelle) ; `uv0/uv1` = sous-région (atlas/sprite). Auto-layout.
4 void Image(NKGuiContext &ctx, uint32 texId, float32 w, float32 h,
5            NKColor tint = NKColor{255, 255, 255, 255}, NKVec2 uv0 = NKVec2{0.f, 0.f},
6            NKVec2 uv1 = NKVec2{1.f, 1.f}) noexcept;
7 // Bouton-image (icône cliquable) : fond + image centrée + survol. Retourne true au
8 // clic.
9 bool ImageButton(NKGuiContext &ctx, const char *idStr, uint32 texId, float32 w,
10 float32 h,
11                 NKColor tint = NKColor{255, 255, 255, 255}, NKVec2 uv0 = NKVec2{0.f,
12 0.f},
13                 NKVec2 uv1 = NKVec2{1.f, 1.f}) noexcept;
```

Le premier paramètre après le contexte est un uint32 texId. Pas un pointeur, pas une NKImage : un simple entier. NKGui ne sait rien de votre image ; il écrit ce nombre dans sa liste de commandes et laisse le backend le résoudre. C'est la conséquence directe de l'architecture décrite au chapitre 1.

16.10.2 L'upload

Le pont vers NKCanvas expose la méthode qui associe un texId à des pixels :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NKGuiCanvasBackend.h:94

```
1 bool UploadImageRGBA(uint32 texId, const uint8 *rgba, int32 w, int32 h);
```

Le nom dit l'exigence : **RGBA**, et implicitement *contigu* (pas de stride). D'où la recopie ligne par ligne que nous avons vue plus haut.

16.10.3 La séquence complète

Voici le bloc réel de la démo, dans son intégralité. C'est l'exemple à recopier :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:290-322

```
1 // — VRAIE image chargée depuis le disque (NKImage) -> texture backend —
2 uint32 imgTexId = 0u;
3 int32 imgW = 0, imgH = 0;
4 {
5     static const char *kCandidates[] = {
6         "Applications/Mou/assets/brand/rihen-logo.png",
7         "Applications/Mou/assets/brand/noge-logo.png",
8         "Resources/Icons/ContentBrowser/FileIcon.png",
9         "../Applications/Mou/assets/brand/rihen-logo.png",
10    };
11    NKImage img;
12    bool ok = false;
13    for (const char *p : kCandidates)
14        if (img.Load(p, 4)) {
15            ok = true;
16            break;
17        }
18    if (ok && img.Width() > 0 && img.Height() > 0 && img.Pixels()) {
19        imgW = img.Width();
20        imgH = img.Height();
21        static NkVector<uint8> rgba;
22        rgba.Resize(static_cast<usize>(imgW) * imgH * 4u);
23        for (int32 y = 0; y < imgH; ++y) {
24            const uint8 *src = img.RowPtr(y);
25            uint8 *dst = rgba.Data() + static_cast<usize>(y) * imgW * 4;
26            for (int32 x = 0; x < imgW * 4; ++x)
27                dst[x] = src[x];
28        }
29        imgTexId = 0x494D4731u; // 'IMG1'
30        backend.UploadImageRGBA(imgTexId, rgba.Data(), imgW, imgH);
31        logger.Info("Image chargée : {0}x{1}", imgW, imgH);
32    } else {
33        logger.Info("Image demo introuvable (cwd) -> section image vide");
34    }
35 }
```

Six observations, dans l'ordre où elles comptent :

1. **Une liste de chemins candidats.** L'application ne sait pas depuis quel répertoire on la lance; elle essaie plusieurs chemins et prend le premier qui marche. Ce n'est pas de la paresse, c'est la réponse honnête à un vrai problème.
2. **NKImage img;** sur la pile, donc aucun Free() à écrire. C'est la voie recommandée.
3. **img.Load(p, 4)** — on force quatre canaux, parce que UploadImageRGBA attend du RGBA.
4. **La recopie ligne par ligne** avec RowPtr(y), pour la raison expliquée plus haut.
5. **imgTexId = 0x494D4731u** — soit 'IMG1' en ASCII. L'identifiant est choisi *par vous*, arbitrairement; il suffit qu'il soit stable et distinct des autres (l'atlas de police, par exemple, utilise 'NKFT'). Un entier lisible en hexadécimal aide énormément au débogage.
6. **Le else journalise.** Sans lui, une image manquante donne simplement... rien. Aucun message. C'est le silence dont parlait le chapitre 1.

Puis, dans la frame, le widget :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:626-637

```
1      Text(ctx, "Image réelle (chargee via NKImage) :");
2      if (imgTexId != 0u && imgW > 0 && imgH > 0) {
3          // Affiche l'image a une largeur fixe en preservant le ratio.
4          const float32 dispW = 150.f;
5          const float32 dispH = dispW * static_cast<float32>(imgH) /
static_cast<float32>(imgW);
6          Image(ctx, imgTexId, dispW, dispH);
7          ctx.SameLine();
8          static int32 ibClicks = 0;
9          if (ImageButton(ctx, "ib", imgTexId, 44.f, 44.f))
10             ++ibClicks;
11     } else {
12         Text(ctx, "(aucune image trouvee)");
13     }
```

Le calcul de `dispH` mérite d'être noté : `Image` ne préserve *pas* le rapport d'aspect, elle dessine exactement le rectangle que vous demandez. Si vous lui donnez une taille carrée pour une image rectangulaire, elle l'écrase sans se plaindre.

Les trois gestes

1. `NKImage img; img.Load(chemin, 4);` — charge et force le RGBA;
 2. `backend.UploadImageRGBA(monId, pixelsContigus, w, h);` — une fois, à l'initialisation;
 3. `Image(ctx, monId, w, h);` — chaque frame, dans la déclaration d'interface.
- La `NKImage` peut mourir après l'étape 2 : le backend a copié les pixels côté GPU. C'est l'entier `monId` qui doit survivre.

16.10.4 Le chemin plus court : `NkTexture` de `NkCanvas`

Si vous n'êtes pas dans une interface `NKGui` mais dans une scène `NkCanvas`, il existe un raccourci qui fait tout :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.h:53-58

```
1 bool Create(NKIRenderer2D &renderer, uint32 width, uint32 height, const NkColor2D &fillColor);
2 bool LoadFromFile(NKIRenderer2D &renderer, const char *path);
3 bool LoadFromImage(NKIRenderer2D &renderer, const NKImage &image, const NkRect2i &area =
    NkRect2i{});
4 bool LoadFromMemory(NKIRenderer2D &renderer, const void *data, uint sizeBytes);
```

Son implémentation montre qu'il n'y a aucune magie — c'est exactement ce que vous auriez écrit :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:78-86

```
1 bool NkTexture::LoadFromFile(NKIRenderer2D &renderer, const char *path) {
2     NKImage img;
3     if (!img.Load(path))
4         return false;
5     return LoadFromImage(renderer, img);
6 }
```

Créer les textures *après* Initialize()

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:17-19

```
1 Active backend dispatch table. Initialement vide (callbacks null) :  
2 les operations sur NkTexture sont des no-ops tant qu'aucun renderer  
3 n'a appele NkTextureSetBackend() a la fin de son Initialize().
```

Autrement dit : une NkTexture créée avant que le renderer soit initialisé ne fait **rien**, et ne dit rien. C'est un invariant d'ordre pur. Créez vos textures après `renderer.Initialize()`, toujours.

16.11 Décoder ailleurs, uploader ici

Décoder un JPEG de huit mégapixels prend des dizaines de millisecondes : bien plus qu'une image à 60 Hz. La tentation est de le faire sur un fil de travail.

Le module ne l'interdit pas, mais il ne le promet pas non plus : le `.jenga` liste `NKThreading` en dépendance, et aucun verrou n'est exposé dans l'API publique. Aucun commentaire du module ne garantit la *thread-safety*. La règle prudente, qui est celle de la visionneuse du dépôt ([Applications/NkImageDemo/src/Demo/ViewerApp.cpp:463](#), « Upload sur le main thread (libere l'image apres) »), tient en une ligne :

Ce qu'il faut retenir

Décodage dans un fil de travail, upload sur le fil principal. Le décodage produit des octets — c'est du calcul pur, isolé. L'upload touche au contexte graphique, qui appartient au fil principal.

16.12 État réel du module

Terminons par l'inventaire honnête, parce que c'est ce qui vous fera gagner du temps.

16.12.1 Ce qui fonctionne

Format	Lecture	Écriture
PNG	oui	oui
JPEG	oui	oui
BMP	oui	oui
TGA	oui	oui
PPM / PGM / PBM	oui	oui
QOI	oui	oui
HDR	oui	oui
EXR	oui	oui (scanline float, compression NONE)
GIF	oui (statique <i>et</i> animé)	oui (palette 256 + LZW)
ICO	oui	aucun encodeur
WebP	oui	par le codec seulement, pas par Save
SVG	oui (rastérisation)	non

S'y ajoutent : le redimensionnement en `NK_NEAREST` et `NK_BILINEAR`, et toutes les primitives de dessin CPU.

16.12.2 Ce qui ne fonctionne pas

- `NkImage::SaveWebP` et `NkImage::SaveSVG` renvoient `false` sans rien faire (`Core/NkImage.cpp:2119-2134`);
- `NK_BICUBIC` et `NK_LANCZOS3` retombent silencieusement sur du bilinéaire (`Core/NkImage.cpp:2901-2903`);
- l'exemple en tête de `src/NkImage/NkImage.h` cite des classes supprimées.

NkImageDemo ne compile plus

Vous trouverez dans le dépôt une application `Applications/NkImageDemo`, déclarée dans `Nkentseu.jenga:1447`. Ne la prenez pas pour modèle d'appel :

`Applications/NkImageDemo/src/Demo/ViewerApp.cpp:450`

```
1      NkImage *img = NkImage::Load(path.CStr(), 4);
```

Il n'existe **aucune** `static NkImage* Load(...)`. Le seul `Load` du module est la méthode d'instance `bool Load(const char*, int32)` (`Core/NkImage.h:262`). Cette démo n'apparaît d'ailleurs pas dans `Build/Bin/Debug-Windows/` : elle ne compile plus contre l'API actuelle. Sa *logique* en revanche reste excellente à lire — parcours de dossier, lecture de GIF animé, *tone mapping* HDR. Lisez-la comme un plan, pas comme une référence d'API.

16.12.3 Les deux classes utilitaires exportées

Pour mémoire, parce que vous les croiserez en lisant les codecs :

- `NkImageStream` (`Core/NkImage.h:933-1043`) — un lecteur/écrivain binaire qui sait lire en *big endian* comme en *little endian* : `ReadU16BE`, `ReadU32LE`, `ReadBytes`, `Skip`, `Seek`, `Tell`, `HasError`, et les `Write*` symétriques. C'est l'outil avec lequel tous les codecs du module sont écrits;
- `NkDeflate` (`Core/NkImage.h:1085-1217`) — la compression *deflate/inflate* du PNG, exposée publiquement : `Decompress`, `DecompressRaw`, `Compress`. Utile bien au-delà des images.

Ce chapitre en bref

`NkImage` décode et encode des pixels, et *rien d'autre* : il ne connaît ni GPU ni fenêtre. Deux familles d'API cohabitent et ne se mélangent pas — la question à se poser est toujours *qui possède les pixels*, car c'est elle qui décide comment on les libère, et les quatre cas ne se devinent pas. Le **pas de ligne** (*stride*) n'est pas la largeur : le supposer produit une image inclinée, un défaut visuellement spectaculaire et à cause discrète. Enfin, tous les formats ne savent pas tout enregistrer — un aller-retour peut perdre la transparence sans se plaindre.

16.13 Exercices

1 — Le round-trip, et le format qui échoue

Écrivez un petit programme console qui :

1. alloue une image RGB24 de 256×256 via `NkImage::Alloc`;
2. la remplit d'un dégradé à la main, en utilisant `Stride()`;
3. la sauvegarde en `.png`, `.bmp`, `.tga`, `.qoi` et `.webp`, en **testant chaque valeur de retour** et en affichant le résultat;
4. recharge le `.png` et compare octet par octet avec l'original.

Une seule des cinq sauvegardes doit échouer. Laquelle, et pourquoi? Inspirez-vous de [Applications/NkImageCodecTest/src/main.cpp](#), qui fait exactement cela en 79 lignes.

2 — Rendre le stride visible

Chargez le même PNG deux fois : une fois avec `Load(path, 3)`, une fois avec `Load(path, 4)`. Pour chacune, affichez `Width()`, `Channels()`, `BytesPP()` et `Stride()`. Choisissez une largeur qui n'est pas multiple de 4 (par exemple 101 pixels) et vérifiez que `Stride() != Width() * BytesPP()` dans le cas à trois canaux.

Puis, volontairement, écrivez la conversion en RGBA avec un `memcpy` global au lieu de `RowPtr`, affichez le résultat, et regardez la diagonale apparaître. Une erreur qu'on a vue une fois ne se refait pas.

3 — Une vignette dans une interface

Reprenez l'application `NKGui` du chapitre 4 et ajoutez-lui un panneau qui :

- charge une image du dépôt (essayez plusieurs chemins candidats);
- en fabrique une vignette de 128 pixels de large avec `Resize`, **en préservant le rapport d'aspect**;
- upload la vignette sous un `texId` de votre choix;
- l'affiche avec `Image`, et affiche à côté ses dimensions d'origine avec `Text`.

Vérifiez ensuite avec un débogueur — ou un simple compteur — que vous avez bien appelé `Free()` exactement une fois sur le résultat de `Resize`, et zéro fois sur l'image de pile.

4 — Mesurer le mensonge

Chargez une photographie assez grande, puis produisez deux vignettes de la même taille : une avec `NK_BILINEAR`, une avec `NK_LANCZOS3`. Sauvegardez les deux en PNG, puis comparez-les octet par octet.

Elles doivent être **rigoureusement identiques**. Vous venez de démontrer vous-même, sans lire le `.cpp`, que le filtre Lanczos n'existe pas. C'est exactement la démarche à appliquer chaque fois qu'une API vous promet quelque chose que vous n'avez pas vu marcher.

Questions de cours

1. Que fait NKImage, et que ne fait-il jamais ?
2. Qu'est-ce que le pas de ligne, et pourquoi ne vaut-il pas toujours la largeur en octets ?
3. Citez les quatre cas de libération et dites ce qui les distingue.
4. Pourquoi un aller-retour d'enregistrement peut-il perdre de l'information sans erreur ?
5. Comment une image chargée arrive-t-elle à l'écran dans une interface NKGui ?

Exercice de logique

Une image chargée s'affiche « en biais », chaque ligne décalée un peu plus que la précédente.

1. Expliquez le mécanisme exact qui produit cette forme, et pourquoi le décalage *s'accumule*.
2. Le décalage est parfois nul pour certaines images et pas pour d'autres. Quelle propriété de l'image décide ? Donnez la condition précise.
3. Un collègue « corrige » en redimensionnant l'image à une largeur multiple de quatre. Ça marche. Dites en une phrase pourquoi c'est le pire des correctifs.

NKFont : polices, atlas et métriques

Afficher du texte est l'opération la plus banale d'une interface et la plus mal comprise. On croit qu'il suffit d'« écrire une chaîne » ; en réalité, il faut convertir des octets UTF-8 en *codepoints*, trouver le dessin de chaque caractère, le rastériser une fois pour toutes dans une grande image, puis émettre deux triangles texturés par lettre en faisant avancer un curseur.

NKFont fait tout cela, et **rien d'autre**. C'est un module remarquablement honnête sur son périmètre : il produit un bitmap en mémoire vive, et vous rend pour chaque caractère un rectangle à l'écran et un rectangle dans le bitmap. Ce qu'on en fait ensuite ne le regarde pas.

17.1 Le module ne connaît ni image, ni GPU, ni fenêtre

Kernel/Runtime/NKFont/NKFont.jenga:35-39

```
1 nkentseudependson(
2   ["NKPlatform", "NKCore", "NKMemory", "NKMath", "NKContainers", "NKThreading",
   "NKLogger"],
3   selfexport="NKFont",
4   extra_includes=["src"],
5 )
```

Ni NKImage, ni NKCanvas, ni NKWindow. NKFont est même plus autonome que NKImage : il ne dépend pas du système de fichiers pour fonctionner, puisqu'il embarque ses propres polices — nous y venons.

Kernel/Runtime/NKFont/ — arborescence

```
1 NKFont/
2   NKFont.jenga
3   src/NKFont/
4     NKFont.h           ( 366 l.) <- EN-TETE PUBLIC PRINCIPAL (NkFontAtlas + NkFont)
5     NkFontAtlas.cpp    (~ 700 l.) <- packing + rasterisation
6     NkFontMesh.cpp     (~ 400 l.) <- extrusion 3D des glyphes
7   Core/
8     NkFontTypes.h      <- alias nkft_*, NkFontCodepoint, NkGlyphId
9     NkFontParser.h/.cpp <- parser TTF/OTF/CFF/WOFF + rasteriseur + SDF
10    NkFontRasterizer.cpp
11    NkFontDetect.h/.cpp <- NkFontProfile / NkFontDetector
12    NkFontSizeCache.h/.cpp <- cache multi-tailles
13  Embedded/
14    NkFontEmbedded.h/.cpp <- 10 polices compressées DANS le binaire
```

La description du .jenga promet six fonctionnalités inexistantes

Kernel/Runtime/NKFont/NKFont.jenga:7-23 annonce fièrement le support de BDF, de Type 1 (PFB/PFA), de WOFF2 avec Brotli, d'une machine virtuelle de *hinting* TrueType, de GSUB pour les ligatures, et de l'algorithme bidirectionnel UAX#9.

Une recherche exhaustive sur [src/](#) donne **zéro occurrence** de BDF, Type1, PFB, Brotli,

Bidi, GSUB et Hinting. La seule mention de WOFF2 est un avertissement disant qu'il n'est pas supporté. GPOS apparaît une fois, pour lire un décalage de table qui n'est ensuite jamais utilisé ([Core/NkFontParser.cpp:993](#)).

La liste honnête est celle que le parser affiche lui-même :

Kernel/Runtime/NKFont/src/NKFont/Core/NkFontParser.h:5-14

```
1 // Formats supportés :
2 // OK TTF SimpleGlyph + CompositeGlyph
3 // OK TTC (TrueType Collection) via faceIndex
4 // OK OTF avec table glyf (contours TrueType)
5 // OK OTF/CFF (Type 2 charstrings – interpréteur intégré)
6 // OK WOFF (décompresseur zlib intégré)
7 // /\ WOFF2 (Brotli – non supporté, dépendance externe requise)
8 // OK cmap format 4 (BMP) et format 12 (full Unicode)
9 // OK kern table format 0
10 // OK SDF generation (Signed Distance Field)
```

C'est déjà beaucoup — un interpréteur CFF Type 2 écrit à la main n'est pas rien. Mais c'est ça, et pas ce que promet le fichier de build.

17.2 Les quatre objets à connaître

17.2.1 NkFontAtlas — le propriétaire de tout

L'atlas est la seule chose que vous créez vous-même. Il possède la texture, les configurations, et les polices.

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:102-133 (champs publics)

```
1 void *texID = nullptr;
2 nkft_int32 texWidth = 0, texHeight = 0;
3 nkft_uint8 *texPixels = nullptr; // ALPHA8, 1 octet/pixel
4 bool texReady = false;
5 nkft_int32 texDesiredWidth = 0;
6 nkft_int32 texGlyphPadding = 2; ///< Padding recommandé >= 2 pour éviter le
    bleeding.
7 bool sdfMode = false;
8 nkft_int32 sdfSpread = 6; ///< Rayon SDF en pixels (4-8 typique).
9 NkFont *fonts[NK_FONT_ATLAS_MAX_FONTS] = {};
10 NkFontConfig configs[NK_FONT_ATLAS_MAX_FONTS];
11 nkft_uint32 fontCount = 0u;
```

Ce sont des champs publics, sans accesseurs : on les lit et on les écrit directement. C'est délibéré et cohérent avec le reste du moteur.

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:134-157

```
1 NkFont* AddFontFromFile(const char* path, nkft_float32 sizePixels, const NkFontConfig* cfg =
    nullptr);
2 NkFont* AddFontFromMemory(const nkft_uint8* data, nkft_size dataSize,
3     nkft_float32 sizePixels, const NkFontConfig* cfg = nullptr);
4 NkFont* AddFontFromMemoryOwned(nkft_uint8* data, nkft_size dataSize,
5     nkft_float32 sizePixels, const NkFontConfig* cfg = nullptr);
6
7 static const nkft_uint32* GetGlyphRangesDefault();
```

```

8 static const nkft_uint32* GetGlyphRangesLatinExtA();
9 static const nkft_uint32* GetGlyphRangesCyrillic();
10 static const nkft_uint32* GetGlyphRangesGreek();
11 static const nkft_uint32* GetGlyphRangesChineseFull();
12
13 bool Build(); // <- rasterise TOUT
14
15 void GetTexDataAsAlpha8 (nkft_uint8** op, nkft_int32* ow, nkft_int32* oh, nkft_int32* ob =
    nullptr);
16 void GetTexDataAsRGBA32 (nkft_uint8** op, nkft_int32* ow, nkft_int32* oh, nkft_int32* ob =
    nullptr);
17
18 void ClearTexData();
19 void Clear();
20 bool IsBuilt() const;

```

Comme NkImage, l'atlas n'est pas copiable : `NkFontAtlas(const NkFontAtlas&) = delete;` (NkFont.h:129).

17.2.2 NkFont — une face rastérisée à une taille

Attention au nom : dans ce module, NkFont n'est pas « une police ». C'est **une police à une taille donnée**, déjà rastérisée. Charger Inter en 18 et en 32 pixels donne *deux* NkFont.

Kernel/Runtime/NkFont/src/NkFont/NkFont.h:211-219

```

1 NkFontAtlas *containerAtlas = nullptr; // L'atlas POSSEDE cette NkFont
2 NkFontConfig config;
3 nkft_float32 fontSize = 0, ascent = 0, descent = 0, lineAdvance = 0, scale = 1;
4 NkFontGlyph glyphs[NK_FONT_MAX_GLYPHS];
5 nkft_uint32 glyphCount = 0u;
6 const NkFontGlyph *fallbackGlyph = nullptr;
7 nkft_float32 fallbackAdvanceX = 0;

```

Le commentaire d'en-tête est catégorique sur la propriété :

Kernel/Runtime/NkFont/src/NkFont/NkFont.h:88-90

```

1 NkFont (le struct produit) n'est pas auto-portant : il est detenu par
2 son `containerAtlas` qui parse le TTF, fournit les glyphes et la
3 texture. NkFont seul ne « se charge » pas.

```

Ce qu'il faut retenir

Les NkFont* que vous recevez appartiennent à l'atlas. **Ne les détruisez jamais vous-même**, et ne les gardez pas au-delà de la vie de l'atlas. Détruire l'atlas détruit toutes ses faces.

17.2.3 NkFontGlyph — la structure qui fait tout le travail

Kernel/Runtime/NkFont/src/NkFont/NkFont.h:32-38

```

1 struct NkFontGlyph {
2     NkFontCodepoint codepoint = 0;
3     nkft_float32 advanceX = 0.f;
4     nkft_float32 x0 = 0, y0 = 0, x1 = 0, y1 = 0; ///< Position quad (pixels écran,
    relative au curseur).

```

```

5         nkft_float32 u0 = 0, v0 = 0, u1 = 0, v1 = 0; ///< UV dans l'atlas [0..1].
6         bool visible = true;
7     };

```

Lisez-la attentivement : c'est la structure la plus importante du chapitre.

- x_0 , y_0 , x_1 , y_1 sont les coins du rectangle à l'écran, **relatifs au curseur** — pas absolus. Vous ajoutez la position du curseur, et c'est tout;
- u_0 , v_0 , u_1 , v_1 sont les coordonnées dans l'atlas, déjà normalisées entre 0 et 1;
- `advanceX` est de combien avancer le curseur après ce caractère;
- `visible` vaut `false` pour l'espace et les caractères qui n'ont pas de dessin : on saute l'émission du quad mais on avance quand même le curseur.

Il n'y a **rien d'autre à calculer**. Pas de métriques à recomposer, pas de conversion d'unités de police en pixels : `Build()` l'a déjà fait.

17.2.4 NkFontConfig — les réglages, posés avant l'ajout

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:44-62

```

1     struct NkFontConfig {
2         const nkft_uint8 *fontData = nullptr;
3         nkft_size fontDataSize = 0u;
4         bool fontDataOwned = false;
5         nkft_int32 fontIndex = 0;
6         nkft_float32 sizePixels = 13.f;
7         nkft_int32 oversampleH = 2;
8         nkft_int32 oversampleV = 1;
9         bool pixelSnapH = false;
10        math::NkVec2f glyphOffset = {0.f, 0.f};
11        nkft_float32 glyphMinAdvanceX = 0.f;
12        nkft_float32 glyphMaxAdvanceX = 1e9f;
13        nkft_float32 glyphExtraSpacing = 0.f;
14        const nkft_uint32 *glyphRanges = nullptr;
15        NkFontCodepoint glyphFallback = 0x003Fu;
16        bool mergeMode = false;
17        nkft_float32 rasterizerMultiply = 1.f;
18        char name[40] = {};
19    };

```

Dix-sept champs, dont deux comptent vraiment au début.

`glyphRanges` est un tableau de *paires* {début, fin} terminé par un zéro. Si vous le laissez à `nullptr`, vous obtenez la plage par défaut. Si vous voulez le latin étendu plus les caractères de dessin de boîtes, vous écrivez :

Exemple écrit pour le cours (API : NkFont.h:56, 134)

```

1     static const uint32 kRanges[] = { 0x0020, 0x00FF, 0x2500, 0x257F, 0 }; // Latin-1 +
    box-drawing
2     NkFontConfig cfg;
3     cfg.glyphRanges = kRanges;
4     NkFont *f = atlas.AddFontFromFile("Resources/Fonts/Inter-Regular.ttf", 20.f, &cfg);

```

`mergeMode` est le mécanisme de *repli* : en ajoutant une seconde police avec `mergeMode = true`, ses glyphes viennent combler ceux qui manquent dans la première, dans la même `NkFont`.

C'est ainsi qu'une police latine peut afficher des idéogrammes ou des émojis. Nous verrons le code réel plus loin.

17.3 Les dix polices embarquées : écrire du texte sans aucun fichier

C'est la meilleure porte d'entrée du module, et sans doute la fonctionnalité la plus sous-estimée du moteur.

Kernel/Runtime/NKFont/src/NKFont/Embedded/NkFontEmbedded.h:49-74

```
1 enum class NkEmbeddedFontId : nkft_uint32 {
2     ProggyClean = 0, ProggyTiny = 1, DroidSans = 2, Karla = 3, Roboto = 4,
3     Cousine = 5, SourceCodePro = 6, DroidSerif = 7, DejaVuSansMono = 8, Inter = 9,
4     Count = 10, Default = ProggyClean,
5 };
```

Ces dix polices sont *réellement* présentes : le fichier `Embedded/NkFontEmbedded.cpp` pèse 2,1 Mo de données compressées, et le registre (`NkFontEmbedded.cpp:20265-20370`) fait pointer chaque entrée sur un tableau non vide. Aucune n'est un espace réservé.

Kernel/Runtime/NKFont/src/NKFont/Embedded/NkFontEmbedded.h:120-158

```
1 static bool IsAvailable(NkEmbeddedFontId id);
2 static const NkEmbeddedFontData* GetData(NkEmbeddedFontId id);
3 static NkFont* AddToAtlas(NkFontAtlas& atlas, NkEmbeddedFontId id,
4     nkft_float32 sizePx = 0.f, const NkFontConfig* cfg = nullptr);
5 static NkFont* AddDefaultFont(NkFontAtlas& atlas, const NkFontConfig* cfg = nullptr);
6 static const char* GetName(NkEmbeddedFontId id);
7 static const NkEmbeddedFontData* GetAll(nkft_int32* outCount);
8 static nkft_uint8* DecompressData(const NkEmbeddedFontData& data, nkft_uint32* outSize =
9     nullptr);
9 static void FreeDecompressedData(nkft_uint8* ptr);
```

Les licences sont déclarées dans le registre : ProggyClean et ProggyTiny sont en MIT ; DroidSans, Roboto, Cousine et DroidSerif en Apache 2.0 ; Karla, SourceCodePro et Inter en OFL 1.1 ; DejaVuSansMono en Bitstream Vera / domaine public. Ce n'est pas un détail décoratif : cela veut dire que vous pouvez distribuer votre exécutable sans vous poser de question.

Ce qu'il faut retenir

Votre première application affichant du texte n'a besoin d'aucun fichier de police, d'aucun dossier de ressources, d'aucun chemin relatif. Une ligne suffit : `NkFontEmbedded::AddToAtlas(atlas, NkEmbeddedFontId::Inter, 18.f)`. Vous n'aurez jamais l'erreur « police introuvable ».

Le dépôt s'en sert d'ailleurs avec un repli en cascade, parce que rien n'oblige une police donnée à être compilée dans un binaire particulier :

Applications/Pong/src/Pong/Render/FontAtlas.cpp:52-58

```
1     NkEmbeddedFontId fontId = NkEmbeddedFontId::ProggyClean;
2     if (NkFontEmbedded::IsAvailable(NkEmbeddedFontId::Karla)) {
3         fontId = NkEmbeddedFontId::Karla;
4     } else if (NkFontEmbedded::IsAvailable(NkEmbeddedFontId::DroidSans)) {
5         fontId = NkEmbeddedFontId::DroidSans;
6     } else if (NkFontEmbedded::IsAvailable(NkEmbeddedFontId::Roboto)) {
```

```

7         fontId = NkEmbeddedFontId::Roboto;
8     }

```

17.4 La séquence canonique en quatre temps

Tout usage de NKFont suit le même ordre, sans exception :

1. `atlas.Clear()` — si l'atlas a déjà servi;
2. `Add*()` — une ou plusieurs fois, une par police et par taille;
3. `atlas.Build()` — **c'est ici que les pixels apparaissent**;
4. `atlas.GetTexDataAsAlpha8(...)` — on récupère le bitmap.

Le pont officiel entre NKFont et NKGui l'applique littéralement :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.cpp:118-133

```

1     bool NkGuiFont::LoadEmbedded(NkEmbeddedFontId id, float32 sizePx, bool extFallback)
2     noexcept {
3         atlas.Clear();
4         face = nullptr;
5         pixels = nullptr; // rechargeable
6         NkFontConfig cfg;
7         cfg.glyphRanges = NkGlyphRanges();
8         face = NkFontEmbedded::AddToAtlas(atlas, id, sizePx, &cfg);
9         if (!face)
10            return false;
11        NkMergeFallback(atlas, sizePx, extFallback); // repli pour les glyphes manquants
12        if (!atlas.Build())
13            return false;
14        int32 bpp = 0;
15        atlas.GetTexDataAsAlpha8(&pixels, &atlasW, &atlasH, &bpp);
16        dirty = (pixels != nullptr && atlasW > 0 && atlasH > 0);
17        return dirty;
18    }

```

L'ordre n'est pas négociable, et l'échec est silencieux

`Build()` refuse de travailler sur un atlas vide et le journalise :

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:330-333

```

1     if (fontCount == 0) {
2         logger.Info("[NkFontAtlas] Build():aucune fonte\n");
3         return false;
4     }

```

Mais `GetTexDataAsAlpha8`, appelé avant `Build()`, ne journalise *rien* : il pose simplement `nullptr` (`NkFontAtlas.cpp:640-642` : `if (!texReady || !texPixels) { *op = nullptr; ... }`). Vous uploadez alors un pointeur nul, le backend ne fait rien, et vous cherchez pendant une heure pourquoi tous vos textes sont invisibles.

Testez toujours le pointeur récupéré, comme le fait `NkGuiFont` avec son champ `dirty`.

17.4.1 Le mécanisme de repli en action

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.cpp:98-113

```
1      static void NkMergeFallback(NkFontAtlas &atlas, float32 sizePx, bool ext) noexcept {
2          if (!ext)
3              return;
4          auto add = [&](const char *path, const uint32 *ranges) {
5              if (!NkFileExists(path))
6                  return;
7              NkFontConfig fb;
8              fb.glyphRanges = ranges;
9              fb.mergeMode = true;
10             atlas.AddFontFromFile(path, sizePx > 0.f ? sizePx : 16.f, &fb);
11         };
12         add(gFbBroad, NkBroadRanges());
13         add(gFbCjk, NkCjkRanges());
14         add(gFbEmoji, NkEmojiRanges());
15     }
```

Trois polices de repli : une couverture large, une pour le chinois-japonais-coréen, une pour les émojis. Chacune n'est ajoutée que si le fichier existe — d'où le `NkFileExists`. Les chemins, eux, sont posés par l'application :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:76-78

```
1  A poser par l'APPLICATION (ex. NKCode, depuis son dossier data/fonts) AVANT
2  de charger les polices. Un chemin nullptr/vide = role desactive.
3  void NkSetFallbackFontPaths(const char* broad, const char* cjk, const char* emoji) noexcept;
```

Avant de charger — pas après. Une fois l'atlas construit, il est trop tard.

17.5 Dessiner une chaîne de caractères

17.5.1 Le patron universel

Voici la boucle que vous écrirez, sous une forme ou une autre, chaque fois que vous rendrez du texte vous-même. C'est celle de NKGui :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:245-266

```
1      while (p < end) {
2          const NkFontCodepoint cp = NkFontDecodeUTF8(&p, end);
3          if (cp == 0u)
4              break;
5          const NkFontGlyph *g = face->FindGlyph(cp);
6          if (!g)
7              continue;
8          if (g->visible) {
9              const float32 x0 = NkGuiPixelSnap(x + g->x0), y0 = NkGuiPixelSnap(y +
10             g->y0);
11             const float32 x1 = x0 + (g->x1 - g->x0), y1 = y0 + (g->y1 - g->y0);
12             if (x1 > xEnd)
13                 break; // troncature simple
14             const uint32 i0 = Vtx({x0 + sTop, y0}, {g->u0, g->v0}, c);
15             const uint32 i1 = Vtx({x1 + sTop, y0}, {g->u1, g->v0}, c);
16             const uint32 i2 = Vtx({x1 + sBot, y1}, {g->u1, g->v1}, c);
17             const uint32 i3 = Vtx({x0 + sBot, y1}, {g->u0, g->v1}, c);
```

```

17         Tri(i0, i1, i2, texId);
18         Tri(i0, i2, i3, texId);
19     }
20     x += g->advanceX;
21 }

```

Décortiquons.

1. `NkFontDecodeUTF8(&p, end)` avance le pointeur `p` et rend le *codepoint*. C'est une fonction libre du module (`NkFont.h:313`), doublée d'une méthode statique `NkFont::DecodeUTF8` (`NkFont.h:235`);
2. `FindGlyph(cp)` cherche le glyphe **avec repli** : si le caractère n'existe pas dans la police, on obtient le glyphe de remplacement plutôt que `nullptr`. La variante `FindGlyphNoFallback` existe si vous voulez savoir la vérité;
3. si visible, on émet quatre sommets et deux triangles. Les positions sont `x + g->x0` et `y + g->y0` : le curseur plus le décalage du glyphe;
4. **en dehors du `if`**, on avance : `x += g->advanceX`. L'espace n'a pas de dessin mais fait avancer le curseur.

17.5.2 Le pixel-snap, et l'invariant subtil

Regardez à nouveau : les positions passent par `NkGuiPixelSnap`. Le commentaire explique pourquoi :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:225-231

```

1  Le curseur de texte accumule des avances FRACTIONNAIRES. Sans arrondi,
2  seul le PREMIER glyphe d'une chaîne tombe sur un pixel entier ; tous les
3  suivants dérivent et échantillonnent l'atlas ENTRE deux texels, ce qui
4  rend le texte uniformément flou.

```

Mais l'arrondi est appliqué avec une précaution qu'on ne devine pas :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:257-266

```

1  On arrondit la POSITION du quad, jamais l'AVANCE : `x` continue
2  d'accumuler la valeur exacte, donc la largeur totale de la chaîne
3  est inchangée et MeasureWidth reste d'accord avec le rendu.

```

Arrondir l'affichage, pas le calcul

Si vous arrondissiez `advanceX`, l'erreur s'accumulerait caractère après caractère et la largeur mesurée ne correspondrait plus à la largeur dessinée — vos alignements, vos centrages et vos troncatures deviendraient tous faux. C'est une leçon générale : **on arrondit à l'affichage, jamais dans l'accumulateur**.

17.5.3 La ligne de base, pas le haut du texte

`y` dans la boucle ci-dessus n'est pas le haut du texte : c'est la **ligne de base** — la ligne sur laquelle « posent » les lettres, et sous laquelle descendent les jambages du `p` et du `g`. Les `g->y0` sont donc négatifs pour la plupart des caractères.

Les métriques nécessaires sont dans la `NkFont`, calculées par `Build()` :

- ascent — hauteur au-dessus de la ligne de base;
- descent — profondeur en dessous;
- lineAdvance — de combien descendre pour la ligne suivante;
- fontSize, scale — la taille demandée et le facteur appliqué.

Donc, pour dessiner à partir d'un coin haut-gauche :

Exemple écrit pour le cours (API : NkFont.h:213)

```
1 float x = posX;
2 float y = posY + font->ascent;      // y = LIGNE DE BASE, pas le haut !
3 // ... boucle de glyphs ...
4 y += font->lineAdvance;             // ligne suivante
```

La constante magique de Pong

Le jeu Pong approxime la montée par un facteur empirique :

Applications/Pong/src/Pong/Render/FontAtlas.cpp:206-208

```
1      const float ascent = fontPx * 0.82f;
```

Cela fonctionne pour *cette* police, à *ces* tailles. Changez de police et tout se décale verticalement. La valeur exacte existe pourtant : c'est font->ascent, calculée par Build() (NkFontAtlas.cpp:377). Utilisez-la.

17.5.4 Mesurer avant de dessiner

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:229-235

```
1      nkft_float32 GetCharAdvance(NkFontCodepoint c) const {
2          const NkFontGlyph *g = FindGlyph(c);
3          return g ? g->advanceX : fallbackAdvanceX;
4      }
5
6      nkft_float32 CalcTextSizeX(const char *text, const char *textEnd = nullptr) const;
7      static NkFontCodepoint DecodeUTF8(const char **text, const char *textEnd);
```

CalcTextSizeX parcourt la chaîne et somme les avances. C'est ce qu'il faut pour centrer un libellé, dimensionner un bouton, ou décider d'une troncature. Notez qu'il n'existe pas de CalcTextSizeY : la hauteur d'une ligne, c'est lineAdvance, point.

17.6 De l'atlas à l'écran

17.6.1 L'atlas est en alpha 8 bits

GetTexDataAsAlpha8 rend un octet par pixel : la *couverture* du glyphe, de 0 (rien) à 255 (plein). Ce n'est pas une image en niveaux de gris à afficher telle quelle — c'est un masque.

Or les rasteriseurs 2D du moteur échantillonnent des textures RGBA. Il faut donc convertir, et le dépôt explique exactement pourquoi la recette est celle-là :

Applications/Pong/src/Pong/Render/FontAtlas.cpp:92-104

```

1      // L'atlas NKFont est alpha8 (1 canal = couverture du glyphe). Le
2      // shader 2D de NKCanvas echantillonne la texture en RGBA puis la
3      // multiplie par la couleur du vertex. On convertit donc l'atlas en
4      // RGBA8 "blanc + alpha=couverture" : texture(1,1,1,cov) * color =
5      // (color.rgb, color.a*cov) => exactement le rendu de texte voulu.
6      const usize pixCount = static_cast<usize>(w) * static_cast<usize>(h);
7      NkVector<uint8> rgba;
8      rgba.Resize(pixCount * 4u);
9      for (usize i = 0; i < pixCount; ++i) {
10         rgba[i * 4 + 0] = 255;
11         rgba[i * 4 + 1] = 255;
12         rgba[i * 4 + 2] = 255;
13         rgba[i * 4 + 3] = pixels[i]; // couverture du glyphe
14     }

```

Blanc partout, alpha = couverture. Multiplié par la couleur du sommet, cela donne exactement la couleur voulue avec la bonne transparence. Vous n'avez pas à écrire cette boucle si vous ne le voulez pas : `GetTexDataAsRGBA32` (`NkFontAtlas.cpp:662-680`) fait rigoureusement la même chose, en allouant un second tampon à la demande.

17.6.2 Le piège d'upload

Applications/Pong/src/Pong/Render/FontAtlas.cpp:107-109

```

1      // NB : surtout PAS Create()+Update() : Update exige gTextureBackend.Update
2      // qui n'est pas cable sur tous les backends -> echec silencieux.

```

Ce commentaire vaut avertissement général : dans `NkCanvas`, préférez `LoadFromImage` en un seul appel plutôt que la paire `Create` + `Update`, tant que tous les backends ne câblent pas `Update`.

17.6.3 Le chemin NKGui, celui que vous emploierez

Dans une interface `NKGui`, tout ce qui précède est encapsulé dans un petit *wrapper* :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:19-69 (abrégé)

```

1      struct NkGuiFont {
2          NkFontAtlas atlas;           ///< possède la texture + les glyphes
3          NkFont *face = nullptr;      ///< face produite (détenue par l'atlas)
4          uint32 texId = 0x4E4B4654u; ///< 'NKFT' - id stable pour le backend
5          uint8 *pixels = nullptr;     ///< atlas alpha8 (détenu par l'atlas)
6          int32 atlasW = 0;
7          int32 atlasH = 0;
8          bool dirty = false;          ///< à (ré)uploader côté backend
9          bool LoadEmbedded(NkEmbeddedFontId id, float32 sizePx, bool extFallback =
10 true) noexcept;
11          bool LoadFromFile(const char *path, float32 sizePx, bool extFallback = true)
12          noexcept;
13          /* Face() TexId() Valid() Ascent() Descent() LineHeight() MeasureWidth() */
14      };

```

et son branchement tient en six lignes :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:281-288

```
1 auto fontPtr = memory::NkMakeUnique<NkGuiFont>();
2 if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f)) {
3     fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
4 }
5 ctx.font = fontPtr.Get();
6 if (fontPtr->Valid()) {
7     backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
8     fontPtr->atlasH);
9 }
```

Notez UploadFontGray8 et non UploadImageRGBA : le backend sait qu'un atlas de police est en un seul canal et fait la conversion lui-même ([Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NKGuiCanvasBackend.h:41](#)). Après quoi ctx.font pointe la face, et tous les widgets de NKGui dessinent leur texte tout seuls.

Trois lignes, trois responsabilités

1. fontPtr->LoadEmbedded(id, taille) — NKFont rasterise;
2. backend.UploadFontGray8(...) — le backend crée la texture GPU;
3. ctx.font = fontPtr.Get() — NKGui sait quelle face utiliser.

Oubliez la deuxième : tous les textes sont invisibles, sans message d'erreur. Oubliez la troisième : les widgets n'affichent aucun libellé, sans message d'erreur non plus.

17.6.4 Changer d'échelle : recharger et ré-uploader

Sur un écran à 150 %, il ne suffit pas d'agrandir les quads : l'atlas doit être rasterisé plus gros, sinon le texte est flou. La démo le fait à la touche F9 :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:461-466

```
1 if (pendingScale > 0.f) { // F9 : appliquer la nouvelle echelle DPI
2     SetUiScale(ctx, pendingScale);
3     if (fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f * pendingScale))
4         backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
5         fontPtr->atlasH);
6     pendingScale = 0.f;
7 }
```

Le ré-upload est obligatoire, et le module le dit :

Kernel/Runtime/NKFont/src/NKFont/Core/NkFontSizeCache.h:62-63

```
1 @note Le rebuild de l'atlas invalide toutes les textures GPU.
2 L'application doit être notifiée pour re-uploader la texture.
```

C'est la raison d'être du drapeau dirty de NkGuiFont et des méthodes NeedsGpuUpload() / ClearGpuUploadFlag() du cache de tailles.

17.7 Le mode SDF : du texte net à toute taille

Un atlas classique est rastérisé à une taille précise; l'agrandir le rend flou. Le mode *Signed Distance Field* stocke, pour chaque texel, la distance au bord du glyphe plutôt que sa couverture. Un *shader* reconstitue alors un bord net à n'importe quelle échelle.

Exemple écrit pour le cours (API : `NkFont.h:112-113, 149`)

```
1 NkFontAtlas atlas;
2 atlas.sdfMode = true; // AVANT Build()
3 atlas.sdfSpread = 6; // 4-8 pixels
4 NkFontEmbedded::AddToAtlas(atlas, NkEmbeddedFontId::Inter, 48.f);
5 atlas.Build();
6 // Utiliser le shader fourni : nkentseu::kNkFontFragSDF (NkFont.h:343)
7 // avec uSmoothing ~ 0.1 / fontSize.
```

Ce mode est réellement branché : `NkFontAtlas.cpp:496-535` alloue un tampon temporaire et appelle `nkfont::NkMakeSDFFromBitmap(...)`.

Et le module fournit les deux *shaders* correspondants, en constantes compilées :

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:324-336

```
1 static constexpr const char *kNkFontFragNormal = R"GLSL(
2 #version 460 core
3 in vec2 vUV;
4 in vec4 vColor;
5 out vec4 fragColor;
6 layout(binding=0) uniform sampler2D uAtlas;
7 void main() {
8     float alpha = texture(uAtlas, vUV).a;
9     if (alpha < 0.01) discard;
10    fragColor = vec4(vColor.rgb, vColor.a * alpha);
11 }
12 )GLSL";
```

`kNkFontFragSDF` (`NkFont.h:343`) est son pendant pour le mode SDF, avec un uniforme `uSmoothing`. Vous n'en aurez pas besoin dans une interface `NKGui` — le backend a déjà les siens — mais ils sont là si vous écrivez votre propre rendu.

17.8 Les limites dures, et l'invariant du `Build()`

Voici la partie que le lecteur pressé saute et regrette.

17.8.1 Des tableaux de taille fixe

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:99-100, 169-170

```
1 static constexpr nkft_uint32 NK_FONT_ATLAS_MAX_FONTS = 16;
2 static constexpr nkft_uint32 NK_FONT_ATLAS_MAX_CUSTOM_RECTS = 64;
3 static constexpr nkft_uint32 NK_FONT_MAX_GLYPHS = 4096u;
4 static constexpr nkft_uint32 NK_FONT_INDEX_SIZE = 256u;
```

`NkFont::glyphs` est un tableau *membre* de 4096 entrées, pas un conteneur dynamique. Conséquences concrètes :

- une face ne peut pas dépasser 4096 glyphes. La plage `GetGlyphRangesChineseFull()` en dépasse largement : elle ne rentrera pas;
- un atlas ne peut pas contenir plus de 16 entrées — et une entrée, c'est *une police à une taille*. Cinq tailles de deux polices, plus trois replis, et vous êtes à treize;
- chaque `NkFont` est un objet volumineux. Ne les copiez pas; manipulez des `NkFont*`, ce que l'API impose de toute façon.

17.8.2 `Build()` n'est ni réentrant ni parallélisable

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:336-337

```
1 static nkfont::NkFontFaceInfo faceInfos[NK_FONT_ATLAS_MAX_FONTS];
2 static nkft_float32 scales[NK_FONT_ATLAS_MAX_FONTS];
```

Ces tableaux sont `static` — partagés par toutes les instances. Et pire, chaque face en garde l'adresse :

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:381-382

```
1 // Stockage du faceInfo pour l'extraction des contours
2 font->m_FaceInfo = &faceInfos[i];
```

Les tableaux de *packing* sont eux aussi statiques (`NkFontAtlas.cpp:381-386`).

Un seul atlas, un seul `Build()`, un seul fil

Trois conséquences, toutes silencieuses :

1. deux `Build()` **en parallèle** sur deux fils écrivent dans les mêmes tableaux : corruption, sans message;
2. un `Build()` sur un atlas B **invalide** les `m_FaceInfo` des faces de l'atlas A. Les fonctions qui s'en servent — `GetGlyphOutlinePoints`, l'extrusion 3D — deviennent fausses. L'atlas A continue pourtant d'afficher du texte normalement : seule l'extraction de contours ment;
3. `Build()` n'est pas réentrant : ne l'appellez pas depuis un *callback* déclenché par lui-même.

La règle à suivre : un seul `NkFontAtlas` par application, sur le fil principal. C'est ce que fait `NKGui`, et c'est ce que vous ferez.

17.8.3 Le `mergeMode` duplique les pointeurs

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:185-189

```
1 Les polices FUSIONNÉES (mergeMode) partagent le MÊME NkFont* que leur police de
2 base (cf. AddFontFromMemoryOwned) : fonts[] contient donc des pointeurs DUPLIQUÉS.
3 Il faut ne détruire chaque pointeur unique QU'UNE fois – sinon double-free / tas
4 corrompu au rechargement (crash zoom).
```

Vous n'aurez pas à gérer cela vous-même — l'atlas s'en charge — mais retenez le symptôme : « crash au zoom » signifie « rechargement d'atlas avec fusion ». Si vous écrivez votre propre `wrap-`

per et que vous parcourez `atlas.fonts[]` pour faire quelque chose sur chaque face, **vous visiterez plusieurs fois le même pointeur**.

17.8.4 Le cache de tailles rend `nullptr` la première fois

Si vous utilisez `NkFontSizeCache` (`Core/NkFontSizeCache.h:85`), lisez ce contrat avant :

Kernel/Runtime/NKFont/src/NKFont/Core/NkFontSizeCache.h:114-117

```
1 Si la taille n'est pas dans le cache :
2   1. Ajoute la fonte à l'atlas.
3   2. Marque l'atlas comme nécessitant un rebuild.
4   3. Retourne nullptr jusqu'au prochain appel après BuildAtlas().
```

Un `GetFont(24.f)` qui renvoie `nullptr` n'est donc pas forcément une erreur : c'est peut-être « revenez à la frame suivante ». Le cache plafonne à `MAX_CACHED_SIZES = 16` tailles (`NkFontSizeCache.h:87`).

17.9 Ce que le module ne fait pas

Pour être complet, et pour vous éviter de chercher :

Fonctionnalité	État réel
WOFF2 / Brotli	absent (<code>Core/NkFontParser.h:11</code>)
GSUB, ligatures	absent
Bidi (UAX#9)	absent
<i>Hinting</i> TrueType	absent
BDF, Type 1 (PFB/PFA)	absent
GPOS	offset lu, jamais exploité
Kerning par paire	existe au parser, non remonté
Faux-gras	non — charger une fonte grasse
<code>NkFontSdf.h</code>	annoncé « futur », n'existe pas

Deux précisions.

Le **kerning** est un vrai cas curieux : le parser sait lire la table kern format 0 et expose `NkGetGlyphKernAdvance` (`Core/NkFontParser.h:171`), mais l'information ne remonte pas jusqu'à `NkFontGlyph`. Le wrapper `NkCanvas` l'écrit franchement : « Le module NKFont n'expose pas le kerning par paire → 0 » (`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkFont.h:87`). Un « AV » restera donc plus espacé que dans un traitement de texte.

Le **gras** n'est pas synthétisé : « bold ignoré (le module ne fait pas de faux-gras ; charger une fonte bold dédiée si nécessaire) » (`.../Resources/NkFont.h:84-85`). Si vous voulez du gras, ajoutez une seconde entrée dans l'atlas — en gardant à l'esprit la limite de seize.

17.10 L'autre wrapper : `renderer::NkFont`

Il existe une seconde façade, côté `NkCanvas`, d'inspiration SFML :


```

1      class NkFont {
2      public:
3          bool LoadFromFile(NkIRenderer2D &renderer, const char *path);
4          bool LoadFromMemory(NkIRenderer2D &renderer, const void *data, usize
sizeBytes);
5          const NkGlyph &GetGlyph(uint32 codepoint, uint32 characterSize, bool bold =
false) const;
6          float32 GetKerning(uint32 first, uint32 second, uint32 characterSize) const;
7          float32 GetLineHeight(uint32 characterSize) const;
8          const NkTexture *GetAtlasTexture(uint32 characterSize) const;
9          bool IsValid() const;
10         void Destroy();
11     private:
12         struct Page {                                // une page = une TAILLE de caractère
13             uint32 characterSize = 0;
14             ::nkentseu::NkFontAtlas *atlas = nullptr;
15             ::nkentseu::NkFont *moduleFont = nullptr;
16             NkTexture texture;
17         };
18     };

```

Attention à l'homonymie : `renderer::NkFont` n'est pas `nkentseu::NkFont`. Le premier est un gestionnaire de pages qui délègue au second :

```

1  Depuis le refactoring 2026-05-28, NKCanvas ne réimplémente plus la
2  rasterisation (ex-FreeType supprimé). renderer::NkFont délègue au module
3  NkFont (nkentseu::NkFontAtlas + nkentseu::NkFont) : une "page" (atlas
4  rasterisé + texture GPU) par taille de caractère demandée.

```

Une page par taille demandée : chaque nouvelle `characterSize` construit un nouvel atlas. Pratique, mais rappelez-vous l'invariant du `Build()` statique — c'est un point à surveiller si vous mélangez ce wrapper avec un atlas à vous.

17.11 Récapitulatif des trois chemins

Chemin	Quand	Ce que vous écrivez
<code>NkGuiFont</code>	interface <code>NKGui</code>	3 lignes, rien de plus
<code>renderer::NkFont + NkText</code>	scène <code>NKCanvas</code>	chargement + dessin
Atlas manuel	rendu maison	atlas, conversion, quads

Le premier est celui de ce cours. Le troisième est celui qui *explique*, et c'est pourquoi nous l'avons détaillé : le jour où votre texte est flou, décalé ou invisible, c'est dans ces quatre étapes que le problème se trouve.

Ce chapitre en bref

`NKFont` ne connaît ni image, ni GPU, ni fenêtre : il transforme une police en *glyphes* rangés dans un atlas, et vous rend des coordonnées. Le passage à l'écran est votre affaire — ou celle du pont. La séquence est toujours la même : charger, demander les caractères voulus, construire l'atlas, dessiner ; et le `Build()` porte un invariant qu'on ne peut pas contourner

— ce qui n’a pas été demandé avant ne sera pas dans l’atlas. Le mode **SDF** donne du texte net à toute taille, au prix d’un atlas plus coûteux à produire.

17.12 Exercices

1 — Le plus court chemin vers un texte à l’écran

Dans une application NKGui minimale, remplacez le chargement de police par une boucle sur les dix polices embarquées : pour chaque `NkEmbeddedFontId` de 0 à 9, testez `IsAvailable`, récupérez son nom avec `GetName`, et affichez la liste dans un panneau. Puis ajoutez un bouton qui passe à la police suivante. Vous devrez recharger l’atlas *et* le ré-uploader dans le backend — l’exercice est là. Vérifiez ce qui se passe si vous oubliez le ré-upload.

2 — Voir l’atlas

Récupérez le bitmap alpha8 avec `GetTexDataAsAlpha8`, enveloppez-le dans une `NkImage` — le chapitre 5 vous a donné tout ce qu’il faut — et sauvegardez-le en PNG. Ouvrez le fichier. Vous verrez les glyphes empilés en étagères, avec deux pixels de marge (`texGlyphPadding`). Comptez-les et comparez avec `face->glyphCount`. Puis recommencez avec une plage de glyphes plus large et observez l’atlas grandir. Enfin, mettez `sdfMode = true` et regardez à quoi ressemble un champ de distance.

3 — Écrire son propre rendu de texte

Sans utiliser NKGui, écrivez une fonction qui prend une `NkFont*`, une chaîne UTF-8, une position et une couleur, et remplit deux tableaux de sommets et d’indices — exactement comme `NkGuiDrawList::AddText`. Testez-la sans GPU : affichez pour chaque caractère son *codepoint*, ses quatre coins et ses UV.

Vérifiez ensuite que la somme des `advanceX` est bien égale à `face->CalcTextSizeX(texte)`. Puis arrondissez volontairement `advanceX` à l’entier et observez l’écart se creuser avec la longueur de la chaîne : vous venez de reproduire le bug contre lequel le commentaire de [NkGuiDrawList.cpp:257-266](#) met en garde.

4 — Toucher les plafonds

Écrivez un programme console qui ajoute des polices à un atlas dans une boucle, en incrémentant la taille d’un pixel à chaque tour, et qui s’arrête quand `AddToAtlas` renvoie `nullptr`. Combien d’itérations ?

Ensuite, tentez d’ajouter une police avec `GetGlyphRangesChineseFull()` et regardez ce que vaut `face->glyphCount` après `Build()`. Comparez avec `NK_FONT_MAX_GLYPHS`. Que se passe-t-il pour les caractères au-delà ? Cherchez la réponse dans le comportement observé, puis dans [NkFontAtlas.cpp](#) — et notez si le module vous a prévenu ou non.

Questions de cours

1. Quels sont les quatre temps de la séquence canonique ?
2. Que se passe-t-il si l’on demande un caractère après le `Build()` ?
3. Qu’est-ce qu’un atlas, et pourquoi n’affiche-t-on pas les glyphes un par un ?

4. Qu'apporte le mode SDF, et que coûte-t-il ?
5. Le dépôt embarque des polices : quel problème cela supprime-t-il ?

Exercice de logique

Une application affiche correctement « Bonjour », mais les accents ressortent en carrés vides. Le même texte s'affiche bien dans un autre écran de la même application.

1. Énumérez au moins trois causes possibles.
2. Le fait que *le même texte* s'affiche bien ailleurs élimine lesquelles ? Justifiez chaque élimination.
3. Décrivez la mesure qui tranche entre celles qui restent — et dites ce qu'elle affiche dans chaque cas, sinon elle ne tranche rien.

NKAudio : faire du bruit

Un bouton qui ne fait aucun bruit quand on clique dessus paraît cassé. C’est irrationnel et c’est ainsi. Ce chapitre vous donne les moyens de corriger cela en une dizaine de lignes, puis explique tout ce qu’il y a derrière — parce que le module va beaucoup plus loin qu’un simple «jouer un fichier».

NKAudio est, des cinq modules d’intégration, celui dont la façade est la plus aboutie. Un seul en-tête suffit :

L’unique inclusion nécessaire

```
1 #include "NKAudio/NkAudio.h"
```

et tout vit dans le *namespace* `nkentseu::audio`.

18.1 Le module, et sa dépendance surprenante

Kernel/Runtime/NKAudio/NKAudio.jenga:30-35

```
1 nkentseudependson(
2     # NKMedia : decodeur Opus from-scratch (codec .opus via NkOpusCodec).
3     ["NKPlatform", "NKCore", "NKMemory", "NKContainers", "NKLogger", "NKThreading",
4     "NKFileSystem", "NKStream", "NKMedia"],
5     selfexport="NKAudio",
6     extra_includes=["src"],
7 )
```

NKAudio dépend de NKMedia, qui dépend lui-même de NKImage. La raison est mince — le décodeur Opus vit dans NKMedia — mais elle est réelle : dans l’ordre de construction, NKImage vient d’abord, puis NKMedia, puis NKAudio. Nous enseignons l’audio avant la vidéo parce que la façade audio est plus simple, mais retenez la direction de la flèche : c’est l’audio qui appelle la vidéo, jamais l’inverse.

Kernel/Runtime/NKAudio/ — arborescence (abrégée)

```
1 NKAudio/
2   src/NKAudio/
3     NkAudio.h                (1427 l.) <- EN-TETE PUBLIC UNIQUE : tout est la
4     NkAudioEngineCore.cpp    <- AudioEngine (PIMPL)
5     NkAudioLoader.cpp        <- AudioLoader (WAV natif + dispatch codecs)
6     NkAudioMixer.cpp · NkAudioGenerator.cpp · NkAudioAnalyzer.cpp
7     NkAudioEffects.{h,cpp}   <- effets DSP concrets
8     NkAudioBackends.{h,cpp}  <- WASAPI / CoreAudio / ALSA / AAudio / WebAudio / Null
9     NkAudioBus.{h,cpp}       <- bus hiérarchiques Master -> SFX/Music/Voice/UI
10    NkAudioCapture.{h,cpp}    <- MICRO (entree)
11    Codecs/ MP3/ · OGG/ · FLAC/ · Opus/
12    Streaming/
13      NkAudioStream.{h,cpp}    <- IAudioStream (pull) + WavStream/MemoryStream
```

```

14 NkAudioStreamPlayer.{h,cpp} <- thread decodeur + ring buffer
15 NkContainerAudioStream.{h,cpp}<- piste audio d'un conteneur video

```

18.2 L'invariant fondateur : un seul format interne

Avant toute chose, comprenez ce que le module fait de vos fichiers :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:258-260

```

1 @note Format interne normalisé : Float32 interleaved stéréo.
2 La conversion est faite au chargement.

```

Peu importe que votre fichier soit un WAV 8 bits mono à 22 050 Hz ou un FLAC 24 bits à 96 000 Hz : après `AudioLoader::Load`, vous avez des flottants 32 bits entrelacés. Cela simplifie énormément la suite — le mixeur n'a qu'un seul cas à traiter — et cela signifie qu'un fichier chargé occupe souvent plus de mémoire que sur le disque.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:262-284

```

1 struct AudioSample {
2     float32 *data = nullptr; ///< Samples Float32 (interleaved)
3     usize frameCount = 0;    ///< Nombre de frames (samples/channels)
4     int32 sampleRate = 48000;
5     int32 channels = 2;
6     AudioFormat format = AudioFormat::UNKNOWN;
7     float32 GetDuration() const;
8     usize GetSampleCount() const;
9     bool IsValid() const;
10    memory::NkAllocator *mAllocator = nullptr; ///< Allocateur responsable
11 };

```

Notez le vocabulaire, qu'il faut fixer une fois pour toutes : une *frame* est un instant, un *sample* est une valeur. En stéréo, une *frame* contient deux *samples*. `frameCount` compte les instants, pas les nombres — d'où `GetSampleCount() = frameCount * channels`.

Notez aussi `mAllocator` : chaque `AudioSample` se souvient de qui l'a alloué. Nous verrons pourquoi c'est vital.

18.3 Le patron canonique en cinq temps

L'application `NkAudioPlayer` ouvre son fichier principal sur la recette, et il n'y a rien à y ajouter :

Applications/NkAudioPlayer/src/main.cpp:2-9

```

1 // Le patron CORRECT pour Lire un fichier audio vers Les haut-parleurs, ET L'afficher :
2 // 1. AudioEngine::Instance().Initialize() -> ouvre Le device + thread de mixage
3 // 2. AudioLoader::Load(path) -> décode WAV/MP3/OGG/FLAC/Opus en AudioSample
4 // 3. engine.Play(sample, params) -> Lance une voix (bus "Music")
5 // 4. boucle : GetPlaybackPosition -> tête de Lecture sur La FORME D'ONDE dessinée
6 // 5. engine.Shutdown()

```

18.3.1 Temps 1 — initialiser le moteur

Applications/NkAudioPlayer/src/main.cpp:113-121

```
1 // — 1) Moteur audio —————
2 audio::AudioEngine &engine = audio::AudioEngine::Instance();
3 audio::AudioEngineConfig cfg;
4 if (!engine.Initialize(cfg)) {
5     logger.Error("[NkAudioPlayer] AudioEngine::Initialize a echoue (device indisponible
6     ?)");
7     return -2;
8 }
9 logger.Info("[NkAudioPlayer] device reel : {0} Hz, {1} canaux", engine.GetSampleRate(),
10 engine.GetChannels());
```

AudioEngine est un **singleton** : Instance() rend toujours la même. Initialize ouvre le périphérique de sortie et lance le fil de mixage.

La ligne de journalisation n'est pas décorative. Le périphérique impose son taux d'échantillonnage et son nombre de canaux; votre cfg n'exprime qu'un souhait. Lisez GetSampleRate() et GetChannels() après Initialize(), jamais avant.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:396-411

```
1 struct AudioEngineConfig {
2     AudioBackendType backend = AudioBackendType::AUTO;
3     int32 sampleRate = AUDIO_DEFAULT_SAMPLE_RATE; // 48000
4     int32 channels = AUDIO_DEFAULT_CHANNELS; // 2
5     int32 bufferSize = AUDIO_DEFAULT_BUFFER_SIZE;
6     int32 maxVoices = AUDIO_MAX_VOICES;
7     bool enableHrtf = false; bool enableDoppler = true;
8     float32 masterVolume = 1.0f;
9     memory::NkAllocator *allocator = nullptr; ///< nullptr = allocateur global
10    bool enableMasterLimiter = true;
11    float32 masterLimiterThresholdDb = -0.5f;
12    ResamplingQuality resamplingQuality = ResamplingQuality::LINEAR;
13};
```

Les valeurs par défaut conviennent. **Ne touchez pas au champ backend** — nous verrons dans un instant pourquoi.

Un seul Initialize, un seul Shutdown

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1071-1073

```
1 @note Initialize() et Shutdown() sont appelés une seule fois
2     depuis le thread principal. Toutes les autres méthodes
3     sont thread-safe via des atomics et queues lock-free.
```

Les contrats sont formels : Initialize exige !IsInitialized(), Shutdown exige IsInitialized() (NkAudio.h:1100-1111). Toutes les autres méthodes, elles, sont appelables de n'importe où.

18.3.2 Temps 2 — décoder le fichier

Applications/NkAudioPlayer/src/main.cpp:115-121

```
1 // — 2) Charge/decode Le fichier —————
2 audio::AudioSample sample = audio::AudioLoader::Load(path.CStr());
3 if (!sample.IsValid()) {
4     logger.Error("[NkAudioPlayer] impossible de charger/decoder : {0}", path.CStr());
5     engine.Shutdown();
6     return -3;
7 }
```

AudioLoader est entièrement statique :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:583-648

```
1 static AudioFormat DetectFormat(const uint8* data, usize size);
2 static AudioFormat DetectFormatFromPath(const char* path);
3 static AudioSample Load(const char* path, memory::NkAllocator* allocator = nullptr);
4 static AudioSample LoadFromMemory(const uint8* data, usize size,
5                                   AudioFormat format = AudioFormat::UNKNOWN,
6                                   memory::NkAllocator* allocator = nullptr);
7 static bool SaveWAV(const char* path, const AudioSample& sample);
8 static void Free(AudioSample& sample);
9 static void ConvertSampleFormat(AudioSample& sample, SampleFormat target);
10 static void Resample(AudioSample& sample, int32 targetSampleRate, bool highQuality = true);
11 static void ConvertChannels(AudioSample& sample, int32 targetChannels);
```

Load détecte le format par le contenu, pas par l'extension : renommer un MP3 en .wav ne le trompe pas.

18.3.3 Temps 3 — lancer une voix

Applications/NkAudioPlayer/src/main.cpp:158-162

```
1 // — 4) Joue + prepare La forme d'onde —————
2 audio::VoiceParams vp;
3 vp.bus = "Music";
4 vp.volume = 1.0f;
5 audio::AudioHandle handle = engine.Play(sample, vp);
```

Une *voix* est une instance de lecture. Jouer trois fois le même AudioSample crée trois voix indépendantes, chacune avec son volume, sa position de lecture et son *handle*.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:346-357

```
1 struct VoiceParams {
2     float32 volume = 1.0f;    float32 pitch = 1.0f;    float32 pan = 0.0f;
3     bool looping = false;    float32 loopStart = 0.0f; float32 loopEnd = -1.0f;
4     float32 fadeInTime = 0.0f; float32 startOffset = 0.0f;
5     int32 priority = 128;
6     AudioSource3D source3d;
7     const char *bus = "SFX";    ///< Bus de routage (SFX/Music/Voice/UI)
8 };
```

Le `AudioHandle` est un simple entier enveloppé (`NkAudio.h:227-249`), avec un `IsValid()` et une conversion implicite en `bool`. Il sert à piloter la voix après coup :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1145-1166

```
1 void Stop(AudioHandle handle, float32 fadeOutTime = 0.0f);
2 void Pause(AudioHandle handle); void Resume(AudioHandle handle);
3 bool IsPlaying(AudioHandle) const; bool IsPaused(AudioHandle) const; bool
  IsLooping(AudioHandle) const;
4 void SetVolume/SetPitch/SetPan/SetLooping(AudioHandle, ...);
5 float32 GetVolume/GetPitch/GetPan(AudioHandle) const;
6 float32 GetPlaybackPosition(AudioHandle) const;
7 void SetPlaybackPosition(AudioHandle, float32 seconds);
```

Play peut échouer sans rien dire

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1125

```
1 @return Handle de contrôle, invalide si pool plein
```

Le nombre de voix simultanées est plafonné par `maxVoices`. Passé ce seuil, `Play` renvoie un *handle* invalide et rien ne se joue. Si vous comptez réutiliser le *handle*, testez `h.IsValid()`. Si vous n'en avez pas besoin — un son de clic, par exemple — ignorer le retour est parfaitement légitime.

18.3.4 Temps 4 — piloter pendant la lecture

Applications/NkAudioPlayer/src/main.cpp:185-191

```
1         else if (k->GetKey() == NkKey::NK_SPACE) {
2             paused = !paused;
3             if (paused)
4                 engine.Pause(handle);
5             else
6                 engine.Resume(handle);
7         }
```

et la détection de fin naturelle :

Applications/NkAudioPlayer/src/main.cpp:254-255

```
1     if (!engine.IsPlaying(handle) && !paused)
2         running = false; // fin naturelle
```

Notez la condition composée : une voix en pause n'est pas « en train de jouer ». Sans le `&& !paused`, appuyer sur la barre d'espace fermerait l'application.

18.3.5 Temps 5 — l'ordre de fermeture, qui n'est pas négociable

Applications/NkAudioPlayer/src/main.cpp:258-259

```
1 engine.Shutdown();
2 audio::AudioLoader::Free(sample);
```

Shutdown() d'abord, **Free()** ensuite. Toujours.

La documentation de Play pose le contrat :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1123

```
1 @param sample Sample source (doit rester valide pendant la lecture)
```

Play ne copie pas vos échantillons : la voix lit *directement* dans `sample.data`, depuis le fil audio. Libérer le `AudioSample` avant d'avoir arrêté le moteur, c'est laisser un fil temps réel lire de la mémoire rendue au système.

Le symptôme est particulièrement déplaisant : cela ne plante pas toujours. Selon ce qui a été réalloué à cet endroit, vous obtiendrez un grésillement, un bruit blanc, un silence — ou un plantage, mais dix secondes plus tard, ailleurs.

Retenez la séquence : arrêter le moteur, puis libérer les données. Toujours dans cet ordre, dans les cinq chemins de sortie de votre programme.

Et la libération elle-même a sa règle :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:284, 620

```
1 memory::NkAllocator *mAllocator = nullptr; ///< Allocateur responsable (jamais nullptr si
   IsValid())
2 @note Doit être appelé avec le même allocateur utilisé pour Load()
```

C'est la même règle que pour `NKImage` : jamais `delete`, jamais `free`. Un `IAudioStream` qu'on n'a confié à personne se détruit avec `memory::NkGetDefaultAllocator().Delete(stream)` — le dépôt le note explicitement, avec la mention du plantage `c000374` en cas de mélange ([Applications/NkAudioDemo/src/main.cpp:44](#)).

18.4 Jouer un son au clic d'un bouton

Voici l'objectif annoncé. Tout est en place; il ne reste qu'à assembler.

18.4.1 Au démarrage — une seule fois

Exemple écrit pour le cours (API : `NkAudio.h:1091, 1104, 595`)

```
1 #include "NKAudio/NkAudio.h"
2 using namespace nkentseu;
3
4 audio::AudioEngine &engine = audio::AudioEngine::Instance();
5 if (!engine.Initialize(audio::AudioEngineConfig{})) {
6     logger.Error("[UI] audio indisponible : les sons d'interface seront muets");
7     // ... mais l'application continue : le son n'est pas vital.
8 }
```

```

9  audio::AudioSample clic = audio::AudioLoader::Load("Resources/Audio/clic.wav");
10 if (!clic.IsValid())
11     logger.Warn("[UI] son de clic introuvable");

```

Remarquez la posture : l'échec de l'audio ne doit pas empêcher l'application de démarrer. Une carte son occupée, un périphérique débranché, une machine sans sortie : ce sont des situations normales.

18.4.2 Dans la frame — quand le widget renvoie true

Exemple écrit pour le cours (API : NkGuiWidgets.h:44, NkAudio.h:1127)

```

1  if (nkgui::Button(ctx, "Valider")) {
2      audio::VoiceParams vp;
3      vp.bus = "UI"; // bus dedie : volume UI réglable a part
4      engine.Play(clic, vp); // handle ignore : son court, « fire and forget »
5      // ... l'action du bouton ...
6  }

```

Trois lignes. C'est tout — et c'est possible parce que Play est *non bloquant* : il inscrit la voix dans le mixeur et rend la main immédiatement. Votre frame ne ralentit pas.

18.4.3 À la fermeture

Exemple écrit pour le cours (API : NkAudio.h:1114, 622)

```

1  engine.Shutdown();
2  audio::AudioLoader::Free(clic);

```

Pourquoi le bus « UI »

Ranger les sons d'interface sur leur propre bus n'est pas de la coquetterie : cela permet à l'utilisateur de les couper sans toucher à la musique, par un simple `engine.GetBus("UI")->SetMute(true)`. Si vous les mettez sur le bus « SFX » par défaut, vous ne pourrez plus les distinguer des bruitages du jeu. Le coût de ce choix est nul ; le coût de sa correction plus tard ne l'est pas.

18.5 Les bus : régler des groupes de sons ensemble

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1246-1251

```

1  Le bus Master est cree automatiquement a Initialize() avec ses
2  4 sous-buses standard : SFX (default), Music, Voice, UI.
3
4  Volume effectif d'une voix = voix.volume * bus.volume *
5                               bus.parent.volume * ... * Master.volume.

```

Les quatre bus existent dès `Initialize()` : vous n'avez rien à créer. Écrire `vp.bus = "Music"` suffit.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1255-1276

```
1 NkAudioBus* GetMasterBus();
2 NkAudioBus* GetBus(const char* name);
3 NkAudioBus* GetOrCreateBus(const char* name, NkAudioBus* parent = nullptr);
```

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioBus.h:53-137 (abrégé)

```
1 static constexpr int32 MAX_EFFECTS = 8;
2 static constexpr int32 MAX_CHILDREN = 16;
3 static constexpr int32 MAX_NAME_LEN = 32;
4
5 NkAudioBus* GetParent() const noexcept;
6 NkAudioBus* GetChild(int32 idx) const noexcept;
7 NkAudioBus* FindDescendant(const char* name) noexcept;
8 void SetVolume(float32) noexcept; float32 GetVolume() const noexcept;
9 float32 GetEffectiveVolume() const noexcept; // produit récursif
10 void SetMute(bool) / IsMuted() / SetSolo(bool) / IsSoloed()
11 bool AddEffect(IAudioEffect*) / RemoveEffect / ClearEffects
12 void SetSidechainFromBus(NkAudioBus* sourceBus, float32 amount = 0.7f,
13 float32 threshold = 0.05f) noexcept;
```

Un cas d'usage typique — baisser la musique sans toucher aux bruitages :

Exemple écrit pour le cours (API : NkAudio.h:1266, NkAudioBus.h:84)

```
1 if (audio::NkAudioBus *bus = engine.GetBus("Music"))
2     bus->SetVolume(0.35f);
```

GetBus renvoie nullptr si le moteur n'est pas initialisé : le if n'est pas facultatif.

Le *sidechain* mérite un mot, parce qu'il résout un problème courant : SetSidechainFromBus(voix, 0.7f) sur le bus Music fait automatiquement baisser la musique quand un dialogue joue. C'est ce que font tous les jeux, et cela tient ici en un appel.

Enfin, le fondu croisé entre deux musiques :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1297

```
1 AudioHandle PlayMusicCrossfade(const AudioSample& newMusic, float32 fadeTime = 2.0f,
2 const VoiceParams& params = VoiceParams{});
```

Le limiteur master, et pourquoi le son ne sature jamais

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:409-411

```
1 bool enableMasterLimiter = true;
2 float32 masterLimiterThresholdDb = -0.5f;
```

Un limiteur est actif par défaut sur la sortie. C'est pourquoi mettre un volume supérieur à 1,0 ne produit pas de distorsion : le signal est compressé au lieu d'être écrêté. C'est un filet de sécurité utile — mais cela veut aussi dire que si vous poussez tous vos volumes, vous n'obtiendrez pas « plus fort », vous obtiendrez « plus compressé ».

18.6 Les fichiers longs : le streaming

Charger une heure de podcast en Float32 stéréo à 48 kHz représente environ 1,4 Go en mémoire. Pour tout ce qui dépasse quelques dizaines de secondes, il faut décoder au fil de l'eau.

Kernel/Runtime/NKAudio/src/NKAudio/Streaming/NkAudioStream.h:42-68

```
1  class IAudioStream {
2      virtual ~IAudioStream() = default;
3      virtual int32 ReadFrames(float32* outBuf, int32 maxFrames) noexcept = 0;
4      virtual bool Seek(nk_int64 frameIdx) noexcept = 0;
5      virtual nk_int64 GetFrameCount() const noexcept = 0;
6      virtual int32 GetSampleRate() const noexcept = 0;
7      virtual int32 GetChannels() const noexcept = 0;
8      virtual bool IsEOF() const noexcept = 0;
9  };
10 IAudioStream* OpenAudioStream(const char* path) noexcept;
```

C'est une interface *pull* : on demande des *frames*, on en reçoit. Il faut donc quelqu'un pour tirer régulièrement, et à l'avance — c'est le rôle du lecteur de flux, qui lance un fil décodeur et remplit un tampon circulaire :

Kernel/Runtime/NKAudio/src/NKAudio/Streaming/NkAudioStreamPlayer.h:41-135 (abrégé)

```
1  class AudioStreamPlayer {
2      bool Init(int32 sampleRate, int32 channels, int32 ringBufferFrames = 88200)
3      noexcept;
4      void Shutdown() noexcept;
5      bool Play(IAudioStream* stream, bool loop = false) noexcept;
6      void Stop() noexcept; void Pause() noexcept; void Resume() noexcept;
7      int32 ReadFrames(float32* outBuf, int32 maxFrames) noexcept;
8      bool IsPlaying() const noexcept;
9      void SetVolume(float32) / GetVolume()
10     void SetSpeed(float32) / GetSpeed()
11     void SeekContent(float32 seconds) noexcept;
12     void FlushRing() noexcept;
13     bool IsActive() const noexcept; bool IsFinished() const noexcept;
14 };
15
```

L'ouverture, telle qu'elle est écrite dans la démo :

Applications/NkAudioDemo/src/main.cpp:27-50 (abrégé)

```
1  static int RunStreamingTest(const char *path) {
2      IAudioStream *stream = OpenAudioStream(path);
3      if (!stream) {
4          logger.Error("[NkAudioDemo] OpenAudioStream echec");
5          return 1;
6      }
7      int32 sampleRate = stream->GetSampleRate();
8      int32 channels = stream->GetChannels();
9
10     AudioStreamPlayer player;
11     if (!player.Init(sampleRate, channels, 88200)) {
12         memory::NkGetDefaultAllocator().Delete(stream); // voir note c0000374
13         return 2;
14     }
15 }
```

```

15     if (!player.Play(stream, /*Loop=*/false)) {
16         player.Shutdown();
17         return 3;
18     }

```

Après **Play**, le lecteur possède le flux : c'est lui qui le détruira. Avant **Play** — donc dans les chemins d'erreur — c'est à vous de le faire, avec l'allocateur du moteur.

Reste à brancher ce lecteur sur le moteur. C'est le rôle du *callback* procédural.

18.7 Le *callback* procédural, et le fil audio

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1140-1141

```

1 using ProceduralCallback = NkFunction<void(float32* buffer, int32 frames, int32 channels)>;
2 AudioHandle PlayProcedural(ProceduralCallback callback, const VoiceParams& params =
    VoiceParams{});

```

Au lieu de lire un `AudioSample` figé, la voix appelle votre fonction chaque fois qu'elle a besoin d'échantillons. C'est ainsi qu'on branche un flux :

Applications/NkVideoPlayer/src/main.cpp:396-402

```

1         audio::VoiceParams vp;
2         vp.bus = "Music";
3         audioHandle = engine.PlayProcedural(
4             [&streamPlayer](float32 *buf, int32 frames, int32 /*channels*/) {
5                 streamPlayer.ReadFrames(buf, frames);
6             },
7             vp);

```

C'est aussi ainsi qu'on écrit un synthétiseur : rien ne vous oblige à lire un fichier, vous pouvez calculer les échantillons.

Ce *callback* tourne sur le fil audio

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1134-1139

```

1 Le callback est appelé depuis le thread audio pour remplir
2 les buffers à la demande. Idéal pour synthétiseur logiciel.
3 @note TEMPS RÉEL : callback ne doit pas allouer/locker

```

« Temps réel » veut dire ici : votre fonction dispose de quelques millisecondes, et si elle les dépasse, l'utilisateur entend un craquement.

Sont donc interdits dans ce *callback* : allouer de la mémoire, prendre un verrou, ouvrir un fichier, journaliser, redimensionner un conteneur. Tout ce qui peut faire attendre un fil peut faire craquer le son.

Ce que vous pouvez faire : lire des tableaux préalloués, calculer, écrire dans le tampon fourni, manipuler des atomiques.

18.7.1 L'ordre de destruction avec un lecteur de flux

Ce commentaire du lecteur vidéo mérite d'être lu deux fois :

Applications/NkVideoPlayer/src/main.cpp:672-676

```
1  streamPlayer.Shutdown() joint le thread décodeur et libère le IAudioStream
2  qu'il possède (ContainerAudioStream/MemoryStream/WavStream) ; l'ordre importe :
3  arrêter l'engine (qui appelle le callback procédural depuis son thread audio)
4  AVANT de détruire streamPlayer, pour ne jamais mixer un stream en cours de
5  destruction.
```

D'où la séquence complète, dans le bon ordre :

Exemple écrit pour le cours (invariant : NkVideoPlayer/src/main.cpp:672-676)

```
1  engine.Shutdown();      // 1) arrête le fil audio -> plus aucun appel au callback
2  player.Shutdown();      // 2) joint le fil décodeur et libère le stream
```

La règle générale des trois fermetures

engine.Shutdown() vient **toujours en premier**, parce que c'est lui qui fait tourner le fil qui touche à tout le reste. Ensuite seulement on détruit ce qu'il lisait : les lecteurs de flux, puis les AudioSample.

18.8 Le micro

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioCapture.h:27-75 (abrégé)

```
1  struct NkCaptureDeviceInfo { /* ... */ bool isDefault = false; };
2  struct NkCaptureConfig {
3      int32 sampleRate = 48000; int32 channels = 1; int32 ringSeconds = 4;
4  };
5  using NkAudioInCallback = NkFunction<void(const float32* interleaved, int32 frames, int32
channels)>;
6
7  class NkAudioCapture {
8      static NkVector<NkCaptureDeviceInfo> EnumerateDevices();
9      bool Open(const NkCaptureConfig& config); void Close();
10     bool Start(); void Stop(); bool IsCapturing() const;
11     int32 Read(float32* out, int32 maxFrames);
12     int32 Available() const;
13     void SetCallback(NkAudioInCallback cb);
14     int32 SampleRate() const; int32 Channels() const;
15     static bool SelfTest();
16 };
```

Deux modes coexistent : le *pull* (Read depuis votre boucle) et le *push* (SetCallback). Le premier est plus simple et suffit largement :

Exemple écrit pour le cours (API : NkAudioCapture.h:53-63)

```
1  #include "NKAudio/NkAudioCapture.h"
2
3  audio::NkAudioCapture cap;
4  audio::NkCaptureConfig ccfg;      // 48000 Hz, mono, ring de 4 s
5  if (!cap.Open(ccfg) || !cap.Start())
6      return 1;
7
```

```

8  NkVector<float32> bloc; bloc.Resize(1024);
9  while (enregistre) {
10     const int32 n = cap.Read(bloc.Data(), 1024);    // 0 si rien de dispo
11     /* ... accumuler les n frames ... */
12 }
13 cap.Stop();
14 cap.Close();

```

Read qui renvoie 0 n'est pas une erreur : c'est « rien de neuf ». Le tampon circulaire fait quatre secondes par défaut ; si vous ne lisez pas assez souvent, les plus anciennes données sont perdues.

NkAudioCapture est l'une des deux seules classes du module à posséder un auto-test ([NkAudioCapture.cpp:930](#)) ; l'autre est NkDenoiser ([NkDenoiser.cpp:304](#)). Il n'y a d'auto-test ni pour AudioEngine, ni pour AudioLoader, ni pour les codecs.

L'application NkMicRecord donne au passage le test de codecs le plus court du dépôt :

Applications/NkMicRecord/src/main.cpp:76-89 (abrégé)

```

1  // Mode décodage : NkMicRecord --decode <in.(wav|mp3|ogg|flac|opus)> <out.wav>
2  // Charge via AudioLoader (auto-détection du format, dont Ogg-Opus) et
3  // réécrit en WAV 16-bit – test end-to-end des codecs NKAudio.
4  if (argc >= 4 && NkString(argv[1]) == NkString("--decode")) {
5      audio::AudioSample smp = audio::AudioLoader::Load(argv[2]);
6      if (!smp.IsValid()) {
7          printf("[DECODE] ERREUR : chargement impossible : %s\n", argv[2]);
8          return 1;
9      }
10     const bool ok = audio::AudioLoader::SaveWAV(argv[3], smp);

```

18.9 Tester sans carte son

Voici la fonctionnalité la plus utile pour apprendre — et la moins connue.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1321

```

1  void RenderToBuffer(float32* outputBuffer, int32 frameCount, int32 channels = 2);

```

Couplée au backend NULL_OUTPUT, elle permet de faire tourner tout le moteur — voix, bus, effets, limiteur — *sans aucun périphérique*, et de récupérer le mixage dans un tableau que vous pouvez inspecter :

Exemple écrit pour le cours (API : NkAudio.h:122-136, 1321)

```

1  audio::AudioEngineConfig cfg;
2  cfg.backend = audio::AudioBackendType::NULL_OUTPUT;    // aucune sortie physique
3  engine.Initialize(cfg);
4  engine.Play(smp);
5
6  NkVector<float32> mix; mix.Resize(48000 * 2);           // 1 s de stereo
7  engine.RenderToBuffer(mix.Data(), 48000, 2);           // appel SYNCHRONE
8  // mix contient maintenant exactement ce qui serait sorti des haut-parleurs.

```

Ce qu'il faut retenir

NULL_OUTPUT + RenderToBuffer transforme l'audio en calcul pur, donc en quelque chose de **vérifiable**. Un test qui vérifie que le mixage n'est pas silencieux, qu'il ne dépasse pas 1,0, ou qu'un bus muet produit bien du zéro, tourne sans matériel et sans attendre. C'est ainsi qu'il faut aborder les exercices de ce chapitre.

18.10 État réel du module

18.10.1 Ce qui fonctionne

Décodage	WAV natif, MP3, OGG Vorbis, FLAC, Opus — tous écrits dans le dépôt
Sortie	WASAPI, CoreAudio, ALSA, AAudio, WebAudio, Null
Capture	NkAudioCapture complet, avec auto-test
Streaming	IAudioStream, AudioStreamPlayer, piste audio de conteneur vidéo
Mixage	bus hiérarchiques, sidechain, fondu croisé musique, limiteur master
3D	atténuation, occlusion, absorption de l'air, HRTF synthétique
Hors ligne	RenderToBuffer

Le décodage mérite une précision, parce que le dépôt se calomnie lui-même : le commentaire de tête de `NkAudioLoader.cpp:3` parle encore de « stubs MP3/OGG/FLAC ». **C'est périmé**. Les quatre décodeurs sont branchés sur de vrais codecs (`NkAudioLoader.cpp:411-429`). Encore un cas où le commentaire ment et le code dit vrai.

18.10.2 Ce qui ne fonctionne pas

Ne jamais demander DIRECTSOUND

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioBackends.cpp:445-451

```
1      bool DirectSoundAudioBackend::Initialize(int32 sr, int32 ch, int32 buf) {
2          mSampleRate = sr;
3          mChannels = ch;
4          mBufferSize = buf;
5          mImpl = memory::NkGetDefaultAllocator().New<DsImpl>();
6          // DirectSoundCreate8, CreateSoundBuffer, etc.
7          return true;
8      }
```

Lisez bien la dernière ligne : `return true`. Le backend annonce le succès sans avoir ouvert quoi que ce soit. Vous obtiendrez un moteur « initialisé », des voix « en lecture », des positions qui avancent — et un silence absolu, sans le moindre message d'erreur.

Laissez `cfg.backend` sur **AUTO**, qui choisit WASAPI sous Windows, CoreAudio sous macOS, ALSA sous Linux. Trois autres valeurs de l'énumération — `PULSE_AUDIO`, `OPEN_SL_ES`, `CUSTOM` — n'ont pas de classe de backend correspondante dans `NkAudioBackends.h` : elles existent dans l'énumération et nulle part ailleurs.

Le rééchantillonnage « haute qualité » est du linéaire

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioLoader.cpp:588-592

```
1 void AudioLoader::ResampleSinc(AudioSample &sample, int32 targetSampleRate) {
2     // Pour l'instant, fallback linéaire
3     // TODO: Implémenter Kaiser-windowed Sinc pour qualité studio
4     ResampleLinear(sample, targetSampleRate);
5 }
```

Donc `AudioLoader::Resample(sample, taux, /*highQuality=*/true)` fait exactement la même chose que `false`. C'est le jumeau exact du piège `NK_LANCZOS3` du chapitre 5 : ça ne plante pas, ça ment.

Par symétrie, les valeurs `SINC_4` et `SINC_8` de `ResamplingQuality` dans `AudioEngineConfig` sont à vérifier avant d'être présentées comme différenciantes.

La bonne nouvelle, c'est que vous n'avez normalement pas à rééchantillonner vous-même :

Applications/NkAudioPlayer/src/main.cpp:15-16

```
1 NOTE (agent NKCode) : le mixage NKAudio convertit le TAUX du fichier vers celui du
2 device (fix mixBuffer/format-device). Un lecteur n'a RIEN à faire côté
   rééchantillonnage.
```

18.10.3 Le reste de la surface, en survol

Le module contient bien davantage que ce chapitre ne couvre. Pour que vous sachiez au moins que cela existe :

- `IAudioEffect` ([NkAudio.h:438](#)) et les effets concrets de `NkAudioEffects.h` — réverbération, écho, chorus, compresseur, filtres, *pitch shift*... Ils s'attachent à une voix (`AddEffect`), à un bus, ou à la sortie (`AddMasterEffect`). Leur propriété reste à l'appelant ([NkAudio.h:1230](#)) : l'effet doit survivre à ce à quoi il est attaché ;
- l'audio 3D : `SetSourcePosition`, `SetListenerOrientation`, `SetSourceOcclusion`. Notez que **c'est l'application qui calcule l'occlusion** — « L'application fait le raycast pour calculer cette valeur » ([NkAudio.h:1177-1178](#)). Pour le HRTF, si vous n'avez pas de fichier de données, `GenerateSyntheticHrtf()` ([NkAudio.h:1211](#)) n'en demande aucun ;
- `AudioGenerator` ([NkAudio.h:695](#)) — synthèse de formes d'onde, enveloppes ADSR ;
- `AudioAnalyzer` ([NkAudio.h:947](#)) — FFT, avec un `FftResult` qui porte les magnitudes et une conversion `BinToFrequency`. De quoi dessiner un spectre en temps réel.

18.11 Relier le son à l'image

NKAudio ne dépend d'aucun module graphique : le raccordement est entièrement à votre charge, et c'est très bien ainsi. Trois motifs reviennent :

1. **Le son au clic** — le widget renvoie `true`, vous appelez `Play` avec `vp.bus = "UI"`. C'est ce que nous avons fait ;
2. **La forme d'onde** — `AudioSample::data` et `frameCount` vous donnent tout pour pré-calculer une enveloppe min/max par colonne de pixels, et `GetPlaybackPosition(handle)` vous

donne la tête de lecture. C'est exactement ce que fait `ComputePeaks` dans `Applications/NkAudioPlayer/src/main.cpp:56-87`, avant d'envoyer un quad par colonne au renderer;

3. Le spectre — `AudioAnalyzer` rend un `FftResult`, une barre par *bin*.

Le calcul de la fraction lue, pour dessiner un curseur de lecture :

`Applications/NkAudioPlayer/src/main.cpp:212-215`

```
1 // Position de Lecture -> fraction [0..1].
2 const float32 pos = engine.GetPlaybackPosition(handle);
3 const float32 frac = (durSec > 0.0) ? math::NkClamp((float32)(pos / durSec), 0.0f,
4 1.0f) : 0.0f;
5 const int32 headCol = (int32)(frac * (float32)cols);
```

Le `NkClamp` n'est pas superflu : la position peut légèrement dépasser la durée en fin de lecture, à cause de la granularité du tampon.

Ce chapitre en bref

NKAudio repose sur un invariant : **un seul format interne**. Tout ce qui entre y est converti, ce qui supprime la combinatoire des conversions et rend le mélange possible. Le patron est en cinq temps, et il ne se raccourcit pas. Les **bus** permettent de régler des groupes ensemble — effets, musique, voix — au lieu de retoucher chaque son. Un fichier long se *diffuse* au lieu d'être chargé. Et le rappel procédural s'exécute sur le **fil audio** : ce qu'on y fait de lent s'entend, sous forme de coupures.

18.12 Exercices

1 — Le son au clic, complet

Ajoutez à votre application `NKGui` trois boutons et trois sons différents, tous sur le bus «UI». Puis ajoutez un `Checkbox` «sons d'interface» qui appelle `engine.GetBus("UI")->SetMute(...)`.

Vérifiez ensuite trois choses : que couper le bus UI n'affecte pas un son joué sur «SFX»; que l'application démarre normalement si vous renommez les fichiers audio pour les rendre introuvables; et que la fermeture n'émet aucun bruit parasite — c'est-à-dire que vous appelez bien `Shutdown()` avant `Free()`.

2 — Mesurer sans écouter

Écrivez un programme console qui initialise le moteur en `NULL_OUTPUT`, joue un son, et rend une seconde de mixage dans un tableau avec `RenderToBuffer`. Calculez et affichez le maximum absolu et la moyenne quadratique du tableau.

Recommencez en réglant le bus sur `SetVolume(0.5f)` : le maximum doit être divisé par deux. Puis avec `SetMute(true)` : le tableau doit être rigoureusement nul. Vous venez d'écrire un test audio automatisable, sans matériel.

3 — Prouver l'ordre de fermeture

Écrivez volontairement la mauvaise séquence : `AudioLoader::Free(sample)`; puis `engine.Shutdown()`; sur un son de plusieurs secondes en cours de lecture. Lancez plusieurs fois et notez ce que vous entendez et ce qui se passe.

Puis remettez le bon ordre. Cet exercice n'a l'air de rien : il vous vaccine contre une classe entière de bugs qu'on ne diagnostique pas au débogueur, parce que le plantage arrive ailleurs et plus tard.

4 — Un synthétiseur en dix lignes

Utilisez `PlayProcedural` avec un *callback* qui remplit le tampon d'une sinusoïde, en gardant la phase dans une variable capturée par référence. Ajoutez un `SliderFloat` `NKGui` qui règle la fréquence.

Attention : le *slider* s'écrit depuis le fil principal et le *callback* lit depuis le fil audio. Réfléchissez à ce que vous partagez — et relisez l'interdiction d'allouer et de verrouiller. Une variable atomique de type `float` est la bonne réponse ; un `NkVector` redimensionné dans le *callback* est la mauvaise.

Questions de cours

1. Pourquoi un format interne unique plutôt que de gérer tous les formats ?
2. Citez les cinq temps du patron canonique.
3. À quoi sert un bus, et que permet-il qu'un réglage par son ne permet pas ?
4. Quand faut-il diffuser un fichier plutôt que le charger ?
5. Que ne doit-on jamais faire dans le rappel audio, et pourquoi ?

Exercice de logique

Un son se déclenche correctement au clavier, et « ne marche pas » au clic de souris — alors que les deux chemins appellent la même fonction de lecture.

1. Le son est-il forcément en cause ? Argumentez, puis dites où vous chercheriez *en premier*.
2. Énumérez trois causes qui n'ont rien à voir avec l'audio.
3. Proposez une mesure qui sépare « le son n'est pas joué » de « le son est joué et inaudible ». Précisez ce qu'elle affiche dans chaque cas.

NKMedia : lire et écrire de la vidéo

Ce chapitre est le plus court des cinq, et c'est délibéré.

NKMedia est de très loin le plus gros module du moteur. Il contient des décodeurs H.264, HEVC, VP8, VP9, AV1, MPEG-2, Theora, AAC, Opus (avec ses deux moitiés CELT et SILK), AMR — tous écrits à la main, sans ffmpeg. Le seul décodeur AV1 pèse 286 Ko de source. Un chapitre pour débutants qui essaierait d'en faire le tour serait illisible et faux.

Nous nous limiterons donc à **quatre classes de façade**, plus une cinquième en bonus. Tout le reste est de la plomberie interne, et vous n'avez aucune raison d'y toucher.

Le périmètre de ce chapitre

- NkMediaProbe — qu'y a-t-il dans ce fichier ?
- NkVideoWriter — écrire une vidéo, en MJPEG ;
- NkVideoReader — la lire image par image ;
- NkVideoRecorder — enregistrer ce que l'application affiche ;
- NkImageSequenceWriter — écrire une séquence d'images.

Cinq classes, aucune ligne de codec. Si vous maîtrisez ces cinq-là, vous savez faire tout ce dont une application a besoin.

19.1 Le module, et une inversion inhabituelle

Kernel/Runtime/NKMedia/NKMedia.jenga:17-22

```
1 nkentseudependson(
2     ["NKCore", "NKPlatform", "NKMemory", "NKContainers", "NKMath", "NKLogger",
3     "NKStream", "NKFileSystem", "NKImage", "NKThreading", "NKTime"],
4     selfexport="NKMedia",
5     extra_includes=["src"],
6 )
```

NKImage est dans la liste, et pour une raison élégante : **le codec MJPEG n'est rien d'autre que le codec JPEG de NKImage appliqué image par image**. Les séquences d'images passent, elles aussi, par les codecs PNG, BMP, TGA et QOI de NKImage. Le chapitre 5 n'était donc pas un préalable arbitraire.

Il n'y a pas d'en-tête parapluie : on inclut le sous-en-tête précis dont on a besoin. Tout vit dans le *namespace* `nkentseu::media`.

Kernel/Runtime/NKMedia/ — arborescence (abrégée)

```
1 NKMedia/
2   NKMedia.jenga · ROADMAP.md (1529 l. — la source de verite sur l'etat reel)
3   src/NKMedia/
4     NkMediaProbe.{h,cpp}      <- detection conteneur + pistes
```

```

5     NkMediaDemux.{h,cpp}      <- extraction des paquets audio
6     Video/
7     NkVideoTypes.h           <- NkVideoInputFormat
8     NkVideoReader.{h,cpp}     <- LECTURE : conteneur -> RGBA8
9     NkVideoWriter.{h,cpp}     <- ECRITURE simple
10    NkVideoRecorder.{h,cpp}    <- CAPTURE A/V threadée
11    NkVideoConverter.{h,cpp}   <- sequence/AVI -> video
12    NkImageSequenceWriter.{h,cpp}
13    Containers/ NkAviWriter · NkMovWriter · NkMp4H264Writer · NkWebmWriter
14    Audio/Containers/NkWavWriter.{h,cpp}
15    Codecs/ Video/{H264,HEVC,VP8,VP9,AV1,Mpeg1,Mpeg2,Theora} · Aac · Opus · AMR

```

19.1.1 Ici, les commentaires *sous-estiment* le module

Nous avons pris l'habitude, depuis le chapitre 5, de nous méfier des commentaires qui promettent trop. NKMedia présente le problème inverse, et il faut le savoir pour ne pas renoncer à des fonctionnalités qui existent.

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoReader.h:8-11

```

1 // - Conteneur MOV/MP4 (ISOBMFF) : piste vidéo -> MJPEG (mjpa/jpeg). H264 = à venir.
2 // H264/AVC (MP4 courant) exige un décodeur complet (gros chantier séparé) : la
3 // Lecture d'une piste avc1 renvoie une erreur claire tant que le décodeur n'existe pas.

```

C'est faux. Le fichier `Video/NkVideoReader.cpp` affecte réellement `info.codec` à "h264" (lignes 1592, 1828, 2002), "hevc" (955, 1563), "vp8" (1572, 1650), "vp9" (1578, 1635), "av1" (1586, 1644), "mpeg2" (1869), "theora" (1944), en plus de "mjpeg", "rawrgb" et "image". Les décodeurs sont branchés.

De même, `Video/NkVideoWriter.h:5-6` annonce «AVI pour l'instant; MP4/WebM à venir» alors que les écrivains MOV/MP4 et WebM sont livrés.

La source de vérité de ce module

Pour NKMedia, la vérité n'est ni dans les en-têtes ni dans les `.jenga` : elle est dans [Kernel/Runtime/NKMedia/ROADMAP.md](#), un document de 1 529 lignes tenu à jour. Prenez le réflexe de l'ouvrir avant de conclure qu'une fonctionnalité manque.

Voici l'essentiel de cette feuille de route, au moment de la rédaction :

Brique	Statut
Probe / démux conteneurs	fini — MP4, WebM, WAV, OGG, MP3, FLAC
Extraction de paquets	fini — MP4 (stb1 et fMP4), WebM
Opus CELT / SILK	fini (SILK exact au bit face à libopus)
Dispatcher Opus	<i>partiel</i> — mode hybride non fait
Décodeur AAC-LC	fini, exact au bit
Muxers (écriture)	fini — AVI, MOV/MP4, WAV, WebM
Vidéo (décodage)	fini — H.264 Main+High, VP8, VP9, HEVC
Vidéo (encodage)	en cours — RAW/MJPEG/MPEG-1 + H.264 baseline
AV1 / MPEG-2 / Theora	restent à faire

Ce sur quoi ce cours ne s'engage pas

- **L'encodage H.264** est marqué « en cours » et limité au profil *baseline*. Il fonctionne — le NkVideoRecorder l'utilise par défaut — mais ce n'est pas un chemin sur lequel bâtir un exercice de débutant. **Restez en MJPEG**;
- **AV1, MPEG-2 et Theora** : le code de décodage existe, la feuille de route ne les déclare pas terminés;
- **Opus en mode hybride** et en stéréo peut être « refusé proprement » : un fichier .opus courant n'est pas garanti;
- **HEVC** refuse proprement les tuiles, le PCM, les échantillonnages 4 :2 :2 et 4 :4 :4;
- **VP8** refuse la segmentation par macrobloc et les partitions multiples.

« Refusé proprement » est une bonne nouvelle : cela signifie une erreur claire plutôt qu'une image corrompue.

19.2 NkMediaProbe : qu'y a-t-il dans ce fichier ?

C'est la classe par laquelle commencer, parce qu'elle ne décode rien : elle lit les en-têtes et vous dit ce que contient le fichier.

Kernel/Runtime/NKMedia/src/NKMedia/NkMediaProbe.h:21-70 (abrégé)

```
1 enum class NkMediaContainer { NK_UNKNOWN, NK_MP4, NK_WEBM, NK_WAV, NK_OGG, NK_MP3, NK_FLAC };
2 enum class NkMediaTrackType { NK_UNKNOWN, NK_AUDIO, NK_VIDEO };
3
4 struct NkMediaTrack {
5     NkMediaTrackType type = NkMediaTrackType::NK_UNKNOWN;
6     NkString codec;           // "aac", "opus", "vorbis", "vp8", "vp9", "h264", "pcm", "mp3", ...
7     int32 sampleRate = 0;     int32 channels = 0;           // audio
8     int32 width = 0;          int32 height = 0;            // video
9     int32 bitsPerSample = 0;  bool pcmBigEndian = false;
10    NkVector<nk_uint8> codecPrivate;                          // OpusHead pour Opus, etc.
11 };
12 struct NkMediaInfo {
13     NkMediaContainer container = NkMediaContainer::NK_UNKNOWN;
14     NkVector<NkMediaTrack> tracks;
15     const char* ContainerName() const;
16     const NkMediaTrack* FirstAudio() const;
17     const NkMediaTrack* FirstVideo() const;
18 };
19 struct NkMediaProbe {
20     static bool Probe(const uint8* data, usize size, NkMediaInfo& out);
21     static bool ProbeFile(const char* path, NkMediaInfo& out);
22     static NkMediaContainer DetectContainer(const uint8* data, usize size);
23     static bool SelfTest();
24 };
```

L'usage réel, tiré de l'application de test :

Applications/NKMediaTest/src/main.cpp:331-353 (abrégé)

```
1 if (argc >= 2) {
2     media::NkMediaInfo info;
3     if (!media::NkMediaProbe::ProbeFile(argv[1], info)) {
4         printf("[ERREUR] probe echoue : %s\n", argv[1]);
```

```

5         return 1;
6     }
7     printf("Conteneur : %s\n", info.ContainerName());
8     printf("Pistes : %d\n", (int)info.tracks.Size());
9     for (uint64 i = 0; i < info.tracks.Size(); ++i) {
10         const media::NkMediaTrack &t = info.tracks[i];
11         printf(" [%d] %-5s codec=%-6s", (int)i, TrackTypeName(t.type), t.codec.CStr());
12         if (t.type == media::NkMediaTrackType::NK_AUDIO)
13             printf(" %d Hz, %d canal(aux)", t.sampleRate, t.channels);
14         if (t.type == media::NkMediaTrackType::NK_VIDEO)
15             printf(" %dx%d", t.width, t.height);
16         printf("\n");
17     }

```

FirstAudio() et FirstVideo() évitent de parcourir soi-même les pistes quand on ne s'intéresse qu'à la principale — ils rendent nullptr s'il n'y en a pas.

Ce qu'il faut retenir

Avant d'ouvrir un fichier avec NkVideoReader, sondez-le. Vous saurez alors si le codec est l'un de ceux que vous savez traiter, et vous pourrez afficher un message utile au lieu d'un échec muet. ProbeFile est instantané : il ne décode aucune image.

19.3 Écrire une vidéo

19.3.1 L'API

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoWriter.h:26-81

```

1  enum class NkVideoCodec { RAW_BGR, MJPEG, MPEG1 };
2  enum class NkVideoContainer { AVI, MOV, ELEMENTARY };
3  // (NkVideoInputFormat { RGB24, RGBA32, BGR24 } vit dans Video/NkVideoTypes.h:16-20)
4
5  struct NkVideoConfig {
6      int32 width = 0; int32 height = 0;
7      int32 fpsNum = 30; int32 fpsDen = 1;
8      NkVideoCodec codec = NkVideoCodec::MJPEG;
9      NkVideoContainer container = NkVideoContainer::AVI;
10     int32 quality = 90; // MJPEG 1..100
11     int32 audioSampleRate = 0; // 0 = video muette
12     int32 audioChannels = 0;
13 };
14 class NkVideoWriter {
15     bool Open(const char* path, const NkVideoConfig& cfg);
16     bool WriteFrame(const uint8* pixels, NkVideoInputFormat fmt);
17     bool AddAudioSamples(const int16* interleaved, usize frameCount);
18     bool Close();
19     bool IsOpen() const; int32 FrameCount() const;
20 };

```

Notez la cadence exprimée en fraction : fpsNum / fpsDen. Pour du 30 images/s, 30/1; pour du 29,97 (NTSC), 30000/1001. Les formats vidéo n'aiment pas les flottants.

19.3.2 Le squelette de référence

Applications/NKVideoTest/src/main.cpp:83-112

```
1  bool MakeVideo(const char *path, media::NkVideoCodec codec, media::NkVideoContainer
   container, int32 w, int32 h,
2      int32 fps, int32 nframes) {
3      media::NkVideoConfig cfg;
4      cfg.width = w;
5      cfg.height = h;
6      cfg.fpsNum = fps;
7      cfg.fpsDen = 1;
8      cfg.codec = codec;
9      cfg.container = container;
10     cfg.quality = 88;
11
12     media::NkVideoWriter vw;
13     if (!vw.Open(path, cfg)) {
14         ::printf(" [ERREUR] ouverture %s\n", path);
15         return false;
16     }
17     uint8 *rgb = (uint8 *)memory::NkAlloc((usize)w * h * 3);
18     for (int32 fr = 0; fr < nframes; ++fr) {
19         RenderFrame(rgb, w, h, fr, nframes);
20         if (!vw.WriteFrame(rgb, media::NkVideoInputFormat::RGB24)) {
21             ::printf(" [ERREUR] trame %d\n", fr);
22             memory::NkFree(rgb);
23             vw.Close();
24             return false;
25         }
26     }
27     memory::NkFree(rgb);
28     vw.Close();
29     return true;
30 }
```

Quatre choses à retenir de ce bloc.

1. **Le tampon est alloué une fois**, hors de la boucle, et réutilisé. Allouer une image par trame est le meilleur moyen de rendre l'encodage plus lent que le décodage;
2. **la mémoire vient de memory::NkAlloc** et repart par memory::NkFree — la règle du chapitre 5 ne change pas;
3. **le chemin d'erreur appelle quand même Close()**;
4. **le format d'entrée est explicite** : NkVideoInputFormat::RGB24. Le writer accepte aussi RGBA32 et BGR24 et fait la conversion.

Close() n'est pas optionnel

Un fichier AVI se termine par un index; un MP4 par un atome moov. Aucun des deux n'est écrit tant que Close() n'a pas été appelé. Sans lui, vous obtenez un fichier de la bonne taille, contenant toutes vos images, et qu'**aucun lecteur au monde n'ouvrira**. Le message typique est «moov atom not found».

Le dépôt prend explicitement cette précaution dans son code de capture :

Applications/NKViewportDemo/src/NKViewportDemo/main.cpp:358-362

```

1 // Robustesse : si la fenêtre est fermée AVANT la fin de l'enregistrement, on
  finalise quand même
2 // (écrit le moov) → le MP4 reste lisible (sinon "moov atom not found").
3 if (rec.IsRecording()) {
4     rec.End();
5 }

```

La même règle vaut pour `NkVideoRecorder::End()` et `NkWebmWriter::Finalize()`. **Prévoyez une finalisation sur tous les chemins de sortie, y compris la fermeture brutale de la fenêtre.**

19.3.3 Deux limites de l'écriture

- **L'audio n'est pas supportée partout** : `Video/NkVideoWriter.h:50-52` précise que la piste audio est « supportée par AVI ... et MOV/MP4 ... Non supportée par ELEMENTARY/MPEG1 (Open échoue si demandée) ». Au moins l'échec est franc ;
- **le sens des pixels** : les encodeurs veulent du *haut en bas* (`Video/NkVideoTypes.h:16-20`). Or OpenGL rend de bas en haut. Nous verrons le paramètre qui corrige cela.

19.3.4 La séquence d'images, à la manière de Blender

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkImageSequenceWriter.h:21-42

```

1 enum class NkImageSeqFormat { PNG, JPEG, BMP, TGA, QOI };
2 struct NkImageSequenceWriter {
3     bool Open(const char* dir, const char* basename, int32 width, int32 height,
4               NkImageSeqFormat fmt, int32 padding = 4, int32 quality = 90);
5     bool WriteFrame(const uint8* pixels, NkVideoInputFormat fmt);
6     bool Close(); int32 FrameCount() const;
7 };

```

Applications/NKVideoTest/src/main.cpp:185-201

```

1 media::NkImageSequenceWriter seq;
2 bool ok = seq.Open(outDir, "nkframe", W, H, media::NkImageSeqFormat::PNG, 4, 90);
3 if (ok) {
4     uint8 *rgb = (uint8 *)memory::NkAlloc((usize)W * H * 3);
5     for (int32 fr = 0; fr < 5 && ok; ++fr) { // 5 images d'exemple
6         RenderFrame(rgb, W, H, fr, N);
7         ok = seq.WriteFrame(rgb, media::NkVideoInputFormat::RGB24);
8     }
9     memory::NkFree(rgb);
10    seq.Close();
11 }

```

Cela produit `nkframe_0000.png`, `nkframe_0001.png`... Cette classe délègue entièrement aux codecs de `NkImage` : c'est le chemin le plus fiable du module, et le plus facile à déboguer — vous pouvez ouvrir chaque image.

Et pour transformer ensuite la séquence en vidéo, il existe un raccourci en une ligne :

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoConverter.h:24-30

```
1 static int32 ImageSequenceToVideo(const char* prefix, int32 digits, const char* suffix,
2                                   int32 first, int32 count, const char* outPath,
3                                   int32 fpsNum, int32 fpsDen, int32 quality = 90);
4 static int32 MjpegAviToVideo(const char* aviPath, const char* outPath, int32 quality = 90);
```

Ces deux fonctions renvoient le *nombre de trames* traitées, pas un booléen : zéro signifie l'échec.

19.4 Lire une vidéo

19.4.1 L'API

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoReader.h:26-70

```
1 struct NkVideoFrame {
2     NkVector<nk_uint8> rgba;           // width*height*4, row-major, haut->bas
3     int32 width = 0;   int32 height = 0;
4     int64 timestampMs = 0;
5     int32 index = -1;
6 };
7 struct NkVideoReaderInfo {
8     int32 width = 0;   int32 height = 0;
9     int32 frameCount = -1;           // -1 = inconnu
10    double fps = 0.0;
11    NkString codec;                  // "mjpeg"|"rawrgb"|"h264"|"hevc"|"vp8"|"vp9"|...
12    NkString container;              // "avi"|"mov"|"mp4"|"sequence"
13 };
14 class NkVideoReader {
15     NkVideoReader(const NkVideoReader&) = delete;           // NON COPIABLE
16     bool Open(const char* path);
17     bool IsOpen() const;
18     const NkVideoReaderInfo& Info() const;
19     bool ReadFrame(NkVideoFrame& out);                       // false = fini
20     bool SeekFrame(int32 index);
21     int32 CurrentIndex() const;
22     void Close();
23     static bool SelfTest();
24 };
```

Le point capital est dans le commentaire de `NkVideoFrame::rgba` : **RGBA8, ligne par ligne, de haut en bas**. C'est exactement le format qu'attendent `NkTexture::Update` et `UploadImageRGBA`. Il n'y a aucune conversion à écrire.

19.4.2 La boucle de lecture

Applications/NkVideoReadTest/src/main.cpp:3041-3070 (abrégé)

```
1 const char *path = argv[1];
2 NkVideoReader rd;
3 if (!rd.Open(path)) {
4     printf(" [K0] impossible d'ouvrir/lire : %s\n", path);
5     return 1;
6 }
7 const NkVideoReaderInfo &in = rd.Info();
8 printf(" conteneur=%s codec=%s %dx%d %.2f fps frames=%d\n", in.container.CStr(),
```

```

    in.codec.CStr(), in.width,
9      in.height, in.fps, in.frameCount);
10
11     int32 count = 0;
12     NkVideoFrame fr;
13     while (rd.ReadFrame(fr)) {
14         /* ... traiter fr.rgba ... */
15         ++count;
16     }

```

Remarquez que `fr` est déclarée *hors* de la boucle et réutilisée : son `NkVector` conserve sa capacité d'une trame à l'autre. La déclarer dans la boucle provoquerait une allocation par image.

`Open` accepte aussi un **dossier d'images** : le champ `container` vaut alors "sequence" et le codec "image". C'est le moyen le plus simple de tester votre code de lecture sans avoir de vidéo sous la main.

19.4.3 Le seek n'est pas ce que vous croyez

Applications/NkVideoReadTest/src/main.cpp:2762-2765

```

1  SeekFrame se contente de repositionner sur l'IDR précédente (voir implémentation) : il
2  faut ensuite redécoder EN AVANT jusqu'à la cible, exactement comme le fait la boucle de
3  rattrapage du lecteur (Applications/NkVideoPlayer).

```

Dans un format à trames inter-dépendantes comme H.264, l'image numéro 1 000 ne peut pas être décodée seule : elle décrit des différences par rapport aux précédentes. `SeekFrame(1000)` vous replace donc sur la dernière image autonome avant la cible, et c'est à vous d'appeler `ReadFrame` jusqu'à ce que `CurrentIndex() >= 1000`.

Sur une séquence d'images ou un AVI MJPEG — où chaque image est indépendante — le seek est exact.

19.4.4 Décoder coûte cher

Une phrase à graver : **ne décidez jamais plus d'une image par image affichée**. Le lecteur du dépôt borne explicitement sa boucle de rattrapage :

Applications/NkVideoPlayer/src/main.cpp:562-620 (extrait)

```

1      if (!paused && !ended) {
2          if (audioOn && !streamPlayer.IsFinished())
3              mediaClock = streamPlayer.GetPositionSeconds();
4          else
5              mediaClock += (float64)(dt * speed);
6          const int32 targetIdx = (int32)(mediaClock * (float64)fps);
7          /* resynchronisation dure si on a plus de 1,5 s de retard */
8          const int32 kHardResyncLagFrames = (int32)(fps * 1.5f);
9          if (!firstFrame && (targetIdx - reader.CurrentIndex()) > kHardResyncLagFrames) {
10             if (reader.SeekFrame(targetIdx) && reader.ReadFrame(fr))
11                 havePendingFrame = true;
12         }
13         NkChrono burstTimer;
14         while ((firstFrame || reader.CurrentIndex() < targetIdx) &&
15             burstTimer.Elapsed().ToMilliseconds() < 150.0) {
16             if (reader.ReadFrame(fr)) { havePendingFrame = true; firstFrame = false; }
17             else if (loop) { /* SeekFrame(0) + ReadFrame */ }
18             else { paused = true; break; }

```

```

19         }
20         if (havePendingFrame)
21             pushFrame(); // upload + dessin UNE SEULE FOIS, avec la dernière image de la
    rafale
22     }

```

Trois garde-fous dans ce seul bloc : la rafale de rattrapage est limitée à 150 ms de temps mur; au-delà d'une seconde et demie de retard on saute carrément; et surtout, **l'upload GPU n'a lieu qu'une fois**, avec la dernière image de la rafale. Décoder dix images pour n'en afficher qu'une est acceptable; en uploader dix ne l'est pas.

L'horloge maîtresse

Notez la première ligne : quand l'audio joue, c'est *lui* qui donne l'heure (`mediaClock = streamPlayer.GetPositionSeconds()`). Ce n'est qu'en son absence qu'on accumule le *delta time*. La raison est physiologique : l'oreille détecte un hoquet audio de 20 ms, l'œil ne voit pas une image manquante. On synchronise donc toujours l'image sur le son, jamais l'inverse.

19.5 Afficher la vidéo

19.5.1 Dans une fenêtre NkCanvas

Le principe : créer **une** texture à la taille de la vidéo, puis la mettre à jour à chaque nouvelle image.

Applications/NkVideoPlayer/src/main.cpp:355-362

```

1 // — 4) Texture RGBA de la taille vidéo + sprite d'affichage —————
2 NkTexture frameTex;
3 if (!frameTex.Create(*target.GetRenderer(), (uint32)vidW, (uint32)vidH, NkColor2D{0, 0,
    0, 255})) {
4     logger.Error("[NkVideoPlayer] echec creation texture {0}x{1}", vidW, vidH);
5     window.Close();
6     return -5;
7 }
8 NkSprite sprite(frameTex);

```

puis, à chaque image décodée :

Applications/NkVideoPlayer/src/main.cpp:424-431

```

1 auto pushFrame = [&]() {
2     rawW = fr.width;
3     rawH = fr.height;
4     rawFrame = fr.rgba;
5     applyLook(fr.rgba.Data(), (uint64)fr.width * fr.height);
6     frameTex.Update(fr.rgba.Data(), (uint32)fr.width, (uint32)fr.height, 0, 0);
7     haveFrame = true;
8 };

```

et le dessin, qui n'a rien de particulier :

Applications/NkVideoPlayer/src/main.cpp:664-668

```

1      target.Clear(NkColor2D{0, 0, 0, 255});
2      if (haveFrame)
3          target.Draw(static_cast<const NkDrawable &>(sprite));
4      target.Display();

```

Ne recréez jamais la texture par image

Create alloue une texture GPU ; Update y recopie des pixels. Appeler Create à chaque frame, c'est allouer et libérer une texture soixante fois par seconde : la mémoire GPU se fragmente et le pilote finit par protester. Créez une fois, mettez à jour ensuite.

19.5.2 Dans une interface

Le motif est le même, avec l'API du backend d'interface. L'IDE du dépôt en donne la version complète, y compris le cas du changement de dimensions :

Applications/NKCode/src/NKCode/Shell/NkVideoViewer.h:70-82

```

1      inline void NkVideoUpload(editorkit::NkEditorShell *shell, NkVideoClip *c) {
2          if (!shell || c->frame.rgba.Empty() || c->frame.width <= 0 || c->frame.height <=
3              0)
4              return;
5          const uint8 *px = c->frame.rgba.Data();
6          if (c->texId == 0 || c->texW != c->frame.width || c->texH != c->frame.height) {
7              c->texId = shell->UploadRGBA(px, c->frame.width, c->frame.height);
8              c->texW = c->frame.width;
9              c->texH = c->frame.height;
10         } else {
11             shell->UpdateRGBA(c->texId, px, c->frame.width, c->frame.height);
12         }
13         c->haveFrame = c->texId != 0;
14     }

```

Premier appel ou changement de taille : UploadRGBA, qui crée. Sinon : UpdateRGBA, qui recopie. C'est exactement la même logique que pour NKImage au chapitre 5, avec une texture qui vit plus longtemps qu'une frame.

Et le cadencement, côté interface :

Applications/NKCode/src/NKCode/Shell/NkVideoViewer.h:153-170

```

1      float32 dt = ctx.input.dt;
2      if (dt <= 0.f || dt > 0.25f)
3          dt = 1.f / 60.f; // garde-fou (onglet revenu au premier plan / hoquet)
4      const float32 frameDur = 1.f / (c->fps * (c->speed > 0.01f ? c->speed : 0.01f));
5      if (c->playing && !c->ended) {
6          c->acc += dt;
7          if (c->acc > frameDur * 4.f)
8              c->acc = 0.f; // evite le rattrapage explosif
9          while (c->acc >= frameDur) {
10             c->acc -= frameDur;
11             if (c->reader->ReadFrame(c->frame)) {
12                 c->index = c->frame.index;
13                 NkVideoUpload(shell, c);
14             }
15             /* ... Loop / fin ... */

```

```

16         }
17     }

```

Les deux garde-fous méritent d'être notés, parce qu'ils correspondent à deux situations réelles : un *delta time* aberrant quand l'onglet revient au premier plan après plusieurs secondes, et un accumulateur qui a débordé et voudrait décoder quarante images d'un coup.

19.6 Enregistrer ce que l'application affiche

19.6.1 L'API

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:39-77

```

1  enum class NkRecorderCodec { H264, MJPEG };
2
3  struct NkVideoRecorder {
4      bool Begin(const char* path, int32 width, int32 height, int32 fpsNum = 60, int32
        fpsDen = 1,
5          int32 qp = 20, int32 maxQueuedFrames = 32,
6          NkRecorderCodec codec = NkRecorderCodec::H264, int32 mjpegQuality = 90);
7      int32 AddAudio(int32 sampleRate, int32 channels, const char* lang3 = nullptr);
8      int32 AddSubtitleTrack(const char* lang3 = nullptr);
9      bool PushVideo(const uint8* pixels, NkVideoInputFormat fmt, bool flipVertical =
        false);
10     void PushAudio(int32 trackIdx, const int16* interleaved, uint32 frames);
11     void AddSubtitle(int32 trackIdx, const char* utf8, uint32 startMs, uint32 durMs);
12     bool End();
13     bool IsRecording() const; int32 FrameCount() const;
14     int32 QueueDepth(); uint64 DroppedFrames(); double EncodeFps();
15 };

```

La différence avec NkVideoWriter est architecturale :

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:5-8

```

1  **Encodage sur un THREAD de fond** : la boucle de rendu pousse les trames capturées
2  dans une file (rapide) et un worker encode en arrière-plan → l'application reste
3  FLUIDE pendant la capture (l'encodeur H.264 est lourd).

```

PushVideo rend la main immédiatement; l'encodage a lieu ailleurs. End() joint ce fil : ne détruisez jamais un recorder sans avoir appelé End().

19.6.2 La capture, de bout en bout

Le point de jonction avec le reste du moteur est délicieusement simple : la capture d'écran est une NkImage.

Applications/NKViewportDemo/src/NKViewportDemo/main.cpp:329-362 (abrégé)

```

1      if (record && target->CaptureToImage(capImg) && capImg.IsValid()) {
2          if (!rec.IsRecording()) {
3              if (rec.Begin("viewport_capture.mp4", capImg.Width(), capImg.Height(), 30, 1,
4                  24)) {
5                  recAudio = rec.AddAudio(kRate, 1, "fre");
6                  const int32 st = rec.AddSubtitleTrack("fre");

```

```

6         rec.AddSubtitle(st, "Capture live NKCanvas (DX11) - Nkentseu", 0, 4000);
7     }
8 }
9     if (rec.IsRecording()) {
10         rec.PushVideo(capImg.Pixels(), media::NkVideoInputFormat::RGBA32);
11         rec.PushAudio(recAudio, recTone.Data(), (uint32)kAudioPerFrame);
12         if (recFrames >= kRecTotal) { rec.End(); running = false; }
13     }
14 }

```

`NkRenderWindow::CaptureToImage(NkImage&)` remplit une `NkImage`, et `capImg.Pixels()` part directement dans `PushVideo`. `NKImage`, `NKCanvas` et `NKMedia` se rejoignent en une ligne.

19.6.3 Trois pièges du recorder

L'ordre des appels

`Video/NkVideoRecorder.h:51-54` : `AddAudio` et `AddSubtitleTrack` sont à appeler « juste après `Begin` (avant les trames) ». Ajouter une piste en cours d'enregistrement n'a pas de sens dans un conteneur MP4, dont la table des pistes est écrite en tête.

En MJPEG, l'audio est ignorée — silencieusement

`Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:36-38`

```
1    conteneur MOV/MP4, **VIDEO SEULE (audio/sous-titres ignorees dans ce mode)**
```

C'est le piège le plus vicieux du module : `AddAudio` vous rend un index parfaitement valide, `PushAudio` accepte vos échantillons sans broncher, et le fichier produit est muet. Rien ne vous prévient.

Vous êtes donc devant un choix : MJPEG (léger, sûr, muet) ou H.264 (avec audio, mais l'encodeur est marqué « en cours »). Pour apprendre, prenez MJPEG et enregistrez l'audio à part.

Le recorder abandonne des trames

`Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:45-46`

```
1    `maxQueuedFrames` BORNE la file video : si l'encodeur (H.264 lourd) prend du retard,
2    les nouvelles trames sont ABANDONNEES (drop-newest) au lieu de gonfler la memoire
3    (temps reel : perdre une trame vaut mieux que geler). Abandons comptes
    (DroppedFrames()).
```

C'est le bon comportement — mais il faut le savoir, sinon on s'étonne qu'une capture de dix secondes à 60 images/s en contienne 400. Surveillez `DroppedFrames()` et `QueueDepth()`, et réglez si nécessaire :

`Applications/Sandbox/src/Demo/main.cpp:1025-1050 (extrait)`

```
1        const bool encoderBusy = recorder && recorder->QueueDepth() >= 24;
2        if (!encoderBusy)
3            (void)recordCapture.EnqueueCopy(...);
```

19.6.4 Le sens des pixels

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:56-57

```
1  `flipVertical` pour framebuffer bottom-up (OpenGL)
```

OpenGL considère l'origine en bas à gauche; les formats vidéo la placent en haut à gauche. Si votre capture provient d'un *framebuffer* OpenGL lu directement, passez `flipVertical = true`. Le symptôme d'un oubli est sans ambiguïté : la vidéo est à l'envers.

19.7 Le démux audio, en deux mots

Nous n'en aurons pas besoin dans ce cours, mais vous croiserez la classe :

Kernel/Runtime/NKMedia/src/NKMedia/NkMediaDemux.h:22-48

```
1  struct NkMediaPacket {
2      usize offset = 0;    // position dans le buffer SOURCE (ne copie pas !)
3      usize size = 0;
4      int64 timestampMs = 0;
5      int64 granule = -1;    // OGG
6      int64 discardPaddingNs = 0;    // WebM
7  };
8  struct NkMediaDemux {
9      static bool ExtractAudioPackets(const uint8* data, usize size, const NkMediaInfo&
10         info,
11                                     NkVector<NkMediaPacket>& out);
12      static bool ExtractAudioPacketsFile(const char* path, NkVector<nk_uint8>& outBytes,
13         NkMediaInfo& outInfo, NkVector<NkMediaPacket>&
14         out);
15      static bool SelfTest();
16  };
```

Les paquets pointent dans le tampon source

`NkMediaPacket` ne contient pas de données : il contient un *décalage*. Le commentaire est explicite — «Un paquet encodé (pointe dans le buffer d'origine; ne copie pas)» (`NkMediaDemux.h:22`), et pour la variante fichier : «outBytes reçoit le buffer (à garder vivant car les paquets pointent dedans)» (`NkMediaDemux.h:47`).

Si votre `NkVector<nk_uint8>` bytes sort de portée avant que vous ayez fini de lire les paquets, vous lisez de la mémoire libérée. Gardez les deux ensemble, dans la même portée.

19.8 Vérifier que tout marche sur votre machine

NKMedia est le module le mieux doté du moteur en auto-tests — il en compte 52. L'application `NkVideoReadTest`, lancée *sans argument*, les enchaîne :

Applications/NkVideoReadTest/src/main.cpp:252-276

```
1  if (argc < 2) {
2      printf(" [self-test] ecrire AVI MJPEG -> relire -> verifier...\n");
3      bool ok = NkVideoReader::SelfTest();
4      printf(" [ %s ] NkVideoReader::SelfTest (AVI MJPEG round-trip)\n", ok ? "OK " :
5      "KO");
6      bool okH264 = NkH264Decoder::SelfTest();
```



```

6      bool okCavlc = NkH264Cavlc::SelfTest();
7      bool okVp9 = NkVp9Decoder::SelfTest();
8      bool okAv1 = NkAv1Decoder::SelfTest();
9      bool okHevc = NkHevcDecoder::SelfTest();
10     bool okHevcCabac = NkHevcCabacState::SelfTest();
11     bool okWav = NkWavWriter::SelfTest();
12     bool okWebm = NkWebmWriter::SelfTest();
13     bool all = ok && okH264 && okCavlc && okVp9 && okAv1 && okHevc && okHevcCabac &&
        okWav && okWebm;
14     printf("=== %s ===\n", all ? "LECTURE VIDEO OPERATIONNELLE" : "ECHEC");
15     return all ? 0 : 1;
16 }

```

Le premier de la liste est le plus parlant : il écrit un AVI MJPEG, le relit, et vérifie que ce qui sort correspond à ce qui est entré. C'est ce *round-trip* qui valide indirectement NkVideoWriter, qui n'a pas d'auto-test propre — pas plus que NkVideoRecorder, NkVideoConverter, NkImageSequenceWriter, ni les écrivains de conteneurs.

Deux détails pratiques :

- ces auto-tests écrivent dans le répertoire courant : `nkvideoreader_selftest.avi`, `nkwavwriter_selftest.wav`, `nkwebmwriter_selftest.webm`. Lancez-les depuis un dossier de travail, pas depuis la racine du dépôt;
- le décodeur HEVC lit une variable d'environnement, `NK_HEVC_THREADS` : absente ou à 0, il choisit tout seul; à 1, il travaille séquentiellement (`NkVideoReadTest/src/main.cpp:249-250`). Utile pour départager un bug de décodage d'un bug de parallélisme.

19.9 Une leçon de performance qui dépasse la vidéo

La feuille de route documente un bug de performance instructif :

[Kernel/Runtime/NKMedia/ROADMAP.md:1447-1458 \(abrégé\)](#)

```

1  le vrai coupable etait `NkVideoReader::ReadFrame` [...] : la gestion du DPB (liste de
2  references) COPIAIT EN PROFONDEUR (au lieu de déplacer) jusqu'a 16 `NkH264Frame`
3  (plans Y/Cb/Cr + grilles de mouvement, ~380 Ko/image) DEUX FOIS par frame decodee
4  [...] cout mesure ~80 ms/frame et CROISSANT [...] >90% du temps total. Fix : traits::NkMove

```

Quatre-vingts millisecondes par image, dont plus de 90 % en copies inutiles. La leçon est transférable bien au-delà de la vidéo : **dans une boucle chaude, une copie profonde qui aurait dû être un déplacement peut représenter l'essentiel du temps d'exécution**. Le champ `NkVideoFrame::rgba` est un conteneur : le copier par valeur à chaque image coûte cher, et le lecteur du dépôt ne le fait que parce qu'il a explicitement besoin d'une copie non retouchée (`Applications/NkVideoPlayer/src/main.cpp:427`).

Le même document ajoute un avertissement de méthode, qui vaut pour toute mesure :

[Kernel/Runtime/NKMedia/ROADMAP.md:1449-1452](#)

```

1  (mesure fiable : chrono mur autour d'un run borne, PAS le champ `t=` de `NkVideoReadTest`
2  qui affiche le PTS video, pas le temps de decodage reel – piège rencontre une fois)

```

Le *PTS* est l'instant auquel une image doit être affichée dans la vidéo. Il n'a rien à voir avec le temps qu'il a fallu pour la décoder. Confondre les deux, c'est mesurer la durée du film au lieu de la vitesse du lecteur.

Ce chapitre en bref

NKMedia lit et écrit des vidéos, entièrement depuis zéro. Avant d'ouvrir un fichier, on le **sonde** : ce qu'il contient réellement ne se déduit pas de son extension. À l'affichage, une vidéo n'est qu'une texture qu'on remplace à chaque image — le rendu n'a aucune notion de « vidéo ». Le `Close()` n'est pas une politesse : sans lui le fichier écrit est *incomplet*, et il s'ouvre parfois quand même, ce qui rend l'oubli difficile à détecter.

19.10 Exercices

1 — Sonder avant d'ouvrir

Écrivez un petit outil en ligne de commande qui prend un chemin, appelle `NkMediaProbe::ProbeFile`, et affiche le conteneur et toutes les pistes. Passez-lui successivement un WAV, un MP3, un MP4 et un fichier texte renommé en `.mp4`. Ajoutez ensuite une règle : si la première piste vidéo n'a pas un codec parmi `"mjpeg"`, `"rawrgb"` ou `"image"`, affichez un avertissement disant que la lecture n'est pas garantie par ce cours. Vous venez d'écrire le garde-fou qui manque à la plupart des applications.

2 — Le *round-trip* complet

Générez une vidéo de 100 images en MJPEG/AVI, avec un motif dont vous pouvez prédire le contenu — par exemple un rectangle qui se déplace d'un pixel par image. Relisez-la ensuite avec `NkVideoReader` et vérifiez que le nombre d'images correspond, et que le rectangle est bien là où vous l'attendez à l'image 50. Recommencez avec le conteneur MOV. Puis essayez `RAW_BGR` et comparez la taille des fichiers : vous aurez une idée concrète de ce que compresser veut dire.

3 — Prouver que `Close()` est obligatoire

Reprenez l'exercice précédent et commentez l'appel à `vw.Close()`. Regardez la taille du fichier produit — elle est presque identique — puis essayez de l'ouvrir avec votre lecteur vidéo habituel, et avec `NkVideoReader`. Cet exercice prend deux minutes et vous fera gagner une soirée le jour où votre application se fermera brutalement en cours d'enregistrement.

4 — Un lecteur cadencé, dans votre interface

Ajoutez à votre application `NKGui` un panneau qui lit une séquence d'images depuis un dossier (c'est le chemin le plus simple : `Open` accepte un dossier) et l'affiche avec le widget `Image` du chapitre 5. Cadencez sur `Info().fps` avec un accumulateur, en reprenant les deux garde-fous de `NkVideoViewer` : *delta time* aberrant et accumulateur débordé. Ajoutez un bouton pause et un `SliderFloat` de vitesse. Vérifiez que vous n'appellez `UploadRGBA` qu'une seule fois — au premier affichage — et `UpdateRGBA` ensuite.

Questions de cours

1. Pourquoi sonder un fichier plutôt que se fier à son extension ?
2. Comment une image décodée arrive-t-elle à l'écran ?

3. Que manque-t-il à un fichier vidéo dont on a oublié le `Close()` ?
4. Où se branche un traitement des pixels dans la chaîne de lecture ?
5. Qu'est-ce qui cadence la lecture, et que se passe-t-il si on l'ignore ?

Exercice de logique

Une vidéo que vous venez d'écrire s'ouvre dans votre lecteur, mais pas dans celui du système.

1. Est-ce forcément un défaut de votre écriture ? Donnez au moins une cause où ce n'en est pas un.
2. Que peut faire votre lecteur qu'un lecteur du système ne fait pas, et pourquoi cette indulgence est-elle un piège pour vous ?
3. Concluez sur la règle générale : pourquoi un producteur ne doit-il *jamais* être validé par son propre consommateur seul ?

NKNetwork : faire parler deux machines

Le réseau est le dernier des cinq modules, et le plus abstrait : rien ne s'affiche, rien ne s'entend, et quand ça ne marche pas, il n'y a souvent aucun message. C'est aussi celui où le périmètre honnête est le plus important à poser, parce que le module contient beaucoup de code que *rien* dans le dépôt n'utilise.

20.1 Où il vit, et ce dont il a besoin

Première singularité : contrairement aux quatre autres, NKNetwork n'est pas dans `Kernel/Runtime` mais dans `Kernel/System`. C'est un service système, au même titre que le système de fichiers ou les fils d'exécution.

Kernel/System/NKNetwork/ — arborescence

```

1 NKNetwork/
2   NKNetwork.jenga · ROADMAP.md
3   tests/.jenga/          <- DOSSIER VIDE (voir plus bas)
4   src/NKNetwork/
5     NKNetwork.h          <- en-tete PARAPLUIE + alias de commodite
6     Core/NkNetDefines.h/.cpp <- types, enums, constantes
7     Transport/
8       NkSocket.h/.cpp     <- socket UDP/TCP brut
9       NkReliableUDP.h/.cpp <- ACK, retransmit, fenetre (usage INTERNE)
10    Protocol/
11      NkBitStream.h/.cpp   <- NkBitWriter / NkBitReader
12      NkConnection.h/.cpp  <- NkConnection + NkConnectionManager
13      Replication/NkNetWorld.h/.cpp
14      RPC/NkRPC.h/.cpp
15      Lobby/NkLobby.h/.cpp
16      HTTP/NkHTTPClient.h/.cpp

```

Tout vit dans le `namespace nkentseu::net`.

Kernel/System/NKNetwork/NKNetwork.jenga:27-42 (abrégé)

```

1 _NET_DEPS = ["NKCore", "NKPlatform", "NKMemory", "NKContainers", "NKLogger", "NKThreading",
2             "NKMath", "NKTime", "NKFileSystem"]
3 _NET_INCLUDES = ["src", "pch"]
4 _NET_DEFINES = []
5 if _WANT_MBEDTLS:
6     _NET_DEPS.append("NKMbedTLS")
7     _NET_INCLUDES.append("%{wks.location}/Externals/Libs/NKMbedTLS/include")
8     _NET_DEFINES.append("NKENTSEU_HTTP_USE_MBEDTLS")

```

Il faut lier ws2_32 sous Windows

Le module utilise les sockets Berkeley, qui s'appellent Winsock sous Windows. Votre *application* doit donc ajouter la bibliothèque à son édition de liens — le module ne le fait

pas pour vous. Les deux consommateurs du dépôt le font : [Applications/NkNetWorldDemo/NkNetWorldDemo.jenga:35-40](#) et [Sandbox/System/NKNetwork/NKNetworkSandbox.jenga:74-78](#) ajoutent tous deux "ws2_32" à leur `links()`.

L'oubli se manifeste par des erreurs d'édition de liens sur des symboles `__imp_socket`, `__imp_WSASStartup...` — obscures si on ne sait pas d'où elles viennent, évidentes ensuite.

20.1.1 Deux avertissements sur la documentation du module

La description du `.jenga` parle d'un autre module

[Kernel/System/NKNetwork/NKNetwork.jenga:3-15](#) décrit « un système de réflexion runtime » avec les fichiers `NkType`, `NkClass`, `NkProperty`, `NkMethod`, `NkRegistry`. C'est la description de **NKReflection**, copiée par erreur. Aucun de ces fichiers n'existe dans `NKNetwork`.

Le dossier `tests/` est vide

[Kernel/System/NKNetwork/tests/](#) ne contient qu'un sous-dossier vide. Le glob `testfiles(["tests/**/*.cpp"])` de [NKNetwork.jenga:92-95](#) ne correspond à aucun fichier : le bloc `with test()` : est un squelette mort.

De plus, il n'existe **aucune** fonction `SelfTest()` dans tout le module — là où `NKMedia` en compte 52 et `NKAudio` 2.

Cela ne veut pas dire que le module n'est pas testé : le test existe, mais c'est une application séparée, et la feuille de route le dit :

[Kernel/System/NKNetwork/ROADMAP.md:131-138](#)

```
1  ### Tests (2026-07-12)
2  - `Sandbox/System/NKNetwork` (cible `SandboxNKNetwork`) : **67 checks verts** –
3    NkBitStream round-trip + overflow, NkNetId Pack/Unpack, NkNetInterpolator,
4    réplication client/serveur sur messages forgés, **bout-en-bout loopback réel**
5    (StartServer + Connect 127.0.0.1, handshake, snapshot → spawn → delta →
6    despawn). Mode manuel `--https <url>` pour valider TLS.
```

Attention au chemin : ce sandbox est dans [Sandbox/System/NKNetwork/](#) — à la racine du dépôt — et **pas** dans [Applications/Sandbox/](#). Ses binaires sont bâtis en Debug et en Release.

20.2 Le périmètre de ce chapitre

Ce que nous enseignons, et pourquoi

Enseigné — parce que du code du dépôt l'exerce réellement :

- `NkSocket` (initialisation de plateforme, et UDP brut pour la découverte réseau);
- `NkAddress`;
- `NkConnectionManager` — la classe centrale;
- `NkBitWriter` / `NkBitReader`;
- `NkNetWorld` — la réplication d'entités.

Non enseigné — parce qu'*aucune* application du dépôt ne s'en sert :

- `NkLobby`, `NkSession`, `NkMatchmaker`, `NkDiscovery` — 67 Ko d'en-tête sans un seul

consommateur;

- NkRPC / NkRPCRouter — la feuille de route le classe « livré (déclaratif) », ce qui n'est pas la même chose que « prouvé »;
- NkReliableUDP en usage direct : il n'est employé qu'en interne par NkConnection.

Ce n'est pas un jugement sur la qualité de ce code. C'est une règle de prudence : un cours ne doit enseigner que ce qu'il peut montrer en train de fonctionner.

Sont également absents, d'après [ROADMAP.md:32-40](#) : la prédiction client et la réconciliation serveur, le relais des entités à autorité client, les transports WebSocket et WebRTC, la compression des instantanés, le chiffrement DTLS, et la traversée de NAT (STUN/TURN/ICE).

20.3 Le socle : la plateforme

Kernel/System/NKNetwork/src/NKNetwork/Transport/NkSocket.h:746-753

```
1 static NkNetResult PlatformInit() noexcept;
2 static void PlatformShutdown() noexcept;
```

Sous Windows, PlatformInit appelle WSStartup. Sous les autres systèmes, il ne fait presque rien — mais il faut l'appeler quand même, sinon votre code ne sera pas portable. La feuille de route est catégorique : « PlatformInit() requis avant usage » ([ROADMAP.md:52-54](#)).

Applications/Pong/src/Pong/Net/NetworkSession.cpp:28-40

```
1 void NetworkSession::PlatformInit() {
2     const auto r = net::NkSocket::PlatformInit();
3     if (r != net::NkNetResult::NK_NET_OK) {
4         logger.Error("[Net] PlatformInit failed: {0}", (int)r);
5     } else {
6         logger.Info("[Net] PlatformInit OK");
7     }
8 }
9
10 void NetworkSession::PlatformShutdown() {
11     net::NkSocket::PlatformShutdown();
12     logger.Info("[Net] PlatformShutdown");
13 }
```

20.3.1 Les codes de retour

Kernel/System/NKNetwork/src/NKNetwork/Core/NkNetDefines.h:268-291

```
1 enum class NkNetResult : uint8 {
2     NK_NET_OK = 0, NK_NET_INVALID_ARG, NK_NET_NOT_CONNECTED, NK_NET_ALREADY_CONNECTED,
3     NK_NET_CONNECTION_REFUSED, NK_NET_TIMEOUT, NK_NET_PACKET_TOO_LARGE, NK_NET_SOCKET_ERROR,
4     NK_NET_BUFFER_FULL, NK_NET_NOT_INITIALIZED, NK_NET_PLATFORM_UNSUPPORTED,
5     NK_NET_AUTH_FAILED, NK_NET_BANNED, NK_NET_UNKNOWN
6 };
7 inline const char* NkNetResultStr(NkNetResult r) noexcept; // -> texte lisible
```

NkNetResultStr existe : utilisez-le dans vos journaux plutôt que d'imprimer un entier. Un « erreur réseau 6 » ne vous dira rien dans six mois.

20.3.2 Les adresses

Kernel/System/NKNetwork/src/NKNetwork/Transport/NkSocket.h:172-353

```
1 class NkAddress {
2     NkAddress(const char* ip, uint16 port);
3     static NkAddress Loopback(uint16 port, Family f = Family::NK_IP_V4) noexcept;
4     static NkAddress Any(uint16 port, Family f = Family::NK_IP_V4) noexcept;
5     static NkAddress Broadcast(uint16 port) noexcept;
6     bool IsValid() const noexcept;
7     NkString ToString() const noexcept;
8 };
```

Trois fabriques nommées, à comprendre une fois pour toutes :

- Loopback(port) — 127.0.0.1, la machine locale. Pour tester;
- Any(port) — « toutes mes interfaces réseau ». C'est ce à quoi un serveur se lie. Any(0) laisse même le système choisir le port;
- Broadcast(port) — 255.255.255.255, tout le réseau local.

Toujours valider une adresse saisie

NkAddress("192.168.1.20", 7777) *analyse* la chaîne. Si l'utilisateur a tapé n'importe quoi, l'adresse est invalide et Connect échouera de façon obscure. Pong teste systématiquement avant ([Applications/Pong/src/Pong/Net/NetworkSession.cpp:122-129](#)) et affiche un message utile.

20.4 NkConnectionManager : la classe du chapitre

C'est par elle que tout passe. Elle fait tenir, derrière une API courte, un protocole UDP fiable complet : poignée de main, accusés de réception, retransmissions, canaux, déconnexion gracieuse.

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkConnection.h:733-790 (abrégé)

```
1 class NkConnectionManager {
2     NkConnectionManager() noexcept = default;
3     ~NkConnectionManager() noexcept; // (Shutdown auto)
4     NkConnectionManager(const NkConnectionManager&) = delete; // NON COPIABLE
5
6     NkNetResult StartServer(uint16 port, uint32 maxClients = 64) noexcept;
7     NkNetResult Connect(const NkAddress& serverAddr, uint16 localPort = 0) noexcept;
8     void Shutdown() noexcept;
9
10    NkNetResult SendTo(NkPeerId peer, const uint8* data, uint32 size, NkNetChannel ch)
11    noexcept;
12    NkNetResult Broadcast(const uint8* data, uint32 size, NkNetChannel ch) noexcept;
13    void DrainAll(NkVector<NkReceiveMsg>& out) noexcept;
14    void Disconnect(NkPeerId peer, const char* reason = nullptr) noexcept;
15    void DisconnectAll(const char* reason = nullptr) noexcept;
16    bool IsServer() const noexcept; bool IsRunning() const noexcept;
17    uint32 ConnectedPeerCount() const noexcept;
18    bool GetConnectionStats(NkPeerId peer, NkConnectionStats& outStats) const noexcept;
19
20    /* NkFunction */ onPeerConnected; ///< (NkPeerId)
21    /* NkFunction */ onPeerDisconnected; ///< (NkPeerId, const char* reason)
```

```

21     uint32 maxConnections = kNkMaxConnections;
22 };

```

20.4.1 Trois invariants de fil d'exécution

Tout le reste du chapitre découle de ces trois faits.

1. StartServer et Connect lancent un fil.

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkConnection.h:762-788

```

1  @note Crée un socket UDP et le lie à l'adresse Any(port).
2  @note Démarre le thread réseau interne pour polling automatique.
3  [...]
4  @note Envoie une déconnexion gracieuse à tous les pairs connectés.
5  @note Attend la fin du thread réseau avant retour (join).

```

Vous n'avez donc pas à interroger le réseau vous-même : un fil le fait en permanence, et Shutdown() l'attend proprement.

2. Les *callbacks* s'exécutent sur ce fil. onPeerConnected et onPeerDisconnected ne sont **pas** appelés depuis votre boucle principale. N'y faites rien de lourd : pas de création d'objets de jeu, pas de manipulation de l'interface, pas d'allocation. Pong s'y limite à un incrément atomique (Applications/Pong/src/Pong/Net/NetworkSession.cpp:60-63).

3. La poignée de main est asynchrone. Connect() renvoie NK_NET_OK *avant* que la connexion soit établie. Ce code de retour signifie « la demande est partie », pas « je suis connecté ». Le seul test valable est de surveiller ConnectedPeerCount() dans une boucle bornée.

20.4.2 Le bout-en-bout, tel qu'il est testé

Sandbox/System/NKNetwork/src/main.cpp:313-334

```

1  void TestLoopback() {
2      NK_CHECK(NkSocket::PlatformInit() == NkNetResult::NK_NET_OK);
3
4      constexpr uint16 kPort = 48213;
5
6      NkConnectionManager server;
7      NK_CHECK(server.StartServer(kPort, 8) == NkNetResult::NK_NET_OK);
8
9      NkConnectionManager client;
10     NK_CHECK(client.Connect(NkAddress("127.0.0.1", kPort)) == NkNetResult::NK_NET_OK);
11
12     // Attente de l'établissement de la connexion (handshake 3-way).
13     bool connected = false;
14     for (int i = 0; i < 500; ++i) {
15         if (server.ConnectedPeerCount() >= 1 && client.ConnectedPeerCount() >= 1) {
16             connected = true;
17             break;
18         }
19         NkChrono::SleepMilliseconds(static_cast<int64>(10));
20     }
21     NK_CHECK(connected);

```


Cinq cents itérations de dix millisecondes : cinq secondes au maximum. La boucle est **bornée** — un test qui attend indéfiniment n'est pas un test. Et on vérifie les *deux* côtés : le serveur voit un pair, le client aussi.

20.4.3 On ne reçoit pas, on draine

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkConnection.h:229-235

```
1 struct NkReceiveMsg {
2     uint8 data[kNkMaxPayloadSize] = {};    ///< buffer INLINE (1380 octets), pas un
        pointeur
3     uint32 size = 0;
4     NkPeerId from;
5     NkNetChannel channel = NkNetChannel::NK_NET_CHANNEL_UNRELIABLE;
6     NkTimestampMs receivedAt = 0;
7 };
```

Le fil réseau accumule les messages; votre boucle les récupère d'un coup :

Sandbox/System/NKNetwork/src/main.cpp:397-410

```
1     bool applied = false;
2     NkVector<NkReceiveMsg> msgs;
3     for (int i = 0; i < 500 && !applied; ++i) {
4         serverWorld.Update(0.02f);
5         msgs.Clear();
6         client.DrainAll(msgs);
7         for (usize m = 0; m < msgs.Size(); ++m) {
8             (void)clientWorld.HandleMessage(msgs[m]);
9         }
10        applied = spawned && clientPawn.x == 12.5f && clientPawn.y == -7.25f;
11        NkChrono::SleepMilliseconds(static_cast<int64>(10));
12    }
```

msgs.Clear() avant chaque DrainAll

DrainAll *ajoute* à votre conteneur. Sans le Clear(), vous retraiterez les anciens messages à chaque tour, indéfiniment.

Et il y a une seconde raison de vider ce conteneur : NkReceiveMsg contient un tampon **en ligne** de 1 380 octets, pas un pointeur. Un NkVector<NkReceiveMsg> de cent éléments pèse 138 Ko. Ne le copiez jamais inutilement, et videz-le entre deux images.

Corollaire de ce tampon fixe : **un message ne peut pas dépasser 1380 octets**. Au-delà, SendTo renvoie NK_NET_PACKET_TOO_LARGE.

Kernel/System/NKNetwork/src/NKNetwork/Core/NkNetDefines.h:145-175

```
1 static constexpr uint32 kNkMaxConnections    = 256u;
2 static constexpr uint32 kNkMaxPacketSize     = 1400u;    ///< taille MTU-safe
3 static constexpr uint32 kNkMaxPayloadSize    = 1380u;
4 static constexpr uint32 kNkMaxChannels       = 8u;
5 static constexpr uint32 kNkMaxRetransmits    = 5u;
6 static constexpr uint32 kNkMaxFragments      = 16u;
```

Ces 1 400 octets ne sont pas arbitraires : c'est la taille sous laquelle un datagramme traverse Internet sans être fragmenté par les routeurs. La fragmentation IP est la première cause de pertes

silencieuses.

Tester `msg.data` avant de le déréférencer

Le dépôt contient deux copies presque identiques du même fichier, et leur **seule** différence est instructive : [Applications/Pong/src/Pong/Net/NetworkSession.cpp:253](#) déréférence `msg.data[0]` sans garde; la copie corrigée ajoute `&& msg.data != nullptr`. Enseignons la version corrigée.

20.5 Choisir son canal

C'est la décision de conception la plus importante du module, et le fichier de définitions la documente mieux que n'importe quel manuel :

Kernel/System/NKNetwork/src/NKNetwork/Core/NkNetDefines.h:505-525

```
1 enum class NkNetChannel : uint8 {
2     NK_NET_CHANNEL_UNRELIABLE = 0,          ///< UDP pur – positions, inputs, animations
3     NK_NET_CHANNEL_RELIABLE_ORDERED,        ///< TCP-like – chat, événements gameplay
4     NK_NET_CHANNEL_RELIABLE_UNORDERED,      ///< Livraison garantie, ordre libre
5     NK_NET_CHANNEL_SEQUENCED,               ///< Seul le plus récent est livré
6     NK_NET_CHANNEL_SYSTEM                   ///< RÉSERVÉ AU MOTEUR – ne pas utiliser
7 };
```

Le raisonnement à tenir est le suivant : **une donnée qui sera de toute façon remplacée dans 16 millisecondes ne mérite pas d'être retransmise**. La position d'un joueur perdue en route est sans importance — la suivante arrive. En revanche, un message de chat perdu est perdu pour toujours.

Vous envoyez	Canal
Position, entrées, animation	UNRELIABLE
Chat, événements de jeu, commandes critiques	RELIABLE_ORDERED
Fichiers, gros blocs où l'ordre importe peu	RELIABLE_UNORDERED
États continus (points de vie, énergie)	SEQUENCED
Rien du tout	SYSTEM

Le canal SYSTEM est réservé au moteur — «usage interne uniquement, comportement non défini si utilisé». Ce n'est pas une suggestion.

Pong choisit son canal par un simple booléen, ce qui est le bon niveau d'abstraction pour une application :

Applications/Pong/src/Pong/Net/NetworkSession.cpp:234-243

```
1 bool NetworkSession::Broadcast(const uint8 *data, uint32 size, uint8 reliable) {
2     if (mConnMgr == nullptr)
3         return false;
4     if (mState.load() != NetworkState::Connected)
5         return false;
6     const auto ch = reliable ? net::NkNetChannel::NK_NET_CHANNEL_RELIABLE_ORDERED
7                             : net::NkNetChannel::NK_NET_CHANNEL_UNRELIABLE;
8     const auto r = mConnMgr->Broadcast(data, size, ch);
9     return r == net::NkNetResult::NK_NET_OK;
10 }
```

Dans le jeu, l'entrée du joueur et l'instantané d'état partent en non fiable (`GameplayScene.cpp:705` et `:849`), tandis que la pause, le but marqué et la demande de revanche partent en fiable (`:227`, `:293`, `:309`).

20.6 NkBitStream : ne pas gaspiller le réseau

Avec 1380 octets par message, chaque bit compte. C'est là qu'intervient la partie la plus élégante du module.

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkBitStream.h:165-352 (abrégé)

```
1 void WriteBool(bool) ; WriteU8/U16/U32/U64 ; WriteI8/I16/I32 ; WriteF32
2 void WriteF32Q(float32 v, float32 minV, float32 maxV, float32 prec) noexcept; // quantifie
3 void WriteInt(int32 v, int32 minV, int32 maxV) noexcept; // borne
4 void WriteVec3f(const NkVec3f&) ; WriteVec3fQ(v, minV, maxV, prec) ; WriteQuatf(const
    NkQuatf&);
5 void WriteString(const char* s, uint32 maxLen = 256) noexcept;
6 void WriteBytes(const uint8* data, uint32 size) noexcept;
7 void WriteBits(uint32 v, uint32 numBits) noexcept;
8 void AlignToByte() noexcept;
9 bool IsOverflowed() const noexcept;
10 uint32 BytesWritten() const;
```

Les méthodes de lecture sont exactement symétriques (`ReadBool`, `ReadU8`, `ReadF32Q`, `ReadInt...`).

20.6.1 L'argument : la quantification

Un `float` coûte 32 bits. Mais si vous savez qu'un score est entre 0 et 999, il tient sur 10 bits. Et si une position est entre -100 et $+100$ au centimètre près, elle tient sur environ 15 bits au lieu de 32.

Exemple écrit pour le cours (API : `NkBitStream.h:237, 252`)

```
1 #include "NKNetwork/Protocol/NkBitStream.h"
2
3 uint8 buf[256];
4 net::NkBitWriter w(buf, sizeof(buf));
5 w.WriteU8(kMonTypeDeMessage);
6 w.WriteInt(scoreJoueur, 0, 999); // 10 bits au lieu de 32
7 w.WriteF32Q(posX, -100.f, 100.f, 0.01f); // quantifie, ~15 bits
8 w.AlignToByte();
9 if (!w.IsOverflowed())
10     client.SendTo(peer, buf, (uint32)w.BytesWritten(),
11         net::NkNetChannel::NK_NET_CHANNEL_RELIABLE_ORDERED);
```

Écriture et lecture doivent être des miroirs exacts

Exemple écrit pour le cours (API : NkBitStream.h:455-635)

```
1 net::NkBitReader r(m.data, m.size);
2 const uint8 type = r.ReadU8();
3 const int32 score = r.ReadInt(0, 999); // MEMES bornes
4 const float32 x = r.ReadF32Q(-100.f, 100.f, 0.01f); // MEMES bornes, MEME
    precision
5 if (r.IsOverflowed()) { /* message tronque : jeter */ }
```

Un flux de bits n'est pas auto-descriptif : il n'y a ni noms de champs, ni balises. Si le lecteur écrit `ReadInt(0, 9999)` là où l'écrivain a écrit `WriteInt(0, 999)`, il ne lira pas le bon nombre de bits, et **tout ce qui suit sera décalé**. Les symptômes sont absurdes : une position correcte suivie d'un identifiant aberrant.

Deux disciplines s'imposent. D'abord, écrire les deux fonctions *côte à côte*, dans le même fichier, immédiatement l'une après l'autre. Ensuite, tester systématiquement `IsOverflowed()` des deux côtés : c'est la seule détection d'erreur dont vous disposez.

Le *round-trip* de test est le meilleur exercice du module, parce qu'il ne demande aucun réseau :

Sandbox/System/NKNetwork/src/main.cpp:47-89 (abrégé)

```
1 void TestBitStream() {
2     uint8 buffer[256];
3     NkBitWriter writer(buffer, sizeof(buffer));
4
5     writer.WriteBool(true);
6     writer.WriteU8(0xAB);
7     writer.WriteU16(0x1234);
8     writer.WriteU32(0xDEADBEEF);
9     writer.WriteU64(0x0123456789ABCDEFu11);
10    writer.WriteI32(-42);
11    writer.WriteF32(3.5f);
12    writer.WriteBits(5u, 3);
13    writer.AlignToByte();
14    const uint8 raw[4] = {1, 2, 3, 4};
15    writer.WriteBytes(raw, 4);
16    NK_CHECK(!writer.IsOverflowed());
17
18    NkBitReader reader(buffer, writer.BytesWritten());
19    NK_CHECK(reader.ReadBool() == true);
20    NK_CHECK(reader.ReadU8() == 0xAB);
21    /* ... symetrique ... */
22    NK_CHECK(!reader.IsOverflowed());
23
24    // Overflow en ecriture comme en lecture.
25    uint8 tiny[2];
26    NkBitWriter tinyWriter(tiny, sizeof(tiny));
27    tinyWriter.WriteU32(0xFFFFFFFFFu);
28    NK_CHECK(tinyWriter.IsOverflowed());
29 }
```

20.7 NkNetWorld : répliquer des entités

Envoyer des octets, c'est bien. Synchroniser l'état d'un monde, c'est le vrai sujet. NkNetWorld fournit la mécanique.

Kernel/System/NKNetwork/src/NKNetwork/Replication/NkNetWorld.h:128-515 (abrégé)

```
1 struct NkNetEntity {
2     NkNetId netId; uint32 prefabId; void* user;
3     WriteStateFn writeState; ///< serialise l'etat - appele cote AUTORITE
4     ReadStateFn readState;   ///< applique un etat reçu - cote replique
5 };
6
7 class NkNetWorld {
8     struct Config { float32 tickRate; uint32 keyframeInterval; /* ... */ };
9     void Init(NkConnectionManager* mgr, bool isServer, Config cfg = {}) noexcept;
10    NkNetId AllocateNetId() noexcept;
11    bool RegisterEntity(const NkNetEntity& desc) noexcept;
12    bool UnregisterEntity(NkNetId netId, bool notifyPeers = true) noexcept;
13    NkNetEntity* FindEntity(NkNetId netId) noexcept;
14    uint32 EntityCount() const noexcept;
15    void Update(float32 dt) noexcept;
16    bool HandleMessage(const NkReceiveMsg& msg) noexcept;
17    uint32 CurrentTick() const noexcept;
18    void DrainInputs(NkPeerId peer, NkVector<NkNetInput>& out) noexcept;
19    SpawnCb onEntitySpawn;   ///< (NkNetId, uint32 prefabId, NkPeerId owner)
20    DespawnCb onEntityDespawn; ///< (NkNetId)
21 };
```

Le modèle tient en trois phrases :

Kernel/System/NKNetwork/src/NKNetwork/Replication/NkNetWorld.h:20-22

```
1 HandleMessage(). Entité inconnue → callback onEntitySpawn (l'application
2 crée son objet et l'enregistre) ; état reçu → readState ; destruction →
3 onEntityDespawn. Les inputs locaux remontent via SendInput().
```

Côté serveur — l'autorité — on décrit comment sérialiser :

Sandbox/System/NKNetwork/src/main.cpp:349-361

```
1 NkNetEntity desc;
2 desc.netId = serverWorld.AllocateNetId();
3 desc.prefabId = 99;
4 desc.user = &serverPawn;
5 desc.writeState = [](void *user, NkBitWriter &w) {
6     TestPawn *p = static_cast<TestPawn *>(user);
7     w.WriteF32(p->x);
8     w.WriteF32(p->y);
9 };
10 NK_CHECK(serverWorld.RegisterEntity(desc));
```

Côté client — la réplique — on réagit à l'apparition d'une entité inconnue :

Sandbox/System/NKNetwork/src/main.cpp:171-203 (abrégé)

```
1 world.onEntitySpawn = [&](NkNetId netId, uint32 prefabId, NkPeerId /*owner*/) {
2     spawnedPrefab = prefabId;
```

```

3      NkNetEntity desc;
4      desc.netId = netId;
5      desc.prefabId = prefabId;
6      desc.user = &pawn;
7      desc.readState = [](void *user, NkBitReader &r) {
8          TestPawn *p = static_cast<TestPawn *>(user);
9          p->x = r.ReadF32();
10         p->y = r.ReadF32();
11     };
12     world.RegisterEntity(desc);
13 };

```

Le prefabId est votre code de type : « ceci est un joueur », « ceci est un projectile ». C'est à vous de décider ce que chaque numéro signifie et de créer l'objet correspondant.

Les deux règles des fonctions d'état

writeState doit être déterministe : « deux appels sur le même état doivent produire les mêmes octets » ([Replication/NkNetWorld.h:124-126](#)). Sinon le calcul de différences envoie des mises à jour perpétuelles pour un état qui n'a pas bougé.

Écrivez des lambdas, pas des pointeurs de fonction nus. Le pont ECS du moteur le documente ([Engine/Noge/src/Noge/ECS/Replication/NkNetWorld.cpp:132-138](#)) : un pointeur de fonction brut résout mal la surcharge du type de rappel.

20.8 Le socket brut : la découverte sur le réseau local

Il reste un cas où l'on descend sous NkConnectionManager : annoncer un serveur pour que les autres machines le trouvent sans qu'on ait à taper une adresse IP.

Kernel/System/NKNetwork/src/NKNetwork/Transport/NkSocket.h:504-715 (abrégé)

```

1 class NkSocket {
2     enum class Type : uint8 { NK_UDP, NK_TCP /* ... */ };
3     NkNetResult Create(const NkAddress& localAddr, Type type = Type::NK_UDP) noexcept;
4     void Close() noexcept;
5     NkNetResult SetNonBlocking(bool v) noexcept;
6     NkNetResult SetBroadcast(bool v) noexcept;
7     NkNetResult SendTo(const void* data, uint32 size, const NkAddress& to) noexcept;
8     NkNetResult RecvFrom(void* buf, uint32 bufSize, uint32& outSize, NkAddress& outFrom)
9         noexcept;
10    bool IsValid() const noexcept;
11 };

```

Le socket qui émet la balise :

Applications/Pong/src/Pong/Net/NetworkDiscovery.cpp:44-63 (abrégé)

```

1     mBeaconSock = new net::NkSocket();
2     // Bind sur une socket UDP IPv4 quelconque (port 0 = Laisse l'OS choisir),
3     // on n'a besoin que d'envoyer. SetBroadcast active SO_BROADCAST
4     // nécessaire pour pouvoir SendTo vers 255.255.255.255.
5     const auto rc = mBeaconSock->Create(net::NkAddress::Any(0),
6     net::NkSocket::Type::NK_UDP);
7     if (rc != net::NkNetResult::NK_NET_OK) { /* ... */ return false; }
8     (void)mBeaconSock->SetNonBlocking(true);
9     const auto rb = mBeaconSock->SetBroadcast(true);

```

```
9         if (rb != net::NkNetResult::NK_NET_OK) { /* ... */ return false; }
```

et celui qui écoute, lié au port de balise sur toutes les interfaces :

Applications/Pong/src/Pong/Net/NetworkDiscovery.cpp:84-96 (abrégé)

```
1         mScanSock = new net::NkSocket();
2         // Bind sur kBeaconPort, INADDR_ANY : on reçoit les broadcasts
3         // émises vers 255.255.255.255:kBeaconPort par d'autres machines du LAN.
4         const auto rc = mScanSock->Create(net::NkAddress::Any(kBeaconPort),
net::NkSocket::Type::NK_UDP);
5         /* ... */
6         (void)mScanSock->SetNonBlocking(true);
7         (void)mScanSock->SetBroadcast(true);
```

Sans SetBroadcast(true), l'émission échoue

Le système d'exploitation refuse par défaut d'envoyer vers une adresse de diffusion — c'est une protection. Il faut l'autoriser explicitement, des deux côtés. C'est l'oubli numéro un de la découverte réseau.

Le *tick*, avec ses deux garde-fous :

Applications/Pong/src/Pong/Net/NetworkDiscovery.cpp:115-160 (abrégé)

```
1         if (mBeaconSock != nullptr) {
2             mBeaconTimer -= dt;
3             if (mBeaconTimer <= 0.0f) {
4                 const auto dst = net::NkAddress::Broadcast(kBeaconPort);
5                 const auto rs = mBeaconSock->SendTo(&mBeaconPkt, sizeof(mBeaconPkt), dst);
6                 mBeaconTimer = kBeaconIntervalSec;
7             }
8         }
9         if (mScanSock != nullptr) {
10            // Boucle : on lit tant qu'il y a des datagrammes pendants.
11            // En non-bloquant, RecvFrom OK avec outSize=0 = rien à lire.
12            constexpr int kMaxPerTick = 32;
13            for (int i = 0; i < kMaxPerTick; ++i) {
14                uint8 buf[256];
15                uint32 received = 0;
16                net::NkAddress from;
17                const auto rr = mScanSock->RecvFrom(buf, sizeof(buf), received, from);
18                if (rr != net::NkNetResult::NK_NET_OK) break;
19                if (received == 0) break;
20                /* filtre magic + version, puis UpdateHost(pkt, from) */
21            }
22        }
```

Deux points à retenir :

1. **en mode non bloquant, un RecvFrom qui réussit avec outSize == 0 signifie «rien à lire», pas «erreur».** C'est la condition d'arrêt normale de la boucle;
2. **la boucle est bornée à 32 datagrammes par image.** Sur un réseau chargé — ou face à un émetteur mal réglé — une boucle non bornée monopoliserait la frame.

Notez enfin le filtrage «magic + version» : un port UDP reçoit tout ce que n'importe qui envoie. Toujours vérifier une signature avant d'interpréter.

20.9 HTTP, et pourquoi HTTPS échoue

Kernel/System/NKNetwork/src/NKNetwork/HTTP/NkHTTPClient.h:383-717 (abrégé)

```
1 struct NkHTTPResponse {
2     uint32 statusCode = 0;
3     NkString body;
4     NkString error;
5     bool IsOK() const noexcept;
6 };
7 class NkHTTPClient {
8     NkHTTPResponse Get(const char* url) noexcept;
9     NkHTTPResponse Post(const char* url, const char* json) noexcept;
10 };
```

HTTPS est désactivé par défaut

Le support TLS est optionnel, activé par une variable d'environnement au moment de la construction :

Kernel/System/NKNetwork/NKNetwork.jenga:21-25

```
1 # TLS/HTTPS opt-in : NK_ENABLE_TLS=1 (ou mbedtls) active le backend mbed-TLS
2 # (bibliotheque NKmbedTLS, C pur, cross-compilee pour toutes les plateformes y
3 # compris iOS). Desactive par défaut -> HTTP simple uniquement, aucune dependance.
4 _TLS_OPT = os.getenv("NK_ENABLE_TLS", "").strip().lower()
5 _WANT_MBEDTLS = _TLS_OPT in ("1", "true", "on", "yes", "mbedtls")
```

Sans cette variable, toute URL en https:// échoue — proprement, au moins :

Kernel/System/NKNetwork/src/NKNetwork/HTTP/NkHTTPClient.cpp:1050

```
1         response.error = "HTTPS not compiled in (rebuild with NK_ENABLE_TLS=1)";
```

Et si vous activez TLS, votre application doit lier NKmbedTLS en plus, comme le fait le sandbox. Pour ce cours, tenons-nous-en à http://.

Le mode de test manuel du sandbox montre l'usage complet :

Sandbox/System/NKNetwork/src/main.cpp:441-451 (abrégé)

```
1 int main(int argc, char **argv) {
2     // Mode HTTPS manuel : SandboxNKNetwork --https <url>
3     // Nécessite un build avec NK_ENABLE_TLS=1 (backend mbedTLS), sinon le
4     // client renvoie proprement "HTTPS not compiled in".
5     if (argc >= 3 && NkString(argv[1]) == NkString("--https")) {
6         NkHTTPClient http;
7         const NkHTTPResponse resp = http.Get(argv[2]);
8         /* ... affichage de resp.statusCode, resp.body.Length(), resp.error.CStr() ... */
9         return (resp.statusCode >= 200 && resp.statusCode < 400) ? 0 : 1;
10    }
```


20.10 Comment structurer une application en réseau

Vous avez maintenant toutes les briques. Reste la question d'architecture, et Pong y répond de la façon qu'il faut copier.

1. **Une classe de session** possède le `NkConnectionManager` et n'expose que des verbes simples : `StartHost`, `StartJoin`, `Broadcast`, `DrainReceived`, `Shutdown`, plus un état atomique (`Idle` / `Hosting` / `Connected`) (`Applications/Pong/src/Pong/Net/NetworkSession.h`);
2. un `Tick(dt)` appelé une fois par image draine le fil réseau et met à jour cet état :

`Applications/Pong/src/Pong/Game/PongApp.cpp:162-169`

```
1 // Tick reseau (drain interne du thread reseau). Aucun cout si
2 // la session est Idle.
3 mNetwork.Tick(dt);
4 // Tick decouverte LAN : emet beacon (host) + drain scan (client).
5 mDiscovery.Tick(dt);
```

3. **les écrans d'interface lisent l'état, jamais le manager.** Aucune scène de Pong ne touche au `NkConnectionManager` : elles appellent `DrainReceived` et consultent `NetworkState`.

Le routage des messages se fait dans la session, par type, avec la garde qui manquait :

`Applications/Pong/src/Pong/Net/NetworkSession.cpp:245-260 (abrégé)`

```
1 void NetworkSession::DrainInternal() {
2     if (mConnMgr == nullptr)
3         return;
4     // Drain brut depuis NkConnectionManager.
5     NkVector<net::NkReceiveMsg> all;
6     mConnMgr->DrainAll(all);
7     for (uint32 i = 0; i < all.Size(); ++i) {
8         const auto &msg = all[i];
9         if (msg.size >= sizeof(netproto::PktHello) && msg.data[0] ==
netproto::kMsgHello) {
10             netproto::PktHello pkt;
11             /* ... recopie des sizeof(pkt) octets de msg.data dans pkt ... */
12             continue;
13         }
14         // Message non-interne : reserve pour le caller via DrainReceived.
15         mPendingForUser.PushBack(msg);
16     }
17 }
```

Notez le test `msg.size >= sizeof(...)` avant de lire le contenu : un pair malveillant, ou simplement une version différente de votre programme, peut envoyer n'importe quoi.

Le découpage à retenir

Le réseau vit sur son fil; la session fait tampon; l'interface ne voit qu'un état et une file de messages. Aucune scène, aucun widget ne doit connaître `NkConnectionManager`. C'est ce qui rend le jeu testable sans réseau et le réseau modifiable sans toucher au jeu.

20.11 L'ordre de fermeture

Sandbox/System/NKNetwork/src/main.cpp:430-433

```
1      client.Shutdown();
2      server.Shutdown();
3      NkSocket::PlatformShutdown();
```

et, avec une couche de réplication au-dessus :

Applications/NkNetWorldDemo/src/main.cpp:205-211

```
1      serverNet.Shutdown();
2      clientNet.Shutdown();
3      client.Shutdown();
4      server.Shutdown();
5      net::NkSocket::PlatformShutdown();
```

La règle est celle des poupées russes : **on éteint du plus haut vers le plus bas**. Les systèmes de réplication d'abord, puis les gestionnaires de connexion — qui joignent leur fil réseau —, puis la plateforme. Inverser reviendrait à fermer Winsock pendant qu'un fil s'en sert encore.

N'incluez pas l'en-tête parapluie dans une application ECS

Voici le piège le plus coûteux du module, documenté deux fois dans le dépôt :

Applications/NkNetWorldDemo/src/main.cpp:20-37

```
1  // PAS L'umbrella NKNetwork/NKNetwork.h : il injecte des alias de commodité
2  // dans `nkentseu` (dont `using NkNetSystem = net::NkNetSystem;` et
3  // `using NkNetEntity = net::NkNetEntity;`) qui entrent en collision directe
4  // avec l'adaptateur ECS de Noge (`nkentseu::NkNetSystem`, composant
5  // `nkentseu::ecs::NkNetEntity`). On inclut donc uniquement les en-têtes
6  // précis nécessaires.
7  #include "Noge/ECS/Replication/NkNetWorld.h"
8  #include "NKECS/World/NkWorld.h"
9  #include "NKNetwork/Transport/NkSocket.h"
10 #include "NKTime/NkChrono.h"
11 #include "NKLogger/NkLog.h"
12
13 using namespace nkentseu;
14 using namespace nkentseu::ecs;
15 // PAS de `using namespace nkentseu::net;` : net::NkNetEntity (descripteur de
16 // réplication NKNetwork) et ecs::NkNetEntity (composant ECS) partagent le
17 // même nom -- même piège que Noge/ECS/Replication/NkNetWorld.cpp.
```

C'est l'exception à la règle générale du moteur — « incluez toujours l'en-tête parapluie » du chapitre 1. Ici, l'en-tête parapluie injecte des alias dans le *namespace* global du moteur, et deux types différents portent le même nom. Incluez les en-têtes précis, et n'ouvrez pas le *namespace* `nkentseu::net` en grand.

Ce chapitre en bref

Le réseau standard offre deux façons de parler : l'une garantit l'ordre et l'arrivée, l'autre ne garantit rien mais n'attend jamais. Nkentseu ne choisit pas à votre place — il expose des **canaux**, et vous décidez par message : une position de joueur périmée ne sert à rien,

un message de chat perdu, si. `NkBitStream` descend au *bit* plutôt qu'à l'octet, parce que sur un réseau ce qu'on n'envoie pas est ce qui coûte le moins cher. `NkNetWorld` réplique des entités par-dessus. Et l'ordre de fermeture n'est pas décoratif : le fermer à l'envers laisse des messages en vol sans destinataire.

20.12 Exercices

1 — Le flux de bits, sans réseau

Écrivez un programme console qui sérialise une petite structure de jeu — un identifiant, un score entre 0 et 999, deux coordonnées entre -100 et $+100$ au centimètre, un booléen — puis la relit et vérifie l'égalité champ par champ. Affichez `BytesWritten()`. Comparez avec ce que coûterait la même structure en écriture brute (`WriteU32`, `WriteF32`). Puis introduisez volontairement une erreur : lisez le score avec les bornes (0, 9999). Observez que ce n'est pas seulement le score qui devient faux, mais tout ce qui suit.

2 — Serveur et client dans le même programme

Reproduisez `TestLoopback` : dans un seul main, démarrez un serveur sur 127.0.0.1, connectez un client, attendez la poignée de main dans une boucle bornée, échangez trois messages sur trois canaux différents, puis fermez dans le bon ordre. Ajoutez de la journalisation dans `onPeerConnected` — et regardez sur quel fil elle s'exécute. Puis, volontairement, supprimez la boucle d'attente et envoyez immédiatement après `Connect()`. Que se passe-t-il, et pourquoi ?

3 — Un chat dans votre interface

Ajoutez à votre application `NKGui` deux boutons « Héberger » et « Rejoindre », un `InputText` pour l'adresse, un second pour le message, et une liste des messages reçus. Respectez l'architecture de `Pong` : une petite classe de session qui possède le manager, un `Tick(dt)` appelé une fois par image, et une interface qui ne voit qu'un état et une file. Utilisez `RELIABLE_ORDERED` — un message de chat perdu ne se rattrape pas. Testez avec deux instances de votre programme sur la même machine, puis, si vous le pouvez, sur deux machines du même réseau.

4 — Mesurer ce que coûte la fiabilité

Reprenez l'exercice 2 et envoyez mille messages, une fois en `UNRELIABLE`, une fois en `RELIABLE_ORDERED`, en comptant ceux qui arrivent et en chronométrant. Sur une boucle locale, vous ne perdrez probablement rien : les deux chiffres seront identiques. C'est un résultat en soi — **tester le réseau sur 127.0.0.1 ne teste pas le réseau**. Consultez ensuite `GetConnectionStats` et voyez ce que le module sait vous dire du temps d'aller-retour et des pertes.

Questions de cours

1. Quelles sont les deux garanties qu'un canal peut offrir, et laquelle coûte de l'attente ?
2. Donnez un message qu'il vaut mieux perdre que retarder, et l'inverse.
3. Qu'apporte `NkBitStream` par rapport à l'écriture octet par octet ?

4. À quoi sert NkNetWorld, et sur quoi s'appuie-t-il ?
5. Pourquoi HTTPS échoue-t-il là où HTTP fonctionne ?

Exercice de logique

Un jeu en réseau « rame » chez un joueur sur quatre, toujours le même.

1. Distinguez trois grandeurs qu'on confond couramment : la latence, la perte, et le débit. Pour chacune, dites quel symptôme elle produit.
2. Le joueur affecté a une bonne connexion mesurée par un test de débit. Cela élimine-t-il quelque chose ? Justifiez.
3. Vous soupçonnez un canal fiable employé là où il ne fallait pas. Décrivez la mesure qui le confirmerait — et dites explicitement ce qu'elle donnerait si le soupçon était *faux*. Un contrôle qui ne peut pas vous contredire ne prouve rien.

NKCamera : lire une caméra

Une caméra ressemble à un fichier vidéo qu'on lirait en direct. Elle n'en est pas un, et la différence tient en une phrase : **avec un fichier, vous choisissez; avec une caméra, vous subissez**. Le format des pixels, la résolution disponible, la mise au point, le sens de l'image — tout cela est décidé par le matériel et par le système, jamais par vous.

Ce chapitre explique donc moins « comment demander » que « comment recevoir ».

21.1 La façade, en huit appels

Kernel/Runtime/NKCamera/src/NKCamera/NkCameraSystem.h

```
1 bool Init();
2 NkVector<NkCameraDevice> EnumerateDevices();
3 bool StartStreaming(const NkCameraConfig &config = {});
4 bool GetLastFrame(NkCameraFrame &outFrame);
5 void SetFrameCallback(NkFrameCallback cb);
6 void StopStreaming();
7 void Shutdown();
```

Ce qu'il faut retenir

Deux façons de recevoir les images, et elles ne se valent pas.

GetLastFrame rend la *dernière* image, celle qui est là au moment où l'on demande. C'est ce qu'il faut pour un affichage : à 60 images par seconde d'affichage et 30 de caméra, on redemande simplement deux fois la même.

SetFrameCallback vous appelle à chaque image, **depuis le fil de la caméra**. C'est ce qu'il faut pour un enregistrement ou une analyse — et ce qu'il ne faut surtout pas employer pour dessiner.

Le piège

Le rappel s'exécute sur un autre fil. Y dessiner, c'est toucher le contexte graphique depuis un fil qui ne le possède pas — selon le backend, un plantage, un écran noir, ou pire : cela marche chez vous.

C'est la même règle que le chapitre NKThreading : *le calcul se parallélise, le dessin non*. Le rappel copie ou analyse; le fil principal dessine.

21.2 Le format n'est pas un choix

C'est le point le plus important du chapitre, et il est écrit dans le code lui-même :

Kernel/Runtime/NKCamera/src/NKCamera/NkCameraTypes.h

```
1 //  TROIS CHAMPS RETIRES LE 2026-08-15 -- ne pas Les reintroduire
2 //  sans Les cabler d'abord :
```

```

3 //
4 // `outputFormat` -- 3 écrivains, 0 Lecteur. Trois applications
5 //   demandaient `NK_PIXEL_RGBA8` et recevaient du NV12 (Windows),
6 //   YUV420 (Android), BGRA8 (Cocoa/UIKit) ou YUYV (Linux) : Le
7 //   format est IMPOSE par la plateforme, jamais choisi.

```

Ce qu'il faut retenir

Un paramètre qui n'est pas honoré est pire qu'un paramètre absent : l'absence force à chercher, la présence dispense de vérifier.

Trois applications écrivaient `outputFormat = NK_PIXEL_RGBA8`, en toute bonne foi, et recevaient autre chose. Rien ne les avertissait — le champ existait, il acceptait la valeur, et il ne la lisait pas.

Le remède appliqué n'a pas été de faire fonctionner le champ (c'est impossible : le format vient du pilote) mais de le **retirer**. Le format réellement livré se lit sur `NkCameraFrame::format`, après coup.

Kernel/Runtime/NKCamera/src/NKCamera/NkCameraTypes.h

```

1 // Un champ mort dans un en-tête public est une promesse que
2 // quelqu'un finira par croire.

```

Le piège

Le même bloc en signale un autre, et celui-là n'a pas été retiré :

« `autoFocus` — lu par le SEUL backend Android. Sur Windows, aucun code de mise au point n'existe : le réglage y est ignoré en silence — invisible sur une webcam à focale fixe, ce qui explique que personne ne l'ait vu. »

Retenez la cause de l'aveuglement : **le défaut était masqué par le matériel de ceux qui auraient pu le voir**. Sur une webcam à focale fixe, un réglage de mise au point qui ne fait rien produit exactement l'image attendue.

21.3 L'image reçue, et ce qu'elle déclare

Kernel/Runtime/NKCamera/src/NKCamera/NkCameraTypes.h

```

1 struct NkCameraFrame {
2     uint32 width = 0;
3     uint32 height = 0;
4     NkPixelFormat format = NkPixelFormat::NK_PIXEL_RGBA8;
5     uint64 timestampUs = 0;
6     uint32 frameIndex = 0;
7     uint32 stride = 0;
8     NkColorRange range = NkColorRange::NK_COLOR_RANGE_UNKNOWN;
9     bool flipHorizontal = false;
10    // ...
11 };

```

Ce qu'il faut retenir

range vaut « *inconnu* » par défaut, et c'est délibéré. Le commentaire d'origine le dit : « Non renseignée = INCONNUE, et c'est voulu : le défaut ne prétend rien. » C'est l'inverse du réflexe habituel, qui est de choisir une valeur « raisonnable ». Une plage supposée à tort décale toutes les couleurs de quelques pour cent — assez pour être faux, trop peu pour être vu. Un *inconnu* déclaré force l'appelant à décider ; un défaut plausible le dispense de savoir qu'il y avait une question.

Ce qu'il faut retenir

Et la conversion choisit sa formule d'après la **PLAGE**, jamais d'après le format. C'est écrit dans le même commentaire, à propos de `ConvertToRGBA8`. Deux images au même format peuvent avoir deux plages différentes ; se fier au format donnerait la bonne réponse la plupart du temps, ce qui est la pire des situations — un défaut qui ne se reproduit que sur certaines caméras.

21.4 Le miroir : une question de géométrie, pas de goût

Kernel/Runtime/NKCamera/src/NKCamera/NkCameraTypes.h

```
1 // Faux par défaut : L'image brute d'un capteur est geometriquement VRAIE,
2 // et c'est la seule utilisable pour de L'AR, de la calibration ou de la
3 // mesure. On ne miroite que pour un affichage de SOI, ou L'habitude prime
4 // sur la geometrie -- et cela reste une demande explicite.
5 bool flipHorizontal = false;
```

Ce qu'il faut retenir

Une caméra frontale affichée telle quelle donne une image « à l'envers » pour l'utilisateur, qui s'attend à un miroir. Mais une image miroitée est *géométriquement fausse* : un marqueur de réalité augmentée s'y place du mauvais côté, une calibration y rend des paramètres symétriques faux, une mesure y est inversée. D'où le choix : **la vérité par défaut, le confort sur demande**. Et le miroir est reporté dans l'image (`NkCameraFrame::flipHorizontal`) afin que la conversion sache ce qui a été demandé — au lieu que chaque consommateur devine.

21.5 Ce qu'il faut faire, dans l'ordre

Geste	Pourquoi
1 <code>Init()</code>	le système peut refuser — vérifiez
2 <code>EnumerateDevices()</code>	il n'y a pas toujours une caméra
3 <code>StartStreaming(cfg)</code>	la résolution demandée n'est qu'un souhait
4 lire <code>frame.format</code>	c'est là qu'on apprend ce qu'on reçoit
5 convertir en <code>RGBA8</code>	le rendu ne connaît que ça
6 téléverser dans une texture	comme pour une vidéo
7 <code>StopStreaming()</code> , <code>Shutdown()</code>	dans l'ordre inverse

Le piège

L'étape 2 n'est pas facultative. Un ordinateur de bureau sans webcam, un téléphone dont l'utilisateur a refusé la permission, une caméra déjà employée par une autre application : les trois sont normaux, et les trois donnent une liste vide.

Une application qui suppose qu'il y a une caméra affiche un écran noir sans explication — et l'utilisateur croit que le programme est cassé.

Ce qu'il faut retenir

À partir de l'étape 5, vous êtes exactement dans le chapitre NKMedia : des pixels RGBA, une texture qu'on remplace, un sprite qui l'affiche. **Le rendu ne sait pas distinguer une caméra d'un fichier vidéo**, et c'est voulu.

Une seule chose diffère : la caméra n'a pas de fin. Il n'y a pas de dernière image, donc pas de « lecture terminée » — seulement un arrêt que vous décidez.

Ce chapitre en bref

Une caméra ne se configure pas, elle se *consulte*. Le format des pixels est imposé par la plateforme et se lit sur l'image reçue, jamais dans la demande — un champ qui prétendait le contraire a été retiré parce qu'il faisait croire à trois applications qu'elles recevaient du RGBA8. La plage de couleur vaut « inconnu » par défaut, et la conversion s'appuie sur elle plutôt que sur le format. Le miroir est faux par défaut, parce que l'image brute est la seule géométriquement vraie. Et à partir de la conversion, tout redevient identique à la lecture d'une vidéo.

Questions de cours

1. Pourquoi ne peut-on pas choisir le format des pixels d'une caméra ?
2. Où lit-on le format réellement reçu ?
3. Quelle différence entre `GetLastFrame` et le rappel d'image, et lequel convient à l'affichage ?
4. Pourquoi range vaut-il « inconnu » plutôt qu'une valeur plausible ?
5. Pourquoi le miroir est-il désactivé par défaut ?

Exercice pratique

Écrivez un aperçu caméra sur `NkCanvasGuiApp` : énumérez les périphériques, laissez choisir dans une liste, démarrez le flux, convertissez et affichez.

Trois exigences, et ce sont elles qui font l'exercice :

1. afficher en clair le format *réellement* reçu, pas celui demandé ;
2. afficher un message lisible quand la liste des périphériques est vide — et le vérifier en débranchant la caméra ;
3. un bouton « miroir » qui bascule `flipHorizontal`.

Exercice de démonstration

Prouvez que le format demandé n'est pas le format reçu.

Demandez explicitement `NK_PIXEL_RGBA8` dans votre configuration — si le champ n'existe

plus, notez-le et passez à la suite. Puis affichez `frame.format` et `frame.stride` pour chaque image.

Comparez ensuite `stride` à `width × 4` : s'ils diffèrent, vos pixels ne sont pas du RGBA8 contigu, et une copie ligne par ligne qui ignorerait le pas produirait l'image inclinée du chapitre NKImage.

Ce que la démonstration doit établir : que l'écart existe sur *votre* machine, avec un chiffre. « Le format peut différer » est une lecture ; « j'ai demandé RGBA8 et reçu NV12, stride 640 pour width 640 » est une mesure.

Exercice de logique

Votre aperçu caméra fonctionne sur votre ordinateur portable et donne une image verdâtre sur celui d'un collègue.

1. Énumérez au moins quatre causes possibles, en distinguant celles qui viennent du format, de la plage de couleur, et de la conversion.
2. Le fait que l'image soit *verdâtre* plutôt que noire ou bruitée élimine-t-il quelque chose ? Justifiez.
3. Proposez une mesure qui tranche — et dites ce qu'elle affiche dans chaque cas. Puis expliquez pourquoi « ça marche chez moi » est ici l'information la plus trompeuse du problème, et non la plus rassurante.

Ce que cette partie vous laisse

Vous savez charger et enregistrer une image, afficher du texte dans une police réelle, jouer un son, lire une vidéo, et parler à une machine distante. Un motif revient dans les cinq : **le module qui produit la donnée ignore celui qui la montre**. Un décodeur vidéo rend des pixels ; il ne sait pas qu'une texture existe. C'est cette ignorance mutuelle qui rend chacun remplaçable — et c'est la propriété que la dernière partie exploite dans chaque application.

PARTIE VI

Cas pratiques

Dix applications, construites de A à Z

Cette partie ne *décrit* pas des applications : elle les **construit**. Chaque chapitre est une suite d'étapes numérotées, du dossier vide au programme qui se lance — rien n'est sauté, et chaque étape produit quelque chose que vous pouvez éprouver avant de passer à la suivante.

Les dix applications *existent* : elles sont dans le dépôt, elles tournent sur sept plateformes, et les extraits cités en sont copiés. Ce que vous construisez n'est donc pas un exercice d'école, c'est le chemin réel qu'a suivi du code réel.

	Application	Étapes	Ce qu'elle ajoute
1–3	Dames, échecs, ludo	13	le squelette, et le banc avant l'écran
4	GemCrush	9	les phases : le clic n'est plus toujours reçu
5–7	Lecteurs image, son, vidéo	9	transformer une donnée pour l'œil
8–9	Éditeur de texte, puis de code	12	modifier au lieu de regarder
10	Mini terminal	10	parler à un programme qui vit

Le premier chapitre pose le squelette — treize étapes, sept fichiers, un ordre qui n'est pas négociable. Les quatre suivants ne le réécrivent pas : ils disent ce qui s'y ajoute, et pourquoi.

L'ordre va du plus fermé au plus ouvert. Un jeu de plateau a des règles finies qu'on peut prouver ; un terminal doit accueillir un programme quelconque, sur trois systèmes, sans jamais figer l'interface.

À la fin de cette partie, vous ne saurez pas seulement comment ces dix applications sont faites : vous saurez en construire d'autres du même genre. C'est la seule chose qu'elle promet, et c'est pour cela que chaque chapitre se termine par une application *à écrire*, pas par un résumé.

Construire un jeu de plateau, de A à Z

Ce chapitre ne décrit pas un jeu : il le **construit**. Vous partez d'un dossier vide et vous arrivez à un jeu de dames complet — règles, damier, adversaire, écran d'ouverture — qui se lance sur Windows, Linux, macOS, Android, iOS, HarmonyOS et dans un navigateur.

Treize étapes, dans l'ordre. Chacune produit quelque chose qui *se construit et se lance* : à aucun moment vous n'écrivez du code que vous ne pouvez pas encore éprouver. Rien n'est sauté.

Ce qu'il faut retenir

L'ordre des étapes est le sujet du chapitre.

On écrit les règles *avant* l'écran, et le banc *avant* le dessin. Ce n'est pas de la discipline : c'est ce qui rend le jeu prouvable. Écrivez le dessin d'abord, et vos règles n'auront plus jamais de verdict indépendant de ce que vous croyez voir.

Ce que nous allons construire

Étape	Fichier	Ce qu'on obtient	Lignes
1	NkDames.jenga	le projet existe	≈60
2	src/main.cpp	une fenêtre qui s'ouvre	12
3–7	Dames/NkDamesRegles.{h,cpp}	le jeu, sans image	538
8	Dames/NkDamesBanc.{h,cpp}	la preuve	272
9	Dames/NkDamesTheme.h	les couleurs	63
9–10	Dames/NkDamesEcran.{h,cpp}	le damier à l'écran	609
11–13	Dames/NkDamesJeu.{h,cpp}	le jeu jouable	477

Total : **1998 lignes**, sept fichiers de code. Les chiffres sont ceux de Applications/NkDames dans le dépôt — ce chapitre en est la reconstruction pas à pas.

22.1 Étape 1 — le dossier et le fichier de projet

L'arborescence à créer

```
1 Applications/NkDames/
2   NkDames.jenga
3   src/
4     main.cpp
5     Dames/
```

Rien d'autre : pas de dossier assets, pas de Resources. Ce jeu ne chargera **aucun fichier à l'exécution** — le damier est dessiné par géométrie et la police est embarquée dans le binaire.

Ce qu'il faut retenir

C'est ce qui permet de démarrer à l'identique sur sept plateformes. Un seul asset à copier, et il faut une chaîne de copie par plateforme, une permission de lecture sur Android, un téléchargement de plus avant la première image sur le Web. Ne créez ce dossier que le jour où vous avez vraiment un fichier à y mettre.

Voici le fichier de projet **minimal et complet** pour construire sur ordinateur :

Applications/NkDames/NkDames.jenga – version minimale, fonctionnelle

```
1  from Jenga import *
2  from jengaconfig import *
3
4  with project("NkDames"):
5      windowedapp()                # pas de console noire derriere la fenetre
6      language("C++")
7      cppdialect("C++17")
8      location(".")
9
10     files(["src/**/*.cpp"])      # tout le .cpp sous src/, recursivement
11
12     nkentseudependson(
13         ["NKEvent", "NKWindow", "NKCanvas", "NKImage", "NKFont", "NKGlad",
14          "NKLogger", "NKMath", "NKTime", "NKStream", "NKGui", "NKFileSystem",
15          "NKSerialization", "NKContainers", "NKMemory", "NKCore",
16          "NKPlatform", "NKThreading", "NKAudio"],
17         extra_includes=["src", "%{NKGlad.location}/include",
18                        "%{wks.location}/Externals"])
19
20     objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
21     targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
22
23     apppublisher("Rihen Universe")
24     appversion("0.1.0")
25     licensefile("../../LICENSE")
26
27     with filter("system:Windows"):
28         windowedapp()
29         usetoolchain(TC_WINDOWS)
30         defines(["WIN32_LEAN_AND_MEAN", "_UNICODE", "UNICODE"])
31         links(["user32", "gdi32", "opengl32", "dwmapi", "shell32",
32              "ole32", "uuid", "avrt", "winmm"])
33
34     with filter("config:Debug"):
35         defines(["_DEBUG", "DEBUG"]); optimize("Off"); symbols(True)
36     with filter("config:Release"):
37         defines(["NDEBUG"]); optimize("Speed"); symbols(False)
```

Ce qu'il faut retenir

`extra_includes=["src", ...]` est la ligne qu'on oublie, et son absence donne fatal error: 'Dames/NkDamesRegles.h' file not found — une erreur qui accuse votre inclusion alors que c'est le *projet* qui ne sait pas où chercher.

Les dix-neuf modules listés ne sont pas décoratifs : chacun manquant produit une erreur d'*édition de liens*, pas de compilation — donc à la toute fin, avec un message qui nomme un symbole et jamais un module.

Le piège

Un projet qui n'est pas enregistré dans l'espace de travail n'existe pas. Ajoutez, dans Nkentsu.jenga à la racine :

Nkentsu.jenga

```
1 with include("Applications/NkDames/NkDames.jenga"):
2     pass
```

Sans cette ligne, jenga build --target NkDames répond que la cible n'existe pas — et l'on relit son .jenga vingt fois, qui est juste.

22.1.1 Les six autres plateformes

Chaque plateforme ajoute un filter qui déclare sa chaîne d'outils et ses bibliothèques. Le tableau suivant résume ce que chacun apporte; le fichier complet est dans le dépôt.

Plateforme	Chaîne	À déclarer en plus
Linux (XLib)	clang-native	X11 Xext Xrandr GL asound
macOS	clang-native	frameworks Cocoa, Metal, AudioToolbox...
Android	android-ndk	id, SDK, ABI, orientation, OpenSLES
iOS	—	frameworks UIKit, Metal, AudioToolbox
HarmonyOS	ohos-ndk	bundlename, hilog_ndk.z
Web	emscripten	ASYNCIFY, USE_WEBGL2

Trois erreurs de ce tableau ont réellement été payées

AudioUnit n'existe pas sur iOS. C'est un framework séparé sur macOS seulement; sur iOS ses symboles vivent dans AudioToolbox. Recopier la liste macOS donne 1d: framework AudioUnit not found. *Un précédent se vérifie sur SA plateforme.*

Sans orientation déclarée, Android et HarmonyOS imposent la leur et l'image sort tournée d'un quart de tour.

ASYNCIFY n'est pas une option de confort sur le Web : sans elle, la boucle ne peut pas rendre la main et l'onglet gèle.

22.2 Étape 2 — la fenêtre qui s'ouvre

Douze lignes, et le jeu se lance déjà.

Applications/NkDames/src/main.cpp

```
1 #include "Dames/NkDamesJeu.h"
2 #include "NKWindow/NKMain.h"
3
4 NKENTSEU_DEFINE_APP_DATA(([() {
5     nkentsu::NkAppData d{};
6     d.appName = "NkDames";
7     d.appVersion = "1.0.0";
8     return d;
9 }));
```

```

10
11 int nkmain(const nkentseu::NkEntryState &state) {
12     return nkentseu::render::NkCanvasApp::Run<nkentseu::jeux::dames::NkDamesJeu>(state);
13 }

```

Ce qu'il faut retenir

Ce fichier ne contiendra jamais de logique. Il déclare le nom et la version, puis rend la main à la coquille. Tout le reste a son fichier — et c'est cette discipline qui fait qu'à l'étape 13 vous saurez encore où ajouter quelque chose.

Le piège

#include "NKWindow/NKMain.h" fournit le point d'entrée natif de chaque plateforme : WinMain, android_main, UIApplicationMain... Sans lui, tout compile et l'édition de liens tombe sur undefined reference to WinMain — une erreur qui ne nomme *aucun* de vos fichiers, et qu'on cherche donc partout sauf ici. Et **n'écrivez pas votre propre main** : nkmain est le point d'entrée portable, avec seize points d'entrée de plateforme derrière lui.

Le piège

NKENTSEU_DEFINE_APP_DATA attend un NkAppData, pas une chaîne. Lui passer "NkDames" donne no viable overloaded '=' — un message qui ne nomme jamais la cause. Le () final de la lambda n'est pas décoratif non plus : sans lui, vous passez la *fonction* au lieu de son résultat.

22.3 Étape 3 — les règles : les types

Créez src/Dames/NkDamesRegles.h. Regardez d'abord ce qu'il inclut, car c'est la décision la plus importante du chapitre :

Applications/NkDames/src/Dames/NkDamesRegles.h

```

1 #include "NKContainers/Sequential/NkVector.h"
2 #include "NKCore/NkTypes.h"

```

Ce qu'il faut retenir

Deux inclusions. Ni fenêtre, ni GPU, ni image, ni police. Conséquence immédiate, et c'est tout l'objet de l'étape 8 : **ces règles se testeront sans ouvrir de fenêtre**, en console, en une seconde, sur une machine sans écran. Rangez une seule règle du côté du dessin, et elle devient invisible au banc. Le jeu marchera peut-être; personne ne pourra le prouver.

Applications/NkDames/src/Dames/NkDamesRegles.h

```

1 namespace nkentseu {
2     namespace jeux {
3
4         enum class NkDamesCamp : uint8 { NK_BLANC = 0, NK_NOIR = 1 };
5

```

```

6      enum class NkDamesPiece : uint8 {
7          NK_VIDE = 0,
8          NK_PION_BLANC, NK_PION_NOIR,
9          NK_DAME_BLANCHE, NK_DAME_NOIRE
10     };
11
12     inline bool NkDamesEstBlanc(NkDamesPiece p) noexcept {
13         return p == NkDamesPiece::NK_PION_BLANC || p == NkDamesPiece::NK_DAME_BLANCHE;
14     }
15     inline bool NkDamesEstNoir(NkDamesPiece p) noexcept {
16         return p == NkDamesPiece::NK_PION_NOIR || p == NkDamesPiece::NK_DAME_NOIRE;
17     }
18     inline bool NkDamesEstDame(NkDamesPiece p) noexcept {
19         return p == NkDamesPiece::NK_DAME_BLANCHE || p == NkDamesPiece::NK_DAME_NOIRE;
20     }
21     inline bool NkDamesAppartient(NkDamesPiece p, NkDamesCamp c) noexcept {
22         return c == NkDamesCamp::NK_BLANC ? NkDamesEstBlanc(p) : NkDamesEstNoir(p);
23     }

```

Ce qu'il faut retenir

Une seule énumération pour la pièce, pas deux champs «couleur» et «type». Deux champs autorisent l'état *aucune couleur mais une dame* — un état qui n'existe pas dans le jeu et qu'il faudrait pourtant traiter partout. Les quatre fonctions inline sont la contrepartie : elles rendent lisible ce que l'énumération compacte. Elles sont dans l'en-tête, donc gratuites.

Puis le coup. Un coup de dames n'est pas un déplacement : c'est *une rafle entière*.

Applications/NkDames/src/Dames/NkDamesRegles.h

```

1      static const int32 NK_DAMES_MAX_PRISES = 20;
2
3      struct NkDamesCoup {
4          int8 depR = -1, depC = -1;    // case de depart
5          int8 arrR = -1, arrC = -1;    // case d'arrivee FINALE
6
7          uint8 nbPrises = 0;
8          int8 prisR[NK_DAMES_MAX_PRISES] = {};
9          int8 prisC[NK_DAMES_MAX_PRISES] = {};
10
11         // Les cases d'atterrissage successives, pour l'affichage.
12         uint8 nbEtapas = 0;
13         int8 etapeR[NK_DAMES_MAX_PRISES] = {};
14         int8 etapeC[NK_DAMES_MAX_PRISES] = {};
15
16         bool EstPrise() const noexcept { return nbPrises > 0; }
17     };

```

Ce qu'il faut retenir

Des tableaux de taille fixe, pas des vecteurs. Un coup est copié des milliers de fois pendant la génération; une allocation par copie coûterait plus que tout le reste du jeu. Le commentaire du dépôt justifie le 20 : «le record théorique connu au jeu international est de 12; 20 laisse une marge sans jamais tronquer une rafle légale — et une troncature silencieuse fausserait le choix de la rafle maximale, donc la règle elle-même». C'est un plafond qui se justifie par la conséquence de son dépassement, pas par une

intuition de confort.

Enfin la classe :

Applications/NkDames/src/Dames/NkDamesRegles.h

```
1      class NkDamesPartie {
2      public:
3          static const int32 NK_TAILLE = 10;
4
5          NkDamesPartie() { Initialiser(); }
6
7          void Initialiser() noexcept;
8
9          NkDamesPiece Case(int32 r, int32 c) const noexcept {
10             return DansDamier(r, c) ? mCases[r][c] : NkDamesPiece::NK_VIDE;
11         }
12         void PoserCase(int32 r, int32 c, NkDamesPiece p) noexcept {
13             if (DansDamier(r, c)) { mCases[r][c] = p; }
14         }
15
16         NkDamesCamp Trait() const noexcept { return mTrait; }
17         void PoserTrait(NkDamesCamp c) noexcept { mTrait = c; }
18
19         void CoupsLegaux(NkVector<NkDamesCoup> &sortie) const;
20         bool Jouer(const NkDamesCoup &coup) noexcept;
21         bool EstTerminee(NkDamesCamp &gagnant) const;
22         int32 CompterPieces(NkDamesCamp camp) const noexcept;
23         void CoupsDepuis(int32 r, int32 c, NkVector<NkDamesCoup> &sortie) const;
24
25         static bool DansDamier(int32 r, int32 c) noexcept {
26             return r >= 0 && r < NK_TAILLE && c >= 0 && c < NK_TAILLE;
27         }
28         // Les cases jouables sont les SOMBRES. Convention : (r+c) impair.
29         static bool CaseJouable(int32 r, int32 c) noexcept {
30             return ((r + c) & 1) != 0;
31         }
32
33     private:
34         void GenererPrises(int32 r, int32 c, NkVector<NkDamesCoup> &sortie) const;
35         void ExplorerPrises(int32 r, int32 c, NkDamesPiece piece, NkDamesCoup
36             &courant,
37                             bool prisesFaites[NK_TAILLE][NK_TAILLE],
38                             NkVector<NkDamesCoup> &sortie) const;
39         void GenererSimples(int32 r, int32 c, NkVector<NkDamesCoup> &sortie) const;
40
41         NkDamesPiece mCases[NK_TAILLE][NK_TAILLE] = {};
42         NkDamesCamp mTrait = NkDamesCamp::NK_BLANC;
43     };
```

Ce qu'il faut retenir

Six questions publiques suffisent à décrire un jeu au tour par tour : initialiser, lire une case, qui joue, quels coups sont légaux, jouer, est-ce fini. Vous retrouverez exactement les mêmes aux échecs et au ludo.

Case() borne elle-même : un indice hors damier rend NK_VIDE au lieu de lire n'importe où. C'est ce qui permet aux générateurs de coups d'explorer sans tester les bords à chaque pas.

Et CoupsLegaux écrit dans un vecteur qu'on lui donne au lieu d'en rendre un : l'appelant

réutilise le même tampon à chaque tour.

22.4 Étape 4 — l'initialisation

Créez `NkDamesRegles.cpp`. Première fonction :

`Applications/NkDames/src/Dames/NkDamesRegles.cpp`

```
1 void NkDamesPartie::Initialiser() noexcept {
2     for (int32 r = 0; r < NK_TAILLE; ++r) {
3         for (int32 c = 0; c < NK_TAILLE; ++c) {
4             mCases[r][c] = NkDamesPiece::NK_VIDE;
5         }
6     }
7     // Rangées 0..3 = noirs (en haut), 6..9 = blancs (en bas).
8     for (int32 r = 0; r < 4; ++r) {
9         for (int32 c = 0; c < NK_TAILLE; ++c) {
10            if (CaseJouable(r, c)) { mCases[r][c] = NkDamesPiece::NK_PION_NOIR; }
11        }
12    }
13    for (int32 r = 6; r < NK_TAILLE; ++r) {
14        for (int32 c = 0; c < NK_TAILLE; ++c) {
15            if (CaseJouable(r, c)) { mCases[r][c] = NkDamesPiece::NK_PION_BLANC; }
16        }
17    }
18    mTrait = NkDamesCamp::NK_BLANC;
19 }
```

Ce qu'il faut retenir

Les rangées 4 et 5 restent vides : c'est ce qui donne 20 pièces par camp sur un damier de 10×10. Et `CaseJouable` est appelée plutôt que réécrite — une seconde définition de «case sombre» quelque part dans le dessin, et un jour les deux diffèrent.

Et le compteur, qui servira à la fois aux règles, à l'affichage et au banc :

`Applications/NkDames/src/Dames/NkDamesRegles.cpp`

```
1 int32 NkDamesPartie::CompterPieces(NkDamesCamp camp) const noexcept {
2     int32 n = 0;
3     for (int32 r = 0; r < NK_TAILLE; ++r) {
4         for (int32 c = 0; c < NK_TAILLE; ++c) {
5             if (NkDamesAppartient(mCases[r][c], camp)) { ++n; }
6         }
7     }
8     return n;
9 }
```

22.5 Étape 5 — les coups simples

En haut du `.cpp`, les quatre diagonales :

Applications/NkDames/src/Dames/NkDamesRegles.cpp

```
1 namespace {
2     // Ordre fixe : il decide, a rafle egale, quel coup l'ordinateur trouve
3     // en premier -- donc il doit etre stable.
4     const int32 kDirR[4] = {-1, -1, 1, 1};
5     const int32 kDirC[4] = {-1, 1, -1, 1};
6 }
```

Ce qu'il faut retenir

Un ordre arbitraire qui a une conséquence observable cesse d'être arbitraire. Changer ces quatre paires ne change aucune règle — mais change le coup que l'IA choisit à égalité, donc la partie entière.

Une constante dont la valeur n'importe pas mais dont *la stabilité* importe : écrivez-le, sinon quelqu'un les réordonnera pour faire joli.

Un pion avance d'une case en diagonale, sans reculer. Une dame glisse.

Applications/NkDames/src/Dames/NkDamesRegles.cpp – structure

```
1 void NkDamesPartie::GenererSimples(int32 r, int32 c, NkVector<NkDamesCoup> &sortie) const {
2     const NkDamesPiece piece = mCases[r][c];
3     const bool dame = NkDamesEstDame(piece);
4     const int32 sens = NkDamesEstBlanc(piece) ? -1 : +1; // Les blancs montent
5
6     for (int32 d = 0; d < 4; ++d) {
7         if (!dame && kDirR[d] != sens) { continue; } // un pion ne recule pas
8         int32 rr = r + kDirR[d], cc = c + kDirC[d];
9         while (DansDamier(rr, cc) && mCases[rr][cc] == NkDamesPiece::NK_VIDE) {
10             NkDamesCoup m;
11             m.depR = (int8)r; m.depC = (int8)c;
12             m.arrR = (int8)rr; m.arrC = (int8)cc;
13             sortie.PushBack(m);
14             if (!dame) { break; } // Le pion s'arrete a 1
15             rr += kDirR[d]; cc += kDirC[d]; // la dame glisse
16         }
17     }
18 }
```

Ce qu'il faut retenir

Une seule fonction pour le pion et la dame, séparés par deux lignes : le continue qui interdit le recul, et le break qui limite à une case. Deux fonctions distinctes auraient divergé au premier correctif.

22.6 Étape 6 — la rafle, et c'est le seul endroit délicat

Une rafle est une *suite* de prises dans le même coup. On l'explore en profondeur.

Applications/NkDames/src/Dames/NkDamesRegles.cpp – l'en-tête du fichier

```
1 // LE POINT DELICAT, ET IL N'Y EN A QU'UN : La rafle.
2 // On l'explore en profondeur, et la piece prise est marquee "deja prise"
3 // pendant l'exploration SANS ETRE RETIREE du damier : elle continue de
4 // BLOQUER le passage. C'est la regle du jeu, et c'est aussi ce qui evite de
```

```

5 // compter deux fois la même pièce. Retirer la pièce pendant l'exploration
6 // produirait des rafles qui n'existent pas -- l'erreur classique, et elle
7 // est silencieuse.

```

Ce qu'il faut retenir

Lisez ce paragraphe deux fois : il contient la totalité de la difficulté.

Une pièce capturée n'est retirée qu'à la fin de la rafle. Pendant l'exploration elle est marquée — pour ne pas être comptée deux fois — mais elle reste sur le damier et **bloque le trajet**.

Si vous la retirez tout de suite, votre programme trouvera des rafles qui n'existent pas : la case libérée laisse passer une dame qui, en vrai, serait arrêtée. Le jeu ne plantera pas. Il trichera.

Applications/NkDames/src/Dames/NkDamesRegles.cpp

```

1 void NkDamesPartie::ExplorerPrises(int32 r, int32 c, NkDamesPiece piece,
2                                     NkDamesCoup &courant,
3                                     bool prisesFaites[NK_TAILLE][NK_TAILLE],
4                                     NkVector<NkDamesCoup> &sortie) const {
5     const NkDamesCamp camp = NkDamesEstBlanc(piece) ? NkDamesCamp::NK_BLANC
6                                     : NkDamesCamp::NK_NOIR;
7     const NkDamesCamp adverse = (camp == NkDamesCamp::NK_BLANC) ? NkDamesCamp::NK_NOIR
8                                     : NkDamesCamp::NK_BLANC;
9     const bool dame = NkDamesEstDame(piece);
10    bool aContinue = false;
11
12    for (int32 d = 0; d < 4; ++d) {
13        // 1. avancer jusqu'à rencontrer une pièce
14        // 2. si elle est adverse ET pas déjà prise ET la case suivante est libre :
15        //     -> marquer, empiler l'étape, RECURSER, puis DEMARQUER
16        //     -> aContinue = true
17    }
18
19    // 3. une rafle qui ne peut plus continuer est un coup complet
20    if (!aContinue && courant.nbPrises > 0) {
21        courant.arrR = (int8)r; courant.arrC = (int8)c;
22        sortie.PushBack(courant);
23    }
24 }

```

Ce qu'il faut retenir

Le « démarquer » après la récursion est ce qui distingue une exploration d'un parcours.

Sans lui, la première branche essayée empoisonne toutes les suivantes : la deuxième croit que la pièce est déjà prise, et la rafle qu'elle aurait trouvée disparaît.

C'est le *retour sur trace (backtracking)*, et il tient en une ligne qu'on oublie une fois sur deux.

Le piège

La case de DÉPART de la rafle est vide « pour de faux ». La pièce y est encore, physiquement, puisqu'on n'a rien joué. Une dame qui revient sur sa propre case de départ en cours de rafle doit pouvoir passer.

C'est le genre de cas qu'on ne rencontre qu'une partie sur cent — donc jamais pendant le développement, et toujours chez un joueur.

22.7 Étape 7 — rafle maximale, jouer, fin de partie

Applications/NkDames/src/Dames/NkDamesRegles.cpp – structure

```
1 void NkDamesPartie::CoupsLegaux(NkVector<NkDamesCoup> &sortie) const {
2     sortie.Clear();
3     // 1. generer TOUTES les prises de toutes les pieces du camp au trait
4     // 2. s'il y en a : garder UNIQUEMENT celles de nbPrises maximal
5     // 3. sinon seulement : generer Les coups simples
6 }
```

Ce qu'il faut retenir

Le filtrage par maximum se fait EN SORTIE, jamais pendant l'exploration. Filtrer trop tôt fait perdre des branches qui allaient devenir plus longues : une rafle de 2 prises peut se prolonger en 5.

C'est une erreur d'optimisation prématurée qui donne un jeu *presque* juste — le pire cas, car il faut une position précise pour la voir.

Applications/NkDames/src/Dames/NkDamesRegles.cpp – structure

```
1 bool NkDamesPartie::Jouer(const NkDamesCoup &coup) noexcept {
2     NkDamesPiece p = Case(coup.depR, coup.depC);
3     if (!NkDamesAppartient(p, mTrait)) {
4         return false; // un refus se DIT
5     }
6     PoserCase(coup.depR, coup.depC, NkDamesPiece::NK_VIDE);
7     for (uint8 i = 0; i < coup.nbPrises; ++i) {
8         PoserCase(coup.prisR[i], coup.prisC[i], NkDamesPiece::NK_VIDE);
9     }
10    // Promotion : SEULEMENT si Le coup S'ACHEVE sur la derniere rangee.
11    if (!NkDamesEstDame(p)) {
12        if (NkDamesEstBlanc(p) && coup.arrR == 0) { p =
13            NkDamesPiece::NK_DAME_BLANCHE; }
14        else if (NkDamesEstNoir(p) && coup.arrR == NK_TAILLE-1) { p =
15            NkDamesPiece::NK_DAME_NOIRE; }
16    }
17    PoserCase(coup.arrR, coup.arrC, p);
18    mTrait = (mTrait == NkDamesCamp::NK_BLANC) ? NkDamesCamp::NK_NOIR : NkDamesCamp::NK_BLANC;
19    return true;
20 }
```

Ce qu'il faut retenir

Jouer rend un booléen, et il faut le lire. « Un refus se dit, il ne se devine pas à l'absence d'effet » : sans ce retour, un coup illégal laisse le damier inchangé et le joueur croit que son clic n'a pas été reçu.

Et la promotion n'a lieu que si le coup S'ACHÈVE sur la dernière rangée. Un pion qui traverse cette rangée *au milieu* d'une rafle ne devient pas dame — règle réelle, et l'une des cinq que toutes les implémentations ratent.

Applications/NkDames/src/Dames/NkDamesRegles.cpp – structure

```
1 bool NkDamesPartie::EstTerminee(NkDamesCamp &gagnant) const {
2     if (CompterPieces(NkDamesCamp::NK_BLANC) == 0) { gagnant = NkDamesCamp::NK_NOIR; return
    true; }
3     if (CompterPieces(NkDamesCamp::NK_NOIR) == 0) { gagnant = NkDamesCamp::NK_BLANC; return
    true; }
4     NkVector<NkDamesCoup> coups;
5     CoupsLegaux(coups);
6     if (coups.Size() == 0) { // bloque = perdu
7         gagnant = (mTrait == NkDamesCamp::NK_BLANC) ? NkDamesCamp::NK_NOIR :
    NkDamesCamp::NK_BLANC;
8         return true;
9     }
10    return false;
11 }
```

Le piège

Un camp sans coup légal a perdu, même s'il lui reste des pièces. Oublier ce cas donne un jeu qui se fige : c'est au joueur de jouer, il ne peut rien jouer, et rien ne le dit. Le symptôme — « le jeu ne répond plus » — ne ressemble en rien à sa cause.

22.8 Étape 8 — le banc. Avant l'écran.

Vous n'avez encore dessiné aucun pixel, et vous allez déjà prouver que votre jeu est juste. C'est possible uniquement parce que l'étape 3 n'a inclus ni fenêtre ni GPU.

Applications/NkDames/src/Dames/NkDamesBanc.h

```
1 namespace nkentseu { namespace jeux { namespace dames {
2     /// Rend 0 si tout passe, non nul sinon. Le code de sortie EST le verdict.
3     int32 NkDamesLancerBanc();
4 }}}}
```

Et on le branche *avant* toute fenêtre, dans le jeu :

Applications/NkDames/src/Dames/NkDamesJeu.cpp – structure

```
1 NkOptional<int> NkDamesJeu::OnCommandLine(const NkVector<NkString> &args) {
2     for (uint64 i = 0; i < args.Size(); ++i) {
3         if (args[i] == "--selftest") {
4             return NkDamesLancerBanc(); // on rend un code : pas de fenetre
5         }
6     }
7     return {}; // rien rendu = on continue normalement
8 }
```

Ce qu'il faut retenir

`OnCommandLine` s'exécute **AVANT** la création de la fenêtre. C'est ce qui permet à `NkDames` `--selftest` de tourner dans une intégration continue sans écran, sans GPU, en une seconde.

Rendre une valeur arrête le programme avec ce code; ne rien rendre le laisse démarrer. Un `NkOptional` plutôt qu'un `int` sentinelle : « pas de réponse » et « le code 0 » sont deux

choses différentes.

Un cas de banc ressemble à ceci — on *fabrique* la position :

Applications/NkDames/src/Dames/NkDamesBanc.cpp – forme d'un cas

```
1 // Cas : La promotion n'a lieu QUE si le coup s'achève sur la dernière rangée.
2 {
3     NkDamesPartie p;
4     for (int32 r = 0; r < 10; ++r)
5         for (int32 c = 0; c < 10; ++c)
6             p.PoserCase(r, c, NkDamesPiece::NK_VIDE); // damier vide : on maîtrise tout
7     p.PoserCase(2, 1, NkDamesPiece::NK_PION_BLANC);
8     p.PoserCase(1, 2, NkDamesPiece::NK_PION_NOIR); // à sauter, arrivée rangée 0
9     p.PoserCase(1, 4, NkDamesPiece::NK_PION_NOIR); // la rafle CONTINUE
10    p.PoserTrait(NkDamesCamp::NK_BLANC);
11
12    NkVector<NkDamesCoup> coups;
13    p.CoupsLegaux(coups);
14    VERIFIER(coups.Size() == 1);
15    p.Jouer(coups[0]);
16    VERIFIER(p.Case(coups[0].arrR, coups[0].arrC) == NkDamesPiece::NK_PION_BLANC);
17    // ^ un PION, pas une dame : la rafle ne s'achève pas rangée 0
18 }
```

Ce qu'il faut retenir

On part d'un damier **VIDE**, jamais de la position initiale. Sur un damier complet, vingt autres coups sont légaux et l'on ne sait plus lequel on mesure. Fabriquer la position est ce qui rend l'assertion *lisible* : `coups.Size() == 1` n'a de sens que parce qu'on a construit la situation où il n'y a qu'un coup.

Les cas négatifs comptent autant que les positifs

Un banc qui vérifie seulement que les coups légaux sont acceptés passe au vert sur des règles qui acceptent **tout**. Écrivez donc aussi : *on ne peut pas jouer une pièce adverse, une rafle de 3 est obligatoire quand une de 2 existe, un pion ne recule pas en déplacement simple*.

La contre-épreuve, et elle n'est pas facultative

Un banc qui n'a jamais rougi est une intention, pas un contrôle. Cassez exprès ce qu'il surveille — retirez le filtre de rafle maximale — et vérifiez qu'il passe au **rouge**, sur le bon cas, avec un code de sortie non nul. Puis restaurez. Le dépôt inscrit cette contre-épreuve dans l'en-tête du banc, avec le cas qui est tombé :

Applications/NkDames/src/Dames/NkDamesBanc.cpp

```
1 // CONTRE-EPREUVE FAITE : en retirant le filtre de rafle maximale, le banc
2 // est passé au ROUGE sur le bon cas, code de sortie 1.
```

Le cas qui a été faux avant d'être juste

Le banc du ludo exigeait d'abord que chaque pas de la piste soit *orthogonal*. Il échouait — et le code avait raison : un vrai plateau de ludo a exactement quatre virages en diagonale, aux coins.

Un banc rouge n'accuse pas toujours le code. Avant de corriger ce qui est mesuré, vérifiez ce que la mesure attendait.

Lancez-le : `NkDames.exe --selftest`. Vous avez un jeu prouvé, et toujours aucun pixel.

22.9 Étape 9 — le thème, puis la géométrie

Une seule source pour les couleurs, dans un en-tête sans code :

Applications/NkDames/src/Dames/NkDamesTheme.h – forme

```
1 struct NkDamesTheme {
2     NkColor fond, caseClaire, caseSombre, liseréSel, dispo;
3     NkColor pionBlanc, pionNoir, contour, texte;
4 };
5 inline const NkDamesTheme &NkDamesThemeDefault();
```

Ce qu'il faut retenir

Aucune couleur écrite ailleurs. Une seule valeur en dur dans le dessin, et le jour où l'on veut un thème clair il faut la retrouver — ce qui se termine toujours par « il en restait une ».

Puis la géométrie. Elle se calcule, elle ne se code pas en dur :

Applications/NkDames/src/Dames/NkDamesEcran.h – forme

```
1 struct NkDamesGeometrie {
2     NkRect damier; // Le carre du jeu
3     float32 caseTaille; // cote d'une case
4     NkRect hud; // La bande d'information
5     NkRect boutons[2]; // Les sieges
6
7     static NkDamesGeometrie Calculer(const renderer::NkLayoutInfo &info);
8 };
```

Le piège

Le bloc se centre sur l'ÉCRAN, pas sur le plateau. Le ludo a d'abord ancré son menu sur le damier — un carré centré — et laissait une bande vide sur tout le tiers supérieur en fenêtre portrait.

Chaque rectangle était juste ; leur *ensemble* était mal placé. C'est un défaut qu'aucun calcul ne signale et qui ne se voit que sur une image.

Ce qu'il faut retenir

`NkLayoutInfo` porte la **zone sûre** — les marges de l'encoche et de l'indicateur de geste. Partez d'elle, jamais de la taille brute de la fenêtre, sinon la première rangée du damier passe sous l'encoche sur téléphone.

22.10 Étape 10 — le dessin

L'écran reçoit une *vue* : ce qu'il doit montrer, jamais l'objet du jeu.

Applications/NkDames/src/Dames/NkDamesEcran.h – forme

```
1 struct NkDamesVue {
2     const NkDamesPartie *partie;
3     const NkVector<NkDamesCoup> *coupsProposes;
4     int32 selR, selC;
5     bool finie;
6     NkDamesCamp gagnant;
7 };
8
9 void NkDamesDessiner(nkgui::NkGuiDrawList &dl, const NkDamesGeometrie &geo,
10     const NkDamesVue &vue);
```

Ce qu'il faut retenir

Le dessin ne prend que des **const**. Il ne peut donc rien modifier — ce n'est plus une discipline, c'est le compilateur qui l'impose.

Une fonction de dessin qui recevrait NkDamesJeu& finirait, un jour de fatigue, par jouer un coup depuis le dessin. Et le défaut serait introuvable : personne ne cherche une règle de jeu dans un fichier d'affichage.

Applications/NkDames/src/Dames/NkDamesEcran.cpp – ordre du dessin

```
1 // 1. Le fond
2 // 2. Les 100 cases (claire / sombre)
3 // 3. Le liseré de la case selectionnee
4 // 4. Les pastilles des destinations possibles
5 // 5. Les pieces (disque + anneau ; une couronne si c'est une dame)
6 // 6. Le HUD : compteurs, trait, boutons de siege
7 // 7. Le bandeau de fin, s'il y a lieu
```

Ce qu'il faut retenir

L'ordre **EST** le résultat. Il n'y a pas de profondeur en 2D : ce qui est dessiné en dernier est devant. Le liseré avant les pièces, et il disparaît dessous.

22.11 Étape 11 — le clic devient un coup

Applications/NkDames/src/Dames/NkDamesJeu.cpp – structure

```
1 bool NkDamesJeu::OnPointer(const NkPointer &p) {
2     if (mSplash.Actif()) { return mSplash.Sauter(p); }
3     if (p.phase != NkPointerPhase::NK_PRESSED) { return false; }
4
5     // 1. Le point est-il dans le damier ? -> (r, c)
6     // 2. si une piece est deja selectionnee et (r,c) est une destination
7     //     proposee -> jouer ce coup, deselectionner
8     // 3. sinon, si (r,c) porte une piece du camp au trait
9     //     -> selectionner, et demander CoupsDepuis(r, c, mCoupsProposes)
10    // 4. sinon -> deselectionner
11    return true;
```

```
12 }
```

Ce qu'il faut retenir

Deux clics, jamais un glisser. Sélectionner puis poser marche à la souris *et* au doigt, sans code différent. Un glisser-déposer demanderait un suivi de mouvement, un seuil de déclenchement, et une gestion de l'annulation.

Et c'est CoupsDepuis qui alimente l'affichage des destinations — la même fonction de règles qui décide. Recalculer les destinations dans l'écran donnerait deux réponses qui divergeraient un jour.

Le piège

NkPointer donne des **pixels client**, pas des pixels écran. Et la géométrie du damier est en pixels client aussi. Les mélanger donne une sélection décalée d'exactly la position de la fenêtre — donc juste quand la fenêtre est en haut à gauche, et fausse partout ailleurs.

22.12 Étape 12 — l'adversaire

Applications/NkDames/src/Dames/NkDamesRegles.h

```
1  /// Choisit un coup pour L'ordinateur. Simple par construction : elle prefere
2  /// la rafle la plus grosse, puis la promotion, puis L'avance.
3  /// Ce n'est pas un adversaire fort, et Le fichier ne pretend pas L'etre.
4  const NkDamesCoup *NkDamesChoisirCoup(const NkDamesPartie &partie,
5                                          const NkVector<NkDamesCoup> &coups,
6                                          uint32 &graine) noexcept;
```

Ce qu'il faut retenir

«**Ce n'est pas un adversaire fort, et le fichier ne prétend pas l'être.**» Une IA de trente lignes qui s'annonce comme telle est honnête; la même sans cette phrase déçoit. Notez la graine passée *par référence* : l'IA départage les ex æquo au hasard, et ce hasard est **reproductible**. Même graine, même partie — donc un défaut d'IA se rejoue.

Applications/NkDames/src/Dames/NkDamesJeu.cpp – structure

```
1  void NkDamesJeu::OnTick(float32 deltaTime) {
2      if (mSplash.Avancer(deltaTime)) { return; } // L'ouverture tient l'ecran
3      if (mAnim.Actif()) { mAnim.Avancer(deltaTime); return; }
4      if (mFinie || EstHumain(TraitIndex())) { return; }
5
6      mAttenteIA -= deltaTime; // Laisser le temps de VOIR
7      if (mAttenteIA <= 0.f) { JouerCoupIA(); mAttenteIA = 0.45f; }
8  }
```

Le piège

Avancer rend « vrai, je tiens l'écran », et il faut s'arrêter là. Sans ce retour, le jeu tournerait *derrière* l'écran d'ouverture : l'IA jouerait ses premiers coups pendant les quatre secondes, et le joueur trouverait la partie déjà entamée en arrivant.

Et une IA qui joue instantanément donne l'impression d'un bug. Les 0,45 seconde d'attente ne servent pas l'IA : elles servent l'œil.

22.13 Étape 13 — l'ouverture, et les modes

Applications/NkDames/src/Dames/NkDamesJeu.cpp

```
1 bool NkDamesJeu::OnGuiInit() {
2     mSplash.PoserJeu("Dames");
3     Rejouer();
4     return true;
5 }
```

Ce qu'il faut retenir

Une ligne pour l'écran d'ouverture. La coquille porte la marque RIHEN et l'animation ; vous ne donnez que le nom du jeu. C'est ce qui rend les ouvertures des quatre jeux du dépôt identiques, à un mot près.

Et elle est **toujours sautable** : « un écran d'ouverture qu'on ne peut pas passer est une taxe payée à chaque lancement, y compris par celui qui teste vingt fois par heure ».

Enfin, les modes. Chaque siège a un contrôleur, et rien d'autre n'en découle :

Applications/NkDames/src/Dames/NkDamesJeu.h

```
1 /// Par défaut : vous Les blancs, L'IA Les noirs.
2 NkControleur mControleur[2] = {NkControleur::NK_HUMAIN, NkControleur::NK_IA};
```

Ce qu'il faut retenir

Trois modes de jeu, et aucun n'est un cas particulier du code : humain/IA, humain/humain à deux sur le même écran, IA/IA en simulation. Un tableau de deux contrôleurs les produit tous les trois.

Écrire `if (modeDuo) ...` aurait donné trois chemins à tester au lieu d'un.

22.14 Construire, lancer, empaqueter

Les commandes, dans l'ordre ou vous les taperez

```
1 jenga build --target NkDames --config Release
2 .\Build\Bin\Release-Windows\NkDames\NkDames.exe --selftest # le banc, 1 s
3 .\Build\Bin\Release-Windows\NkDames\NkDames.exe # le jeu
4
5 jenga build --target NkDames --platform web
6 jenga package --platform windows --project NkDames --output dist/
```

Le piège

jenga package nomme sa sortie d'après le PROJET, pas d'après la plateforme. Empaqueter Windows puis Web dans le même dossier produit deux fois NkDames.zip, et le

second écrase le premier *en silence*, code de sortie 0.
Donnez un dossier de sortie par plateforme. Et **ouvrez l'archive** : c'est en y trouvant un .wasm là où l'on attendait un .exe que le défaut se voit, jamais dans la sortie de la commande.

Ce qu'il faut retenir

Un APK Android de ce jeu n'a légitimement pas de `classes.dex`. C'est une *NativeActivity* : il n'y a pas de code Java à convertir. Un contrôle qui exige un dex déclarera cassé un APK qui s'installe et fonctionne.

22.15 Ce qui change pour les échecs et le ludo

Le squelette ne bouge pas. Ce qui change tient dans une colonne.

	Ce qui est identique	Sa difficulté propre
Dames	les 7 fichiers, les 6 questions	la <i>rafle maximale</i>
Échecs	les 7 fichiers, les 6 questions	roque, prise en passant, échec et mat
Ludo	les 7 fichiers, les 6 questions	4 sièges, un dé, des sièges éteints

	NkDames	NkEchecs	NkLudo
Fichiers	10	9	9
Lignes	1 998	2 121	2 130
Cas de banc	16	14	23

Ce qu'il faut retenir

Trois jeux, trois cents lignes d'écart. Ce n'est pas une coïncidence : c'est le squelette qui domine, et la règle propre qui est petite.
Apprendre à construire l'un, c'est savoir construire les trois — et c'est exactement ce que ce chapitre voulait vous laisser.

La leçon du ludo, qui dépasse le ludo

Un siège peut être Humain, IA, ou **désactivé**. Sauter un siège éteint est une règle de *tour* — elle vit donc dans les règles, pas dans l'écran.
Mise du côté du dessin, elle aurait été invisible au banc. Et le banc est le seul endroit qui vérifie qu'une partie **se termine** : sans lui, un tour qui donne la main à un siège sans pions ferait attendre le jeu indéfiniment — et le symptôme serait « le jeu se fige », à des lieues de sa cause.

Ce chapitre en bref

Treize étapes, sept fichiers, et un ordre qui n'est pas négociable : le projet, la fenêtre, les règles, le **banc**, puis seulement l'écran, l'entrée, l'adversaire. Les règles n'incluent que deux en-têtes — c'est ce qui rend le banc possible, et le banc est ce qui distingue un jeu qui marche d'un jeu dont on *sait* qu'il marche. Le dessin ne reçoit que du `const`, l'écran ne décide de rien, et le fichier de jeu est le seul à modifier l'état. Les échecs et le ludo se

construisent avec les mêmes sept fichiers et les mêmes six questions.

Questions de cours

1. Pourquoi écrit-on le banc avant le dessin, et qu'est-ce qui le rend possible ?
2. Quelles sont les deux seules inclusions du fichier de règles ?
3. Pourquoi une pièce capturée n'est-elle pas retirée du damier pendant l'exploration d'une rafle ?
4. Pourquoi le filtrage par rafle maximale se fait-il en sortie et non pendant l'exploration ?
5. Que se passe-t-il si l'on oublie le cas « camp sans coup légal » ?
6. Où s'exécute `OnCommandLine`, et pourquoi est-ce décisif ?

Exercice pratique

Construisez le **morpion** en suivant les treize étapes, dans l'ordre, sans en sauter une.
Contraintes :

1. votre `NkMorpionRegles.h` n'inclut que `NkVector.h` et `NkTypes.h` ;
2. votre banc tourne *avant* que le moindre pixel soit dessiné, et contient au moins deux cas négatifs — on ne joue pas sur une case occupée, et une grille pleine sans alignement est une partie nulle ;
3. les trois modes (humain/IA, humain/humain, IA/IA) sortent d'un tableau de deux contrôleurs, sans un seul `if` de mode.

Puis construisez-le pour le Web et ouvrez-le dans un navigateur.

Exercice de démonstration

Prouvez que votre banc de morpion mesure vraiment quelque chose.
Cassez exprès la détection d'alignement — ne testez que les lignes, pas les colonnes.
Lancez : il doit passer au **rouge**, sur le cas des colonnes, avec un code de sortie non nul.
Restaurez, relancez, notez le retour au vert.

Recopiez les deux sorties dans votre compte rendu : c'est la paire qui prouve, pas le vert seul.

□ Si la mutation *ne casse rien*, ne concluez pas trop vite que le contrôle manque : vérifiez d'abord que votre jeu d'essai pouvait exprimer l'écart. Une grille où l'alignement gagnant est toujours une ligne ne peut pas voir un défaut sur les colonnes.

Exercice de logique

Un joueur signale : « au ludo, quand je désactive deux sièges, le jeu se fige après quelques tours ».

1. Sans lire le code, énumérez au moins trois causes possibles, en distinguant celles qui vivraient dans les *règles* de celles qui vivraient dans *l'écran*.
2. Pour chacune, dites quel cas de banc l'aurait attrapée.
3. Concluez sur celles qu'aucun banc de règles ne peut voir — et dites par quel autre moyen on les prend.

Construire un jeu animé, de A à Z

Le chapitre précédent a produit un jeu au tour par tour : on clique, l'état change, on redessine. Ce chapitre construit un jeu **animé** — des gemmes qui s'échangent, tombent, disparaissent en cascade — et cela change une chose fondamentale : **le jeu n'est plus toujours prêt à recevoir un clic**.

Nous reprenons le squelette de l'étape 1 à l'étape 13 et nous lui ajoutons ce qui manque. Onze étapes de plus, et vous obtenez GemCrush : trente niveaux, progression enregistrée, sons synthétisés, didacticiel.

Fichier	Rôle	Lignes
NkGem.{h,cpp}	une gemme : couleur, forme, état	313
NkGemBoard.{h,cpp}	le plateau et ses phases	1 343
NkMatchFinder.{h,cpp}	trouver les alignements	426
NkGemLevels.{h,cpp}	les trente niveaux	536
Ui/NkGemArt	dessiner une gemme	612
Ui/NkGemScreens	les quatre écrans	568
Ui/NkGemHud	score, objectifs, coups restants	1 003
Ui/NkGemAudio	les sons, <i>synthétisés</i>	642
Ui/NkGemTutorial	le didacticiel	353
Ui/NkGemTheme.h	les couleurs	181
Ui/NkGemSplash	l'ouverture	298

22 fichiers, 8 091 lignes pour un jeu dont la règle tient en une phrase : *échangez deux gemmes voisines, alignez-en trois ou plus, elles disparaissent*.

Ce qu'il faut retenir

Où passent les huit mille lignes ? Regardez la colonne de droite : la règle — gemme, plateau, alignements — pèse 2 082 lignes. Tout le reste, écrans, HUD, son, didacticiel, progression, en pèse 6 009.

Quand un jeu grossit, c'est le côté écran qui grossit. C'est la leçon la plus utile de ce chapitre, et il faut la savoir avant de commencer, pas après.

23.1 Étape 14 — la gemme, et ce qu'elle ne fait pas

Applications/Gemcrush/src/Gemcrush/NkGem.h – l'en-tête du fichier

```
1 // v2 : NkGem NE CONNAIT PAS NkEvent. Elle ne fait pas son propre test de
2 //      survol -- c'est NkGemBoard qui resout la case concernee en O(1) via
3 //      WorldToCell() et appelle les hooks ci-dessous.
4 //
```

```

5 // v3 : NkGem NE DESSINE PLUS. Elle portait trois NkShape plus quatre formes
6 //      d'overlay, soit SEPT objets de rendu par gemme.

```

Ce qu'il faut retenir

Deux capacités retirées à la gemme, et chacune pour une raison différente.

Le survol : soixante-quatre gemmes qui testent chacune si la souris est dessus, c'est soixante-quatre tests par image. Le plateau, lui, calcule la case directement depuis la position — une division. **Ce n'est pas une optimisation : c'est le bon propriétaire de la question.** La gemme ne sait pas où elle est dans la grille ; le plateau, oui.

Le dessin : sept objets de rendu par gemme, et surtout **deux chemins de dessin** qui « auraient divergé au premier changement de thème, et rien ne l'aurait signalé ».

Le piège

Une classe qui « se dessine elle-même » est le réflexe naturel, et c'est presque toujours faux dans un jeu. Le dessin d'une gemme dépend du thème, de l'animation en cours, du niveau, de la sélection — toutes choses que la gemme ne connaît pas et qu'elle devrait donc recevoir en paramètre. À ce moment-là, autant sortir la fonction.

23.2 Étape 15 — la machine à phases

C'est la différence avec un jeu de plateau, et il faut l'écrire avant tout le reste :

Applications/Gemcrush/src/Gemcrush/NkGemBoard.h

```

1 enum class NkGemBoardPhase : uint8 {
2     NK_BOARD_PHASE_IDLE,           ///< Pret a recevoir une entree
3     NK_BOARD_PHASE_SWAPPING,       ///< Deux gemmes s'echangeant (animation)
4     NK_BOARD_PHASE_REVERTING,      ///< L'echange n'a rien aligne -> retour anime
5     NK_BOARD_PHASE_RESOLVING_MATCH, ///< Gemmes alignees en train de disparaitre
6     // ... chute, remplissage, cascade
7 };

```

Ce qu'il faut retenir

Un clic n'est acceptable que dans la phase IDLE. Sans cette machine, un joueur qui clique pendant une chute déclenche un second échange par-dessus le premier : deux animations sur les mêmes cases, un état incohérent, et un plateau qui finit par afficher des gemmes qui n'existent plus.

C'est la panne la plus classique des jeux animés, et elle est *impossible* dès qu'une énumération de phase existe.

Applications/Gemcrush/src/Gemcrush/NkGemBoard.h

```

1 bool IsIdle() const noexcept;
2 bool TrySwap(int32 rowA, int32 columnA, int32 rowB, int32 columnB);
3 void Update(float32 deltaTime);

```

Ce qu'il faut retenir

TrySwap, pas Swap. Le nom dit que ça peut échouer — gemmes non voisines, plateau occupé — et le booléen le fait dire. Comparez avec Jouer du chapitre précédent : la même décision, le même mot.

Et Update(deltaTime) est ce qui fait avancer les phases. Oubliez de l'appeler, et le plateau reste bloqué dans SWAPPING pour toujours — le jeu se fige, silencieusement, exactement comme le ludo dont le tour ne revenait jamais.

23.3 Étape 16 — trouver les alignements

Applications/Gemcrush/src/Gemcrush/NkMatchFinder.h – forme

```
1  /// Trouve TOUS Les alignements de 3 ou plus, horizontaux et verticaux.
2  /// Ne modifie rien : elle REND ce qu'elle a trouvé.
3  void NkTrouverAlignements(const NkGemBoard &plateau, NkVector<NkGemMatch> &sortie);
```

Ce qu'il faut retenir

Chercher et supprimer sont deux opérations, et il faut les séparer. Une fonction qui supprimerait au fur et à mesure raterait les alignements qui se croisent : une gemme appartenant à une ligne et à une colonne serait retirée par la première, et la seconde ne la verrait plus.

Le résultat serait un jeu qui compte parfois mal — une fois sur cinquante.

Le piège

Une gemme peut appartenir à deux alignements. En croix, elle compte pour les deux, et la marquer deux fois double le score.

C'est pourquoi la suppression se fait sur un *ensemble* de cases, construit à partir de tous les alignements trouvés — jamais alignement par alignement.

23.4 Étape 17 — la cascade

Quand des gemmes disparaissent, celles du dessus tombent, de nouvelles arrivent en haut — et il peut se former un nouvel alignement, sans que le joueur ait rien fait.

La boucle de phases, en clair

```
1  IDLE --clic--> SWAPPING --alignement ?--> RESOLVING --> CHUTE --> REMPLISSAGE
2                      | non                      |
3                      v                          |
4          REVERTING -----> IDLE          <---- alignement ? --+
5                      oui : RESOLVING (cascade)
```

Ce qu'il faut retenir

La cascade est une boucle, et elle doit terminer. Chaque tour retire au moins trois gemmes; le plateau est fini; donc elle termine. Écrivez ce raisonnement en commentaire — c'est la seule chose qui garantit que votre jeu ne bouclera pas indéfiniment sur un plateau pathologique.

Le compteur de cascade (GetCascadeCount) sert au score *et* au son : la troisième cascade d'affilée sonne plus haut que la première.

Ce qu'il faut retenir

FindHintMove cherche un coup possible. Deux usages, et le second est le vrai : montrer un indice au joueur qui hésite, et **détecter qu'il n'y a plus aucun coup** — auquel cas Shuffle() rebat le plateau.

Sans cette détection, une grille sans coup possible bloque la partie sans rien dire. Le joueur clique partout et rien ne se passe.

23.5 Étape 18 — les quatre écrans

Applications/Gemcrush/src/Gemcrush/Ui/NkGemScreens.h

```
1 enum class GemScreen : uint8 { Splash, Menu, Map, Game };
```

Ce qu'il faut retenir

Une énumération, et une seule variable qui dit où l'on est. Le dessin principal aiguille dessus.

Sans elle, on finit avec quatre booléens — menuOuvert, enPartie, surLaCarte... — qui peuvent être vrais en même temps, ce qui n'a aucun sens et arrive tout de même. Une énumération rend l'état impossible à contredire.

La structure du dessin, une fois pour toutes

```
1 void OnDraw(nkgui::NkGuiDrawList &dl) override {
2     switch (mEcran) {
3         case GemScreen::Splash: mSplash.Dessiner(dl, ...); break;
4         case GemScreen::Menu:   DessinerMenu(dl);         break;
5         case GemScreen::Map:    DessinerCarte(dl);         break;
6         case GemScreen::Game:   DessinerPartie(dl);        break;
7     }
8 }
```

Le piège

Le switch doit couvrir tous les cas, et le compilateur peut le vérifier — si vous n'ajoutez pas de default. Avec un default, ajouter un cinquième écran compile en silence et n'affiche rien.

Un default sur une énumération fermée transforme une erreur de compilation en écran noir.

23.6 Étape 19 — la progression

Trente niveaux, des étoiles, un fichier de sauvegarde.

Ce qu'il faut retenir

Les cas de recette du jeu vérifient trois choses sur la progression, et la troisième est celle qu'on oublie : elle est **écrite, relue à neuf, et effaçable**.

Le piège

« **La progression est relue à neuf** » n'est pas la même chose que « **la progression se relit** ».

Un aller-retour qui compare le fichier à lui-même certifie une amputation reproductible : si l'enregistrement perd un champ, la relecture le perd aussi, et le contrôle reste vert.

Un contrôle d'aller-retour porte *deux* exigences :

1. **stabilité** : le second enregistrement égale le premier;
2. **conservation** : le contenu attendu est encore là, *champ par champ*, sur un document qui les porte tous.

C'est la seconde qu'on oublie, et sans elle « octet pour octet » certifie une perte reproductible.

Ce qu'il faut retenir

Les trente niveaux sont déterministes : même graine, même plateau. Sans cela, aucun cas de banc n'est écrivable — on ne peut pas vérifier qu'un niveau est jouable si son plateau change à chaque lancement.

Déterministe ne veut pas dire prévisible pour le joueur : il ne rejoue pas le même niveau assez souvent pour le mémoriser, et le hasard qui compte est celui de ses propres coups.

23.7 Étape 20 — le son, sans un seul fichier

Ce qu'il faut retenir

NkGemAudio synthétise ses sons : il calcule les échantillons au lieu de lire un .wav. Le jeu ne charge donc **aucun fichier à l'exécution** — « à l'identique sur les six plateformes sans chaîne de copie d'assets ».

Le prix : les sons sont simples. Le gain : rien à copier, rien à emballer, rien à précharger sur le Web, et un jeu qui démarre pareil partout.

Pour un jeu de puzzle, l'échange est bon. Pour un jeu à musique, il ne le serait pas — et c'est une décision à prendre au début, pas à la fin.

Le principe, en une fonction

```
1 // Une note = une sinusoïde qui s'éteint. Trois lignes de calcul par
2 // échantillon, et zéro octet sur le disque.
3 for (uint32 i = 0; i < n; ++i) {
4     const float32 t = (float32)i / (float32)frequenceEchantillonnage;
5     const float32 enveloppe = NkExp(-t * declin);
6     echantillons[i] = NkSin(2.f * NK_PI_F * hauteur * t) * enveloppe * volume;
7 }
```

Le piège

2.f * NK_PI_F, et non NK_TAU_F. Cette dernière n'existe pas dans NKMath — et NK_PI tout court non plus : il n'y a que NK_PI_F et NK_PI_D, suffixés par leur précision. Le dépôt a déjà payé cette absence. Écrire une constante « qui devrait exister » donne une erreur de compilation, ce qui est le bon cas ; l'écrire dans un cours donnerait un exemple qui ne compile pas.

Le piège

L'enveloppe n'est pas un détail esthétique. Une sinusoïde qui s'arrête net produit un *clic* audible — une discontinuité dans le signal. Le NkExp décroissant la supprime. C'est la première chose qu'on entend quand on l'oublie, et la dernière à laquelle on pense.

23.8 Étape 21 — le didacticiel

Un fichier entier — 353 lignes — pour apprendre au joueur à échanger deux gemmes.

Ce qu'il faut retenir

Ce n'est pas du superflu. Un jeu dont la règle tient en une phrase n'est pas un jeu qu'on comprend en une phrase : il faut montrer le geste, une fois, au bon moment, puis *se taire*. Le didacticiel de GemCrush s'efface seul et devient muet au niveau 7 — il ne répète pas ce qui est acquis. C'est cette seconde moitié qui est difficile à écrire, et c'est elle qui distingue une aide d'un harcèlement.

23.9 Étape 22 — les cas de recette d'un jeu animé

Cas	Ce qu'il prouve
carte : dessin et clic d'accord	le clic tombe là où le nœud est dessiné
les 30 niveaux sont déterministes	même graine, même plateau
étoiles : accord affichage/verdict	ce qui s'affiche est ce qui est calculé
ouverture : 4,1 s, sautable en 2 appuis	le splash ne bloque pas
didacticiel : s'efface seul, muet au niveau 7	il ne harcèle pas
progression : écrite, relue à neuf, effaçable	la sauvegarde tient

Ce qu'il faut retenir

Deux de ces cas ne parlent pas de la règle du jeu. « Le clic tombe là où le nœud est dessiné » et « ce qui s'affiche est ce qui est calculé » vérifient un *accord entre deux chemins*. C'est la forme de banc la plus rentable quand un jeu grossit : dès que deux chemins doivent produire le même verdict — le dessin et le clic, l'affichage et le calcul — l'un est le contrôle de l'autre. S'ils appliquent chacun leur copie de la règle, rien ne signale le jour où elles divergent.

Le piège

Un jeu animé ne se teste pas en appelant ses fonctions. Un cas qui appelle TrySwap directement prouve la *règle* d'échange ; il ne prouve pas qu'un clic y mène.

Faites partir le cas de la *position du clic*, passez par la conversion en case, et regardez le plateau. Ce détournement est exactement ce qui rend visible une géométrie fautive — et une géométrie fautive est le défaut numéro un de ce genre de jeu.

Ce chapitre en bref

Un jeu animé ajoute neuf étapes au squelette du chapitre précédent, et la première est la plus importante : une **machine à phases**, sans laquelle un clic reçu pendant une animation corrompt l'état. Chercher les alignements et les supprimer sont deux opérations séparées, sinon les croix se comptent mal. Les écrans se tiennent par une énumération, jamais par des booléens qui se contredisent. Le son synthétisé supprime toute chaîne d'assets — une décision à prendre au début. Et les bancs les plus utiles ne vérifient plus la règle mais *l'accord entre deux chemins*.

Questions de cours

1. Pourquoi une machine à phases est-elle indispensable dès qu'il y a animation ?
2. Pourquoi NkGem ne fait-elle plus son propre test de survol ?
3. Pourquoi séparer « trouver les alignements » de « les supprimer » ?
4. Que se passe-t-il si l'on met un default dans le switch des écrans ?
5. Quelles sont les deux exigences d'un contrôle d'aller-retour ?
6. Pourquoi les trente niveaux doivent-ils être déterministes ?

Exercice pratique

Construisez le cœur d'un jeu d'alignement, dans l'ordre des étapes 14 à 17.

A. La gemme et le plateau 8×8 — *sans dessiner*. **B.** La recherche d'alignements de trois ou plus, horizontaux et verticaux, qui *rend* le résultat. **C.** La chute et le remplissage. **D.** La machine à phases, et un TrySwap qui refuse hors IDLE.

Vérifiez par un banc : un plateau fabriqué à la main dont vous connaissez les alignements attendus, et le plateau d'après la chute, comparé case par case. Seulement ensuite, dessinez.

Exercice de démonstration

Prouvez que votre détection d'alignements ne trouve pas ce qui n'existe pas.

Construisez un plateau *sans aucun alignement* : la fonction doit rendre zéro. Puis — et c'est la partie qui compte — un plateau avec deux gemmes identiques côte à côte seulement : elle doit rendre *encore* zéro.

Pourquoi ce second cas : une fonction qui compterait les paires au lieu des triplets passerait le premier test et raterait le second. Un cas qui doit rendre zéro prouve autant qu'un cas qui doit trouver.

Ajoutez enfin le cas en croix : une gemme dans une ligne et une colonne alignées. Comptez les gemmes retirées — pas les alignements.

Exercice de logique

Votre jeu se fige de temps en temps, plateau figé, aucun clic ne répond.

1. En vous appuyant sur la machine à phases, énumérez trois façons dont cela peut arriver

— une par phase où l'on peut rester coincé.

2. Pour chacune, dites quelle information affichée à l'écran permettrait de la distinguer des deux autres en une seconde.
3. Un collègue propose de forcer le retour à IDLE après trois secondes dans n'importe quelle phase. Expliquez pourquoi ce correctif est plus dangereux que le défaut — et ce qu'il rend impossible à découvrir.

Construire un lecteur, de A à Z

Un lecteur est la première application où vous ne *décidez* de rien : vous ouvrez un fichier, vous le transformez, vous l'affichez. Tout le travail est dans la transformation — et c'est justement là que se cachent les erreurs qui donnent une image inclinée, une forme d'onde plate ou une vidéo qui saccade.

Nous construisons **trois** lecteurs — images, son, vidéo — avec le même squelette. Neuf étapes, et vous les avez tous les trois.

	Ce qu'il lit	Lignes
NkAudioPlayer	un son, et sa forme d'onde	263
NkVideoPlayer	une vidéo, et son audio	684
NkImageDemo	une image, et <i>ce qu'il ne faut pas faire</i>	2 369

Ce qu'il faut retenir

Les étapes 1 et 2 du chapitre 21 ne changent pas : le dossier, le `.jenga`, le `main.cpp` de douze lignes. Ce qui change commence à la troisième — et ce chapitre ne réécrit que ce qui change.

Aux dépendances du `.jenga`, ajoutez selon le lecteur : `NkImage` (déjà là), `NkAudio` (déjà là), et `NkMedia` pour la vidéo.

24.1 Étape 1 — ouvrir un fichier, et le dire quand ça rate

Le squelette d'ouverture, commun aux trois

```

1 bool OnGuiInit() override {
2     const NkString chemin = mArguments.Size() > 1 ? mArguments[1] : NkString();
3     if (chemin.Empty()) {
4         mMessage = "Usage : NkLecteur <fichier>";
5         return true; // on démarre quand meme, et on le DIT
6     }
7     if (!Ouvrir(chemin)) {
8         mMessage = NkFormat("Impossible de lire : {0}", chemin);
9         return true; // fenetre ouverte, message lisible
10    }
11    return true;
12 }
```

Ce qu'il faut retenir

On démarre même quand le fichier manque. Rendre `false` fermerait l'application avant qu'un message soit lisible — l'utilisateur voit une fenêtre qui clignote et disparaît, et il n'a aucun moyen d'apprendre pourquoi.

Un repli se crie, mais ne verrouille pas l'utilisateur dehors.

Le piège

Ne déduisez pas le format de l'extension. Un .jpg peut être un PNG renommé, et un .avi peut contenir n'importe quoi.

Pour la vidéo, NkMedia offre NkMediaProbe : on *sonde* le fichier avant de l'ouvrir. Pour l'image, le décodeur lit la signature des premiers octets. Dans les deux cas, **l'en-tête est la vérité, l'extension n'est qu'un indice.**

24.2 Étape 2 — le son : réduire un million d'échantillons

Un fichier de trois minutes en 44 100 Hz contient environ huit millions d'échantillons. Votre fenêtre en fait mille de large. Il faut donc en montrer huit mille par colonne — et le *comment* n'est pas un détail.

Applications/NkAudioPlayer/src/main.cpp

```
1 static void ComputePeaks(const audio::AudioSample &s, int32 cols,
2                          NkVector<WavePeak> &out) {
3     out.Clear();
4     if (!s.data || s.frameCount <= 0 || cols <= 0) { return; }
5     out.Resize((uint64)cols);
6     const int32 ch = s.channels > 0 ? s.channels : 1;
7
8     for (int32 c = 0; c < cols; ++c) {
9         const uint64 f0 = (uint64)((double)c / cols * (double)s.frameCount);
10        uint64 f1 = (uint64)((double)(c + 1) / cols * (double)s.frameCount);
11        if (f1 <= f0) { f1 = f0 + 1; }
12        if (f1 > (uint64)s.frameCount) { f1 = (uint64)s.frameCount; }
13
14        float32 lo = 1.0f, hi = -1.0f;
15        for (uint64 f = f0; f < f1; ++f) {
16            float32 m = 0.0f;
17            for (int32 k = 0; k < ch; ++k) { m += s.data[f * (uint64)ch + (uint64)k]; }
18            m /= (float32)ch;
19            if (m < lo) { lo = m; }
20            if (m > hi) { hi = m; }
21        }
22        if (hi < lo) { lo = 0.0f; hi = 0.0f; }
23        out[(uint64)c].lo = lo;
24        out[(uint64)c].hi = hi;
25    }
26 }
```

Ce qu'il faut retenir

Chaque colonne garde le minimum et le maximum de sa tranche, puis se dessine comme un trait vertical de l'un à l'autre.

Pourquoi pas la moyenne : une onde sonore oscille autour de zéro. La moyenne d'un passage fort et celle d'un silence sont toutes deux proches de zéro — vous obtiendriez une ligne plate, techniquement juste et parfaitement inutile.

Ce qui porte l'information, c'est l'**amplitude**, donc l'écart. C'est un cas d'école du piège général : *un agrégat peut être exact et ne rien décrire.*

Le piège

Trois détails de cette fonction, et chacun est un défaut évité :

lo = 1.0f, hi = -1.0f — à l'envers des bornes attendues. C'est la seule initialisation correcte d'une recherche de minimum et de maximum : la première valeur rencontrée les remplace tous les deux. Initialiser à zéro serait faux — un signal entièrement négatif rendrait **hi = 0**, un maximum qui n'existe pas dans les données.

if (f1 <= f0) f1 = f0 + 1; — sur un fichier plus court que la fenêtre, deux colonnes tomberaient sur la même tranche vide.

if (hi < lo) { lo = 0; hi = 0; } — la tranche vide. Sans elle, une colonne sans échantillon garderait 1 et -1, et dessinerait un trait pleine hauteur à partir de rien.

24.3 Étape 3 — dessiner, et comprendre ce qu'on dessine

Applications/NkAudioPlayer/src/main.cpp

```
1 static void PushQuad(NkVector<NkVertex> &v, float32 x0, float32 y0,
2                     float32 x1, float32 y1, const NkColor2D &col) {
3     NkVertex a{x0, y0, 0, 0, col.r, col.g, col.b, col.a};
4     NkVertex b{x1, y0, 0, 0, col.r, col.g, col.b, col.a};
5     NkVertex c{x1, y1, 0, 0, col.r, col.g, col.b, col.a};
6     NkVertex d{x0, y1, 0, 0, col.r, col.g, col.b, col.a};
7     v.PushBack(a); v.PushBack(b); v.PushBack(c);
8     v.PushBack(a); v.PushBack(c); v.PushBack(d);
9 }
```

Ce qu'il faut retenir

Quatre coins, six sommets. Le GPU ne connaît que des triangles : un rectangle est le triangle *abc* plus le triangle *acd*. Les sommets *a* et *c* sont donc envoyés *deux fois* — c'est normal, c'est le prix d'une liste de triangles.

C'est exactement ce que fait `AddRectFilled` pour vous. Le voir écrit une fois explique tout le reste : pourquoi un dessin coûte en *nombre de sommets*, et pourquoi regrouper les rectangles dans un même tampon est ce qui rend un rendu rapide.

Ce qu'il faut retenir

Mille colonnes, c'est mille rectangles — soit un seul appel de dessin si vous les poussez tous dans le même tampon avant de vider. Une forme d'onde n'est donc pas coûteuse, contrairement à ce qu'on craint.

24.4 Étape 4 — la vidéo : la chaîne complète

Six lignes, et elles sont dans l'en-tête du lecteur du dépôt :

Applications/NkVideoPlayer/src/main.cpp

```
1 // 1. NkWindow          -> fenetre native (Win/Linux/macOS/...)
2 // 2. NkRenderWindow     -> cible de rendu 2D NKCanvas
3 // 3. NkVideoReader      -> decode chaque image en RGBA
4 // 4. NkTexture          -> recoit les pixels RGBA de l'image courante (Update)
5 // 5. NkSprite           -> affiche la texture, mise a l'echelle
```



```
6 // 6. boucle cadencee au fps de la video (Clear -> Draw -> Display)
```

Ce qu'il faut retenir

Il n'y a aucune API «vidéo» dans le rendu. Une vidéo, pour l'écran, c'est une texture qu'on remplace à chaque image.

Le décodage — H.264, MJPEG, conteneurs — est entièrement dans NKMedia, et le rendu n'en sait rien. C'est la séparation la plus profitable de tout ce cours : *le module qui produit des pixels ignore le module qui les montre.*

Applications/NkVideoPlayer/src/main.cpp

```
1 media::NkVideoReader reader;
2 // ... ouverture ...
3 const media::NkVideoReaderInfo &vin = reader.Info(); // dimensions, fps, duree
4
5 // puis, une fois par image :
6 frameTex.Update(fr.rgbData(), (uint32)fr.width, (uint32)fr.height, 0, 0);
```

Ce qu'il faut retenir

Une seule ligne de téléversement, appelée une fois par image. Tout le reste du fichier — 684 lignes — est de la cadence, de la navigation et de l'étalonnage de couleur.

24.5 Étape 5 — la cadence, et pourquoi elle n'est pas un Sleep

Le principe

```
1 mHorloge += deltaTime;
2 const float32 periode = 1.f / vin.fps;
3 while (mHorloge >= periode && reader.ReadFrame(fr)) {
4     mHorloge -= periode;
5     frameTex.Update(fr.rgbData(), fr.width, fr.height, 0, 0);
6 }
```

Ce qu'il faut retenir

Un **while**, pas un **if**. Si une image a mis trop longtemps à arriver — une image-clé coûteuse, un coup de charge système — il faut en rattraper plusieurs d'un coup, sinon la vidéo prend un retard qu'elle ne rattrapera jamais.

Et `mHorloge -= periode` plutôt que `mHorloge = 0` : remettre à zéro perdrait le reste et ferait dériver la lecture de quelques pour cent.

Le piège

N'attendez jamais avec un **Sleep** dans la boucle. Le lecteur cesserait de répondre : pas de redimensionnement, pas de clic, et sur le Web l'onglet gèle.

La cadence se tient en *comptant le temps écoulé*, pas en le dépensant.

24.6 Étape 6 — traiter les pixels, et à quel endroit

Le lecteur du dépôt applique un étalonnage — noir et blanc, sépia, table .cube — et l’endroit où il le fait est un enseignement à lui seul :

Applications/NkVideoPlayer/src/main.cpp (commentaire d’origine)

```
1 // cablage est le meme : transformer les pixels juste avant frameTex.Update
```

Ce qu’il faut retenir

Un traitement d’image se branche entre le décodeur et la texture. Ni avant — le décodeur ne doit rien savoir de vos couleurs — ni après : une fois téléversés, les pixels sont sur le GPU et ne vous appartiennent plus.

Cette position est un *point d’extension* : ajouter un filtre ne demande de toucher à rien d’autre. Écrivez-le en commentaire à cet endroit, c’est ce qui guidera le prochain lecteur du fichier.

24.7 Étape 7 — l’image : le pas de ligne

Ce qu’il faut retenir

Le pas de ligne (*stride*) n’est pas la largeur en octets. Un décodeur peut aligner chaque ligne sur quatre octets : une image de 641 pixels en RGB occupe $641 \times 3 = 1923$ octets utiles, mais 1924 par ligne.

Copier en supposant « largeur $\times$ octets » décale chaque ligne d’un octet de plus que la précédente : **l’image sort inclinée**, en escalier, avec les couleurs mélangées.

Le piège

Le défaut est intermittent, et c’est ce qui le rend cher. Il ne se voit que si la largeur n’est pas un multiple de l’alignement — donc jamais sur une image de 640 ou 1024 pixels, et toujours sur celle de l’utilisateur.

Corriger en redimensionnant l’image à une largeur multiple de quatre « marche » et est le pire des correctifs : il masque la cause et la laisse en place pour le prochain format.

24.8 Étape 8 — rendre les pixels, et les quatre cas

Ce qu’il faut retenir

La question n’est jamais « comment libère-t-on ceci ? » mais « qui l’a alloué ? ».

Quatre cas, et ils ne se devinent pas : une NkImage sur la pile se détruit seule ; une allouée par Alloc() se rend par Free() sans delete ; un tampon rendu par un encodeur vient de NkAlloc et se rend par NkFree ; et Free() appelé sur un objet de la pile exécute une libération *sur une adresse de pile*.

Ce dernier a réellement fait planter une démo du dépôt.

24.9 Étape 9 — la barre de transport

Ce qu'elle doit porter, et rien de plus

```
1 // [<<] [ Lecture / pause ] [>>] #####----- 01:23 / 04:56
```

Ce qu'il faut retenir

La position affichée vient du lecteur, jamais d'un compteur à vous. Un compteur local dérive dès la première image sautée, et l'écart grandit sans jamais se corriger. Après un déplacement dans le fichier, c'est encore le lecteur qui dit où il est réellement arrivé — il se cale sur l'image-clé précédente, qui n'est pas l'endroit demandé.

24.10 Ce que NkImageDemo enseigne malgré lui

NkImageDemo fait 2 369 lignes pour afficher une image — neuf fois le lecteur audio. La raison est visible dans ses fichiers :

Fichier	Ce que c'est
Render/GLContext	un contexte OpenGL, à la main
Render/GLRenderer2D	un renderer 2D, à la main
Render/Texture2D	une texture, à la main
Render/FontAtlas	un atlas de police, à la main
Render/SafeArea	la zone sûre, à la main
ViewerApp	le visualiseur proprement dit

Ce qu'il faut retenir

Cinq de ses six fichiers réimplémentent ce que NKCanvas porte déjà.

C'est le doublon dans sa forme la plus coûteuse : le code marche, personne n'a tort, et pourtant chaque correction apportée à NkRenderWindow ne parviendra jamais à GLRenderer2D.

Avant d'écrire un mécanisme, cherchez qui le porte déjà — et commencez par la couche du dessous. Un grep lancé dans son propre dossier ne trouve jamais ce que la bibliothèque porte.

Le piège

Les trois lecteurs du dépôt sont antérieurs à NkCanvasApp. Ils ouvrent la fenêtre eux-mêmes, dans nkmain, et tiennent leur propre boucle.

Ce n'est pas une raison de les ignorer — ils montrent exactement ce que la coquille vous épargne — mais c'en est une de ne pas les recopier tels quels. Construisez les vôtres sur NkCanvasGuiApp, comme aux étapes 1 et 2 du chapitre 21.

Ce chapitre en bref

Un lecteur ne décide de rien : tout son travail est une transformation. On ouvre en sondant le contenu, pas l'extension, et l'on démarre même quand ça rate — avec un message lisible. Réduire un signal pour l'afficher demande le *minimum* et le *maximum*, jamais la

moyenne. Un rectangle plein est deux triangles et six sommets. Une vidéo, pour le rendu, n'est qu'une texture qu'on remplace, cadencée par un `while` qui rattrape et jamais par un `Sleep`; le traitement des pixels se branche entre le décodeur et la texture. Le pas de ligne n'est pas la largeur. Et `NkImageDemo` montre ce que coûte un doublon : 2 369 lignes dont cinq sixièmes existaient déjà un étage plus bas.

Questions de cours

1. Pourquoi démarre-t-on l'application même quand le fichier est illisible ?
2. Pourquoi une forme d'onde s'affiche-t-elle en min/max et non en moyenne ?
3. Pourquoi `lo` est-il initialisé à 1 et `hi` à -1 ?
4. Combien de sommets pour un rectangle plein, et pourquoi pas quatre ?
5. Pourquoi la cadence vidéo emploie-t-elle un `while` et non un `if` ?
6. Où se branche un traitement d'image, et pourquoi pas ailleurs ?

Exercice pratique

Construisez votre lecteur audio en suivant les étapes 1 à 3 et 9, sur `NkCanvasGuiApp`. Il doit : accepter un fichier en argument, afficher un message lisible quand il n'y arrive pas, dessiner la forme d'onde en min/max, jouer le son, et montrer un curseur de lecture qui avance.

Aucun appel à `NkWindow` ni à `NkRenderWindow` dans votre code. Comptez les lignes, et comparez aux 263 du lecteur du dépôt : dites lesquelles ont disparu et pourquoi. Puis ajoutez la vidéo : étapes 4 à 6.

Exercice de démonstration

Prouvez que la moyenne ne convient pas.

Ajoutez un second mode d'affichage qui dessine la *moyenne* de chaque tranche au lieu du min/max, et une touche pour basculer. Chargez un morceau réel et capturez les deux images.

Ce que la démonstration doit établir : non pas que la seconde est « moins jolie », mais qu'elle est *plate* — donc qu'elle ne distingue pas un passage fort d'un passage faible. Mesurez-le : affichez l'écart entre la plus grande et la plus petite valeur affichée, dans les deux modes.

Exercice de logique

Une vidéo à 25 images par seconde dure 4 minutes. Le décodage d'une image prend en moyenne 12 ms, le téléversement 2 ms, le dessin 1 ms.

1. La lecture tiendra-t-elle la cadence ? Justifiez par le calcul.
2. Une image sur cinquante prend 300 ms à décoder (image-clé). La lecture saccade-t-elle ? Que faudrait-il changer, *sans* accélérer le décodeur ?
3. Vous ajoutez un filtre couleur qui coûte 9 ms par image. Où le mettre — et si vous n'aviez le droit de l'appliquer qu'une image sur deux, la vidéo serait-elle acceptable ? Argumentez sur ce que voit l'œil, pas sur les chiffres seuls.

Construire un éditeur de texte, puis de code

Jusqu'ici, l'utilisateur *regardait*. Ici il **modifie**. Tout change : il faut un curseur, une sélection, un défilement, une annulation — et il faut décider comment le texte est rangé en mémoire, décision qui gouvernera tout le reste.

Nous construisons l'éditeur en douze étapes. Aux étapes 1 à 9 vous avez un éditeur de texte utilisable ; aux étapes 10 à 12 il devient un éditeur de code.

La référence est `NkCodeEditor.h` du dépôt : **5 012 lignes**, plus `NkSyntax.h` (1 422) pour la coloration.

Ce qu'il faut retenir

Les étapes 1 et 2 du chapitre 21 ne changent pas. Ajoutez seulement, aux dépendances du `.jenga`, `NKGui` et `NKFileSystem` — ils y sont déjà — et créez `src/Editeur/`.

25.1 Étape 1 — décider comment ranger le texte

C'est la première décision, et elle est irréversible en pratique.

Représentation	Bon marché	Cher
une seule chaîne	lire tout d'un coup	insérer au milieu
vecteur de lignes	éditer une ligne, dessiner	insérer une ligne au début
tampon à trou (<i>gap</i>)	frappe au clavier	sauts lointains

Ce qu'il faut retenir

Choisissez d'après l'opération la plus fréquente, pas d'après la plus élégante. Un éditeur passe son temps à *dessiner des lignes* et à *éditer celle du curseur* ; il n'insère presque jamais un million de lignes en tête. `NKCode` prend donc le **vecteur de lignes**. L'opération chère n'arrive jamais.

Applications/NKCode/src/NKCode/Editor/NkCodeEditor.h

```
1 // Une ligne = caracteres bruts (PAS de '\0').
2 using NkCodeLine = NkVector<char>;
```

Le piège

« **PAS de '\0'** » : le commentaire d'origine le dit en majuscules. Une ligne est une plage `[begin, end)`, pas une chaîne C. Passer `line.Data()` à une fonction qui attend une chaîne terminée lirait au-delà de la ligne, jusqu'au premier zéro rencontré dans le tas — donc parfois rien d'anormal, et par-

fois un plantage. Le pire des deux mondes.

25.2 Étape 2 — le document, et où vit l'état

Applications/NKCode/src/NKCode/Editor/NkCodeEditor.h

```
1 struct NkCodeDoc {
2     NkVector<NkCodeLine> lines;
3     int32 curLine = 0, curCol = 0;    // curseur (col = index caractere)
4     int32 selLine = 0, selCol = 0;    // ancre de selection (== curseur si vide)
5     // ... defilement, etat d'annulation
6 };
```

L'en-tête explique *pourquoi* le curseur est là et pas ailleurs :

Applications/NKCode/src/NKCode/Editor/NkCodeEditor.h

```
1 // etat curseur/selection/scroll EMBARQUE dans Le document -> survit au
2 // changement d'onglet ET au re-dock (etat hors-frame).
```

Ce qu'il faut retenir

L'interface est en mode immédiat : elle ne garde rien d'une image à l'autre. Un état rangé dans le widget disparaîtrait au premier changement d'onglet — vous reviendriez sur votre fichier, curseur au début, défilement en haut.

Ce qui doit survivre à la disparition du widget vit dans le document. C'est la règle générale du mode immédiat, et l'éditeur en est le cas le plus visible.

Ce qu'il faut retenir

La sélection est une **ANCRE** plus le curseur, pas un couple début/fin. L'ancre ne bouge pas pendant Maj+flèches ; le curseur, oui. La sélection est l'intervalle entre les deux, dans l'ordre où il se trouve.

Stocker « début » et « fin » obligerait à savoir lequel des deux bouge — et c'est exactement l'information que l'ancre porte gratuitement.

25.3 Étape 3 — charger et enregistrer

La forme

```
1 bool Charger(const NkString &chemin, NkCodeDoc &doc);    // decoupe sur \n
2 bool Enregistrer(const NkString &chemin, const NkCodeDoc &doc);
```

Le piège

Les fins de ligne : \n ou \r\n. Un fichier Windows en porte deux octets, un fichier Unix un seul.

Découper naïvement sur \n laisse un \r à la fin de chaque ligne. Il est invisible à l'affichage — et il ressort à l'enregistrement, doublé, puis quadruplé au suivant.

Décidez à l'ouverture, retenez ce que le fichier employait, et réécrivez la même chose. Un

éditeur qui change les fins de ligne rend un diff illisible.

25.4 Étape 4 — ne dessiner que ce qui se voit

Le calcul, avant toute boucle de dessin

```
1 const float32 hauteurLigne = mPolice.LineHeight();
2 const int32 premiere = (int32)(doc.scrollY / hauteurLigne);
3 const int32 combien = (int32)(zone.h / hauteurLigne) + 2; // +2 : Les moities
4 const int32 derniere = NkMin(premiere + combien, (int32)doc.lines.Size());
5
6 for (int32 i = premiere; i < derniere; ++i) { DessinerLigne(dl, doc.lines[i], ...); }
```

Ce qu'il faut retenir

Le coût du dessin devient indépendant de la taille du fichier : cinquante lignes, qu'il y en ait cent ou cent mille.

Le +2 n'est pas de la superstition : la première ligne visible est souvent coupée en haut, la dernière en bas. Sans lui, une bande vide apparaît au bord pendant le défilement.

Ce qu'il faut retenir

L'en-tête de NKCode assume une approximation et l'écrit : « largeur H approximée sur les lignes visibles ». La largeur de défilement horizontal n'est calculée que sur ce qui est à l'écran, parce que mesurer cent mille lignes à chaque image coûterait plus que le bénéfice. **Une approximation écrite est une décision; une approximation tue est un défaut en attente.**

25.5 Étape 5 — le curseur, et la conversion des coordonnées

Les deux conversions, et il faut les deux

```
1 // ecran -> document : ou l'utilisateur a-t-il cliqué ?
2 int32 ligne = (int32)((p.y - zone.y + doc.scrollY) / hauteurLigne);
3 int32 col = ColonneLaPlusProche(doc.lines[ligne], p.x - zone.x + doc.scrollX);
4
5 // document -> ecran : ou dessiner le caret ?
6 float32 x = zone.x - doc.scrollX + LargeurTexte(doc.lines[curLine], 0, curCol);
7 float32 y = zone.y - doc.scrollY + curLine * hauteurLigne;
```

Le piège

Les deux conversions doivent être inverses l'une de l'autre, et rien ne le vérifie. Si le dessin soustrait le défilement et que le clic ne l'ajoute pas, la sélection est juste en haut du fichier et fausse plus bas — donc juste pendant tout le développement.

C'est le même piège que le liseré de sélection peint en espace document au lieu d'espace écran : deux chemins qui convertissent entre les mêmes espaces **doivent partager leur fonction de conversion**, pas la recopier.

Ce qu'il faut retenir

ColonneLaPlusProche arrondit au caractère le plus proche, pas au caractère survolé. Cliquer sur la moitié droite d'une lettre place le curseur *après* elle — c'est ce que fait tout éditeur, et ne pas le faire donne une impression de décalage d'un caractère.

25.6 Étape 6 — la saisie

Entrée	Effet sur le document
caractère	insérer à (curLine, curCol), ++curCol
Entrée	couper la ligne en deux, descendre
Retour arr.	si curCol > 0 effacer; sinon <i>fusionner</i>
Suppr	symétrique, vers l'avant
flèches, Home, End	déplacer, et poser l'ancre <i>sauf</i> avec Maj

Ce qu'il faut retenir

Les deux cas de bord sont la fusion de lignes. Retour arrière en colonne 0 ne supprime pas un caractère : il colle la ligne courante à la précédente et place le curseur à la jointure. Suppr en fin de ligne fait le symétrique. Ce sont les deux seules opérations qui changent le *nombre* de lignes en dehors d'Entrée. Elles sont oubliées une fois sur deux, et le symptôme est « la touche ne fait rien en début de ligne ».

Le piège

Toute saisie avec sélection non vide commence par supprimer la sélection. Taper une lettre alors qu'un bloc est sélectionné doit le remplacer. Oublier cette règle donne un éditeur où la sélection ne sert qu'à copier — et le défaut ne se voit pas tant qu'on ne teste que la frappe simple.

25.7 Étape 7 — le défilement automatique

Ce qu'il faut retenir

Après toute opération qui bouge le curseur, ramenez-le dans la vue. Sinon la flèche « bas » sort de l'écran et l'utilisateur tape à l'aveugle. La règle est simple et il faut les deux moitiés : si le curseur est au-dessus de la vue, faire remonter le défilement à sa ligne ; s'il est en dessous, le descendre jusqu'à ce que sa ligne soit la dernière visible.

25.8 Étape 8 — l'annulation

Applications/NKCode/src/NKCode/Editor/NkCodeEditor.h

```
1 struct UndoState { /* ... */};
```

Ce qu'il faut retenir

Deux stratégies, et la plus simple est souvent la bonne.

Instantané : une copie du document avant chaque groupe de modifications. Simple, sûr, coûteux en mémoire — mais un fichier source fait quelques dizaines de kilo-octets, donc cent états tiennent dans quelques méga-octets.

Journal d'opérations : « inséré x en (ligne, colonne) », et on sait le défaire. Économe, et bien plus difficile à rendre juste : chaque opération a besoin de son inverse exact, et un seul inverse faux corrompt tout l'historique sans prévenir.

Commencez par l'instantané. Ne passez au journal que si une *mesure* — pas une intuition — montre que la mémoire pose problème.

Le piège

Un groupe d'annulation n'est pas une frappe. Si chaque caractère tapé crée un état, l'utilisateur devra faire trente annulations pour effacer un mot.

Le regroupement se fait sur un *critère* : une pause de frappe, un changement de ligne, un changement de nature d'opération (saisie puis effacement). Ce critère est un choix de conception, pas un détail — et c'est celui que les utilisateurs remarquent en premier.

Ce qu'il faut retenir

L'instantané doit contenir le curseur. Annuler et retrouver le curseur ailleurs est désorientant : l'utilisateur veut revenir à *l'endroit* où il était, pas seulement au texte.

25.9 Étape 9 — la gouttière

Numéros de ligne à gauche, alignés à droite dans une colonne dont la largeur dépend du nombre de chiffres du fichier.

Ce qu'il faut retenir

Calculez la largeur sur le plus grand numéro, pas sur une constante. C'est exactement le piège de la table des matières de ce livre : une boîte dimensionnée pour trois chiffres écrase le texte à partir de la ligne 1000.

Et le défaut n'apparaît que sur les gros fichiers — donc jamais pendant que vous écrivez l'éditeur.

Vous avez un éditeur de texte utilisable. Les trois étapes suivantes en font un éditeur de code.

25.10 Étape 10 — décrire un langage plutôt que le programmer

Applications/NKCode/src/NKCode/Editor/NkSyntax.h

```
1 struct NkLangSpec {
2     NkString name;
3     NkVector<NkString> exts;           ///< ".css", ".scss"... (minuscules)
4     NkString lineComment;           ///< "//", "#", "--", ";" ("\" = aucun)
5     NkString blockOpen, blockClose; ///< "/*", "*/" ou "<!--", "-->"
6     NkString strChars;               ///< delimitateurs de chaines
7     NkVector<NkString> keywords;     ///< tries
8     NkVector<NkString> types;        ///< tries
9 };
```

Ce qu'il faut retenir

Un langage n'est pas du code : c'est une donnée. Ajouter la coloration du Lua, du Rust ou du CSS ne demande pas d'écrire un colorateur — seulement de remplir un `NkLangSpec`. Un colorateur écrit langage par langage aurait donné dix fonctions qui divergent; une description en donne dix *données* qui ne peuvent pas diverger, puisqu'un seul code les lit.

C'est le test de votre étape 10 : si ajouter un second langage vous fait écrire une ligne de code, elle n'est pas finie.

Ce qu'il faut retenir

keywords et types sont **triés** : la recherche est dichotomique. Sur un langage à 90 mots-clés, testés pour chaque mot de chaque ligne visible, la différence entre parcours linéaire et dichotomie se voit à l'œil pendant le défilement.

25.11 Étape 11 — tokeniser une ligne, sans allouer

Applications/NKCode/src/NKCode/Editor/NkSyntax.h

```
1 // Tokenise UNE ligne et emet des plages colorees [begin,end) via un callback
2 // Sans allocation : gap-filling (les zones non colorees -> couleur texte)
```

Ce qu'il faut retenir

Deux décisions de performance dans deux lignes de commentaire.

Une ligne à la fois : la coloration se fait au moment de dessiner, sur les lignes visibles seulement. Rien n'est stocké, donc rien ne peut être périmé après une modification — pas de cache à invalider, pas de cache qui ment.

Sans allocation, par remplissage de trous : au lieu de produire la liste de toutes les plages — y compris celles qui gardent la couleur par défaut — le tokeniseur n'émet que les plages *colorées*, et ce qui reste entre elles prend la couleur du texte.

25.12 Étape 12 — l'état d'entrée de ligne

Le piège

Un commentaire de bloc ne tient pas dans une ligne. /* ouvert ligne 10 et fermé ligne 40 : les trente lignes du milieu sont dans le commentaire, et *rien dans leur texte ne le dit*. Un tokeniseur strictement par ligne ne peut pas le savoir : il lui faut un **état d'entrée de ligne** — « suis-je déjà dans un commentaire ? » — calculé depuis le début du fichier, ou depuis un point de reprise connu.

C'est la seule vraie difficulté de la coloration syntaxique, et c'est celle qu'on découvre en dernier, quand un fichier entier devient vert.

Ce qu'il faut retenir

Le remède pratique : garder, pour chaque ligne, un octet d'état de sortie. Le calculer une fois au chargement, et le mettre à jour à partir de la ligne modifiée jusqu'à ce que l'état redevienne identique à l'ancien — ce qui arrive presque toujours en une ou deux lignes. Recalculer tout le fichier à chaque frappe marche aussi, et se voit dès dix mille lignes.

Ce chapitre en bref

Un éditeur repose sur douze étapes, et les quatre premières décident du reste : la représentation du texte (choisie d'après l'opération *la plus fréquente*), le lieu de l'état d'édition (dans le document, jamais dans le widget), les fins de ligne (retenues et restituées), et le dessin restreint aux lignes visibles. Viennent ensuite les deux conversions écran/document, qui **doivent être inverses** et donc partager leur code ; la saisie, dont les deux cas de bord sont les fusions de lignes ; l'annulation, par instantané avant tout journal. Passer au code n'ajoute qu'une chose : la coloration — et le bon réflexe est de *décrire* chaque langage par une donnée. La seule vraie difficulté qui reste est l'état d'entrée de ligne.

Questions de cours

1. Citez trois représentations d'un texte éditable et l'opération que chacune rend chère.
2. Pourquoi l'état du curseur ne peut-il pas vivre dans le widget ?
3. Pourquoi une ligne ne se termine-t-elle pas par '\0' ?
4. Que doivent faire Retour arrière en colonne 0 et Suppr en fin de ligne ?
5. Pourquoi les deux conversions écran/document doivent-elles partager leur code ?
6. Pourquoi un tokeniseur par ligne a-t-il besoin d'un état d'entrée de ligne ?

Exercice pratique

Construisez votre éditeur en suivant les douze étapes, sans en sauter une.

Étape A (1 à 9) — le texte. Ouvrir un fichier, l'afficher avec ses numéros de ligne, un curseur, les flèches, Home, End, la saisie, Entrée, Retour arrière, Suppr, la sélection à la souris et au clavier, le défilement à la molette, l'annulation, et l'enregistrement.

Étape B (10 à 12) — le code. Un NkLangSpec pour un langage, et la coloration des mots-clés, types, chaînes, nombres, commentaires de ligne.

Le critère de réussite de l'étape B est binaire : ajoutez un *second* langage sans écrire une seule ligne de code de coloration. Si vous n'y arrivez pas, votre étape 10 n'était pas assez

« décrite ».

Exercice de démonstration

Prouvez que votre éditeur ne dessine que les lignes visibles.

Ajoutez un compteur de lignes réellement dessinées, affiché à l'écran. Ouvrez un fichier de dix lignes, puis un de cent mille lignes générées. Le compteur doit être *le même* dans les deux cas.

Mesurez ensuite le temps d'une image avec NkChrono, sur les deux fichiers, dix fois chacun.

Ce que la démonstration doit établir : que l'écart entre les deux temps reste sous le bruit de mesure — et il vous faut donc *d'abord* mesurer ce bruit, en comparant deux séries sur le *même* fichier. Sans ce témoin, vous ne pouvez pas dire si un écart de 3 % est un effet ou du hasard.

Exercice de logique

Un utilisateur tape Bonjour, appuie sur Entrée, tape monde, puis fait trois annulations.

1. Décrivez l'état obtenu selon trois politiques de regroupement : par caractère, par mot, par ligne.
2. Laquelle vous paraît la meilleure ? Défendez-la, puis donnez le cas où elle surprend l'utilisateur.
3. Le document est ensuite *rechargé depuis le disque* par une action extérieure. L'historique d'annulation doit-il survivre ? Argumentez dans les deux sens, tranchez, et dites ce que votre choix rend impossible.

Construire un mini terminal, de A à Z

Un terminal semble simple : on tape une commande, on lit la réponse. C'est trompeur. Il doit lancer un programme, lui parler *pendant* qu'il tourne, comprendre un langage de contrôle vieux de cinquante ans, et ne jamais figer l'interface — sur trois systèmes qui ne s'y prennent pas du tout de la même façon.

Dix étapes, en trois niveaux qui se justifient l'un l'autre. À la fin de l'étape 3 vous avez quelque chose d'utile ; à la fin de l'étape 10, un vrai terminal.

Fichier	Rôle	Lignes
NkProcess.h	lancer une commande, lire sa sortie	171
NkPty.{h,cpp}	tenir un shell <i>interactif</i> vivant	496
NkTerm.h	interpréter le flux, en faire une grille	631

Ce qu'il faut retenir

Les étapes 1 et 2 du chapitre 21 ne changent pas. Ajoutez NKThreading aux dépendances — il y est déjà — et créez src/Terminal/.

Niveau 1 — lancer une commande sans geler

26.1 Étape 1 — le fil d'arrière-plan et la file

Applications/NKCode/src/NKCode/Project/NkProcess.h

```
1 // Start(cmd) lance la commande sur un THREAD d'arrière-plan (stdout+stderr
2 // fusionnés) ; Le thread pousse chaque ligne dans une file THREAD-SAFE ;
3 // L'UI appelle Drain() chaque frame pour récupérer les nouvelles lignes sans
4 // geler.
```

La forme, en trois méthodes

```
1 class NkProcess {
2     public:
3         bool Start(const NkString &cmd);
4         void Drain(NkVector<NkString> &out); // vide la file, rend la main
5         bool Running() const;
6     private:
7         void FilLecteur(); // tourne dans un AUTRE fil
8         NkMutex mMutex;
9         NkVector<NkString> mLignes;
10 };
```

Ce qu'il faut retenir

C'est le patron « producteur lent, interface fluide », et il vaut bien au-delà des terminaux. Un fil d'arrière-plan lit le programme et pousse ses lignes dans une file protégée par un mutex. L'interface, à chaque image, appelle `Drain` : elle prend ce qui est arrivé, ou rien, et rend la main immédiatement.

Elle n'*attend* jamais. Une compilation de trois minutes défile ligne à ligne pendant que la fenêtre reste vivante.

Applications/NKCode/src/NKCode/Project/NkProcess.h

```
1 void Drain(NkVector<NkString> &out) {
2     threading::NkScopedLock<NkMutex> lk(mMutex);
3     for (/* ... */) { out.PushBack(mLines[i]); }
4     // ... la file est videe
5 }
```

Le piège

Le verrou est pris pendant la copie, pas pendant la lecture du programme. C'est la règle du chapitre `NKThreading` appliquée : *verrouiller le temps de toucher la donnée, pas le temps de travailler*.

Si le fil gardait le verrou pendant qu'il attend la ligne suivante — ce qui peut durer des secondes — l'interface bloquerait à chaque image en essayant de le prendre. Vous auriez payé un fil pour obtenir un programme séquentiel qui saccade.

Une exception à la règle du dépôt, et elle est écrite

`NkProcess.h` inclut `<cstdio>`, alors que le dépôt interdit la bibliothèque standard. Son en-tête le dit et le justifie :

« WRAPPER MAISON DÉSIGNÉ pour les PIPES process : les appels `_popen/fgets/fgetc` de bas niveau sont volontairement conservés dans cette famille de wrappers (pas d'équivalent `NkFile` pour un flux de processus). »

Comparez au lecteur vidéo du chapitre 23, qui incluait les mêmes en-têtes *sans rien dire*. Le code est identique ; ce qui diffère est décisif.

Une exception écrite est une décision — on peut la discuter, la lever, la dater. Une exception muette est une dette que le prochain lecteur prendra pour un exemple à suivre.

26.2 Étape 2 — afficher, et savoir s'arrêter

Dans OnTick

```
1 void OnTick(float32) override {
2     NkVector<NkString> arrivees;
3     mProcessus.Drain(arrivees);
4     for (uint64 i = 0; i < arrivees.Size(); ++i) { mLignes.PushBack(arrivees[i]); }
5     if (mLignes.Size() > kMaxLignes) { /* jeter les plus anciennes */ }
6 }
```

Le piège

Bornez l'historique dès la première version. Une commande bavarde — une compilation complète, un `find` / — produit des centaines de milliers de lignes. Sans plafond, la mémoire monte jusqu'à ce que le système tue le programme.

Le symptôme est « le terminal a planté au bout de dix minutes », et la cause est à l'endroit qu'on n'a pas écrit.

Ce qu'il faut retenir

NkProcess lance une commande, la lit, et meurt. C'est déjà utile — un bouton « Construire » qui affiche la sortie en direct — et c'est tout ce dont une partie des applications ont besoin.

Ne passez au niveau 2 que si vous voulez un *vrai* terminal.

Niveau 2 — un shell qui reste vivant

26.3 Étape 3 — pourquoi un tube ne suffit pas

Un `cd` lancé par `NkProcess` ne change rien : le processus meurt, et le suivant repart du même dossier. Les variables aussi s'oublient. Il faut garder *le même* shell.

Applications/NKCode/src/NKCode/Project/NkPty.h

```
1 // Contrairement a NkProcess (un _popen jetable par commande), NkPty Lance UN
2 // shell (powershell / wsl -d <distro> / bash / cmd) attache a un pseudo-console
3 // Windows (ConPTY). On ECRIT les frappes clavier dans son entree et on LIT en
4 // continu sa sortie (avec sequences VT : couleurs, déplacement curseur, clear).
5 // => Le shell affiche lui-meme son invite ; `clear` efface ; pas de boite de
6 // saisie separee.
```

Ce qu'il faut retenir

« **Le shell affiche lui-même son invite** » est le renversement complet.

Le débutant écrit une zone de saisie, un bouton *Exécuter*, et une zone de sortie. C'est une *imitation* de terminal : l'invite est fausse, les couleurs sont perdues, `vim` ne marche pas, `clear` n'efface rien.

Un vrai terminal ne fabrique rien. Il transmet les frappes au shell et affiche ce que le shell renvoie — *y compris son invite*. Il n'est pas l'auteur de ce qu'on voit; il en est le verre.

26.4 Étape 4 — le pseudo-terminal

Un programme se comporte différemment selon qu'il écrit dans un fichier ou vers un humain : vers un fichier, il retire les couleurs et l'interactivité. Un tube ordinaire ressemble à un fichier — d'où des sorties ternes et `vim` qui refuse de démarrer.

Un **pseudo-terminal** est un tube qui se *présente* comme un terminal : il a une taille en lignes et colonnes, il accepte d'être redimensionné, et le programme d'en face le croit.

Applications/NKCode/src/NKCode/Project/NkPty.h

```
1 bool Start(const NkString &cmdline, int16 cols, int16 rows,
2           const NkString &cwd = NkString());
```

```

3 void Write(const char *data, usize len); // Les frappes clavier
4 void Write(const char *s);
5 void Resize(int16 cols, int16 rows); // quand le panneau change de taille

```

Ce qu'il faut retenir

Quatre fonctions. C'est toute l'interface — et c'est ce qui rendra l'étape 5 possible.

Le piège

Resize n'est pas cosmétique. Le programme d'en face décide sa mise en page d'après la taille qu'on lui a déclarée. Oubliez de l'informer, et htop dessinera pour 80 colonnes dans une fenêtre qui en fait 200 — ou débordera dans l'autre sens.

Un terminal qui ne réagit pas au redimensionnement n'est pas « moins abouti » : il est faux dès la première fenêtre de taille inhabituelle.

26.5 Étape 5 — isoler le code de plateforme

Applications/NKCode/src/NKCode/Project/NkPty.cpp (structure)

```

1 #if defined(_WIN32)
2     // ConPTY : CreatePseudoConsole, PROC_THREAD_ATTRIBUTE_PSEUDOCONSOLE
3 #else
4     #if defined(__APPLE__)
5         // <util.h>
6     #else
7         // <pty.h> -- forkpty
8     #endif
9 #endif

```

Ce qu'il faut retenir

Trois systèmes, trois mécanismes, un seul en-tête. NkPty.h n'expose que Start, Write, Read, Resize : rien de ce qui suit ne connaît ConPTY ni forkpty.

L'en-tête dit d'ailleurs *pourquoi* le .cpp existe : « isolée dans NkPty.cpp pour ne pas polluer les en-têtes GUI avec les macros de windows.h ». Ce n'est pas une préférence de style — windows.h définit des macros comme min, max et None qui cassent le code qui les inclut par accident.

Là où le portage est le plus sale, l'interface doit être la plus étroite.

Le piège

None n'est pas une macro théorique. X11 la définit à 0, et NKGui l'emploie comme énumérateur — ce qui a empêché NKGui de compiler sous Linux pendant des mois. Le correctif a dû être posé à la porte d'entrée du module, parce qu'un undef par fichier ne protège pas ceux qui l'incluent.

Niveau 3 — comprendre ce que le shell renvoie

26.6 Étape 6 — la grille de cellules

Le shell ne renvoie pas du texte : il renvoie du texte *mêlé d'ordres*. ESC[31m veut dire « à partir d'ici, écris en rouge », ESC[2J « efface l'écran », ESC[10;5H « place le curseur ligne 10, colonne 5 ».

Applications/NKCode/src/NKCode/Project/NkTerm.h

```
1 struct NkTermCell {
2     uint32 cp = 0x20;           // codepoint Unicode (espace)
3     NkColor fg = {204, 204, 204, 255};
4     NkColor bg = {0, 0, 0, 0};  // a==0 => fond par défaut
5 };
6
7 class NkTerm {
8     public:
9         using Line = NkVector<NkTermCell>;
10        NkTerm() { Resize(80, 24); }
11        void Resize(int16 cols, int16 rows);
12        // ... Feed(octets), TotalLines(), LineAt(i)
13    };
```

Ce qu'il faut retenir

L'émulateur transforme un flux en état. Il consomme des octets et maintient une grille : à chaque case, un caractère et deux couleurs. Le dessin, ensuite, est trivial — une boucle sur des cellules, en police à chasse fixe.

C'est la séparation qui rend l'affaire faisable : d'un côté un automate qui ne dessine rien et *se teste sans fenêtre*; de l'autre un dessin qui ne comprend rien et ne peut donc pas se tromper.

Vous reconnaissez le découpage règles/écran du chapitre 21, appliqué à un terminal.

Ce qu'il faut retenir

Notez `bg = {0,0,0,0}` avec le commentaire « `a==0` ⇒ fond par défaut ». Une cellule ne dit pas « fond noir » : elle dit « je n'ai pas d'avis ».

C'est ce qui permet au terminal de suivre le thème de l'éditeur. Une valeur **absente** et une valeur **noire** sont deux choses différentes, et les confondre est l'erreur qui rend une interface non thématable.

26.7 Étape 7 — l'automate, et son état persistant

Les quatre états, et ils suffisent pour commencer

```
1 enum class Etat : uint8 {
2     TEXTE,           // on recopie les caractères dans la grille
3     ESC,             // on vient de lire 0x1B
4     CSI,             // on a lu ESC[ ; on accumule les paramètres
5     OSC              // on a lu ESC] ; on consomme jusqu'au terminateur
6 };
```

Le piège

Une séquence d'échappement peut arriver coupée en deux. La lecture rend ce qui est disponible : ESC[3 dans un paquet, 1m dans le suivant.

Un analyseur qui traite chaque paquet indépendamment affichera [3 puis 1m en clair, aléatoirement, une fois de temps en temps — le défaut qui ne se reproduit jamais chez le développeur et toujours chez l'utilisateur.

L'automate doit donc garder son état d'un appel à l'autre. C'est le même problème que le commentaire de bloc du chapitre 24 : *un analyseur travaille sur un flux, pas sur des morceaux*.

Ce qu'il faut retenir

Ce qu'il faut interpréter en premier, et dans cet ordre : SGR (les couleurs), ED (effacer l'écran), CUP (placer le curseur), puis les déplacements relatifs.

Tout le reste se CONSOMME et s'ignore — mais se consomme. Une séquence inconnue qu'on laisse passer s'affiche en clair et salit la sortie; une séquence inconnue qu'on avale proprement ne se voit pas.

26.8 Étape 8 — dessiner la grille

La boucle, et elle est courte

```
1 for (int32 l = 0; l < visibles; ++l) {
2     const NkTerm::Line &ligne = mTerm.LineAt(premiere + l);
3     for (int32 c = 0; c < mTerm.Cols(); ++c) {
4         const NkTermCell &cel = ligne[c];
5         if (cel.bg.a != 0) { dl.AddRectFilled(CaseRect(l, c), cel.bg); }
6         if (cel.cp != 0x20) { dl.AddText(..., cel.cp, cel.fg); }
7     }
8 }
```

Ce qu'il faut retenir

Une police à chasse fixe est obligatoire, et ce n'est pas esthétique : la grille suppose que toutes les cellules ont la même largeur. Avec une police proportionnelle, les colonnes ne s'alignent plus et tout dessin ASCII se disloque.

Et l'on saute l'espace et le fond transparent : sur un écran de terminal typique, les trois quarts des cellules sont vides. Les dessiner quand même multiplie le travail par quatre pour un résultat identique.

26.9 Étape 9 — le clavier, et le sens du flux

Ce que fait la saisie – et ce qu'elle ne fait PAS

```
1 void OnKey(const NkKeyEvent &k) {
2     // On ECRIT dans le shell. On ne touche JAMAIS a la grille.
3     if (k.text) { mPty.Write(k.text); }
4     else if (k.code == NK_ENTER) { mPty.Write("\r"); }
5     else if (k.code == NK_LEFT) { mPty.Write("\x1b[D"); }
6     // ...
7 }
```

Ce qu'il faut retenir

La saisie n'écrit jamais dans la grille. Elle envoie au shell, le shell renvoie l'écho, et c'est cet écho qui apparaît.

Écrire soi-même le caractère tapé donnerait un double affichage — le vôtre puis celui du shell — et casserait les mots de passe, que le shell n'écho justement pas.

Le flux va dans un seul sens : clavier → shell → grille → écran. Toute boucle courte est un défaut.

Ce qu'il faut retenir

Les touches de direction s'envoient comme des séquences : ESC[D pour la gauche. Le même langage sert dans les deux sens — c'est pourquoi votre automate de l'étape 7 vous aura appris à les écrire.

26.10 Étape 10 — redimensionner, et l'historique

Dans OnLayout

```
1 const int16 cols = (int16)(zone.w / mLargeurCar);
2 const int16 rows = (int16)(zone.h / mHauteurLigne);
3 if (cols != mTerm.Cols() || rows != mTerm.Rows()) {
4     mTerm.Resize(cols, rows); // la grille
5     mPty.Resize(cols, rows); // ET le shell -- Les DEUX
6 }
```

Le piège

Les deux, toujours. Redimensionner la grille sans prévenir le shell donne un affichage juste sur des données calculées pour l'ancienne taille : les lignes se replient au mauvais endroit et l'écran devient illisible après le premier programme plein écran.

Ce qu'il faut retenir

L'historique (**scrollback**) se conserve au redimensionnement, mais l'écran se recadre. C'est ce que fait `NkTerm::Resize` du dépôt : « conserve le scrollback; recadre l'écran ». Reflower tout l'historique à la nouvelle largeur est ce que font les terminaux les plus aboutis, et c'est un chantier à soi seul — ne le promettez pas dans votre première version.

Ce chapitre en bref

Un terminal se construit en trois niveaux qui se justifient l'un l'autre : *lancer sans geler* (fil d'arrière-plan, file protégée, Drain par image, historique borné); *tenir un shell vivant* via

un pseudo-terminal — c'est le shell qui affiche son invite, pas vous, et il faut lui déclarer sa taille; *interpréter* le flux VT en une grille de cellules, avec un automate qui garde son état d'un paquet à l'autre. Le code spécifique à chaque système tient dans un seul .cpp derrière quatre fonctions. Et le flux va dans un seul sens : ce que vous tapez part au shell, ce qui s'affiche revient de lui.

Questions de cours

1. Pourquoi Drain plutôt qu'attendre la fin de la commande?
2. Pourquoi faut-il borner l'historique dès la première version?
3. Pourquoi un tube ordinaire ne suffit-il pas pour un shell interactif?
4. Que se passe-t-il si l'on n'appelle jamais Resize — sur la grille? sur le shell?
5. Pourquoi la saisie n'écrit-elle pas dans la grille?
6. Pourquoi une cellule a-t-elle un fond d'alpha nul plutôt qu'un fond noir?

Exercice pratique

Construisez votre mini terminal en suivant les dix étapes, dans l'ordre. Chaque niveau doit se lancer avant que vous n'attaquiez le suivant.

Niveau 1 (étapes 1–2). Une zone de saisie, un bouton, une zone de sortie. *Vérifiez que l'interface reste fluide* pendant une commande de trente secondes.

Niveau 2 (étapes 3–5). Un shell persistant. `cd` doit désormais avoir un effet sur la commande suivante — c'est votre critère de réussite, et il est binaire.

Niveau 3 (étapes 6–10). Interprétez au minimum SGR et ED. Le reste des séquences se consomme et s'ignore — mais se consomme, jamais ne s'affiche.

Exercice de démonstration

Prouvez que votre analyseur survit à une séquence coupée.

Écrivez un banc *sans fenêtre* — vous pouvez, puisque NkTerm ne dessine rien — qui alimente votre émulateur avec `"ESC[31mROUGE"` en un seul morceau, puis avec le même texte découpé à **chaque octet**, un appel par caractère. Comparez la grille obtenue, cellule par cellule.

Les deux doivent être identiques. Faites ensuite varier le point de coupure sur *toutes* les positions possibles et vérifiez les n résultats.

Pourquoi la variation exhaustive : une coupure à un seul endroit peut tomber juste par chance. C'est la coupure *au milieu du nombre* — `ESC[3` puis `1m` — qui casse les analyseurs naïfs, et rien ne garantit que votre essai unique tombe dessus.

Exercice de logique

Votre terminal affiche parfois `[0m` en clair au milieu de la sortie — environ une fois sur cent lancements, jamais au même endroit.

1. Énumérez au moins trois causes possibles.
2. Pour chacune, dites quelle mesure la distinguerait des autres — une mesure qui donne un résultat *différent* selon la cause, sinon elle ne tranche rien.
3. Un collègue propose de filtrer `[0m` avant l'affichage. Expliquez pourquoi ce correctif est plus dangereux que le défaut, en une phrase.

Projet final : NkAtelier

Vous avez maintenant traversé toute la pile : la fenêtre, les événements, le rendu 2D, les widgets, les images, les polices, le son, la vidéo, le réseau. Ce dernier chapitre ne présente aucune API nouvelle. Il fait une seule chose, et c'est la plus utile : **il assemble**.

Nous allons construire, du fichier de build jusqu'à l'exécutable qui s'affiche, une petite application appelée NkAtelier. Elle tient en un seul `main.cpp`, elle ne dépend d'**aucun fichier externe** — pas une police, pas une image, pas un son — et elle prouve, à l'écran, que cinq modules coopèrent.

27.1 Ce que nous allons construire

NkAtelier est une fenêtre avec trois panneaux.

1. **Panneau « Image »** — une image *fabriquée à l'exécution* par les primitives de dessin de NkImage, redimensionnée en vignette, envoyée au backend et affichée par le widget Image. Un curseur change une couleur ; l'image est refabriquée et ré-uploadée.
2. **Panneau « Son »** — trois boutons qui jouent trois sons *synthétisés* par AudioGenerator, sur le bus « UI », avec une case à cocher pour couper ce bus.
3. **Panneau « Vidéo »** — au démarrage, l'application *écrit* une petite vidéo MJPEG dans un fichier AVI, puis la *relit* image par image et l'affiche, cadencée, dans l'interface. Boutons lecture, pause et retour au début.

Le tout avec du texte rendu par une police embarquée.

Pourquoi aucun actif externe

C'est un choix pédagogique délibéré, et il vaut d'être expliqué.

Une application d'apprentissage qui exige un `logo.png` et un `clic.wav` échoue à la première exécution, parce que le répertoire courant n'est pas celui qu'on croit. Le lecteur passe alors une heure sur un problème de chemin relatif au lieu d'apprendre le moteur — c'est exactement ce que la démo du dépôt contourne avec sa liste de chemins candidats ([Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:294-299](#)).

Ici, chaque ressource est **produite** :

- la police vient de `NkFontEmbedded` (chapitre 6) ;
- l'image est dessinée par `NkImage::FillRect` et consorts (chapitre 5) ;
- le son est synthétisé par `AudioGenerator::GenerateTone` (chapitre 7) ;
- la vidéo est écrite par `NkVideoWriter` avant d'être relue (chapitre 8).

L'application ne peut pas échouer faute de fichier. C'est également ce qui la rend facile à donner à quelqu'un d'autre.

27.2 Étape 0 — l'arborescence

Un module ou une application du dépôt, c'est un dossier avec un `.jenga`. Créez :

Arborescence à créer

```
1 Applications/NkAtelier/  
2   NkAtelier.jenga  
3   src/  
4     main.cpp
```

C'est tout. Un seul fichier source; c'est suffisant pour ce que nous faisons, et cela évite de mélanger l'apprentissage du moteur avec celui d'une organisation de projet.

27.3 Étape 1 — le fichier de build

27.3.1 Le contenu

Le fichier ci-dessous est modelé sur `Applications/NKGuiDemo/NKGuiDemo.jenga`, allégé de ce dont nous n'avons pas besoin (la réflexion, la détection de Vulkan) et augmenté de ce qu'il nous faut (`NKAudio`, `NKMedia`).

Applications/NkAtelier/NkAtelier.jenga — écrit pour le cours, modelé sur NKGuiDemo.jenga:41-95

```
1 #!/usr/bin/env python3  
2 # -*- coding: utf-8 -*-  
3 """NkAtelier – projet final du cours : image + texte + son + video dans une UI NKGui."""  
4  
5 from Jenga import *  
6 from jengaconfig import *  
7  
8  
9 with project("NkAtelier"):  
10     windowedapp()  
11     language("C++")  
12     cppdialect("C++17")  
13     location(".")  
14  
15     files(["src/**/*.cpp"])  
16  
17     nkentseudependson(  
18         ["NKGui", "NKCanvas", "NKWindow", "NKEvent", "NKGlad",  
19          "NKFont", "NKImage", "NKAudio", "NKMedia",  
20          "NKStream", "NKFileSystem", "NKLogger", "NKMath", "NKTime",  
21          "NKContainers", "NKMemory", "NKCore", "NKPlatform", "NKThreading"],  
22         extra_includes=["src", "%{NKGlad.location}/include"],  
23     )  
24  
25     objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}-%{prj.name}")  
26     targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}-%{prj.name}")  
27  
28     # ===== Windows =====  
29     with filter("system:Windows && !options:windows-runtime=uwp && !system:XboxSeries &&  
!system:XboxOne"):  
30         windowedapp()  
31         usetoolchain(TC_WINDOWS)  
32         defines(["WIN32_LEAN_AND_MEAN", "_UNICODE", "UNICODE"])  
33         links([  
34             "user32", "gdi32", "opengl32", "dwmapi", "shell32",
```

```

35         "uuid", "ole32",
36         "d3d11", "d3d12", "dxgi", "d3dcompiler",
37     ])
38
39     # ===== Linux (XLib) =====
40     with filter("system:Linux && options:linux-backend=xlib || system:Linux &&
!options:linux-backend && !options:headless"):
41         windowedapp()
42         usetoolchain("clang-native")
43         defines(["NKENTSEU_FORCE_WINDOWING_XLIB_ONLY"])
44         links(["pthread", "X11", "Xext", "GL"])
45
46     # ===== macOS =====
47     with filter("system:macOS"):
48         windowedapp()
49         usetoolchain("clang-native")
50         frameworks(["Cocoa", "QuartzCore", "OpenGL"])
51
52     with filter("config:Debug"):
53         defines(["_DEBUG", "DEBUG"])
54         optimize("Off")
55         symbols(True)
56     with filter("config:Release"):
57         defines(["NDEBUG"])
58         optimize("Speed")
59         symbols(False)

```

27.3.2 Ce qu'il faut comprendre dans ce fichier

- `windowedapp()` — application fenêtrée, pas console. Sous Windows, cela évite qu'une console noire s'ouvre derrière votre fenêtre;
- `nkentseudependson([...])` — la liste des modules. Elle *doit* contenir tout ce que vous incluez, transitivement ou non;
- `extra_includes` — NKGLad est nécessaire parce que le backend OpenGL de NkCanvas expose ses en-têtes dans les vôtres;
- les blocs `with filter(...)` — la configuration par plateforme. Ne les inventez pas : recopiez-les d'une application existante qui fonctionne.

La liste des dépendances n'est pas décorative

Un module absent de `nkentseudependson` ne provoque pas une erreur claire « module manquant ». Il produit soit une erreur de compilation sur un en-tête introuvable — encore lisible —, soit une erreur d'édition de liens sur des symboles non résolus, qui ne dit pas quel module ajouter.

Le réflexe : quand un symbole `nkentseu::quelquechose` manque à l'édition de liens, cherchez dans quel module il est déclaré, et ajoutez ce module.

27.3.3 Enregistrer le projet dans l'espace de travail

Un `.jenga` isolé n'est pas construit. Il faut l'inclure depuis le descripteur racine, `Nkentseu.jenga`, à côté des autres applications :

Nkentseu.jenga — ajout, forme identique à Nkentseu.jenga:1394-1396

```
1 with include("Applications/NkAtelier/NkAtelier.jenga"):
2
3     pass
```

27.4 Étape 2 — le squelette : fenêtre, contexte, backend, police

27.4.1 Les inclusions

Applications/NkAtelier/src/main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:7-31)

```
1 #include "NKWindow/NKWindow.h"
2 #include "NKWindow/NKMain.h"
3 #include "NKEvent/NKWindowEvent.h"
4 #include "NKEvent/NKMouseEvent.h"
5 #include "NKEvent/NKKeyboardEvent.h"
6
7 #include "NKCanvas/Core/NKContextDesc.h"
8 #include "NKCanvas/Core/NKGraphicsApi.h"
9 #include "NKCanvas/Renderer/Targets/NKRenderWindow.h"
10 #include "NKCanvas/UI/NKGuiCanvasBackend.h" // pont NKGui -> NKCanvas (header-only)
11
12 #include "NKGui/NKGui.h" // TOUJOURS L'en-tête parapluie de NKGui
13 #include "NKImage/Core/NKImage.h"
14 #include "NKAudio/NKAudio.h"
15 #include "NKMedia/Video/NKVideoWriter.h"
16 #include "NKMedia/Video/NKVideoReader.h"
17
18 #include "NKTime/NKClock.h"
19 #include "NKMemory/NKUniquePtr.h"
20 #include "NKMemory/NKMemory.h"
21 #include "NKLogger/NKLog.h"
22 #include "NKMath/NKColor.h"
23
24 using namespace nkentseu;
25 using namespace nkentseu::nkgui;
26 using namespace nkentseu::renderer;
```

Deux remarques d'ordre général.

D'abord, on inclut l'en-tête parapluie de NKGui (NKGui/NKGui.h) et jamais un en-tête interne — le chapitre 1 en a donné la raison : c'est lui qui neutralise les macros de X11 sous Linux.

Ensuite, on n'inclut pas d'en-tête parapluie pour NKMedia : il n'en existe pas. On prend les deux sous-en-têtes précis dont on a besoin.

27.4.2 Fenêtre et cible de rendu

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:240-263)

```
1 int nkmain(const NkEntryState &state) {
2     (void)state;
3
4     NKWindow window;
5     NKWindowConfig cfg;
6     cfg.title = "NkAtelier - projet final du cours Nkentseu";
7     cfg.width = 1180;
```



```

8     cfg.height = 760;
9     cfg.centered = true;
10    cfg.resizable = true;
11    if (!window.Create(cfg))
12        return -1;
13
14    NkContextDesc desc;
15    desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
16    if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
17    #if defined(NKENTSEU_PLATFORM_WINDOWS)
18        desc.api = NkGraphicsApi::NK_GFX_API_DX11;
19    #else
20        desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
21    #endif
22    }
23    auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
24    if (!target || !target->IsValid())
25        return -1;

```

Le point d'entrée s'appelle `nkmain` et vient de `NKWindow/NKMain.h` : c'est lui qui gère les différences de démarrage entre plateformes.

Si rien ne s'affiche, changez de backend

Le tableau du chapitre 1 rappelait que, à la date de sa rédaction, seul le backend OpenGL était validé à l'exécution; DX11 et le rasteriseur logiciel avaient un écran noir signalé, Vulkan et DX12 des plantages ([Kernel/Runtime/NKCanvas/ROADMAP.md](#), lignes 185-200 et 601-607).

Ces états sont datés et le dépôt bouge — mais si votre fenêtre s'ouvre noire, **la première chose à essayer n'est pas de relire votre code** : c'est de remplacer `NK_GFX_API_DX11` par `NK_GFX_API_OPENGL` et de relancer. Cinq secondes, et cela élimine la moitié des causes possibles.

27.4.3 Contexte NKGui et backend

main.cpp — écrit pour le cours (modèle : `NKGuiDemo/main.cpp:264-279`)

```

1     auto ctxPtr = memory::NkMakeUnique<NkGuiContext>();
2     if (!ctxPtr)
3         return -1;
4     NkGuiContext &ctx = *ctxPtr;
5     ctx.Init(static_cast<int32>(cfg.width), static_cast<int32>(cfg.height));
6     SetCurrentContext(&ctx);
7
8     NkGuiCanvasBackend backend;
9     if (!backend.Init(target->GetRenderer()))
10        return -1;

```

L'ordre compte : le backend reçoit le renderer, donc le renderer doit exister. Et, souvenez-vous du chapitre 5, toutes les textures doivent être créées *après* l'initialisation du renderer — ce qui est le cas ici, puisque nos uploads viendront plus bas.

27.4.4 La police

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:281-288)

```
1  auto fontPtr = memory::NkMakeUnique<NkGuiFont>();
2  if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 17.f)) {
3      fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
4  }
5  ctx.font = fontPtr.Get();
6  if (fontPtr->Valid()) {
7      backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
8      fontPtr->atlasH);
9  } else {
10     logger.Error("[NkAtelier] aucune police embarquee disponible : textes invisibles");
11 }
```

Trois lignes utiles et un repli, exactement comme au chapitre 6. Le `else` n'est pas superflu : sans lui, une police manquante donnerait une interface entièrement muette, sans le moindre indice.

27.5 Étape 3 — les événements et la boucle

27.5.1 Le branchement des entrées

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:326-347)

```
1  auto &events = NkEvents();
2  bool running = true;
3  events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
4  });
5  events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
6      ctx.input.mousePos = {static_cast<float32>(e->GetX()),
7      static_cast<float32>(e->GetY())};
8  });
9  events.AddEventCallback<NkMouseButtonPressEvent>([&](NkMouseButtonPressEvent *e) {
10     if (e->GetButton() == NkMouseButton::NK_MB_LEFT)
11         ctx.input.mouseDown[0] = true;
12 });
13 events.AddEventCallback<NkMouseButtonReleaseEvent>([&](NkMouseButtonReleaseEvent *e) {
14     if (e->GetButton() == NkMouseButton::NK_MB_LEFT)
15         ctx.input.mouseDown[0] = false;
16 });
17 events.AddEventCallback<NkMouseWheelVerticalEvent>([&](NkMouseWheelVerticalEvent *e) {
18     ctx.input.wheel += static_cast<float32>(e->GetDeltaY());
19 });
```

C'est le strict minimum pour des boutons, des curseurs et du défilement. Si vous ajoutez un champ de saisie, il vous faudra en plus `NkTextInputEvent` et la traduction des touches d'édition — la démo en donne le modèle complet ([Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:367-405](#)).

27.5.2 La boucle, et l'invariant d'ordre

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:429-470)

```
1  NkClock clock;
2  uint32 lastW = 0, lastH = 0;
3
4  while (running && window.IsOpen()) {
5      float32 dt = clock.Tick().delta;
6      if (dt <= 0.f) dt = 1.f / 60.f;
7      if (dt > 0.1f) dt = 0.1f;          // garde-fou : fenetre deplacee, machine chargee
8
9      while (NkEvent *ev = NkEvents().PollEvent()) {
10         (void)ev;                      // Le travail est fait par les callbacks
11     }
12     if (!running)
13         break;
14
15     // Synchroniser la swapchain sur la taille de la FENETRE.
16     const math::NkVec2u wsz = target->GetWindow().GetSize();
17     if (wsz.x > 0 && wsz.y > 0 && (wsz.x != lastW || wsz.y != lastH)) {
18         target->OnResize(wsz.x, wsz.y);
19         lastW = wsz.x;
20         lastH = wsz.y;
21     }
22     const math::NkVec2u sz = target->GetSize();
23     if (sz.x > 0 && sz.y > 0) {
24         ctx.viewW = static_cast<int32>(sz.x);
25         ctx.viewH = static_cast<int32>(sz.y);
26     }
27
28     ctx.BeginFrame(dt);
29     // ... les trois panneaux, sections suivantes ...
30     ctx.EndFrame();
31
32     target->Clear();
33     backend.Submit(ctx.dl, sz.x, sz.y);
34     backend.Submit(ctx.dlOverlay, sz.x, sz.y);
35     target->Display();
36 }
```

L'invariant d'ordre, une dernière fois

événements → BeginFrame(dt) → widgets → EndFrame() → Clear → Submit ×2 → Display.

Les deux Submit ne sont pas une coquille : le second envoie la couche des popups et menus, qui doit se dessiner par-dessus tout le reste. L'oublier donne une application où les menus déroulants passent derrière les panneaux — un bug qu'on cherche longtemps parce qu'il ressemble à un problème de profondeur.

Le bornage de dt mérite aussi un mot. Quand l'utilisateur déplace la fenêtre, le système bloque la boucle : le *delta time* suivant peut valoir plusieurs secondes. Toutes vos animations feraient alors un bond. Un plafond à 100 ms coûte une ligne et supprime une catégorie entière de bizarreries.

27.6 Étape 4 — l'image (NKImage)

27.6.1 Fabriquer l'image

Nous n'avons pas de fichier; nous dessinons. Voici la fonction, écrite pour ce cours, qui produit une image et l'envoie au backend :

main.cpp — écrit pour le cours (API : `NkImage.h:232, 587-765, 404, 775`)

```
1 // Fabrique une image 512x512, en tire une vignette, et l'uploade sous `texId`.
2 // Renvoie false si quoi que ce soit echoue.
3 static bool ConstruireVignette(NkGuiCanvasBackend &backend, uint32 texId, float32 teinte,
4                               int32 &outW, int32 &outH) {
5     // 1) Une image sur la PILE : rien a liberer.
6     NkImage source;
7     if (!source.Create(512, 512, math::NkColor(18, 20, 26, 255), 4))
8         return false;
9
10    // 2) Dessin CPU (chapitre 5). Toutes ces primitives sont inline dans l'en-tete.
11    const uint8 r = static_cast<uint8>(60.f + 195.f * teinte);
12    const uint8 g = static_cast<uint8>(200.f - 120.f * teinte);
13    source.FillRect(40, 40, 432, 120, math::NkColor(r, g, 90, 255));
14    source.DrawRect(40, 40, 432, 120, math::NkColor(255, 255, 255, 255));
15    for (int32 i = 0; i < 12; ++i)
16        source.DrawCircle(256, 300, 20 + i * 12, math::NkColor(0, 220, 255, 255 - i * 18));
17    source.DrawLine(0, 0, 511, 511, math::NkColor(255, 255, 255, 90));
18    source.DrawLine(0, 511, 511, 0, math::NkColor(255, 255, 255, 90));
19
20    // 3) Vignette : Resize renvoie un NkImage* -> il FAUT le liberer.
21    NkImage *vignette = source.Resize(192, 192, NkResizeFilter::NK_BILINEAR);
22    if (!vignette)
23        return false;
24
25    // 4) Aplattir en RGBA CONTIGU (Le stride ne vaut pas forcément w*4 : chapitre 5).
26    static NkVector<uint8> rgba; // statique : on evite de realloquer a chaque appel
27    const int32 w = vignette->Width();
28    const int32 h = vignette->Height();
29    rgba.Resize(static_cast<usize>(w) * h * 4u);
30    for (int32 y = 0; y < h; ++y) {
31        const uint8 *src = vignette->RowPtr(y);
32        uint8 *dst = rgba.Data() + static_cast<usize>(y) * w * 4u;
33        for (int32 x = 0; x < w * 4; ++x)
34            dst[x] = src[x];
35    }
36
37    const bool ok = backend.UploadImageRGBA(texId, rgba.Data(), w, h);
38    outW = w;
39    outH = h;
40
41    vignette->Free(); // OBLIGATOIRE : issu d'une fabrique
42    return ok;       // ~NkImage() libere `source`
43 }
```

Quatre points de vigilance sont concentrés dans cette fonction, et ce sont exactement les quatre du chapitre 5 :

1. source est sur la pile, donc rien à libérer;
2. Resize renvoie un pointeur, donc Free() obligatoire — y compris si vous ajoutez un return anticipé après cette ligne;
3. la copie passe par RowPtr(y), pas par un memcpy global;
4. le tampon rgba est static : on ne réalloue pas 147 Ko à chaque changement de curseur.

27.6.2 L'appeler, et l'afficher

Avant la boucle :

main.cpp — écrit pour le cours

```
1   constexpr uint32 kTexVignette = 0x56494731u;    // 'VIG1'
2   int32 vigW = 0, vigH = 0;
3   float32 teinte = 0.35f;
4   bool vignetteOk = ConstruireVignette(backend, kTexVignette, teinte, vigW, vigH);
5   float32 teinteAppliquee = teinte;
```

Dans la frame :

main.cpp — écrit pour le cours (API : NkGuiWidgets.h:39, 65, 99, 140)

```
1   SetNextWindowSize(ctx, 340.f, 420.f);
2   if (Begin(ctx, "Image (NKImage)")) {
3       Text(ctx, "Image dessinée à l'exécution, sans aucun fichier.");
4       Separator(ctx);
5
6       if (vignetteOk)
7           Image(ctx, kTexVignette, static_cast<float32>(vigW),
8 static_cast<float32>(vigH));
9       else
10          Text(ctx, "(construction de l'image échouée)");
11
12      Separator(ctx);
13      SliderFloat(ctx, "Teinte", teinte, 0.f, 1.f);
14      // On ne refabrique QUE si la valeur a changé : sinon on redessinerait
15      // et re-uploaderait 512x512 pixels à chaque frame pour rien.
16      if (teinte != teinteAppliquee) {
17          vignetteOk = ConstruireVignette(backend, kTexVignette, teinte, vigW, vigH);
18          teinteAppliquee = teinte;
19      }
20      EndWindow(ctx);
21  }
```

Ne refaites pas le travail à chaque image

Le test `teinte != teinteAppliquee` est la seule chose qui sépare une application fluide d'une application qui redessine et ré-uploade une texture soixante fois par seconde sans raison.

C'est un réflexe à prendre : dans une interface en mode immédiat, *déclarer* un widget est bon marché, mais *réagir* à un widget doit être conditionné au changement. Les widgets qui renvoient `true` en cas de modification — `SliderFloat`, `Checkbox`, `DragFloat` — sont faits pour cela.

27.7 Étape 5 — le son (NKAudio)

27.7.1 Synthétiser trois sons

AudioGenerator produit des `AudioSample` sans aucun fichier. C'est ce que font quatre applications du dépôt, dont Pong :

Applications/Pong/src/Pong/Audio/AudioManager.cpp:31-45

```

1      static AudioSample MakePaddleHit() {
2          AudioSample s = AudioGenerator::GenerateTone(
3              /*frequency*/ 880.0f,
4              /*duration */ 0.06f,
5              /*waveform */ WaveformType::SINE,
6              /*sampleRate*/ 48000,
7              /*amplitude*/ 0.85f);
8          AdsrEnvelope env;
9          env.attackTime = 0.002f; // attack immediate
10         env.decayTime = 0.02f;
11         env.sustainLevel = 0.5f;
12         env.releaseTime = 0.03f;
13         AudioGenerator::ApplyEnvelope(s, env);
14         return s;
15     }

```

L'enveloppe ADSR n'est pas un raffinement optionnel : sans elle, un son qui commence et s'arrête brutalement produit un clic audible — l'oreille entend la discontinuité. Deux millisecondes d'attaque suffisent à le supprimer.

Adaptons pour notre atelier :

main.cpp — écrit pour le cours (API : NkAudio.h:707, 758 ; modèle : Pong/AudioManager.cpp:31-45)

```

1  static audio::AudioSample FaireSon(float32 freq, float32 duree, audio::WaveformType forme) {
2      audio::AudioSample s = audio::AudioGenerator::GenerateTone(freq, duree, forme, 48000,
3          0.75f);
4      audio::AdsrEnvelope env;
5      env.attackTime = 0.003f;
6      env.decayTime = 0.02f;
7      env.sustainLevel = 0.5f;
8      env.releaseTime = 0.04f;
9      audio::AudioGenerator::ApplyEnvelope(s, env);
10     return s;

```

27.7.2 Initialiser, jouer, couper

Avant la boucle :

main.cpp — écrit pour le cours (API : NkAudio.h:1091, 1104)

```

1      audio::AudioEngine &engine = audio::AudioEngine::Instance();
2      const bool audioOk = engine.Initialize(audio::AudioEngineConfig{}); // backend AUTO
3      if (!audioOk)
4          logger.Error("[NkAtelier] audio indisponible : l'application continue en silence");
5
6      audio::AudioSample sonGrave, sonMedium, sonAigu;
7      if (audioOk) {
8          logger.Info("[NkAtelier] device audio : {0} Hz, {1} canaux", engine.GetSampleRate(),
9              engine.GetChannels());
10         sonGrave = FaireSon(220.f, 0.08f, audio::WaveformType::SQUARE);
11         sonMedium = FaireSon(440.f, 0.07f, audio::WaveformType::SINE);
12         sonAigu = FaireSon(880.f, 0.06f, audio::WaveformType::SINE);
13     }
14     bool sonsUiActifs = true;

```

Dans la frame :

main.cpp — écrit pour le cours (API : NkGuiWidgets.h:44-45 ; NkAudio.h:1127, 1266)

```
1      SetNextWindowSize(ctx, 340.f, 260.f);
2      if (Begin(ctx, "Son (NkAudio)")) {
3          if (!audioOk) {
4              Text(ctx, "Moteur audio indisponible sur cette machine.");
5          } else {
6              Text(ctx, "Trois sons synthetises, joues sur le bus UI.");
7              Separator(ctx);
8
9              audio::VoiceParams vp;
10             vp.bus = "UI";           // bus dedie : coupable sans toucher a la musique
11
12             if (Button(ctx, "Grave (220 Hz)"))
13                 engine.Play(sonGrave, vp);
14             if (Button(ctx, "Medium (440 Hz)"))
15                 engine.Play(sonMedium, vp);
16             if (Button(ctx, "Aigu (880 Hz)"))
17                 engine.Play(sonAigu, vp);
18
19             Separator(ctx);
20             if (Checkbox(ctx, "Sons d'interface", sonsUiActifs)) {
21                 if (audio::NkAudioBus *bus = engine.GetBus("UI"))
22                     bus->SetMute(!sonsUiActifs);
23             }
24         }
25         EndWindow(ctx);
26     }
```

Remarquez que Play est appelé sans récupérer le *handle* : ce sont des sons courts, on ne les pilotera pas. Remarquez aussi le if autour de GetBus : il renvoie nullptr si le moteur n'est pas initialisé.

N'écrivez jamais `cfg.backend = DIRECTSOUND`

Rappel du chapitre 7 : le backend DirectSound est un stub qui renvoie true sans rien ouvrir (Kernel/Runtime/NkAudio/src/NkAudio/NkAudioBackends.cpp:445-451). Vous auriez un moteur « initialisé », des voix « en lecture », et le silence complet. Laissez AUTO.

27.8 Étape 6 — la vidéo (NKMedia)

C'est la partie la plus ambitieuse, et elle est faite d'un aller-retour : on écrit une vidéo, puis on la relit.

27.8.1 Écrire la vidéo

main.cpp — écrit pour le cours (modèle : NKVideoTest/src/main.cpp:83-112)

```
1 // Ecrit une petite video MJPEG/AVI de `nframes` images. Renvoie true si OK.
2 static bool EcrireVideoDemo(const char *chemin, int32 w, int32 h, int32 fps, int32 nframes) {
3     media::NkVideoConfig cfg;
4     cfg.width = w;
5     cfg.height = h;
6     cfg.fpsNum = fps;
7     cfg.fpsDen = 1;
```

```

8   cfg.codec = media::NkVideoCodec::MJPEG;           // Le chemin le plus sur (chapitre 8)
9   cfg.container = media::NkVideoContainer::AVI;
10  cfg.quality = 88;
11
12  media::NkVideoWriter vw;
13  if (!vw.Open(chemin, cfg)) {
14      logger.Error("[NkAtelier] impossible d'ouvrir {0} en ecriture", chemin);
15      return false;
16  }
17
18  // Un SEUL tampon, reutilise. RGB24 : 3 octets par pixel, contigu, haut->bas.
19  uint8 *rgb = static_cast<uint8 *>(memory::NkAlloc(static_cast<usize>(w) * h * 3));
20  if (!rgb) {
21      vw.Close();
22      return false;
23  }
24
25  bool ok = true;
26  for (int32 f = 0; f < nframes && ok; ++f) {
27      // Motif : un carre qui traverse l'image + un fond qui defile.
28      const int32 cx = (w - 60) * f / (nframes > 1 ? nframes - 1 : 1);
29      for (int32 y = 0; y < h; ++y) {
30          uint8 *ligne = rgb + static_cast<usize>(y) * w * 3;
31          for (int32 x = 0; x < w; ++x) {
32              const bool dansCarre = (x >= cx && x < cx + 60 && y >= h / 2 - 30 && y < h /
33  2 + 30);
34              ligne[x * 3 + 0] = dansCarre ? 255 : static_cast<uint8>((x + f * 3) & 0xFF);
35              ligne[x * 3 + 1] = dansCarre ? 140 : static_cast<uint8>((y * 2) & 0xFF);
36              ligne[x * 3 + 2] = dansCarre ? 0 : 60;
37          }
38          ok = vw.WriteFrame(rgb, media::NkVideoInputFormat::RGB24);
39      }
40
41      memory::NkFree(rgb);
42      vw.Close();           // OBLIGATOIRE : ecrit l'index AVI. Sans lui : fichier illisible.
43      return ok;
44  }

```

Notez que `Close()` est appelé **sur tous les chemins**, y compris quand `ok` est devenu `false`. C'est la règle du chapitre 8 : sans finalisation, le fichier existe et n'est lisible par personne.

Notez aussi la construction de la ligne : `rgb + y * w * 3`. Ici, pas de *stride* aligné — nous fabriquons nous-mêmes un tampon contigu, c'est ce que `WriteFrame` attend. Le *stride* est une notion de `NkImage`, pas de `NkVideoWriter`.

27.8.2 Relire la vidéo

Avant la boucle :

main.cpp — écrit pour le cours (API : `NkVideoReader.h:54-66`)

```

1   constexpr uint32 kTexVideo = 0x56494431u;           // 'VID1'
2   const char *kCheminVideo = "nkatelier_demo.avi";
3
4   media::NkVideoReader lecteur;
5   bool videoOk = false;
6   double videoFps = 25.0;
7   int32 videoW = 0, videoH = 0, videoNb = -1;
8

```



```

9   if (EcrireVideoDemo(kCheminVideo, 320, 240, 25, 75)) { // 3 secondes
10      if (lecteur.Open(kCheminVideo)) {
11         const media::NkVideoReaderInfo &in = lecteur.Info();
12         videoW = in.width;
13         videoH = in.height;
14         videoNb = in.frameCount;
15         videoFps = (in.fps > 1.0) ? in.fps : 25.0;
16         videoOk = true;
17         logger.Info("[NkAtelier] video : {0} {1} {2}x{3} @ {4} fps",
18                     in.container.CStr(), in.codec.CStr(), in.width, in.height, in.fps);
19      } else {
20         logger.Error("[NkAtelier] video ecrite mais illisible : {0}", kCheminVideo);
21      }
22   }
23
24   media::NkVideoFrame trame; // DECLAREE HORS BOUCLE : son buffer est reutilise
25   float32 accumulateur = 0.f;
26   bool videoEnLecture = true;
27   bool videoFinie = false;
28   int32 videoIndex = -1;

```

Le fait que la lecture soit tentée juste après l'écriture est en soi un test : si Open échoue, c'est que l'écriture a mal tourné.

27.8.3 Cadencer et afficher

main.cpp — écrit pour le cours (modèle : NKCode/Shell/NkVideoViewer.h:153-170)

```

1   SetNextWindowSize(ctx, 400.f, 420.f);
2   if (Begin(ctx, "Video (NKMedia)")) {
3       if (!videoOk) {
4           Text(ctx, "Video indisponible.");
5       } else {
6           // — Cadencement —————
7           const float32 dureeImage = 1.f / static_cast<float32>(videoFps);
8           if (videoEnLecture && !videoFinie) {
9               accumulateur += dt;
10              if (accumulateur > dureeImage * 4.f)
11                  accumulateur = 0.f; // evite le rattrapage explosif
12              while (accumulateur >= dureeImage) {
13                  accumulateur -= dureeImage;
14                  if (lecteur.ReadFrame(trame)) {
15                      videoIndex = trame.index;
16                      // Une seule image decodee -> un seul upload.
17                      backend.UploadImageRGBA(kTexVideo, trame.rgba.Data(),
trame.width, trame.height);
18                  } else {
19                      videoFinie = true; // fin du fichier
20                      break;
21                  }
22              }
23          }
24
25          // — Affichage —————
26          if (videoIndex >= 0)
27              Image(ctx, kTexVideo, static_cast<float32>(videoW),
static_cast<float32>(videoH));
28          else
29              Text(ctx, "(premiere image en attente)");
30

```

```

31         Separator(ctx);
32         Text(ctx, videoFinie ? "Etat : terminee" : (videoEnLecture ? "Etat : lecture"
: "Etat : pause"));
33
34         BeginHBox(ctx);
35         if (Button(ctx, videoEnLecture ? "Pause" : "Lecture"))
36             videoEnLecture = !videoEnLecture;
37         if (Button(ctx, "Debut")) {
38             if (lecteur.SeekFrame(0)) {
39                 videoFinie = false;
40                 accumulateur = 0.f;
41                 videoIndex = -1;
42             }
43         }
44         EndHBox(ctx);
45     }
46     EndWindow(ctx);
47 }

```

Les trois règles de la lecture vidéo, réunies

1. **Un seul upload par image affichée.** La boucle while peut décoder plusieurs trames si l'application a pris du retard, mais on n'uploade que celle qu'on va montrer;
2. **l'accumulateur est borné ($> \text{dureeImage} * 4$) :** sans ce garde-fou, un décrochage de deux secondes déclencherait le décodage de cinquante images d'un coup, ce qui aggraverait le décrochage;
3. **NkVideoFrame est déclarée hors boucle.** Son NkVector garde sa capacité; la déclarer à l'intérieur provoquerait une allocation par image.

Le rebouclage n'est exact que sur les formats non compressés en temps

SeekFrame(0) fonctionne bien ici parce que notre vidéo est en MJPEG : chaque image est indépendante. Sur du H.264, SeekFrame(n) ne fait que revenir à l'image autonome précédente, et il faut redécoder vers l'avant jusqu'à la cible (chapitre 8). Notre choix du MJPEG n'était donc pas seulement un choix de prudence : il simplifie aussi la navigation.

L'upload répété dépend de NkTexture::Update

Regardez l'implémentation du pont : UploadImageRGBA crée la texture au premier appel, puis appelle tex->Update(...) aux suivants ([Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:94-117](#)). C'est exactement le bon comportement — on ne recrée pas la texture par image.

Mais souvenez-vous de l'avertissement de Pong (chapitre 6) : «Update exige gTextureBackend.Update qui n'est pas câblé sur tous les backends → échec silencieux» ([Applications/Pong/src/Pong/Render/FontAtlas.cpp:107-109](#)).

Si votre vignette statique s'affiche mais que la vidéo reste figée sur sa première image, la cause est probablement là — et la solution est de changer de backend graphique, pas de réécrire votre code de lecture.

27.9 Étape 7 — la fermeture

C'est le moment où l'on récolte ce qu'on a semé dans les cinq chapitres précédents. Chaque module a son ordre, et ils s'imbriquent.

main.cpp — écrit pour le cours (invariants : NkAudio.h:1114 ; NkAudioPlayer/main.cpp:258-259)

```
1 // — Fermeture, dans cet ordre exact —————
2
3 // 1) Video : fermer Le lecteur (il possede son etat interne).
4 lecteur.Close();
5
6 // 2) Audio : ARRETER LE MOTEUR AVANT de Liberer Les samples.
7 if (audioOk) {
8     engine.Shutdown();
9     audio::AudioLoader::Free(sonGrave);
10    audio::AudioLoader::Free(sonMedium);
11    audio::AudioLoader::Free(sonAigu);
12 }
13
14 // 3) Interface.
15 SetCurrentContext(nullptr);
16 ctx.Shutdown();
17 return 0;
18 // Les NkUniquePtr (target, ctxPtr, fontPtr) se detruisent ici, dans L'ordre
19 // inverse de Leur declaration. Le backend et l'atlas de police suivent.
20 }
```

Module	Ce qu'on ferme	Piège si l'ordre est inversé
NKMedia	lecteur.Close()	fichier verrouillé
NKAudio	Shutdown() puis Free()	lecture après libération
NKImage	rien (pile) ou Free()	fuite ou double libération
NKFont	rien (l'atlas possède tout)	double libération si fusion
NKGui	ctx.Shutdown()	—

Le principe unique derrière tous ces ordres

On éteint d'abord ce qui **tourne**, ensuite ce qui est **lu**.

Le moteur audio a un fil qui lit vos échantillons; le gestionnaire de connexion a un fil qui lit vos tampons; l'enregistreur vidéo a un fil qui lit vos images. Dans les trois cas, la fonction d'arrêt joint le fil — et ce n'est qu'après qu'on a le droit de libérer ce qu'il lisait. Si vous ne deviez retenir qu'une phrase de ces six chapitres, ce serait celle-là : elle explique l'ordre de fermeture de NKAudio, de NKNetwork et de NKMedia d'un seul coup.

27.10 Compiler et lancer

Ligne de commande (voir README.md:147-163)

```
1 jenga build --target NkAtelier --config Debug
```

L'exécutable atterrit dans `Build/Bin/Debug-Windows/NkAtelier/` — ou `Debug-Linux`, `Debug-macOS` selon la plateforme, suivant le `targetdir` que nous avons écrit dans le `.jenga`.

Pour la version optimisée :

Ligne de commande

```
1 jenga build --target NkAtelier --config Release
```

Le répertoire courant au lancement

Notre application écrit `nkatelier_demo.avi` dans le **répertoire courant**, pas à côté de l'exécutable. Selon que vous la lancez depuis un terminal, depuis un explorateur de fichiers ou depuis un environnement de développement, ce répertoire diffère.

Ce n'est pas grave ici — nous écrivons *et* lisons au même endroit dans la même exécution — mais c'est exactement le mécanisme qui fait qu'un `assets/logo.png` « introuvable » l'est seulement dans certaines conditions de lancement. Si vous voulez voir le fichier produit, lancez l'application depuis un terminal, dans un dossier que vous avez choisi.

Souvenez-vous aussi que les auto-tests de NKMedia écrivent, eux aussi, dans le répertoire courant (chapitre 8).

27.11 Le catalogue des silences

Voici la partie la plus utile de ce chapitre. Toutes les pannes ci-dessous ont un point commun : aucune ne produit de message d'erreur. Gardez cette liste sous la main.

Symptôme	Cause à vérifier en premier
Fenêtre noire, rien du tout	Backend graphique. Essayez <code>NK_GFX_API_OPENGL</code> .
Tout s'affiche sauf les textes	Atlas de police non uploadé, ou <code>ctx.font</code> non affecté.
Les menus passent <i>derrière</i>	<code>Second Submit(ctx.dlOverlay, ...)</code> oublié.
L'image apparaît « penchée »	<code>memcpy</code> global au lieu de <code>RowPtr(y)</code> .
L'image est vide	Texture créée avant <code>renderer.Initialize()</code> .
Aucun son, mais tout indique que ça joue	Backend <code>DIRECTSOUND</code> demandé explicitement.
Grésillement ou plantage à la fermeture	<code>Free(sample)</code> appelé avant <code>engine.Shutdown()</code> .
Le fichier vidéo est illisible	<code>Close()</code> non appelé sur le <code>NkVideoWriter</code> .
La vidéo reste sur la première image	<code>NkTexture::Update</code> non câblé dans ce backend.
La vidéo est à l'envers	<code>flipVertical</code> avec un framebuffer OpenGL.
Vignette de mauvaise qualité malgré Lanczos	<code>NK_LANCZOS3</code> retombe sur du bilinéaire.
Aucun pair ne se connecte	Poignée de main asynchrone : attendre <code>ConnectedPeerCount()</code> .
Erreurs d'édition de liens <code>_imp_socket</code>	<code>ws2_32</code> absent du <code>links()</code> .

La méthode générale

Ces silences ont tous la même parade, et c'est une habitude à prendre plutôt qu'une astuce : **journalisez ce que vous chargez, et journalisez aussi ce que vous ne chargez**

pas.

Toutes les fonctions d'initialisation de ce chapitre ont un `else` qui écrit une ligne. Cela paraît excessif quand tout fonctionne. Le jour où quelque chose manque, cette ligne vous fait gagner une heure — et c'est précisément ce que fait la démo du dépôt : « Image demo introuvable (cwd) -> section image vide » (`Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:422`).

27.12 Le programme, vu de haut

Récapitulons la structure complète, dans l'ordre du fichier :

Applications/NkAtelier/src/main.cpp — plan d'ensemble

```
1 // --- inclusions (etape 2) ---
2
3 // --- fonctions utilitaires ---
4 static bool ConstruireVignette(backend, texId, teinte, &w, &h); // etape 4
5 static audio::AudioSample FaireSon(freq, duree, forme); // etape 5
6 static bool EcrireVideoDemo(chemin, w, h, fps, nframes); // etape 6
7
8 int nkmain(const NkEntryState &state) {
9     // 1. fenetre + cible de rendu // etape 2
10    // 2. contexte NKGui + backend // etape 2
11    // 3. police embarquee + upload atlas // etape 2
12    // 4. callbacks d'evenements // etape 3
13    // 5. image : premiere construction + upload // etape 4
14    // 6. audio : Initialize + synthese des trois sons // etape 5
15    // 7. video : ecriture du fichier puis ouverture en lecture // etape 6
16
17    while (running && window.IsOpen()) {
18        // dt + pompage des evenements + synchro swapchain // etape 3
19        ctx.BeginFrame(dt);
20        // panneau Image // etape 4
21        // panneau Son // etape 5
22        // panneau Video (cadencement + upload + affichage) // etape 6
23        ctx.EndFrame();
24        // Clear + Submit x2 + Display // etape 3
25    }
26
27    // fermeture : video, puis audio (Shutdown avant Free), puis UI // etape 7
28 }
```

Environ 300 lignes en tout. C'est peu pour cinq modules — et c'est le point. La difficulté du moteur n'est pas dans la quantité de code à écrire, elle est dans les ordres à respecter : ordre d'initialisation, ordre de la frame, ordre de fermeture.

27.13 Pour aller plus loin

Trois extensions, par difficulté croissante. Chacune réutilise un chapitre précédent sans rien introduire de nouveau.

27.13.1 Enregistrer ce que l'atelier affiche

Ajoutez un bouton « Enregistrer » qui, à chaque frame, capture la fenêtre et pousse l'image dans un `NkVideoRecorder` en mode MJPEG :

```

1  if (enregistrement && target->CaptureToImage(capture) && capture.IsValid()) {
2      if (!rec.IsRecording())
3          rec.Begin("nkatelier_capture.mp4", capture.Width(), capture.Height(),
4                  30, 1, 22, 32, media::NkRecorderCodec::MJPEG, 92);
5      if (rec.IsRecording())
6          rec.PushVideo(capture.Pixels(), media::NkVideoInputFormat::RGBA32);
7  }
8  // ... et, sur TOUS les chemins de sortie :
9  if (rec.IsRecording())
10     rec.End();

```

Attention aux deux pièges du chapitre 8 : en mode MJPEG l'audio est ignorée silencieusement, et `End()` doit être appelé même si la fenêtre est fermée brutalement.

27.13.2 Ajouter le réseau

Ajoutez à `NkAtelier` un quatrième panneau : deux boutons « Héberger » et « Rejoindre », et une liste de messages. Chaque clic sur un bouton de son envoie un message aux autres instances, qui jouent le même son.

Vous aurez besoin d'ajouter `NKNetwork` aux dépendances du `.jenga` et `ws2_32` au `links()` sous Windows. Le chapitre 9 donne la structure à suivre : une petite classe de session, un `Tick(dt)` par frame, et une interface qui ne voit qu'un état.

27.13.3 Faire du décodage un travail de fond

Notre vidéo fait 320 par 240 pixels : le décodage est instantané. Essayez avec une vidéo de 1920 par 1080, et vous verrez la boucle hoqueter.

La solution — décoder sur un fil, uploader sur le fil principal — est celle du chapitre 5, appliquée à la vidéo. C'est un excellent exercice, mais commencez par **mesurer** : entourez `ReadFrame` d'un chronomètre mural et regardez. Et souvenez-vous de l'avertissement du chapitre 8 : mesurez le temps mur, pas le *PTS* de la vidéo.

Ce chapitre en bref

Une application complète n'est pas la somme des chapitres : c'est leur *ordre*. Créer avant d'employer, fermer dans le sens inverse de l'ouverture, et n'ouvrir une ressource qu'après ce dont elle dépend. Le « catalogue des silences » est la partie la plus utile de ce chapitre : dans chaque module, il existe des façons d'échouer qui ne disent rien — un fichier absent, un identifiant nul, une passe sans attachement — et la seule parade est de *vérifier ce qu'on reçoit* plutôt que de supposer que l'absence d'erreur vaut succès.

27.14 Exercices

1 — Faire tomber chaque panneau, un par un

Pour chacun des quatre modules, cassez volontairement une chose et notez ce que vous observez :

- n'appellez pas `UploadFontGray8`;
- supprimez le vignette->`Free()` et lancez l'application quelques minutes en bougeant le curseur de teinte (surveillez la mémoire);

- inversez `Shutdown()` et `Free()` à la fermeture;
- supprimez le `vw.Close()` dans `EcrireVideoDemo`.

Écrivez, pour chaque cas, la phrase que vous auriez aimé lire dans un journal. Puis ajoutez-la vraiment à votre code. Vous venez d'écrire votre propre catalogue des silences.

2 — Une vraie vignette de fichier

Ajoutez un cinquième panneau qui charge une image *depuis le disque* avec `NkImage::Load`, en essayant plusieurs chemins candidats comme le fait la démo du dépôt. Affichez le format d'origine (`SourceFormat()`), les dimensions, le nombre de canaux et le *stride*. Faites-en une vignette de 128 pixels de large en préservant le rapport d'aspect, et affichez côte à côte l'original réduit et la vignette. Puis ajoutez un bouton qui sauvegarde la vignette en PNG et un autre qui tente de la sauvegarder en WebP. Affichez les deux valeurs de retour côte à côte. Que constatez-vous ?

3 — Le mélangeur

Ajoutez trois `SliderFloat` — Master, UI, Music — reliés aux bus correspondants par `GetBus(...)->SetVolume(...)`, en n'appelant le *setter* que lorsque la valeur change. Ajoutez ensuite un son long en boucle sur le bus « Music » (`vp.looping = true`) et vérifiez, à l'oreille, la formule du chapitre 7 : le volume effectif est le produit du volume de la voix, de celui du bus, et de tous ses parents jusqu'au Master. Vérifiez enfin que couper « UI » ne touche pas à « Music ».

4 — Une chaîne complète, de l'image à la vidéo et retour

Modifiez `EcrireVideoDemo` pour qu'elle ne dessine plus ses pixels à la main, mais qu'elle utilise une `NkImage` et ses primitives de dessin — comme `ConstruireVignette` — puis qu'elle envoie `image.Pixels()` à `WriteFrame`. Attention : `WriteFrame` attend un tampon **contigu**, et `NkImage::Pixels()` a un *stride* aligné sur 4 octets. Choisissez donc des dimensions telles que `stride == width * bpp`, ou bien recopiez ligne par ligne. Vérifiez votre raisonnement en affichant les deux valeurs avant d'écrire la première image. Relisez ensuite la vidéo produite et affichez-la : vous aurez fait circuler les mêmes pixels à travers `NKImage`, le codec JPEG, le conteneur AVI, le décodeur, et enfin le GPU. C'est toute la chaîne de ce cours en une seule fonction.

5 — Le vôtre

Reprenez `NkAtelier` et transformez-le en quelque chose qui vous est utile : une visionneuse de dossier d'images, un métronome, un petit lecteur de séquences, un banc d'essai de vos propres filtres d'image. Une seule contrainte, et c'est celle de tout ce cours : **pour chaque fonction du moteur que vous emploierez, vérifiez dans la source qu'elle fait bien ce que son nom annonce**. Vous savez maintenant où regarder — le `.cpp`, pas le commentaire ; le `ROADMAP.md`, pas le `.jenga` — et vous savez que la réponse est parfois « non ».

Questions de cours

1. Dans quel ordre crée-t-on fenêtre, contexte, backend et police ? Pourquoi pas un autre ?

2. Pourquoi ferme-t-on dans l'ordre inverse de l'ouverture ?
3. Citez trois « silences » du catalogue, et ce qui les rend difficiles à voir.
4. Que faut-il faire quand une taille de fenêtre change ?
5. Qu'est-ce qui, dans ce programme, ne pourrait pas être écrit sans les chapitres précédents ?

Exercice de logique

Votre application se lance, la fenêtre s'ouvre, tout est noir. Aucun message d'erreur.

1. Énumérez au moins cinq causes possibles, en les rangeant par étage de la pile.
2. Pour chacune, dites ce qui la distinguerait des autres — une observation dont le résultat *diffère* selon la cause.
3. Dans quel ordre feriez-vous ces vérifications ? Justifiez par le *coût* de chacune, pas par sa probabilité — et expliquez pourquoi ce critère est le bon quand on ne sait rien.

Ce que cette partie vous laisse

Dix applications, et une seule structure : ce qui *décide* ne dessine pas, ce qui *dessine* ne décide pas, et ce qui *prouve* n'a besoin ni de l'un ni de l'autre. C'est cette séparation qui rend un banc possible — et un banc est la seule chose qui distingue un programme qui marche d'un programme dont on *sait* qu'il marche. Vous avez aussi vu, dans du code réel, une dette assumée et une dette muette : la différence n'est pas dans le code, elle est dans ce qui est écrit à côté.